

OCA OCPP 2.1: Part 6 - Test case document Charging Station Management System for OCTT

DRAFT Thu May 21 09:44:04 2026 +0200

Table of Contents

- 1. Introduction
- 2. A Security
- 3. B Provisioning
- 4. C Authorization
- 5. D Local Authorization List Management
- 6. E Transactions
- 7. F Remote Control
- 8. G Availability
- 9. H Reservation
- 10. I Tariff and Cost
- 11. J Meter Values
- 12. K Smart Charging
- 13. L Firmware Management
- 14. M Certificate Management
- 15. N Diagnostics
- 16. O Display Message
- 17. P Data Transfer
- 18. Q Bidirectional Power Transfer
- 19. R DER Control
- 20. S Battery Swapping
- 21. Reusable states

Copyright © 2010 – 2026 Open Charge Alliance. All rights reserved.

Copyright and Restrictions on Circulation and Use

This OCPP 2.1 Test Cases Draft document (set) is protected by copyright held by the Open Charge Alliance, and is made available only for the purpose of reviewing this draft by OCA participants / members during the review period, to make comments/proposals for the purpose of improving the specification and by users of the OCTT for the purpose of running the OCTT software with their implementation(s) / system.

All other rights are reserved; in particular any party receiving this document (set) is explicitly prohibited from distributing it in any form.

Following the release of any subsequent version (whether draft or final) of the OCPP 2.1 (Part 6) Test cases specification set by OCA, receiving parties are prohibited from using the content of this draft for any purpose, including particularly as a basis for protocol or test tool implementations, unless specific written permission by OCA is given or a separate license for this is acquired.

Version History

Version	Reviewed by	Modified by	Description
OCPP 2.1 Edition 3 DRAFT	2026-05-22	Open Charge Alliance	OCPP 2.1 Edition 3 DRAFT.
OCPP 2.1 Edition 2	2025-12-03	Open Charge Alliance	OCPP 2.1 Edition 2. This is the first published version of OCPP 2.1 Part 6.

DRAFT

1. Introduction

About this document

This document is created to describe a set of valid test cases for OCPP 2.1. These test cases can be executed using a Test System for OCPP 2.1. The scenarios in the tool are described in detail including the expected behaviour of the System Under Test (SUT). This document is divided in chapters, each describing an OCPP functional block as can be found in the official OCPP specification. These are:

- A. Security
- B. Provisioning
- C. Authorization
- D. Local Authorization List Management
- E. Transactions
- F. Remote Control
- G. Availability
- H. Reservation
- I. Tariff and Cost
- J. Meter Values
- K. Smart Charging
- L. Firmware Management
- M. Certificate Management
- N. Diagnostics
- O. Display Message
- P. Data Transfer
- Q. Bidirectional Power Transfer
- R. DER Control
- S. Battery Swapping

The scenarios in this document are also part of the OCA certification process of OCPP. Please refer to OCPP 2.1 Part 5 - Certification Profiles for more information about the relation between certification profiles and the test scenarios in this document.

Conventions

The following conventions / rules apply to all test cases, unless explicitly mentioned otherwise. These will not be mentioned separately at every test case.

- The OCPP specification is always leading.
- This document does not specify which tests need to be passed for certification, this will be specified in a separate document.
- All messages shall comply with the OCPP 2.1 schemas from the OCPP specification.
- The messages are to be sent as mentioned in the scenario details.
- Validations will be mentioned and grouped per step.
- Messages, datatypes and configuration variables will convey to the following formatting rules:
 - Datatypes, messages and configuration variables are displayed bold.
 - Values are displayed italic.

General pre/post conditions & tool validations

General conditions/validations are overruled by testcase specific conditions/validations, unless specifically stated otherwise.

General pre conditions:

The following pre conditions apply to all test cases, unless explicitly mentioned otherwise.

- The Configuration variable **TxCtrlr.TxStartPoint** is *"EVConnected,Authorized"*
- The Configuration variable **TxCtrlr.TxStopPoint** is *"EVConnected"*
- The Configuration variable **AuthCtrlr.AuthEnabled** is *true*
- The Configuration variable **AuthCtrlr.AuthorizeRemoteStart** is *false*
- The Configuration variable **AdditionalRootCertificateCheck** is *false*
- The Configuration variable **AllowNewSessionsPendingFirmwareUpdate** is *false*
- The Configuration variable **AlignedDataSendDuringIdle** is *false*

General tool rules/validations:

- The list of ChargingSchedulePeriod elements in a chargingSchedule SHALL be ordered by increasing values of ChargingSchedulePeriod.startPeriod. This means the list is in chronological order.
- The CSMS SHALL NOT set phaseToUse in a SetChargingProfileRequest when numberPhases is other than 1.

2. A Security

TC_A_01_CSMS: Basic Authentication - Valid username/password combination

Test case name	Basic Authentication - Valid username/password combination
Test case Id	TC_A_01_CSMS
Use case Id(s)	<u>A00</u> , <u>B01</u>
Requirement(s)	<u>A00.FR.204</u> , <u>B01.FR.02</u>
System under test	CSMS
Description	The Charging Station uses Basic authentication to authenticate itself to the CSMS, when using security profile 1 or 2.
Purpose	To verify whether the CSMS is able to validate the (valid) Basic authentication credentials provided by the Charging Station at the connection request.
Prerequisite(s)	The CSMS supports security profile 1 and/or 2

Before (Preparations)

Configuration State:

The CSMS must have a password configured that equals the configured BasicAuthPassword at the Test System.

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a HTTP upgrade request with an Authorization header, containing a username/password combination. <u>Note(s):</u> - The Authorization header is formatted as follows: AUTHORIZATION: Basic <Base64 encoded(<Configured ChargingStationId>:<Configured BasicAuthPassword>)>	2. The CSMS validates the username/password combination AND upgrades the connection to a (secured) WebSocket connection.
3. The Test System sends a BootNotificationRequest	4. The CSMS responds with a BootNotificationResponse
5. The Test System notifies the CSMS about the current state of all connectors.	6. The CSMS responds accordingly.

Tool validations

* Step 4:

Message: **BootNotificationResponse**

- **status** must be *Accepted*

Tool validations**Post scenario validations:**

N/a

TC_A_02_CSMS: Basic Authentication - Username does not equal ChargingStationId

Test case name	Basic Authentication - Username does not equal ChargingStationId
Test case Id	TC_A_02_CSMS
Use case Id(s)	<u>A00</u>
Requirement(s)	<u>A00.FR.204</u>
System under test	CSMS
Description	The Charging Station uses Basic authentication to authenticate itself to the CSMS, when using security profile 1 or 2.
Purpose	To verify whether the CSMS is able to validate the (invalid) Basic authentication credentials provided by the Charging Station at the connection request.
Prerequisite(s)	The CSMS supports security profile 1 and/or 2

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

1. The Test System sends a HTTP upgrade request with an Authorization header, containing a username/password combination.

Note(s):

- The Authorization header is formatted as follows:
 AUTHORIZATION: Basic <Base64 encoded(<Configured ChargingStationId> + Invalid:<Configured basicAuthPassword>)>

CSMS

2. The CSMS validates the username/password combination AND rejects the connection upgrade request.

Tool validations

N/a

Post scenario validations:

N/a

TC_A_03_CSMS: Basic Authentication - Invalid password

Test case name	Basic Authentication - Invalid password
Test case Id	TC_A_03_CSMS
Use case Id(s)	<u>A00</u>
Requirement(s)	<u>A00.FR.204</u>
System under test	CSMS
Description	The Charging Station uses Basic authentication to authenticate itself to the CSMS, when using security profile 1 or 2.
Purpose	To verify whether the CSMS is able to validate the (invalid) Basic authentication credentials provided by the Charging Station at the connection request.
Prerequisite(s)	The CSMS supports security profile 1 and/or 2

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a HTTP upgrade request without an Authorization header.	2. The CSMS rejects the connection upgrade request.
3. The Test System sends a HTTP upgrade request with an Authorization header, containing a username/password combination. <u>Note(s):</u> - The Authorization header is formatted as follows: <i>AUTHORIZATION: Basic <Base64 encoded(<Configured ChargingStationId>:<randomly chosen identifierString with a sufficiently high entropy, consisting of minimum 16 and maximum 40 characters (alpha-numeric characters and the special characters allowed by identifierString)>></i>	4. The CSMS validates the username/password combination AND rejects the connection upgrade request.

Tool validations

N/a

Post scenario validations:

N/a

TC_A_04_CSMS: TLS - server-side certificate - Valid certificate

Test case name	TLS - server-side certificate - Valid certificate
Test case Id	TC_A_04_CSMS
Use case Id(s)	A00
Requirement(s)	A00.FR.306,A00.FR.307,A00.FR.312,A00.FR.318,A00.FR.321,A00.FR.502,A00.FR.503,A00.FR.507,A00.FR.508,A00.FR.510
System under test	CSMS
Description	The CSMS uses a server-side certificate to identify itself to the Charging Station, when using security profile 2 or 3.
Purpose	To verify whether the CSMS is able to provide a valid server certificate and setup a secured WebSocket connection.
Prerequisite(s)	The CSMS supports security profile 2 and/or 3

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System terminates the connection and initiates a TLS handshake and sends a Client Hello to the CSMS.	2. The CSMS responds with a Server Hello With the <Configured server certificate>
3. The Test System performs the following actions: Send client certificate Client Key Exchange Certificate verify Change Cipher Spec Finished <u>Note(s):</u> - The client certificate is only sent when the CSMS uses security profile 3.	4. The CSMS performs the following actions: Change Cipher Spec Finished
5. The Test System sends a HTTP upgrade request to the CSMS <u>Note(s):</u> - The HTTP request only contains a username/password combination when the CSMS uses security profile 2.	6. The CSMS upgrades the connection to a (secured) WebSocket connection.
7. The Test System sends a BootNotificationRequest with reason PowerUp chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	8. The CSMS responds with a BootNotificationResponse

Main (Test scenario)

9. The Test System notifies the CSMS about the current state of all connectors.

Message: **StatusNotificationRequest**

- **connectorStatus** *Available*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Available"*

- **component.name** *"Connector"*

- **variable.name** *"AvailabilityState"*

10. The CSMS responds accordingly.

Tool validations

* Step 3:

The Test System validates the following before finishing the TLS handshake:

- The CSMS must use TLS version 1.2 or above

At least the following set of cipher suites must be supported:

TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256

AND

TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384

AND

TLS_RSA_WITH_AES_128_GCM_SHA256

AND

TLS_RSA_WITH_AES_256_GCM_SHA384

- When using RSA or DSA the key must be at least 2048 bits long.

and when using elliptic curve cryptography the key must be at least 224 bits long.

- The received server side certificate must be transmitted in the X.509 format encoded in Privacy-Enhanced Mail (PEM) format.

- The certificate must include a serial number.

- The subject field of the certificate must contain a commonName RDN which consists of the FQDN of the endpoint of the server.

NOTE: If one of the above validations fails, the Test System can still proceed with the next steps of the testcase (if it is able to), but the testcase will FAIL and the Test System reports why it failed.

* Step 8:

Message: **BootNotificationResponse**

with **status** *Accepted*

Post scenario validations:

N/a

TC_A_06_CSMS: TLS - server-side certificate - TLS version too low

Test case name	TLS - server-side certificate - TLS version too low
Test case Id	TC_A_06_CSMS
Use case Id(s)	<u>A00</u>
Requirement(s)	<u>A00.FR.314,A00.FR.315,A00.FR.409,A00.FR.416,A00.FR.417,A00.FR.418</u>
System under test	CSMS
Description	The CSMS uses a server-side certificate to identify itself to the Charging Station, when using security profile 2 or 3.
Purpose	To verify whether the CSMS is able to terminate the connection when it notices the used TLS version is lower than 1.2.
Prerequisite(s)	The CSMS supports security profile 2 and/or 3

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System terminates the connection and initiates a TLS handshake with a TLS version lower than 1.2 and sends a Client Hello to the CSMS.	2. The CSMS notices that the TLS version is lower than 1.2 and terminates the connection.
3. The Test System initiates a TLS handshake with TLS version 1.2 or higher and sends a Client Hello to the CSMS.	4. The CSMS responds with a Server Hello With the <Configured server certificate>
5. The Test System performs the following actions: Send client certificate Client Key Exchange Certificate verify Change Cipher Spec Finished <u>Note(s):</u> - The client certificate is only sent when the CSMS uses security profile 3.	6. The CSMS performs the following actions: Change Cipher Spec Finished
7. The Test System sends a HTTP upgrade request to the CSMS <u>Note(s):</u> - The HTTP request only contains a username/password combination when the CSMS uses security profile 2.	8. The CSMS upgrades the connection to a (secured) WebSocket connection.

Main (Test scenario)	
9. The Test System sends a BootNotificationRequest with reason <i>PowerUp</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	10. The CSMS responds with a BootNotificationResponse
11. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest - connectorStatus <i>Available</i> Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Available"</i> - component.name <i>"Connector"</i> - variable.name <i>"AvailabilityState"</i>	12. The CSMS responds accordingly.
Tool validations	
* Step 10: Message: BootNotificationResponse - status <i>Accepted</i>	
Post scenario validations: N/a	

TC_A_07_CSMS: TLS - Client-side certificate - valid certificate

Test case name	TLS - Client-side certificate - valid certificate
Test case Id	TC_A_07_CSMS
Use case Id(s)	<u>A00</u>
Requirement(s)	<u>A00.FR.409,A00.FR.410,A00.FR.415,A00.FR.416,A00.FR.421</u>
System under test	CSMS
Description	The Charging Station uses a client-side certificate to identify itself to the CSMS, when using security profile 3.
Purpose	To verify whether the CSMS is able to receive a client certificate provided by a Charging Station and setup a secured WebSocket connection.
Prerequisite(s)	The CSMS supports security profile 3

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System terminates the connection and initiates a TLS handshake and sends a Client Hello to the CSMS.	2. The CSMS responds with a Server Hello With the <Configured server certificate>
3. The Test System performs the following actions: Send <Configured client certificate> Client Key Exchange Certificate verify Change Cipher Spec Finished	4. The CSMS performs the following actions: Change Cipher Spec Finished
5. The Test System sends a HTTP upgrade request to the CSMS	6. The CSMS upgrades the connection to a (secured) WebSocket connection.
7. The Test System sends a BootNotificationRequest with reason <i>PowerUp</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	8. The CSMS responds with a BootNotificationResponse

Main (Test scenario)

9. The Test System notifies the CSMS about the current state of all connectors.

Message: **StatusNotificationRequest**

- **connectorStatus** *Available*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Available"*

- **component.name** *"Connector"*

- **variable.name** *"AvailabilityState"*

10. The CSMS responds accordingly.

Tool validations

* Step 3:

The Test System validates the following before finishing the TLS handshake:

- The CSMS must use TLS version 1.2 or above

At least the following set of cipher suites must be supported:

TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256

AND

TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384

AND

TLS_RSA_WITH_AES_128_GCM_SHA256

AND

TLS_RSA_WITH_AES_256_GCM_SHA384

* Step 8:

Message: **BootNotificationResponse**

with **status** *Accepted*

Post scenario validations:

N/a

TC_A_08_CSMS: TLS - Client-side certificate - Invalid certificate

Test case name	TLS - Client-side certificate - Invalid certificate
Test case Id	TC_A_08_CSMS
Use case Id(s)	<u>A00</u>
Requirement(s)	<u>A00.FR.405,A00.FR.407,A00.FR.409,A00.FR.410</u>
System under test	CSMS
Description	The Charging Station uses a client-side certificate to identify itself to the CSMS, when using security profile 3.
Purpose	To verify whether the CSMS is able to terminate the connection when the received client certificate is invalid.
Prerequisite(s)	- The CSMS supports security profile 3 - This testcase can be executed multiple times, using different kinds of invalid certificates: Unknown certificate expired certificate certificate with commonName that does not equal the serial number of the Charging Station.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System initiates a TLS handshake and sends a Client Hello to the CSMS.	2. The CSMS responds with a Server Hello With a server certificate
3. The Test System performs the following actions: Send <Configured invalid client certificate> Client Key Exchange Certificate verify Change Cipher Spec Finished	4. The CSMS deems the client certificate invalid and terminates the connection.
5. The Test System initiates a TLS handshake and sends a Client Hello to the CSMS.	6. The CSMS responds with a Server Hello With a server certificate

Main (Test scenario)	
7. The Test System performs the following actions: Send <Configured client certificate> Client Key Exchange Certificate verify Change Cipher Spec Finished	8. The CSMS performs the following actions: Change Cipher Spec Finished
9. The Test System sends a HTTP upgrade request to the CSMS	10. The CSMS upgrades the connection to a (secured) WebSocket connection.
11. The Test System sends a BootNotificationRequest with reason <i>PowerUp</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	12. The CSMS responds with a BootNotificationResponse
13. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest - connectorStatus <i>Available</i> Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Available"</i> - component.name <i>"Connector"</i> - variable.name <i>"AvailabilityState"</i>	14. The CSMS responds accordingly.
Tool validations	
* Step 12: Message: BootNotificationResponse with status <i>Accepted</i>	
Post scenario validations: N/a	

TC_A_09_CSMS: Update Charging Station Password for HTTP Basic Authentication - Accepted

Test case name	Update Charging Station Password for HTTP Basic Authentication - Accepted
Test case Id	TC_A_09_CSMS
Use case Id(s)	<u>A01</u>
Requirement(s)	<u>A01.FR.02</u> , <u>A01.FR.03</u>
System under test	CSMS
Description	This test case defines how to use the BasicAuthPassword, the password used to authenticate Charging Stations in security profile 1 (Basic Authentication) and security profile 2 (TLS with Basic Authentication)
Purpose	To verify if the CSMS is able to successfully set the new BasicAuthPassword and only accepts the new credentials as described at the OCPP specification.
Prerequisite(s)	The CSMS supports security profile 1 and/or 2

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetVariablesResponse with status Accepted	1. The CSMS sends a SetVariablesRequest with: setVariableData[1]: - variable.name = "BasicAuthPassword" - component.name = "SecurityCtrlr" - attributeValue = "<NewPassword>"
3. The Test System sends a HTTP upgrade request with an Authorization header, containing a username/password combination (with the new <i>BasicAuthPassword</i>). <u>Note(s):</u> - The Authorization header is formatted as follows: <i>AUTHORIZATION: Basic <Base64 encoded(<Configured ChargingStationId>:<NEW BasicAuthPassword>)></i>	4. The CSMS validates the username/password combination AND upgrades the connection to a (secured) WebSocket connection.
5. The Test System sends a BootNotificationRequest	6. The CSMS responds with a BootNotificationResponse
7. The Test System notifies the CSMS about the current state of all connectors.	8. The CSMS responds accordingly.

Tool validations

* Step 1:

Message: **SetVariableRequest**

- **variable.name** = *"BasicAuthPassword"*

- **component.name** = *"SecurityCtrlr"*

* Step 6:

Message: **BootNotificationResponse**

- **status** must be *Accepted*

Post scenario validations:

N/a

TC_A_10_CSMS: Update Charging Station Password for HTTP Basic Authentication - Rejected

Test case name	Update Charging Station Password for HTTP Basic Authentication - Rejected
Test case Id	TC_A_10_CSMS
Use case Id(s)	<u>A01</u>
Requirement(s)	<u>A01.FR.02</u> , <u>A01.FR.04</u> , <u>A01.FR.05</u>
System under test	CSMS
Description	This test case defines how to use the BasicAuthPassword, the password used to authenticate Charging Stations in security profile 1 (Basic Authentication) and security profile 2 (TLS with Basic Authentication)
Purpose	To verify if the CSMS keeps accepting the old credentials and keeps communication when the new BasicAuthPassword is rejected as described at the OCPP specification.
Prerequisite(s)	The CSMS supports security profile 1 and/or 2

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetVariablesResponse with status <i>Rejected</i>	1. The CSMS sends a SetVariablesRequest with: setVariableData[1]: - variable.name = "BasicAuthPassword" - component.name = "SecurityCtrlr" - attributeValue = "<NewPassword>"
	3. The Test System sends a HTTP upgrade request with an Authorization header, containing a username/password combination (with the old <i>BasicAuthPassword</i>). <u>Note(s):</u> - The Authorization header is formatted as follows: AUTHORIZATION: Basic <Base64 encoded(<Configured ChargingStationId>:<OLD Configured BasicAuthPassword>)>
4. The CSMS validates the username/password combination AND upgrades the connection to a (secured) WebSocket connection.	
5. The Test System sends a BootNotificationRequest	6. The CSMS responds with a BootNotificationResponse
	7. The Test System notifies the CSMS about the current state of all connectors.

Main (Test scenario)

8. The CSMS responds accordingly.

Tool validations

* Step 1:

Message: **SetVariableRequest**

- **variable.name** = "BasicAuthPassword"

- **component.name** = "SecurityCtrlr"

* Step 6:

Message: **BootNotificationResponse**

- **status** must be *Accepted*

Post scenario validations:

N/a

TC_A_11_CSMS: Update Charging Station Certificate by request of CSMS - Success - Charging Station Certificate

Test case name	Update Charging Station Certificate by request of CSMS - Success - Charging Station Certificate
Test case Id	TC_A_11_CSMS
Use case Id(s)	<u>A02</u> & <u>F06</u>
Requirement(s)	<u>A02.FR.11</u> , <u>A02.FR.14</u> , <u>F06.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to update its charging station certificate using the TriggerMessageRequest message.
Purpose	To verify if the CSMS is able to request the Charging Station to update its Charging Station Certificate.
Prerequisite(s)	The CSMS supports security profile 3

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. Execute Reusable State <i>RenewChargingStationCertificate</i>	
2. The Test System disconnects its current connection and reconnects to the CSMS with the new certificate.	3. The CSMS accepts the incoming connection request using the new certificate.

Tool validations

N/a

Post scenario validations:

The Test System and the CSMS are connected.

TC_A_12_CSMS: Update Charging Station Certificate by request of CSMS - Success - V2G Certificate

Test case name	Update Charging Station Certificate by request of CSMS - Success - V2G Certificate
Test case Id	TC_A_12_CSMS
Use case Id(s)	<u>A02</u> & <u>F06</u>
Requirement(s)	<u>A02.FR.11</u> , <u>F06.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to update its charging station certificate using the TriggerMessageRequest message.
Purpose	To verify if the CSMS is able to request the Charging Station to update its V2G Certificate.
Prerequisite(s)	The CSMS supports ISO 15118.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse With status Accepted	1. The CSMS sends a TriggerMessageRequest
3 The Test System sends a SignCertificateRequest With csr Generated CSR based on: - <Configured Country> - <Configured Organization> - <Configured OrganizationalUnit> certificateType V2GCertificate	4. The CSMS responds with a SignCertificateResponse
6. The Test System responds with a CertificateSignedResponse With status Accepted	5. The CSMS sends a CertificateSignedRequest

Tool validations

* Step 1:

Message: **TriggerMessageRequest**

- **requestedMessage** *SignV2GCertificate*

* Step 4:

Message: **SignCertificateResponse**

- **status** *Accepted*

* Step 5:

Message: **CertificateSignedRequest**

- **certificateChain** *<Certificate generated from the received CSR from step 3 and signed by the V2G Root or SubCA certificate from the configured V2G certificate chain>*

NOTE: The Test System will validate the certificate, but if the following validation fail, the testcase will NOT FAIL, because generating the certificate is probably not be done by the CSMS.

- The key must be at least 224 bits long.

- The received certificate must be transmitted in the X.509 format encoded in Privacy-Enhanced Mail (PEM) format.

Post scenario validations:

N/a

TC_A_14_CSMS: Update Charging Station Certificate by request of CSMS - Invalid certificate

Test case name	Update Charging Station Certificate by request of CSMS - Invalid certificate
Test case Id	TC_A_14_CSMS
Use case Id(s)	<u>A02</u>
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS is able to request the Charging Station to update its charging station certificate using the TriggerMessageRequest message.
Purpose	To verify if the CSMS is able to handle a Charging Station rejecting the new Charging Station certificate.
Prerequisite(s)	The CSMS supports security profile 3

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a TriggerMessageRequest
2. The Test System responds with a TriggerMessageResponse With status Accepted	
3 The Test System sends a SignCertificateRequest With csr <Configured CSR> certificateType ChargingStationCertificate	4. The CSMS responds with a SignCertificateResponse
6. The Test System responds with a CertificateSignedResponse With status Rejected	5. The CSMS sends a CertificateSignedRequest
7. The Test System sends a SecurityEventNotificationRequest with type = InvalidChargingStationCertificate	8. The CSMS responds with a SecurityEventNotificationResponse

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** *SignChargingStationCertificate*

* Step 4:

Message: **SignCertificateResponse**- **status** *Accepted*

Tool validations**Post scenario validations:**

N/a

TC_A_19_CSMS: Upgrade Charging Station Security Profile - Accepted

Test case name	Upgrade Charging Station Security Profile - Accepted
Test case Id	TC_A_19_CSMS
Use case Id(s)	<u>A05</u>
Requirement(s)	<u>A05.FR.04</u> , <u>A05.FR.07</u>
System under test	CSMS
Description	The CSMS updates the connection details on the Charging Station, to increase the security profile level.
Purpose	To verify if the CSMS is able to set a new network connection profile at one of the by the Charging Station defined configuration slots with a higher security profile than currently configured.
Prerequisite(s)	- Security profile must be set to 1 or 2. - If Security profile is set to 1, then a trusted certificate must be installed.

Before (Preparations)

Configuration State:

N/a

Memory State:

If configured <Security profile> is 2, then *RenewChargingStationCertificate*

The Test System uses this certificate during the TLS handshake when connecting with security profile 3.

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to set a new *NetworkConnectionProfile* with a security profile level one higher than currently configured

2. The Test System responds with a **SetNetworkProfileResponse**
With **status Accepted**

1. The CSMS sends a **SetNetworkProfileRequest**

Manual Action: Request the CSMS to change the *NetworkConfigurationPriority* to one that contains the *configurationSlot* of the new *NetworkConnectionProfile* from step 1

4. The Test System responds with a **SetVariablesResponse** with **status Accepted**

3. The CSMS sends a **SetVariablesRequest**

Manual Action: Request the CSMS to reboot the Charging Station

6. The Test System responds with a **ResetResponse**
with **status Accepted**

5. The CSMS sends a **ResetRequest**

Main (Test scenario)	
7. The Test System reconnects to the CSMS using the new NetworkProfile, containing the upgraded security profile <Configured securityProfile + 1>.	8. The CSMS accepts the connection attempt.
9. Execute Reusable State Booted	
10. The Test System reconnects to the CSMS using the original NetworkProfile, containing the lower security profile. <u>Note(s):</u> - This is done to ensure that the CSMS does not accept a connection using the lower security profile anymore.	11. The CSMS shall not accept the connection attempt.
12. The Test System reconnects to the CSMS using the new NetworkProfile, containing the upgraded security profile <Configured securityProfile + 1>. <u>Note(s):</u> - This is done to restore the connection before ending the testcase.	13. The CSMS accepts the connection attempt.

Tool validations
* Step 1: Message SetNetworkProfileRequest - connectionData.messageTimeout <Configured messageTimeout> - connectionData.ocppInterface <Configured ocppInterface> - connectionData.ocppTransport JSON - connectionData.ocppVersion OCPP20 - connectionData.securityProfile <Configured securityProfile + 1> * Step 3: Message SetVariablesRequest setVariableData: - variable.name = "NetworkConfigurationPriority" - component.name = "OCPPCommCtrlr" - attributeValue = <contains configurationSlot provided at step 1> * Step 11: When upgrading a Charging Station to a higher security profile, a CSMS has several options regarding which endpoint to use. This affects the way the CSMS is able to detect it needs to reject the incoming connection attempt. In case of having upgraded from security profile 2 to 3, but there is an incoming connection attempt using security profile 2: When the same endpoint is used, then it depends on the CSMS endpoint configuration. - When the CSMS does a full switch and only allows TLS handshakes when a client certificate is provided, then the TLS handshake is rejected. - When the CSMS only requires this Charging Station to use a client certificate, then it accepts the TLS handshake (because it will be unable to detect which Charging Station is connecting) and it rejects the HTTP request to establish the WebSocket connection. When a different port or a whole different endpoint is used for the upgrade, then on the original endpoint the CSMS accepts the TLS handshake and it rejects the HTTP request to establish the WebSocket connection (because this Charging Station is not allowed to connect with security profile 2 anymore). In case of security profile 1, the case is always the same. The CSMS shall always reject the HTTP request to establish the WebSocket connection, because TLS is required for this Charging Station. Post scenario validations: The Test System and the CSMS are connected.

3. B Provisioning

TC_B_01_CSMS: Cold Boot Charging Station - Accepted

Test case name	Cold Boot Charging Station - Accepted
Test case Id	TC_B_01_CSMS
Use case Id(s)	<u>B01</u>
Requirement(s)	<u>B01.FR.02</u>
System under test	CSMS
Description	The booting mechanism allows a Charging Station to provide some general information about the Charging Station to the CSMS on startup AND it allows the Charging Station to request whether it is allowed to start sending other OCPP messages.
Purpose	To verify whether the CSMS is able to accept the communications of a registered Charging Station.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *Booted*

Tool validations

N/a

Post scenario validations:

N/a

TC_B_02_CSMS: Cold Boot Charging Station - Pending

Test case name	Cold Boot Charging Station - Pending
Test case Id	TC_B_02_CSMS
Use case Id(s)	<u>B02</u>
Requirement(s)	<u>B02.FR.01</u> , <u>B02.FR.06</u>
System under test	CSMS
Description	The booting mechanism allows a Charging Station to provide some general information about the Charging Station to the CSMS on startup AND it allows the Charging Station to request whether it is allowed to start sending other OCPP messages. The CSMS may respond to the BootNotificationRequest with status <i>Pending</i> . The <i>Pending</i> status can indicate that the CSMS wants to retrieve or set certain information on the Charging Station before it will accept the Charging Station.
Purpose	To verify whether the CSMS is able to accept the communications of a registered Charging Station.
Prerequisite(s)	The CSMS is configured to first respond to a BootNotificationRequest with status Pending .

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a BootNotificationRequest with reason <i>PowerUp</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	2. The CSMS responds with a BootNotificationResponse
Note(s): - If the interval in the BootNotificationResponse equals 0, the Test System will wait <Configured heartbeatInterval> seconds, before sending another BootNotificationRequest. - If the interval in the BootNotificationResponse > 0, the Test System will wait <Interval provided at the BootNotificationResponse> seconds, before sending another BootNotificationRequest. - During this interval, the CSMS may send messages to retrieve information from the Charging Station (as described in use cases <u>B06</u> , <u>B07</u> , <u>B08</u>) or change its configuration by SetVariablesRequest (as described in use case <u>B05</u>). The Test System will respond to these messages.	
3. The Test System sends a BootNotificationRequest with reason <i>PowerUp</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	4. The CSMS responds with a BootNotificationResponse

Main (Test scenario)

5. The Test System notifies the CSMS about the current state of all connectors.

Message: **StatusNotificationRequest**
with **connectorStatus** *Available*
Message: **NotifyEventRequest**
with **trigger** *Delta*
actualValue *"Available"*
component.name *"Connector"*
variable.name *"AvailabilityState"*

6. The CSMS responds accordingly.

Tool validations

* Step 2:

Message: **BootNotificationResponse**
- **status** *Pending*

* Step 3:

Message: **BootNotificationResponse**
- **status** *Accepted*

Post scenario validations:

N/a

TC_B_30_CSMS: Cold Boot Charging Station - Pending/Rejected - SecurityError

Test case name	Cold Boot Charging Station - Pending/Rejected - SecurityError
Test case Id	TC_B_30_CSMS
Use case Id(s)	<u>B02/B03</u>
Requirement(s)	<u>B02.FR.09, B03.FR.07</u>
System under test	CSMS
Description	The booting mechanism allows a Charging Station to provide some general information about the Charging Station to the CSMS on startup AND it allows the Charging Station to request whether it is allowed to start sending other OCPP messages. The CSMS may respond to the BootNotificationRequest with status <i>Pending</i> or <i>Rejected</i> . During this state, the Charging Station is not allowed to send RPC Framework: CALL message that is NOT a BootNotificationRequest or in case of status <i>Pending</i> , a message triggered by one of the following messages: TriggerMessageRequest, GetBaseReportRequest, GetReportRequest.
Purpose	To verify whether the CSMS is able to handle unauthorized messages from the Charging Station by responding with a SecurityError.
Prerequisite(s)	The CSMS is configured to first respond to a BootNotificationRequest with status <i>Pending</i> or <i>Rejected</i> .

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a BootNotificationRequest with reason <i>PowerUp</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	2. The CSMS responds with a BootNotificationResponse
3. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest with connectorStatus <i>Available</i> Message: NotifyEventRequest with trigger <i>Delta</i> actualValue "Available" component.name "Connector" variable.name "AvailabilityState"	4. The CSMS responds with RPC Framework: CALLERROR: SecurityError.

Tool validations

* Step 2:

Message: **BootNotificationResponse**

- **status** *Pending* OR *Rejected*

Post scenario validations:

N/a

TC_B_31_CSMS: Cold Boot Charging Station - Pending/Rejected - TriggerMessage

Test case name	Cold Boot Charging Station - Pending/Rejected - TriggerMessage
Test case Id	TC_B_31_CSMS
Use case Id(s)	<u>B02</u> , <u>F06</u>
Requirement(s)	N/a
System under test	CSMS
Description	The booting mechanism allows a Charging Station to provide some general information about the Charging Station to the CSMS on startup AND it allows the Charging Station to request whether it is allowed to start sending other OCPP messages.
Purpose	To verify whether the CSMS is able to send a TriggerMessageRequest to trigger a BootNotificationRequest, before the interval expired.
Prerequisite(s)	The CSMS is configured to first respond to a BootNotificationRequest with status <i>Pending</i> or <i>Rejected</i> .

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a BootNotificationRequest with reason <i>PowerUp</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	2. The CSMS responds with a BootNotificationResponse
4. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	3. The CSMS sends a TriggerMessageRequest
5. The Test System sends a BootNotificationRequest with reason <i>Triggered</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	6. The CSMS responds with a BootNotificationResponse

Main (Test scenario)

7. The Test System notifies the CSMS about the current state of all connectors.

Message: **StatusNotificationRequest**
with **connectorStatus** *Available*
Message: **NotifyEventRequest**
with **trigger** *Delta*
actualValue *"Available"*
component.name *"Connector"*
variable.name *"AvailabilityState"*

8. The CSMS responds accordingly.

Tool validations

* Step 2:

Message: **BootNotificationResponse**
- **status** *Pending OR Rejected*

* Step 3:

Message: **TriggerMessageRequest**
- **requestedMessage** *BootNotification*

* Step 6:

Message: **BootNotificationResponse**
- **status** *Accepted*

Post scenario validations:

N/a

TC_B_06_CSMS: Get Variables - single value

Test case name	Get Variables - single value
Test case Id	TC_B_06_CSMS
Use case Id(s)	<u>B06</u>
Requirement(s)	<u>B06.FR.01</u> , <u>B06.FR.02</u> , <u>B06.FR.03</u> , <u>B06.FR.04</u> , <u>B06.FR.10</u> , <u>B06.FR.11</u>
System under test	CSMS
Description	Get the value of two of the required variables of OCPPCommCtrlr
Purpose	To test getting single value using GetVariablesRequest for one of the mandatory component/variable combinations that must exist in the DM implementation.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: GetVariablesResponse	1. Manually request CSMS to get data for: - OCPPCommCtrlr.OfflineThreshold

Tool validations

* Step 1:

Message: **GetVariablesRequest** with (in arbitrary order)**getVariableData[0]:**

- **attributeType** is at least absent or **attributeType** = *Actual*, but *Target*, *MinSet*, and *MaxSet* are also allowed
- **variable.name** = *"OfflineThreshold"*
- **component.name** = *"OCPPCommCtrlr"*

Post scenario validations:

Manually validate that CSMS has correctly read the requested variables.

TC_B_07_CSMS: Get Variables - multiple values

Test case name	Get Variables - multiple values
Test case Id	TC_B_07_CSMS
Use case Id(s)	<u>B06</u>
Requirement(s)	<u>B06.FR.01</u> , <u>B06.FR.02</u> , <u>B06.FR.03</u>
System under test	CSMS
Description	Get the value of two of the required variables of OCPPCommCtrlr
Purpose	To test getting multiple values using GetVariablesRequest for one of the mandatory component/variable combinations that must exist in the DM implementation.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: GetVariablesResponse	1. Manually request CSMS to get data for: - OCPPCommCtrlr.OfflineThreshold - AuthCtrlr.AuthorizeRemoteStart

Tool validations

* Step 1:

Message: **GetVariablesRequest** with (in arbitrary order)**getVariableData[0]:**

- **attributeType** is at least absent or **attributeType** = *Actual*, but *Target*, *MinSet*, and *MaxSet* are also allowed
- **variable.name** = *"OfflineThreshold"*
- **component.name** = *"OCPPCommCtrlr"*

getVariableData[1]:

- **attributeType** is at least absent or **attributeType** = *Actual*, but *Target*, *MinSet*, and *MaxSet* are also allowed
- **variable.name** = *"AuthorizeRemoteStart"*
- **component.name** = *"AuthCtrlr"*

Post scenario validations:

Manually validate that CSMS has correctly read the requested variables.

TC_B_08_CSMS: Get Variables - limit to maximum number of values

Test case name	Get Variables - limit to maximum number of values
Test case Id	TC_B_08_CSMS
Use case Id(s)	B06
Requirement(s)	B06.FR.05
System under test	CSMS
Description	Do not request more variables than supported by <code>MaxItemsPerMessageGetVariables</code> .
Purpose	To test that CSMS does not request more variables than the Charging Station reported to support in the variable <code>MaxItemsPerMessageGetVariables</code> .
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

Configure (using `getVariablesRequest`) `Component.Variable.Instance DeviceDataCtrlr.ItemsPerMessage.GetVariables` at value 4.

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with GetVariablesResponse with with a list of 4 GetVariableResultType items and a GetVariableResponse with 1 GetVariableResultType item.	1. Manually request CSMS for 5 variables: - <code>DeviceDataCtrlr.ItemsPerMessage[GetReport]</code> - <code>DeviceDataCtrlr.ItemsPerMessage[GetVariables]</code> - <code>DeviceDataCtrlr.BytesPerMessage[GetReport]</code> - <code>DeviceDataCtrlr.BytesPerMessage[GetVariables]</code> - <code>AuthCtrlr.AuthorizeRemoteStart</code>

Tool validations

* Step 1:

Message: **GetVariablesRequest** for 4 variables and a **GetVariablesRequest** for 1 variable (in arbitrary order):
for `component.name = "DeviceDataCtrlr"`

- `variable.name = "ItemsPerMessage"` with `variable.instance = "GetReport"`
- `variable.name = "ItemsPerMessage"` with `variable.instance = "GetVariables"`
- `variable.name = "BytesPerMessage"` with `variable.instance = "GetReport"`
- `variable.name = "BytesPerMessage"` with `variable.instance = "GetVariables"`

and for `component.name = "AuthCtrlr"`

- `variable.name = "AuthorizeRemoteStart"`

Post scenario validations:

Test System validates that not more than `ItemsPerMessageGetVariables` elements are requested in one **GetVariablesRequest** message by CSMS.

TC_B_09_CSMS: Set Variables - single value

Test case name	Set Variables - single value
Test case Id	TC_B_09_CSMS
Use case Id(s)	<u>B05</u>
Requirement(s)	<u>B05.FR.01</u> , <u>B05.FR.02</u> , <u>B05.FR.03</u> , <u>B05.FR.10</u> , <u>B05.FR.12</u>
System under test	CSMS
Description	Set the value of one of the required variables of OCPPCommCtrlr
Purpose	To test setting a single value using SetVariablesRequest for one of the mandatory component/variable combinations that must exist in the DM implementation.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: SetVariablesResponse	1. Manually request CSMS to set data for: - OCPPCommCtrlr.OfflineThreshold

Tool validations

* Step 1:

Message: **SetVariablesRequest** with (in arbitraty order):**setVariableData[1]:**- **variable.name** = "OfflineThreshold"- **component.name** = "OCPPCommCtrlr"- **attributeValue** = "123"- **attributeType** is absent or **attributeType** = Actual**Post scenario validations:**

Manually validate that CSMS has correctly set the requested variables.

TC_B_10_CSMS: Set Variables - multiple values

Test case name	Set Variables - multiple values
Test case Id	TC_B_10_CSMS
Use case Id(s)	<u>B05</u>
Requirement(s)	<u>B05.FR.01</u> , <u>B05.FR.02</u> , <u>B05.FR.03</u>
System under test	CSMS
Description	Set the value of two of the required variables of OCPPCommCtrlr
Purpose	To test setting multiple values using SetVariablesRequest for one of the mandatory component/variable combinations that must exist in the DM implementation.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: SetVariablesResponse	1. Manually request CSMS to set data for: - OCPPCommCtrlr.OfflineThreshold - AuthCtrlr.AuthorizeRemoteStart

Tool validations

* Step 1:

Message: **SetVariablesRequest** with (in arbitrary order):**setVariableData[1]:**- **variable.name** = "OfflineThreshold"- **component.name** = "OCPPCommCtrlr"- **attributeValue** = "123"- **attributeType** is absent or **attributeType** = Actual**setVariableData[2]:**- **variable.name** = "AuthorizeRemoteStart"- **component.name** = "AuthCtrlr"- **attributeValue** = "false"- **attributeType** is absent or **attributeType** = Actual

Post scenario validations:

Manually validate that CSMS has correctly set the requested variables.

TC_B_12_CSMS: Get Base Report - ConfigurationInventory

Test case name	Get Base Report - ConfigurationInventory
Test case Id	TC_B_12_CSMS
Use case Id(s)	<u>B07</u>
Requirement(s)	<u>B07.FR.07</u>
System under test	CSMS
Description	CSMS requests a ConfigurationInventory base report.
Purpose	To test that CSMS supports the ConfigurationInventory base report.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

2. Test System responds with:
GetBaseReportResponse

1. *Manually instruct CSMS to retrieve a ConfigurationInventory report.*

Tool validations

* Step 1:

Message: **GetBaseReportRequest** with:- **requestId** has integer value ≥ 0 - **reportBase** = *ConfigurationInventory*

Post scenario validations:

CSMS receives all **NotifyReportRequest** message for this *requestId* and is able to show the result of configuration inventory to an operator.

TC_B_13_CSMS: Get Base Report - FullInventory

Test case name	Get Base Report - FullInventory
Test case Id	TC_B_13_CSMS
Use case Id(s)	<u>B07</u>
Requirement(s)	<u>B07.FR.08</u>
System under test	CSMS
Description	CSMS requests a FullInventory base report.
Purpose	To test that CSMS supports the FullInventory base report.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: GetBaseReportResponse	1. Manually instruct CSMS to retrieve a FullInventory report.

Tool validations

* Step 1:

GetBaseReportRequest with:

- **requestId** has integer value ≥ 0
- **reportBase** = *FullInventory*

Post scenario validations:

CSMS receives all **NotifyReportRequest** message for this *requestId* and is able to show the result of full inventory to an operator.

TC_B_14_CSMS: Get Base Report - SummaryInventory

Test case name	Get Base Report - SummaryInventory
Test case Id	TC_B_14_CSMS
Use case Id(s)	<u>B07</u>
Requirement(s)	<u>B07.FR.09</u>
System under test	CSMS
Description	CSMS requests a SummaryInventory base report.
Purpose	To test that CSMS supports the SummaryInventory base report.
Prerequisite(s)	CSMS implementation supports the optional SummaryInventory report

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

2. Test System responds with:
GetBaseReportResponse

1. *Manually instruct CSMS to retrieve a SummaryInventory report.*

Tool validations

* Step 1:

GetBaseReportRequest with:

- **requestId** has integer value ≥ 0
- **reportBase** = *SummaryInventory*

Post scenario validations:

CSMS receives all **NotifyReportRequest** message for this *requestId* and is able to show the result of summary inventory to an operator.

TC_B_18_CSMS: Get Custom Report - with componentCriteria and component/variables

Test case name	Get Custom Report - with componentCriteria and component/variables
Test case Id	TC_B_18_CSMS
Use case Id(s)	<u>B08</u>
Requirement(s)	<u>B08.FR.01</u> , <u>B08.FR.03</u>
System under test	CSMS
Description	CSMS requests a report of components that match both the component criteria and the given list of components and variables.
Purpose	To test that CSMS supports requesting a report for both the component criteria and a given list of components and optionally with variables and that it handles an empty result set.
Prerequisite(s)	

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: GetReportResponse with status <i>EmptyResultSet</i>	1. Manually instruct CSMS to get the value of: - EVSE #1::AvailabilityState - from all <i>Problem</i> components
4. Test System responds with: GetReportResponse with status <i>Accepted</i>	3. Manually instruct CSMS to get the value of: - EVSE #1::AvailabilityState - from all <i>Available</i> components
5. Test System responds with: NotifyReportRequest	6. CSMS sends NotifyReportResponse

Tool validations

* Step 1:

Message: **GetReportRequest**- **componentCriteria** = *Problem*- **componentVariable[0].component.name** = "EVSE"- **componentVariable[0].component.evse.id** = 1- **componentVariable[0].variable.name** = "AvailabilityState"

Tool validations

* Step 3:

Message: **GetReportRequest**

- **componentCriteria** is *Available*
- **componentVariable[0].component.name** = "EVSE"
- **componentVariable[0].component.evse.id** = 1
- **componentVariable[0].variable.name** = "AvailabilityState"

Post scenario validations:

N/A

TC_B_20_CSMS: Reset Charging Station - Without ongoing transaction - OnIdle

Test case name	Reset Charging Station - Without ongoing transaction - OnIdle
Test case Id	TC_B_20_CSMS
Use case Id(s)	<u>B11</u>
Requirement(s)	<u>B11.FR.04</u>
System under test	CSMS
Description	This test case covers how the CSMS can request the Charging Station to reset itself by sending a ResetRequest without any ongoing transaction. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to reboot the Charging Station with **type_OnIdle**

2. The Test System responds with a **ResetResponse** with **status Accepted**

1. The CSMS sends a **ResetRequest**

3. The Test System sends a **BootNotificationRequest**

4. The CSMS responds with a **BootNotificationResponse**

5. The Test System notifies the CSMS about the current state of all connectors.

Message: **StatusNotificationRequest**

- **connectorStatus Available**

Message: **NotifyEventRequest**

- **trigger Delta**

- **actualValue "Available"**

- **component.name "Connector"**

- **variable.name "AvailabilityState"**

6. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ResetRequest**

- **evseId** must be omitted

* Step 4:

Message **BootNotificationResponse**

- **status** *Accepted*

Post scenario validations:

- N/a

TC_B_21_CSMS: Reset Charging Station - With Ongoing Transaction - OnIdle

Test case name	Reset Charging Station - With Ongoing Transaction - OnIdle
Test case Id	TC_B_21_CSMS
Use case Id(s)	B12
Requirement(s)	B12.FR.01, B12.FR.03, E07.FR.03
System under test	CSMS
Description	This test case covers how the CSMS can remotely request the Charging Station to reset itself by sending a ResetRequest during a transaction. When ResetRequest "OnIdle" is send the charging stations schedules a reboot after all transactions are stopped. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is EnergyTransferStarted

Main (Test scenario)

Charging Station	CSMS
<u>Manual Action:</u> Request the CSMS to reboot the Charging Station with status OnIdle	
2. The Test System responds with a ResetResponse with status Scheduled	1. The CSMS sends a ResetRequest with status OnIdle
3. The Test System sends a TransactionEventRequest . - eventType Updated - triggerReason StopAuthorized - transactionInfo.chargingState EVConnected - idToken.idToken <Configured valid_idtoken_idtoken> - idToken.type <Configured valid_idtoken_type>	4. The CSMS responds with a TransactionEventResponse .
5. The Test System sends a TransactionEventRequest . - eventType Ended - triggerReason EVCommunicationLost - transactionInfo.chargingState Idle - transactionInfo.stoppedReason EVDisconnected	6. The CSMS responds with a TransactionEventResponse .
7. The Test System sends a BootNotificationRequest with reason ScheduledReset	8. The CSMS responds with a BootNotificationResponse

Main (Test scenario)

9. The Test System notifies the CSMS about the current state of all connectors.

Message: **StatusNotificationRequest**

- **connectorStatus** *Available*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Available"*

- **component.name** *"Connector"*

- **variable.name** *"AvailabilityState"*

10. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ResetRequest**

- **type** *OnIdle*

- **evseId** must be omitted

* Step 8:

Message **BootNotificationResponse**

- **status** *Accepted*

Post scenario validations:

- N/a

TC_B_22_CSMS: Reset Charging Station - With Ongoing Transaction - Immediate

Test case name	Reset Charging Station - With Ongoing Transaction - Immediate
Test case Id	TC_B_22_CSMS
Use case Id(s)	<u>B12</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS can remotely request the Charging Station to reset itself by sending a ResetRequest during a transaction. When ResetRequest "Immediate" is send the charging stations will try to stop all transactions before rebooting. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *EnergyTransferStarted***Main (Test scenario)**

Charging Station	CSMS
<u>Manual Action:</u> Request the CSMS to reboot the Charging Station with status Immediate	
2. The Test System responds with a ResetResponse with status Accepted	1. The CSMS sends a ResetRequest with status Immediate
3. The Test System sends a TransactionEventRequest . - eventType <i>Ended</i> - triggerReason <i>ResetCommand</i> - transactionInfo.chargingState <i>EVConnected</i> - transactionInfo.stoppedReason <i>ImmediateReset</i> - idToken is omitted	4. The CSMS responds with a TransactionEventResponse .
5. The Test System sends a BootNotificationRequest with reason RemoteReset	6. The CSMS responds with a BootNotificationResponse

Main (Test scenario)

7. The Test System notifies the CSMS about the current state of all connectors.

For <Configured connectorId>:

Message: **StatusNotificationRequest**

- **connectorStatus** *Occupied*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Occupied"*

- **component.name** *"Connector"*

- **variable.name** *"AvailabilityState"*

For <Other connector(s)>:

Message: **StatusNotificationRequest**

- **connectorStatus** *Available*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Available"*

- **component.name** *"Connector"*

- **variable.name** *"AvailabilityState"*

8. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ResetRequest**

- **type** *Immediate*

- **evseId** must be omitted

* Step 6:

Message **BootNotificationResponse**

- **status** *Accepted*

Post scenario validations:

- N/a

TC_B_25_CSMS: Reset EVSE - Without ongoing transaction

Test case name	Reset EVSE - Without ongoing transaction
Test case Id	TC_B_25_CSMS
Use case Id(s)	<u>B11</u>
Requirement(s)	<u>B11.FR.04</u>
System under test	CSMS
Description	This test case covers how the CSMS can request the Charging Station to reset an EVSE by sending a ResetRequest without any ongoing transaction. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to reboot an EVSE with status OnIdle

2. The Test System responds with a **ResetResponse** with **status Accepted**

1. The CSMS sends a **ResetRequest** with **status OnIdle** and **evseID** <Configured evseId>

Tool validations

* Step 1:

Message **ResetRequest**- **type** OnIdle- **evseId** <Configured evseId>

Post scenario validations:

- N/a

TC_B_26_CSMS: Reset EVSE - With Ongoing Transaction - OnIdle

Test case name	Reset EVSE - With Ongoing Transaction - OnIdle
Test case Id	TC_B_26_CSMS
Use case Id(s)	<u>B12</u>
Requirement(s)	<u>B12.FR.07</u>
System under test	CSMS
Description	This test case covers how the CSMS can remotely request the Charging Station to reset an EVSE by sending a ResetRequest during a transaction. When ResetRequest "OnIdle" is send the charging stations schedules a reboot after all transactions are stopped. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
<u>Manual Action:</u> Request the CSMS to reboot the charging EVSE with status OnIdle	
2. The Test System responds with a ResetResponse with status Scheduled	1. The CSMS sends a ResetRequest with status OnIdle and evseID <Configured evseId>
3. The Test System sends a TransactionEventRequest . - eventType <i>Updated</i> - triggerReason <i>StopAuthorized</i> - transactionInfo.chargingState <i>EVConnected</i> - idToken.idToken <Configured valid_idtoken_idtoken> - idToken.type <Configured valid_idtoken_type>	4. The CSMS responds with a TransactionEventResponse .
5. The Test System sends a TransactionEventRequest . - eventType <i>Ended</i> - triggerReason <i>EVCommunicationLost</i> - transactionInfo.chargingState <i>Idle</i> - transactionInfo.stoppedReason <i>EVDisconnected</i>	6. The CSMS responds with a TransactionEventResponse .

Tool validations

* Step 1:

Message **ResetRequest**

- **type** *OnIdle*

- **evseId** *<Configured evseId>*

Post scenario validations:

- N/a

TC_B_27_CSMS: Reset EVSE - With Ongoing Transaction - Immediate

Test case name	Reset EVSE - With Ongoing Transaction - Immediate
Test case Id	TC_B_27_CSMS
Use case Id(s)	<u>B12</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS can remotely request the Charging Station to reset an EVSE by sending a ResetRequest during a transaction. When ResetRequest "Immediate" is send the charging stations will try to stop all transactions before rebooting. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *EnergyTransferStarted***Main (Test scenario)****Charging Station****CSMS**Manual Action: Request the CSMS to reboot the charging EVSE with status Immediate

2. The Test System responds with a **ResetResponse** with **status Accepted**

1. The CSMS sends a **ResetRequest** with **status Immediate** and **evseId** <Configured evseId>

3. The Test System sends a **TransactionEventRequest**.

- **eventType** *Ended*
- **triggerReason** *ResetCommand*
- **transactionInfo.chargingState** *EVConnected*
- **transactionInfo.stoppedReason** *ImmediateReset*

4. The CSMS responds with a **TransactionEventResponse**.

Tool validations

* Step 1:

Message **ResetRequest**- **type** *Immediate*- **evseId** <Configured evseId>**Post scenario validations:**

N/a

DRAFT

TC_B_42_CSMS: Set new NetworkConnectionProfile - Accepted

Test case name	Set new NetworkConnectionProfile - Accepted
Test case Id	TC_B_42_CSMS
Use case Id(s)	<u>B09</u>
Requirement(s)	<u>B09.FR.01</u>
System under test	CSMS
Description	The CSMS updates the connection details on the Charging Station. For instance in preparation of a migration to a new CSMS.
Purpose	To verify if the CSMS is able to set a new network connection profile at one of the by the Charging Station defined configuration slots.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetNetworkProfileResponse With status Accepted	1. The CSMS sends a SetNetworkProfileRequest

Tool validations

* Step 1:

Message **SetNetworkProfileRequest**

- **configurationSlot** is <Configured configurationSlot>
- **connectionData.messageTimeout** <Configured messageTimeout>
- **connectionData.ocppCsmsUrl** <Configured ocppCsmsUrl>
- **connectionData.ocppInterface** <Configured ocppInterface>
- **connectionData.ocppTransport** JSON
- **connectionData.ocppVersion** OCPP20
- **connectionData.securityProfile** <Configured securityProfile>

Post scenario validations:

- N/a

TC_B_44_CSMS: Set new NetworkConnectionProfile - Failed

Test case name	Set new NetworkConnectionProfile - Failed
Test case Id	TC_B_44_CSMS
Use case Id(s)	<u>B09</u>
Requirement(s)	<u>B09.FR.03</u>
System under test	CSMS
Description	The CSMS updates the connection details on the Charging Station. For instance in preparation of a migration to a new CSMS.
Purpose	To verify if the CSMS is able to handle a Charging Station responding with status Failed, when setting a new network connection profile at one of the by the Charging Station defined configuration slots.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

2. The Test System responds with a **SetNetworkProfileResponse**
With **status Failed**

CSMS

1. The CSMS sends a **SetNetworkProfileRequest**

Tool validations

N/a

Post scenario validations:

- N/a

TC_B_58_CSMS: WebSocket Subprotocol negotiation

Test case name	WebSocket Subprotocol validation
Test case Id	TC_B_58_CSMS
Use case Id(s)	Part 4 - JSON over WebSockets implementation guide
Requirement(s)	Section 3.1.2. OCPP version
System under test	CSMS
Description	OCPP-J imposes extra constraints on the WebSocket subprotocol, detailed in the following section 3.1.2.
Purpose	To verify whether the CSMS is able to select a supported OCPP version, when also a different unsupported version is supported by the Charging Station and relays this selection via the Sec-Websocket-Protocol header.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System disconnects the WebSocket connection and reconnects by sending a HTTP upgrade request with the header; Sec-WebSocket-Protocol: ocpp0.1	2. The CSMS rejects the connection attempt and does NOT upgrade the connection to a WebSocket connection.
3. The Test System disconnects the WebSocket connection and reconnects by sending a HTTP upgrade request with the header; Sec-WebSocket-Protocol: ocpp0.1,ocpp<Selected OCPP version>	4. The CSMS accepts the connection attempt and upgrades the connection to a WebSocket connection.

Tool validations

* Step 4:

The authorization header of the HTTP upgrade response must contain the header Sec-Websocket-Protocol, and it must comply to the following:

- The header is formatted as follows; Sec-WebSocket-Protocol: ocpp<Selected OCPP version>

Post scenario validations:

N/a

TC_B_100_CSMS: Set new NetworkConnectionProfile - Identity and password

Test case name	Set new NetworkConnectionProfile - Identity and password
Test case Id	TC_B_100_CSMS
Use case Id(s)	<u>B09</u>
Requirement(s)	<u>B09.FR.01</u> , <u>B09.FR.14</u>
System under test	CSMS
Description	The CSMS updates the connection details on the Charging Station. For instance in preparation of a migration to a new CSMS.
Purpose	To verify if the CSMS is able to set a new network connection profile that contains basicAuthPassword and identity at one of the by the Charging Station defined configuration slots.
Prerequisite(s)	CSMS uses the SetNetworkProfileRequest to update a network connection profile.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**Manual Action: Request CSMS to send a network profile including identity and password.

2. The Test System responds with a
SetNetworkProfileResponse
 With **status Accepted**

1. The CSMS sends a **SetNetworkProfileRequest**

Tool validations

* Step 1:

Message **SetNetworkProfileRequest**

- **configurationSlot** is <Configured configurationSlot>
- **connectionData.messageTimeout** <Configured messageTimeout>
- **connectionData.ocppCsmsUrl** <Configured ocppCsmsUrl>
- **connectionData.ocppInterface** <Configured ocppInterface>
- **connectionData.ocppTransport** JSON
- **connectionData.ocppVersion** OCPP21
- **connectionData.securityProfile** <Active securityProfile>
- **connectionData.identity** <Configured identity>
- **connectionData.basicAuthPassword** <Configured basicAuthPassword>

Post scenario validations:

- N/a

DRAFT

TC_B_103_CSMS: Reset ImmediateAndResume - With Ongoing Transaction - Resuming Energy Transfer

Test case name	Reset ImmediateAndResume - With Ongoing Transaction - Resuming Energy Transfer
Test case Id	TC_B_103_CSMS
Use case Id(s)	<u>B13</u>
Requirement(s)	<u>B13.FR.31</u>
System under test	CSMS
Description	This test case covers how the CSMS can remotely request the Charging Station to reset itself by sending a ResetRequest during a transaction. When ResetRequest "Immediate" is send the charging stations will try to stop all transactions before rebooting. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to reboot the Charging Station with status *ImmediateAndResume*

2. The Test System responds with a **ResetResponse** with **status** *Accepted*

1. The CSMS sends a **ResetRequest** with **status** *ImmediateAndResume*

3. The Test System sends a **TransactionEventRequest**.
- **eventType** *Updated*
- **triggerReason** *ResetCommand*
- **idToken** is omitted

4. The CSMS responds with a **TransactionEventResponse**.

5. The Test System sends a **BootNotificationRequest** with **reason** *RemoteReset*

6. The CSMS responds with a **BootNotificationResponse**

Main (Test scenario)	
7. The Test System notifies the CSMS about the current state of all connectors. For <Configured connectorId>: Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Occupied"</i> - component.name <i>"Connector"</i> - variable.name <i>"AvailabilityState"</i> For <Other connector(s)>: Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Available"</i> - component.name <i>"Connector"</i> - variable.name <i>"AvailabilityState"</i>	8. The CSMS responds accordingly.
9 The Test System sends a SecurityEventNotificationRequest with - type <i>ResetOrReboot</i>	10 The CSMS responds with a SecurityEventNotificationResponse
11. The Test System sends a TransactionEventRequest. - eventType <i>Updated</i> - triggerReason <i>TxResumed</i> - transactionInfo.chargingState <i>Charging</i> - idToken is omitted	12. The CSMS responds with a TransactionEventResponse.

Tool validations
* Step 1: Message ResetRequest - type <i>ImmediateAndResume</i> * Step 6: Message BootNotificationResponse - status <i>Accepted</i>
Post scenario validations: - N/a

TC_B_104_CSMS: Reset ImmediateAndResume - Without ongoing transaction

Test case name	Reset ImmediateAndResume - Without ongoing transaction
Test case Id	TC_B_104_CSMS
Use case Id(s)	<u>B13</u>
Requirement(s)	
System under test	CSMS
Description	This test case covers how the CSMS can request the Charging Station to reset itself by sending a ResetRequest without any ongoing transaction. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	CSMS has support for resuming transactions

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
<u>Manual Action:</u> Request the CSMS to reboot the Charging Station with type ImmediateAndResume	
2. The Test System responds with a ResetResponse with status <i>Accepted</i>	1. The CSMS sends a ResetRequest
3. The Test System sends a BootNotificationRequest	4. The CSMS responds with a BootNotificationResponse
5. The Test System notifies the CSMS about the current state of all connectors. Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue "Available" - component.name "Connector" - variable.name "AvailabilityState"	6. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ResetRequest**

- **type** *ImmediateAndResume*

* Step 4:

Message **BootNotificationResponse**

- **status** *Accepted*

Post scenario validations:

- N/a

TC_B_105_CSMS: Set new NetworkConnectionProfile - Add new NetworkConfiguration using SetVariables

Test case name	Set new NetworkConnectionProfile - Add new NetworkConfiguration using SetVariables
Test case Id	TC_B_105_CSMS
Use case Id(s)	B09
Requirement(s)	B09.FR.29. B09.FR.30
System under test	Charging Station
Description	The CSMS updates the connection details on the Charging Station. For instance in preparation of a migration to a new CSMS.
Purpose	To verify that CSMS can set NetworkConfiguration and correctly sets the value of Apn/VpnEnabled.
Prerequisite(s)	CSMS is able to set the NetworkConfiguration component Test System reports NetworkConfiguration variables as ReadWrite.

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): N/a

Main (Test scenario)	
Charging Station	CSMS
<u>Note:</u> In the following steps <configurationSlot> equals <Configured non-active configurationSlot>	
<u>Manual action:</u> Request CSMS to set a network configuration that includes APN connection details, but no VPN details.	
2. The Test system responds with SetVariablesResponse(s) with - setVariableResult.attributeStatus = <i>Accepted</i> for each variable	1. CSMS sends a SetVariablesRequest on component NetworkConfiguration [< <i>Configured non-active configurationSlot</i> >]

Tool validations
* Step 2: <i>(The following can be sent in one or multiple messages.)</i> Message SetVariablesRequest for component NetworkConfiguration[<<i>Configured non-active configurationSlot</i>>] with: - OcppCsmsUrl = <<i>Configured ocppCsmsUrl</i>> - OcppInterface = <i>Any</i> - OcppTransport = <i>JSON</i> - OcppVersion = <i>OCPP21</i> - MessageTimeout = <<i>Configured messageTimeout</i>> - SecurityProfile = <i>2</i> - Identity = <<i>Configured charging station identity</i>> - BasicAuthPassword = <i>PasswordOfSufficientLength</i> - VpnEnabled = <i>false</i> - ApnEnabled = <i>true</i> - Apn = <<i>Configured APN URL</i>> - ApnUserName = <<i>Configured APN username</i>> - ApnPassword = <<i>Configured APN password</i>> - ApnAuthentication one of <i>PAP, CHAP, NONE, AUTO</i>
Post scenario validations: - N/a

TC_B_116_CSMS: Reset ImmediateAndResume - With Ongoing Transaction and SmartCharging - resuming energytransfer

Test case name	Reset ImmediateAndResume - With Ongoing Transaction and SmartCharging - resuming energytransfer
Test case Id	TC_B_116_CSMS
Use case Id(s)	<u>B13</u>
Requirement(s)	<u>B13.FR.31</u>
System under test	CSMS
Description	This test case covers how the CSMS can remotely request the Charging Station to reset itself by sending a ResetRequest during a transaction. When ResetRequest "ImmediateAndResume" is sent the charging stations will try to stop all transactions before rebooting. This could for example be necessary if the Charging Station is not functioning correctly.
Purpose	To verify if the CSMS is able to perform the reset mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:
N/a

Memory State:
N/a

Reusable State(s):
State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

Manual action: trigger CSMS to set a Charging Profile of type TxProfile on <transactionId>

2. The Test System responds with a **SetChargingProfileResponse** with **status Accepted**

1. The CSMS sends a **SetChargingProfileRequest**

Manual Action: Request the CSMS to reboot the Charging Station with status ImmediateAndResume

4. The Test System responds with a **ResetResponse** with **status Accepted**

3. The CSMS sends a **ResetRequest**

5. The Test System sends a **TransactionEventRequest**.
- **eventType Updated**
- **triggerReason ResetCommand**

6. The CSMS responds with a **TransactionEventResponse**.

Main (Test scenario)	
7. The Test System sends a BootNotificationRequest with reason <i>RemoteReset</i>	8. The CSMS responds with a BootNotificationResponse
9. The Test System notifies the CSMS about the current state of all connectors. For <Configured connectorId>: Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Occupied"</i> - component.name <i>"Connector"</i> - variable.name <i>"AvailabilityState"</i> For <Other connector(s)>: Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Available"</i> - component.name <i>"Connector"</i> - variable.name <i>"AvailabilityState"</i>	10. The CSMS responds accordingly.
11. The Test System sends a SecurityEventNotificationRequest with - type <i>ResetOrReboot</i>	12. The CSMS responds with a SecurityEventNotificationResponse
13. The Test System sends a TransactionEventRequest . - eventType <i>Updated</i> - triggerReason <i>TxResumed</i> - idToken is omitted	14. The CSMS responds with a TransactionEventResponse .
16. The Test System responds with a SetChargingProfileResponse with status <i>Accepted</i>	15. The CSMS sends a SetChargingProfileRequest
Tool validations	
* Step 1: Message SetChargingProfileRequest - chargingProfile.chargingProfilePurpose <i>TxProfile</i> * Step 3: Message ResetRequest - type <i>ImmediateAndResume</i> * Step 8: Message BootNotificationResponse - status <i>Accepted</i> * Step 15: Message: SetChargingProfileRequest - chargingProfile.chargingProfilePurpose <i>TxProfile</i>	
Post scenario validations: - N/a	

4. C Authorization

TC_C_02_CSMS: Local start transaction - Authorization Invalid/Unknown

Test case name	Local start transaction - Authorization Invalid/Unknown
Test case Id	TC_C_02_CSMS
Use case Id(s)	<u>C01</u> , <u>C04</u> , <u>C06</u>
Requirement(s)	<u>C01.FR.07</u> OR <u>C04.FR.01</u> OR <u>C06.FR.04</u>
System under test	CSMS
Description	When a Charging Station needs to charge an EV, it needs to authorize the EV Driver first at the CSMS before the charging can be started or stopped.
Purpose	To verify whether the CSMS is able to report that an idToken is NOT valid.
Prerequisite(s)	N/a

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): N/a

Main (Test scenario)	
Charging Station	CSMS
1. The Test System sends an AuthorizeRequest with idToken.idToken <Configured invalid_idtoken_idtoken> idToken.type <Configured invalid_idtoken_type>	2. The CSMS responds with an AuthorizeResponse

Tool validations
* Step 2: Message: AuthorizeResponse - idTokenInfo.status <i>Invalid</i> or <i>Unknown</i>
Post scenario validations: - N/a

TC_C_06_CSMS: Local start transaction - Authorization Blocked

Test case name	Local start transaction - Authorization Blocked
Test case Id	TC_C_06_CSMS
Use case Id(s)	<u>C01</u>
Requirement(s)	<u>C01.FR.07</u>
System under test	CSMS
Description	When a Charging Station needs to charge an EV, it needs to authorize the EV Driver first at the CSMS before the charging can be started or stopped.
Purpose	To verify whether the CSMS is able to report that an idToken is Blocked.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

The IdToken configured as Blocked at the Test System, must be set as Blocked at the CSMS.

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest with idToken.idToken <Configured blocked_idtoken_idtoken> idToken.type <Configured blocked_idtoken_type>	2. The CSMS responds with an AuthorizeResponse

Tool validations

* Step 2:

Message: **AuthorizeResponse**

- **idTokenInfo.status** *Blocked* or *Invalid*

Post scenario validations:

TC_C_07_CSMS: Local start transaction - Authorization Expired

Test case name	Local start transaction - Authorization Expired
Test case Id	TC_C_07_CSMS
Use case Id(s)	<u>C01</u>
Requirement(s)	<u>C01.FR.07</u>
System under test	CSMS
Description	When a Charging Station needs to charge an EV, it needs to authorize the EV Driver first at the CSMS before the charging can be started or stopped.
Purpose	To verify whether the CSMS is able to report that an idToken is Expired.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

The IdToken configured as Expired at the Test System, must be set as Expired at the CSMS.

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest with idToken.idToken <Configured expired_idtoken_idtoken> idToken.type <Configured expired_idtoken_type>	2. The CSMS responds with an AuthorizeResponse

Tool validations

* Step 2:

Message: **AuthorizeResponse**

- **idTokenInfo.status** *Expired* or *Invalid*

Post scenario validations:

TC_C_08_CSMS: Authorization through authorization cache - Accepted

Test case name	Authorization through authorization cache - Accepted
Test case Id	TC_C_08_CSMS
Use case Id(s)	<u>C12</u>
Requirement(s)	<u>C12.FR.03</u>
System under test	CSMS
Description	This test case describes how the EV Driver is authorized to start a transaction while the Charging Station uses Cached IdToken. This enables the EV Driver to Online start a transaction by using the Authorization Cache in which the Charging Station can respond faster, as no AuthorizeRequest is being sent.
Purpose	To verify if the CSMS is able to respond correctly when an idToken which has status "Accepted" in the charging stations cache is presented according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station

1. The Test System sends a **TransactionEventRequest** with
 - **triggerReason** *Authorized*
 - **idToken** *<Valid id token configured in Authorization Cache>*
 - **eventType** *Updated*

Note(s):

- **TxStartPoint** contains *ParkingBayOccupancy*

CSMS

2. The CSMS responds with a **TransactionEventResponse**

Tool validations

* Step 2:

Message **TransactionEventResponse**

- **idTokenInfo.status** *Accepted*

Post scenario validations:

- N/a

TC_C_20_CSMS: Authorization through authorization cache - Invalid

Test case name	Authorization through authorization cache - Invalid
Test case Id	TC_C_20_CSMS
Use case Id(s)	<u>C12</u>
Requirement(s)	<u>C12.FR.03</u>
System under test	CSMS
Description	This test case describes how the EV Driver is authorized to start a transaction while the Charging Station uses Cached IdToken. This enables the EV Driver to Online start a transaction by using the Authorization Cache in which the Charging Station can respond faster, as no AuthorizeRequest is being sent.
Purpose	To verify if the CSMS is able to respond correctly when an idToken, which has status "Invalid" in the charging stations cache but not in the CSMS, is presented according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:State is *EVConnectedPreSession***Main (Test scenario)**

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with - triggerReason <i>Authorized</i> - idToken.idToken <i><Configured invalid_idtoken_idtoken></i> - idToken.type <i><Configured invalid_idtoken_type></i> - eventType <i>Updated</i> <u>Note(s)</u>: - TxStartPoint contains <i>ParkingBayOccupancy</i>	2. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 2:

Message **TransactionEventResponse**- **idTokenInfo.status** *Invalid* or *Unknown***Post scenario validations:**

- N/a

TC_C_37_CSMS: Clear Authorization Data in Authorization Cache - Accepted

Test case name	Clear Authorization Data in Authorization Cache - Accepted
Test case Id	TC_C_37_CSMS
Use case Id(s)	<u>C11</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the Charging Station autonomously stores a record of previously presented identifiers that have been successfully authorized by the CSMS in the Authorization Cache. (Successfully meaning: a response received on a message containing an IdToken)
Purpose	To verify if the CSMS is able to request the Charging Station to clear all identifiers from the Authorization Cache according to the mechanism as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

2. The Test System responds with a **ClearCacheResponse** with status *Accepted*

CSMS

1. The CSMS sends a **ClearCacheRequest**

Tool validations

- N/a

Post scenario validations:

- N/a

TC_C_38_CSMS: Clear Authorization Data in Authorization Cache - Rejected

Test case name	Clear Authorization Data in Authorization Cache - Rejected
Test case Id	TC_C_38_CSMS
Use case Id(s)	<u>C11</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the Charging Station autonomously stores a record of previously presented identifiers that have been successfully authorized by the CSMS in the Authorization Cache. (Successfully meaning: a response received on a message containing an IdToken)
Purpose	To verify if the CSMS is able to request the Charging Station to clear all identifiers from the Authorization Cache according to the mechanism as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

2. The Test System responds with a **ClearCacheResponse** with status *Rejected*

CSMS

1. The CSMS sends a **ClearCacheRequest**

Tool validations

- N/a

Post scenario validations:

- N/a

TC_C_39_CSMS: Authorization by GroupId - Success

Test case name	Authorization by GroupId - Success
Test case Id	TC_C_39_CSMS
Use case Id(s)	<u>C09</u>
Requirement(s)	<u>C09.FR.02</u> , <u>C09.FR.03</u>
System under test	CSMS
Description	This test case covers how a Charging Station can authorize an action for an EV Driver based on GroupId information. This could for example be used if 2 people regularly use the same EV: they can use their own IdToken (e.g. RFID card), and can deauthorize transactions that were started with the other idToken (with the same GroupId).
Purpose	To verify if the CSMS is able to correctly handle the Authorization of idTokens with the same GroupId according to the mechanism as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

Two valid idTokens with the same GroupId are configured

Reusable State(s):state is *EVConnectedPreSession***Main (Test scenario)**

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest with idToken.idToken <Configured valid_idtoken2_idtoken> idToken.type <Configured valid_idtoken2_type>	2. The CSMS responds with an AuthorizeResponse
3. The Test System sends a TransactionEventRequest with - triggerReason <i>Authorized</i> - idToken.idToken <Configured valid_idtoken_idtoken> - idToken.type <Configured valid_idtoken_type> if transaction was already started - eventType <i>Updated</i> else - eventType <i>Started</i>	4. The CSMS responds with a TransactionEventResponse
5. Execute Reusable State <i>EnergyTransferStarted</i>	
6. The Test System sends an AuthorizeRequest with idToken.idToken <Configured valid_idtoken2_idtoken> idToken.type <Configured valid_idtoken2_type>	7. The CSMS responds with an AuthorizeResponse

Main (Test scenario)	
8. The Test System sends a TransactionEventRequest with - triggerReason <i>StopAuthorized</i> - idToken.idToken <i><Configured valid_idtoken2_idtoken></i> - idToken.type <i><Configured valid_idtoken2_type></i> - eventType <i>Updated</i>	9. The CSMS responds with a TransactionEventResponse
10. Execute Reusable State <i>EVConnectedPostSession</i>	
11. Execute Reusable State <i>EVDisconnected</i>	

Tool validations
* Step 2: Message AuthorizeResponse - idTokenInfo.status <i>Accepted</i> - idTokenInfo.groupIdToken.idToken <i><Configured groupIdToken></i> * Step 4: Message TransactionEventResponse - idTokenInfo.status <i>Accepted</i> - idTokenInfo.groupIdToken.idToken <i><Configured groupIdToken></i> * Step 7: Message AuthorizeResponse - idTokenInfo.status <i>Accepted</i> - idTokenInfo.groupIdToken.idToken <i><Configured groupIdToken></i> * Step 9: Message TransactionEventResponse - idTokenInfo.status <i>Accepted</i> - idTokenInfo.groupIdToken.idToken <i><Configured groupIdToken></i>
Post scenario validations: - N/a

TC_C_40_CSMS: Authorization by GroupId - Success with Local Authorization List

Test case name	Authorization by GroupId - Success with Local Authorization List
Test case Id	TC_C_40_CSMS
Use case Id(s)	<u>C09</u>
Requirement(s)	<u>C09.FR.02</u> , <u>C09.FR.03</u>
System under test	CSMS
Description	This test case covers how a Charging Station can authorize an action for an EV Driver based on GroupId information. This could for example be used if 2 people regularly use the same EV: they can use their own IdToken (e.g. RFID card), and can deauthorize transactions that were started with the other idToken (with the same GroupId).
Purpose	To verify if the CSMS is able to correctly handle the Authorization of idTokens with the same GroupId which are located in the Local Authorization List according to the mechanism as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

Two valid idTokens with same GroupId are configured

Reusable State(s):state is *EVConnectedPreSession***Main (Test scenario)**

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with - triggerReason <i>Authorized</i> - idToken.idToken <i><Configured valid_idtoken_idtoken></i> (with a configured GroupId) which is configured in the local Authorization List - idToken.type <i><Configured valid_idtoken_type></i> (with a configured GroupId) which is configured in the local Authorization List If transaction was already started - eventType <i>Updated</i> else - eventType <i>Started</i>	2. The CSMS responds with a TransactionEventResponse
3. Execute Reusable State <i>EnergyTransferStarted</i>	

Main (Test scenario)	
5. The Test System sends a TransactionEventRequest with - triggerReason <i>StopAuthorized</i> - idToken.idToken <Configured valid_idtoken2_idtoken> (with same configured GroupId) which is configured in the local Authorization List> - idToken.type <Configured valid_idtoken2_type> - eventType <i>Updated</i>	6. The CSMS responds with a TransactionEventResponse
7. Execute Reusable State <i>EVConnectedPostSession</i>	
8. Execute Reusable State <i>EVDisconnected</i>	

Tool validations
* Step 2: Message TransactionEventResponse - idTokenInfo.status <i>Accepted</i> - idTokenInfo.groupIdToken.idToken <Configured groupIdToken>
* Step 6: Message TransactionEventResponse - idTokenInfo.status <i>Accepted</i> - idTokenInfo.groupIdToken.idToken <Configured groupIdToken>
Post scenario validations: - N/a

TC_C_43_CSMS: Authorization by GroupId - Invalid status with Local Authorization List

Test case name	Authorization by GroupId - Invalid status with Local Authorization List
Test case Id	TC_C_43_CSMS
Use case Id(s)	<u>C09</u>
Requirement(s)	<u>C09.FR.02</u> , <u>C09.FR.03</u>
System under test	CSMS
Description	This test case covers how a Charging Station can authorize an action for an EV Driver based on GroupId information. This could for example be used if 2 people regularly use the same EV: they can use their own IdToken (e.g. RFID card), and can deauthorize transactions that were started with the other idToken (with the same GroupId).
Purpose	To verify if the CSMS is able to correctly handle the Authorization of idTokens with the same GroupId which are located in the Local Authorization List according to the mechanism as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

Two known valid idTokens with same GroupId are configured.

Reusable State(s):

state is *EVConnectedPreSession*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with - triggerReason <i>Authorized</i> - idToken.idToken <i><Configured valid_idtoken_idtoken></i> - idToken.type <i><Configured valid_idtoken_type></i> if transaction was already started - eventType <i>Updated</i> else - eventType <i>Started</i>	2. The CSMS responds with a TransactionEventResponse
3. Execute Reusable State <i>EnergyTransferStarted</i>	
4. The Test System sends an AuthorizeRequest with - idToken.idToken <i><Configured valid_idtoken2_idtoken></i> - idToken.type <i><Configured valid_idtoken2_type></i>	5. The CSMS responds with an AuthorizeResponse
6. The Test System sends a TransactionEventRequest with - triggerReason <i>StopAuthorized</i> - idToken.idToken <i><Configured valid_idtoken2_idtoken></i> - idToken.type <i><Configured valid_idtoken2_type></i> - eventType <i>Updated</i>	7. The CSMS responds with a TransactionEventResponse

Main (Test scenario)

8. Execute **Reusable State** *EVConnectedPostSession*

9. Execute **Reusable State** *EVDisconnected*

Tool validations

* Step 1:

Message **TransactionEventResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<Configured groupIdToken>*

* Step 5:

Message **AuthorizeResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<Configured groupIdToken>*

* Step 7:

Message **TransactionEventResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<Configured groupIdToken>*

Post scenario validations:

- N/a

TC_C_47_CSMS: Stop Transaction with a Master Pass - With UI - All transactions

Test case name	Stop Transaction with a Master Pass - With UI - All transactions
Test case Id	TC_C_47_CSMS
Use case Id(s)	C16
Requirement(s)	C16.FR.01
System under test	CSMS
Description	This test case covers how somebody with a Master Pass (User) can stop (selected) ongoing transactions, so the cable becomes unlocked. This Master Pass can be configured in: MasterPassGroupId. This could for example be usefull for Law Enforcement officials.
Purpose	To verify if the CSMS is able to correctly respond on a request to stop all transactions when an idToken which has the MasterPass as GroupId is used and the user has selected to stop all transactions in the User Interface according to the mechanism as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

An idToken with the MastersPass as GroupId is configured

Reusable State(s):

State is *EnergyTransferStarted* for EVSE 1 with idToken valid idTokenState is *EnergyTransferStarted* for EVSE 2 with idToken valid idToken2

Main (Test scenario)

Charging Station	CSMS
<u>Manual action:</u> trigger CSMS to set the masterPassGroupId	
2. The Test System responds with SetVariablesResponse with attributeStatus = <i>Accepted</i>	1. the CSMS sends SetVariablesRequest with: - variable.name = "MasterPassGroupId" - component.name = "AuthCtrlr" - attributeValue = <Specified masterPassGroupId>
3. The Test System sends an AuthorizeRequest with idToken.idToken <Configured masterpass_idtoken_idtoken> idToken.type <Configured masterpass_idtoken_type>	4. The CSMS responds with an AuthorizeResponse
5. The Test System sends a TransactionEventRequest with - transactionInfo.stoppedReason MasterPass - idToken.idToken <Configured masterpass_idtoken_idtoken> - idToken.type <Configured masterpass_idtoken_type> - eventType Ended for both EVSE	6. The CSMS responds with a TransactionEventResponse for both EVSE

Tool validations

* Step 4:

Message **AuthorizeResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<masterPassGroupId from step 1>*

* Step 6:

Message **TransactionEventResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<masterPassGroupId from step 1>*

Post scenario validations:

- N/a

TC_C_48_CSMS: Stop Transaction with a Master Pass - With UI - With UI - Specific transactions

Test case name	Stop Transaction with a Master Pass - With UI - With UI - Specific transactions
Test case Id	TC_C_48_CSMS
Use case Id(s)	<u>C16</u>
Requirement(s)	<u>C16.FR.01</u>
System under test	CSMS
Description	This test case covers how somebody with a Master Pass (User) can stop (selected) ongoing transactions, so the cable becomes unlocked. This Master Pass can be configured in: MasterPassGroupId. This could for example be usefull for Law Enforcement officials.
Purpose	To verify if the CSMS is able to correctly respond on a request to stop a transaction when an idToken which has the MasterPass as GroupId is used and the user has selected to stop one transaction in the User Interface according to the mechanism as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

An idToken with the MastersPass as GroupId is configured

Reusable State(s):

State is *EnergyTransferStarted* for all EVSE

Main (Test scenario)

Charging Station	CSMS
<u>Manual action:</u> trigger CSMS to set the masterPassGroupId	
2. The Test System responds with SetVariablesResponse with attributeStatus = <i>Accepted</i>	1. the CSMS sends SetVariablesRequest with: - variable.name = "MasterPassGroupId" - component.name = "AuthCtrlr" - attributeValue = <Specified masterPassGroupId>
3. The Test System sends an AuthorizeRequest with idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	4. The CSMS responds with an AuthorizeResponse
5. The Test System sends a TransactionEventRequest with - transactionInfo.stoppedReason MasterPass - idToken.idToken <Configured masterpass_idtoken_idtoken> - idToken.type <Configured masterpass_idtoken_type> - eventType Ended	6. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 4:

Message **AuthorizeResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<masterPassGroupId from step 1>*

* Step 6:

Message **TransactionEventResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<masterPassGroupId from step 1>*

Post scenario validations:

- N/a

TC_C_49_CSMS: Stop Transaction with a Master Pass - Without UI

Test case name	Stop Transaction with a Master Pass - Without UI
Test case Id	TC_C_49_CSMS
Use case Id(s)	C16
Requirement(s)	C16.FR.02
System under test	CSMS
Description	This test case covers how somebody with a Master Pass (User) can stop (selected) ongoing transactions, so the cable becomes unlocked. This Master Pass can be configured in: MasterPassGroupId. This could for example be usefull for Law Enforcement officials.
Purpose	To verify if the CSMS is able to correctly respond on a request to stop all transactions when an idToken which has the MasterPass as GroupId is used and the Charging Station does not have a User Interface according to the mechanism as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

An idToken with the MastersPass as GroupId is configured

Reusable State(s):

State is *EnergyTransferStarted* for EVSE 1 with idToken valid idTokenState is *EnergyTransferStarted* for EVSE 2 with idToken valid idToken2

Main (Test scenario)

Charging Station	CSMS
<u>Manual action:</u> trigger CSMS to set the masterPassGroupId	
2. The Test System responds with SetVariablesResponse with attributeStatus = <i>Accepted</i>	1. the CSMS sends SetVariablesRequest with: - variable.name = "MasterPassGroupId" - component.name = "AuthCtrlr" - attributeValue = <Specified masterPassGroupId>
3. The Test System sends an AuthorizeRequest with idToken.idToken <Configured masterpass_idtoken_idtoken> idToken.type <Configured masterpass_idtoken_type>	4. The CSMS responds with an AuthorizeResponse
5. The Test System sends a TransactionEventRequest with - transactionInfo.stoppedReason MasterPass - idToken.idToken <Configured valid_idtoken_idtoken> - idToken.type <Configured valid_idtoken_type> - eventType Ended for both EVSE	6. The CSMS responds with a TransactionEventResponse for both EVSE

Tool validations

* Step 4:

Message **AuthorizeResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<masterPassGroupId from step 1>*

* Step 6:

Message **TransactionEventResponse**

- **idTokenInfo.status** *Accepted*

- **idTokenInfo.groupIdToken.idToken** *<masterPassGroupId from step 1>*

Post scenario validations:

- N/a

TC_C_50_CSMS: Authorization using Contract Certificates 15118 - Online - Local contract certificate validation - Accepted

Test case name	Authorization using Contract Certificates 15118 - Online - Local contract certificate validation - Accepted
Test case Id	TC_C_50_CSMS
Use case Id(s)	<u>C07</u>
Requirement(s)	<u>C07.FR.04</u>
System under test	CSMS
Description	The Charging Station is able to authorize with contract certificates when it supports ISO 15118.
Purpose	To verify if the CSMS is able to validate the certificate hash data and the provided eMAID.
Prerequisite(s)	- The configured eMAID is known by the CSMS as valid. - The configured contract certificate is valid. - The CN of the configured contract certificate equals the configured eMAID. - iso15118CertificateHashData has a responder URL that points to an OCSP service for Test System. - CSMS does not have a cached OCSP response for the contract certificate.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured eMAID> idToken.type <i>eMAID</i> iso15118CertificateHashData contains <hashes from configured (V2G) certificate chain>	2. The CSMS sends an OCSP request to responder URL of iso15118CertificateHashData to check validity
3. The Test System OCSP service responds that certificate is valid.	4. The CSMS responds with a AuthorizeResponse
5. The Test System sends a TransactionEventRequest With triggerReason <i>Authorized</i>	6. The CSMS responds with a TransactionEventResponse
5. Execute Reusable State <i>EnergyTransferStarted</i>	

Tool validations

* Step 2:

CSMS sends an OSCP request for **iso15118CertificateHashData**

* Step 3:

Test System checks that received request for **iso15118CertificateHashData** is valid

* Step 4: Message: **AuthorizeResponse**

- **idTokenInfo.status** *Accepted*

- **certificateStatus** *Accepted*

* Step 4:

Message: **TransactionEventResponse**

- **idTokenInfo.status** *Accepted*

Post scenario validations:

N/a

TC_C_51_CSMS: Authorization using Contract Certificates 15118 - Online - Local contract certificate validation - Rejected

Test case name	Authorization using Contract Certificates 15118 - Online - Local contract certificate validation - Rejected
Test case Id	TC_C_51_CSMS
Use case Id(s)	<u>C07</u>
Requirement(s)	<u>C07.FR.16</u>
System under test	CSMS
Description	The Charging Station is able to authorize with contract certificates when it supports ISO 15118.
Purpose	To verify if the CSMS is able to validate the certificate hash data and the provided eMAID.
Prerequisite(s)	- The configured eMAID is known by the CSMS as valid. - The contract certificate is revoked. - iso15118CertificateHashData has a responder URL that points to an OCSP service for Test System. - CSMS does not have a cached OCSP response for the contract certificate.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type> iso15118CertificateHashData contains <hashes from configured (V2G) certificate chain>	2. The CSMS sends an OCSP request to responder URL of iso15118CertificateHashData to check validity
3. The Test System OCSP service responds that certificate is valid.	4. The CSMS responds with a AuthorizeResponse

Tool validations

* Step 2:

CSMS sends an OCSP request for **iso15118CertificateHashData**

* Step 3:

Test System checks that received request for **iso15118CertificateHashData** is valid* Step 4: Message: **AuthorizeResponse**- **idTokenInfo.status** *Invalid*- **certificateStatus** *CertificateRevoked*

Tool validations**Post scenario validations:**

EV is not authorized and shall not charge:

Charging Station does not send **TransactionEventRequest** with:

- **triggerReason** = *Authorized* or **chargingState** = *Charging*

TC_C_52_CSMS: Authorization using Contract Certificates 15118 - Online - Central contract certificate validation - Accepted

Test case name	Authorization using Contract Certificates 15118 - Online - Central contract certificate validation - Accepted
Test case Id	TC_C_52_CSMS
Use case Id(s)	<u>C07</u>
Requirement(s)	<u>C07.FR.04,C07.FR.05</u>
System under test	CSMS
Description	The Charging Station is able to authorize with contract certificates when it supports ISO 15118.
Purpose	To verify if the CSMS is able to validate the provided certificate and eMAID. The field iso15118CertificateHashData is not provided to force CSMS to calculate certificate hash data for the OSCP request.
Prerequisite(s)	- The configured eMAID is known by the CSMS as valid. - The contract certificate is signed by the configured V2GRoot or MORoot certificate at the CSMS. - Contract certificate has a responder URL that points to an OSCP service for Test System. - CSMS does not have a cached OSCP response for the contract certificate.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type> iso15118CertificateHashData is absent certificate from keystore	2. The CSMS sends an OSCP request to responder URL of certificate to check validity
3. The Test System OSCP service responds that certificate is valid.	4. The CSMS responds with a AuthorizeResponse
5. The Test System sends a TransactionEventRequest With triggerReason <i>Authorized</i>	6. The CSMS responds with a TransactionEventResponse
5. Execute Reusable State <i>EnergyTransferStarted</i>	

Tool validations

* Step 2:

CSMS sends an OSCP request for **certificate**

* Step 3:

Test System checks that received request for **certificate** is valid AND key type = ECDSA AND certificate chain contains at least one subCA

* Step 4: Message: **AuthorizeResponse**

- **idTokenInfo.status** *Accepted*

- **certificateStatus** *Accepted*

* Step 6:

Message: **TransactionEventResponse**

- **idTokenInfo.status** *Accepted*

Post scenario validations:

N/a

TC_C_103_CSMS: Authorization with prepaid card - success

Test case name	Authorization with prepaid card - success
Test case Id	TC_C_103_CSMS
Use case Id(s)	<u>C17</u>
Requirement(s)	<u>C17.FR.01</u> , <u>C17.FR.03</u>
System under test	CSMS
Description	This test case verifies if the CSMS is able to authorize a prepaid card for a transaction.
Purpose	To verify if the CSMS is able to authorize a prepaid card for a transaction.
Prerequisite(s)	- CSMS supports prepaid cards. - Prepaid card with positive amount is available.

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): N/a

Main (Test scenario)	
Charging Station	CSMS
1. The Test System sends a AuthorizeRequest with idToken.idToken <prepaid card id> idToken.type = <prepaid card id type>	2. The CSMS responds a AuthorizeResponse
3. The Test System sends a TransactionEventRequest with - eventType Started - idToken.idToken <prepaid card id> - idToken.type = <prepaid card id type>	4. The CSMS responds with a TransactionEventResponse

Tool validations
* Step 2: Message AuthorizeResponse - idTokenInfo.status Accepted - idTokenInfo.cacheExpiryDateTime <current date/time> * Step 4: Message TransactionEventResponse - idTokenInfo.status Accepted - idTokenInfo.cacheExpiryDateTime <cacheExpiryDateTime from step 2> - transactionLimit.maxCost = <not omitted>

TC_C_104_CSMS: Authorization with prepaid card - no credit

Test case name	Authorization with prepaid card - no credit
Test case Id	TC_C_104_CSMS
Use case Id(s)	<u>C17</u>
Requirement(s)	<u>C17.FR.02</u>
System under test	CSMS
Description	This test case verifies if the CSMS is able to authorize a prepaid card for a transaction.
Purpose	To verify if the CSMS is able to reject a prepaid card for a transaction.
Prerequisite(s)	- CSMS supports prepaid cards. - Prepaid card with an amount of "0" is available.

Before
(Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main
(Test scenario)

Charging Station	CSMS
1. The Test System sends a AuthorizeRequest with idToken.idToken <prepaid card no credit id> idToken.type = <prepaid card no credit id type>	2. The CSMS responds a AuthorizeResponse

Tool validations

* Step 2:

Message **AuthorizeResponse**- **idTokenInfo.status** NoCredit- **idTokenInfo.cacheExpiryDateTime** <current date/time>

TC_C_108_CSMS: Integrated Payment Terminal - VAT number validation

Test case name	Integrated Payment Terminal - VAT number validation
Test case Id	TC_C_108_CSMS
Use case Id(s)	<u>C18</u>
Requirement(s)	<u>C18.FR.09</u> , <u>C18.FR.10</u>
System under test	CSMS
Description	To start/authorize a transaction from a payment terminal connected directly to the Charging Station.
Purpose	To verify that the CSMS can validate VAT numbers correctly
Prerequisite(s)	* CSMS supports VAT number validation

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Valid VAT number with EVSE ID

1. The Test System sends a **VatNumberValidationRequest** with **vatNumber** = **<validVatNumber>**
evseId = 1

2. The CSMS validates given VAT number and responds with **VatNumberValidationResponse**

Invalid VAT number with EVSE ID

3. The Test System sends a **VatNumberValidationRequest** with **vatNumber** = **<invalidVatNumber>**
evseId = 1

4. The CSMS validates given VAT number and responds with **VatNumberValidationResponse**

Valid VAT number without EVSE ID

5. The Test System sends a **VatNumberValidationRequest** with **vatNumber** = **<validVatNumber>**
evseId = **<omitted>**

6. The CSMS validates given VAT number and responds with **VatNumberValidationResponse**

Invalid VAT number without EVSE ID

7. The Test System sends a **VatNumberValidationRequest** with **vatNumber** = **<invalidVatNumber>**
evseId = **<omitted>**

8. The CSMS validates given VAT number and responds with **VatNumberValidationResponse**

Tool validations

* Step 2:

Message **VatNumberValidationResponse**

- **vatNumber** must be *<validVatNumber>*
- **evseId** must be *1*
- **status** must be *Accepted*

* Step 4:

Message **VatNumberValidationResponse**

- **vatNumber** must be *<invalidVatNumber>*
- **evseId** must be *1*
- **status** must be *Rejected*

* Step 6:

Message **VatNumberValidationResponse**

- **vatNumber** must be *<validVatNumber>*
- **evseId** must be *<omitted>*
- **status** must be *Accepted*

* Step 8:

Message **VatNumberValidationResponse**

- **vatNumber** must be *<invalidVatNumber>*
- **evseId** must be *<omitted>*
- **status** must be *Rejected*

Post scenario validations:

- N/a

TC_C_113_CSMS: Integrated Payment Terminal - Cancellation after start of transaction - stopped by EV driver

Test case name	Integrated Payment Terminal - Cancellation after start of transaction - stopped by EV driver
Test case Id	TC_C_113_CSMS
Use case Id(s)	<u>C20</u>
Requirement(s)	<u>C20.FR.05</u>
System under test	CSMS
Description	To inform the CSMS that payment has been canceled and the authorization amount has been released.
Purpose	To verify if the CSMS is able to handle Cancellation of a local payment terminal authorization after a transaction has started when the EV driver stops the session at the Charging Station.
Prerequisite(s)	- CSMS supports Charging Stations with integrated payment terminals. - Test System has PaymentCtrlr.AuthorizeDirectPayment = <i>false</i> - Test System has PaymentCtrlr.AuthorizationAmount = <i>50.00</i> - Test System has PaymentCtrlr.PaymentDetails = "CardLast4Digits" - Test System has PaymentCtrlr.SettlementByCSMS = <i>false</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Note: Starts and immediately ends a transaction

1. The Test System sends a **TransactionEventRequest** with
eventType *Started*
triggerReason *Authorized*
idToken *<pspRef>*
idToken.type *DirectPayment*
idToken.additionalInfo[0].additionalIdToken = *1234*
idToken.additionalInfo[0].type = *Last4Digits*
transactionInfo.transactionId *<transactionId>*
transactionInfo.transactionLimit.maxCost *50.00*

2. The CSMS responds with a **TransactionEventResponse**

Main (Test scenario)	
3. The Test System sends a TransactionEventRequest with eventType <i>Ended</i> idToken <i><pspRef></i> idToken.type <i>DirectPayment</i> idToken.additionalInfo[0].additionalIdToken <i>= 1234</i> idToken.additionalInfo[0].type <i>= Last4Digits</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.stoppedReason <i>Local</i> triggerReason <i>StopAuthorized</i> costDetails.totalCost.currency <i>EUR</i> costDetails.totalCost.typeOfCost <i>NormalCost</i> costDetails.totalCost.total <i>0</i> costDetails.totalUsage.energy <i>0</i> costDetails.totalUsage.chargingTime <i>0</i> costDetails.totalUsage.idleTime <i>0</i>	4. The CSMS responds with a TransactionEventResponse
5. The Test System sends a NotifySettlementRequest with transactionId <i><transactionId></i> pspRef <i><pspRef></i> status <i>Canceled</i> settlementTime <i><Equal to timestamp step 3></i> settlementAmount <i>0</i> receiptUrl <i><omitted></i>	6. The CSMS responds with a NotifySettlementResponse

Tool validations
N/a
Post scenario validations:
N/a

TC_C_117_CSMS: Settlement at end of transaction - settled by CSMS

Test case name	Settlement at end of transaction - settled by CSMS, receipt by CSMS
Test case Id	TC_C_117_CSMS
Use case Id(s)	<u>C21</u>
Requirement(s)	<u>C21.FR.03</u> , <u>C21.FR.05</u>
System under test	CSMS
Description	Settle the amount of charging session via CSMS.
Purpose	To verify if the CSMS is able to settle a transaction directly via PSP.
Prerequisite(s)	- CSMS has an interface to the PSP. - CSMS supports Charging Stations with integrated payment terminals. - Test System has PaymentCtrlr.Enabled = <i>true</i> - Test System has PaymentCtrlr.AuthorizeDirectPayment = <i>false</i> - Test System has PaymentCtrlr.AuthorizeAmount = <i>50.00</i> - Test System has PaymentCtrlr.PaymentDetails = "CardLast4Digits" - Test System has PaymentCtrlr.SettlementByCSMS = <i>true</i> - Value of PaymentCtrlr.ReceiptByCSMS is irrelevant.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with eventType <i>Started</i> idToken <i><PspRef></i> idToken.type <i>DirectPayment</i> transactionInfo.transactionId <i><transactionId></i> idToken.additionalInfo[0].additionalIdToken = <i><not omitted></i> idToken.additionalInfo[0].type = <i>CardLast4Digits</i> transactionInfo.transactionLimit.maxCost <i>50.00</i>	2. The CSMS responds with a TransactionEventResponse

Main (Test scenario)

3. The Test System sends a **TransactionEventRequest** with
eventType *Ended*
idToken *<PspRef>*
idToken.type *DirectPayment*
triggerReason *StopAuthorized*
transactionInfo.stoppedReason *Local*
transactionInfo.transactionId *<transactionId>*
costDetails.totalCost.currency *EUR*
costDetails.totalCost.fixed.exclTax *15.00*
costDetails.totalCost.fixed.inclTax *18.15 EUR*
costDetails.totalCost.fixed.taxRate.type *MyTax*
costDetails.totalCost.fixed.taxRate.tax *21*
costDetails.totalUsage.energy *123*
costDetails.totalUsage.chargingTime *5*
costDetails.totalUsage.idleTime *0*
costDetails.chargingPeriods *<omitted>*

4. The CSMS responds with a **TransactionEventResponse**

Note(s):

- The CSMS settles the total cost of the transaction with the payment service provider for amount of 18.15 EUR.

Tool validations

N/a

Post scenario validations:

N/a

TC_C_118_CSMS: Settlement at end of transaction - settled by CS, receipt by CSMS

Test case name	Settlement at end of transaction - settled by CS, receipt by CSMS
Test case Id	TC_C_118_CSMS
Use case Id(s)	<u>C21</u>
Requirement(s)	<u>C21.FR.03</u>
System under test	CSMS
Description	In order to test that Charging Station supports settlement at the end of a transaction where the settlement is handled by the Charging Station and the receipt is provided by the CSMS.
Purpose	To verify that the CSMS can properly handle a transaction where the settlement is done by the Charging Station and the CSMS provides the receipt.
Prerequisite(s)	- CSMS supports settlement at the end of a transaction. - CSMS supports Charging Stations with integrated payment terminals. - Test System has PaymentCtrlr.Enabled = <i>true</i> - Test System has PaymentCtrlr.AuthorizeDirectPayment = <i>false</i> - Test System has PaymentCtrlr.AuthorizationAmount = <i>50.00</i> - Test System has PaymentCtrlr.PaymentDetails = <i>"CardLast4Digits"</i> - Test System has PaymentCtrlr.SettlementByCSMS = <i>false</i> - Test System has PaymentCtrlr.ReceiptByCSMS = <i>true</i> - CSMS lets the CS handle the settlement at the end of a transaction, but the CSMS provides receipts.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with eventType <i>Started</i> idToken <i><configured PspRef></i> idToken.type <i>DirectPayment</i> transactionInfo.transactionId <i><transactionId></i> idToken.additionalInfo[0].additionalIdToken = <i><not omitted></i> idToken.additionalInfo[0].type = <i>CardLast4Digits</i> transactionInfo.transactionLimit.maxCost <i>50.00</i>	2. The CSMS responds with a TransactionEventResponse

Main (Test scenario)	
3. The Test System sends a TransactionEventRequest with eventType <i>Ended</i> triggerReason <i>StopAuthorized</i> idToken <i><configured PspRef></i> idToken.type <i>DirectPayment</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.stoppedReason <i>Local</i> costDetails.totalCost.currency <i>EUR</i> costDetails.totalCost.fixed.exclTax <i>15.00</i> costDetails.totalCost.fixed.inclTax <i>18.15 EUR</i> costDetails.totalCost.fixed.taxRate.type <i>MyTax</i> costDetails.totalCost.fixed.taxRate.tax <i>21</i> costDetails.totalUsage.energy <i>123</i> costDetails.totalUsage.chargingTime <i>5</i> costDetails.totalUsage.idleTime <i>0</i> costDetails.chargingPeriods <i><omitted></i>	4. The CSMS responds with a TransactionEventResponse
5. The Test System sends a NotifySettlementRequest with transactionId <i><transactionId></i> pspRef <i><configured PspRef></i> status <i>Settled</i> settlementTime <i><not omitted></i> settlementAmount <i>18.15</i> receiptUrl <i><omitted></i>	6. The CSMS responds with a NotifySettlementResponse
Tool validations	
* Step 6: Message NotifySettlementResponse - receiptUrl must be <i><not omitted></i>	
Post scenario validations: - Verify that the receipt URL provided by the CSMS is valid and accessible.	

TC_C_119_CSMS: Settlement - is rejected or fails - Failed

Test case name	Settlement - is rejected or fails - Failed
Test case Id	TC_C_119_CSMS
Use case Id(s)	<u>C22</u>
Requirement(s)	...
System under test	CSMS
Description	To inform the CSMS that the transaction settlement has been rejected or otherwise failed to be successfully settled.
Purpose	To verify if the CSMS is able handle if the settlement failed according the Charging Station.
Prerequisite(s)	- CSMS supports Charging Stations with integrated payment terminals. - Test System has PaymentCtrlr.Enabled is <i>true</i> - Test System has PaymentCtrlr.AuthorizationAmount = <i>50.00</i> - Test System has PaymentCtrlr.PaymentDetails = "CardLast4Digits" - Test System has PaymentCtrlr.SettlementByCSMS is <i>false</i> - Test System has PaymentCtrlr.AuthorizeDirectPayment is <i>false</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with eventType <i>Started</i> idToken <i><configured PspRef></i> idToken.type <i>DirectPayment</i> transactionInfo.transactionId <i><transactionId></i> idToken.additionalInfo[0].additionalIdToken = <i><not omitted></i> idToken.additionalInfo[0].type = <i>CardLast4Digits</i> transactionInfo.transactionLimit.maxCost <i>50.00</i>	2. The CSMS responds with a TransactionEventResponse

Main (Test scenario)	
3. The Test System sends a TransactionEventRequest with eventType <i>Ended</i> idToken <i><configured PspRef></i> idToken.type <i>DirectPayment</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.stoppedReason <i>Local</i> triggerReason <i>StopAuthorized</i> costDetails.totalCost.currency <i>EUR</i> costDetails.totalCost.fixed.exclTax <i>15.00</i> costDetails.totalCost.fixed.inclTax <i>18.15</i> costDetails.totalCost.fixed.taxRate.type <i>MyTax</i> costDetails.totalCost.fixed.taxRate.tax <i>21</i> costDetails.totalUsage.energy <i>123</i> costDetails.totalUsage.chargingTime <i>5</i> costDetails.totalUsage.idleTime <i>0</i> costDetails.chargingPeriods <i><omitted></i>	4. The CSMS responds with a TransactionEventResponse
5. The Test System sends a NotifySettlementRequest with transactionId <i><transactionId></i> pspRef <i><configured PspRef></i> status <i>Failed</i> settlementTime <i><not omitted></i> settlementAmount <i>18.15</i> receiptUrl <i><omitted></i>	6. The CSMS responds with a NotifySettlementResponse

Tool validations
N/a
Post scenario validations:
N/a

TC_C_120_CSMS: Settlement - is rejected or fails - Rejected

Test case name	Settlement - is rejected or fails - Rejected
Test case Id	TC_C_120_CSMS
Use case Id(s)	<u>C22</u>
Requirement(s)	...
System under test	CSMS
Description	To inform the CSMS that the transaction settlement has been rejected or otherwise failed to be successfully settled.
Purpose	To verify if the CSMS is able to handle if the settlement is rejected according the Charging Station.
Prerequisite(s)	- CSMS supports Charging Stations with integrated payment terminals. - Test System has PaymentCtrlr.Enabled is <i>true</i> - Test System has PaymentCtrlr.AuthorizationAmount = 50.00 - Test System has PaymentCtrlr.PaymentDetails = "CardLast4Digits" - Test System has PaymentCtrlr.SettlementByCSMS is <i>false</i> - Test System has PaymentCtrlr.ReceiptByCSMS is <i>false</i> - Test System has PaymentCtrlr.AuthorizeDirectPayment is <i>false</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with eventType <i>Started</i> idToken <i><configured PspRef></i> idToken.type <i>DirectPayment</i> transactionInfo.transactionId <i><transactionId></i> idToken.additionalInfo[0].additionalIdToken = <i><not omitted></i> idToken.additionalInfo[0].type = <i>CardLast4Digits</i> transactionInfo.transactionLimit.maxCost 50.00	2. The CSMS responds with a TransactionEventResponse

Main (Test scenario)	
3. The Test System sends a TransactionEventRequest with eventType <i>Ended</i> idToken <i><configured PspRef></i> idToken.type <i>DirectPayment</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.stoppedReason <i>Local</i> triggerReason <i>StopAuthorized</i> costDetails.totalCost.currency <i>EUR</i> costDetails.totalCost.fixed.exclTax <i>15.00</i> costDetails.totalCost.fixed.inclTax <i>18.15 EUR</i> costDetails.totalCost.fixed.taxRate.type <i>MyTax</i> costDetails.totalCost.fixed.taxRate.tax <i>21</i> costDetails.totalUsage.energy <i>123</i> costDetails.totalUsage.chargingTime <i>5</i> costDetails.totalUsage.idleTime <i>0</i> costDetails.chargingPeriods is <i><omitted></i>	4. The CSMS responds with a TransactionEventResponse
5. The Test System sends a NotifySettlementRequest with transactionId <i><transactionId></i> pspRef <i><configured PspRef></i> status <i>Rejected</i> settlementTime <i><not omitted></i> settlementAmount <i>18.15</i> receiptUrl <i><omitted></i>	6. The CSMS responds with a NotifySettlementResponse
Tool validations	
N/a	
Post scenario validations:	
N/a	

TC_C_125_CSMS: Ad hoc payment via stand-alone payment terminal - central cost calculation

Test case name	Ad hoc payment via stand-alone payment terminal - central cost calculation
Test case Id	TC_C_125_CSMS
Use case Id(s)	<u>C24</u>
Requirement(s)	<u>C24.FR.01</u> , <u>C24.FR.02</u> , <u>C24.FR.10</u>
System under test	CSMS
Description	In order to test that Charging Station supports ad hoc payment via a stand-alone payment terminal with central cost calculation.
Purpose	To verify that the CSMS can properly handle ad hoc payments made via a stand-alone payment terminal when using central cost calculation.
Prerequisite(s)	- CSMS is connected with a stand-alone payment terminal - Test System has AuthCtrlr.AuthorizeRemoteStart = <i>false</i> - Test System has TariffCostCtrlr.Enabled[Cost] = <i>false</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Present a payment card to the payment terminal Kiosk for Charging Station with identity <Configured unique Charging Station identifier>

2. The Test System responds with a **RequestStartTransactionResponse** with status *Accepted*

1. The CSMS sends a **RequestStartTransactionRequest**

3. The Test System sends a **TransactionEventRequest** with
eventType *Started*
idToken <idToken from step 1>
idToken.type *DirectPayment*
idToken.additionalInfo <additionalInfo>
transactionInfo.transactionId <transactionId>
meterValue[0].timestamp <not omitted>
meterValue[0].sampledValue[0].value *10000*
meterValue[0].sampledValue[0].context *Transaction.Begin*

4. The CSMS responds with a **TransactionEventResponse**

Settlement

Main (Test scenario)

5. The Test System sends a **TransactionEventRequest** with
eventType *Ended*
idToken *<idToken from step 1>*
idToken.type *DirectPayment*
transactionInfo.transactionId *<transactionId>*
transactionInfo.transactionLimit.maxCost *<not omitted>*
transactionInfo.stoppedReason *Local*
triggerReason *StopAuthorized*
meterValue[0].timestamp *<not omitted>*
meterValue[0].sampledValue[0].value *15000*
meterValue[0].sampledValue[0].context *Transaction.End*

6. The CSMS responds with a **TransactionEventResponse**

Note(s):

- The CSMS sends a message to Payment Kiosk or PSP to settle the cost for amount calculated for 5000 Wh

Tool validations

* Step 1:

Message **RequestStartTransactionRequest**

- **evseId** must be *<evse>*
- **remoteStartId** must be *<not omitted>*
- **idToken.idToken** *<Not empty>*
- **idToken.type** must be *DirectPayment*
- **idToken.additionalInfo.additionalIdToken** must be *<Configured CardLast4Digits>*
- **idToken.additionalInfo.type** must be *CardLast4Digits*

* Step 4:

Message **TransactionEventResponse**

- **transactionLimit.maxCost** *<not omitted>*

* Step 6:

Message **TransactionEventResponse**

- **totalCost** *<not omitted>*

Post scenario validations:

N/a

TC_C_126_CSMS: Ad hoc payment via stand-alone payment terminal - local cost calculation

Test case name	Ad hoc payment via stand-alone payment terminal - local cost calculation
Test case Id	TC_C_126_CSMS
Use case Id(s)	<u>C24</u>
Requirement(s)	<u>C24.FR.01</u> , <u>C24.FR.02</u> , <u>C24.FR.10</u>
System under test	CSMS
Description	In order to test that Charging Station supports ad hoc payment via a stand-alone payment terminal with local cost calculation.
Purpose	To verify that the CSMS can properly handle ad hoc payments made via a stand-alone payment terminal when using local cost calculation.
Prerequisite(s)	- CSMS is connected with a stand-alone payment terminal - Test System has AuthCtrlr.AuthorizeRemoteStart = <i>false</i> - Test System has TariffCostCtrlr.Enabled[Cost] = <i>true</i> - Test System has TariffCostCtrlr.Currency = "EUR"

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Present a payment card to the payment terminal Kiosk for Charging Station with identity <Configured unique Charging Station identifier>

2. The Test System responds with a **RequestStartTransactionResponse** with **status** *Accepted*

1. The CSMS sends a **RequestStartTransactionRequest**

3. The Test System sends a **TransactionEventRequest** with **eventType** *Started*
idToken <IdToken from step 1>
idToken.type *DirectPayment*
idToken.additionalInfo <additionalInfo>
transactionInfo.transactionId <transactionId>

4. The CSMS responds with a **TransactionEventResponse**

Settlement

Main (Test scenario)

5. The Test System sends a **TransactionEventRequest** with
eventType *Ended*
idToken *<IdToken from step 1>*
idToken.type *DirectPayment*
transactionInfo.transactionId *<transactionId>*
transactionInfo.transactionLimit.maxCost *<not omitted>*
transactionInfo.stoppedReason *Local*
triggerReason *StopAuthorized*
costDetails.totalCost.currency *EUR*
costDetails.totalCost.fixed.exclTax *15.00*
costDetails.totalCost.fixed.inclTax *18.15 EUR*
costDetails.totalCost.fixed.taxRate.type *MyTax*
costDetails.totalCost.fixed.taxRate.tax *21*
costDetails.totalUsage.energy *123*
costDetails.totalUsage.chargingTime *5*
costDetails.totalUsage.idleTime *0*
costDetails.chargingPeriods *<omitted>*

6. The CSMS responds with a **TransactionEventResponse**

Note(s):

- The CSMS sends a message to Payment Kiosk or PSP to settle the cost for amount: 18.15 EUR

Tool validations

* Step 1:

Message **RequestStartTransactionRequest**

- **evseId** must be *<evse>*
- **remoteStartId** must be *<not omitted>*
- **idToken.idToken** must be *<Not empty>*
- **idToken.type** must be *DirectPayment*
- **idToken.additionalInfo.additionalIdToken** must be *<Configured CardLast4Digits>*
- **idToken.additionalInfo.type** must be *CardLast4Digits*

* Step 4:

Message **TransactionEventResponse**

transactionLimit.maxCost = *<not omitted>*

Post scenario validations:

N/a

TC_C_131_CSMS: Ad hoc payment via static or dynamic QR code - success

Test case name	Ad hoc payment via static or dynamic QR code - success
Test case Id	TC_C_131_CSMS
Use case Id(s)	<u>C25</u>
Requirement(s)	<u>C25.FR.07</u> , <u>C25.FR.09</u> , <u>C25.FR.20</u> , <u>C25.FR.21</u> , <u>C25.FR.23</u> , <u>C25.FR.24</u> , <u>C25.FR.40</u> , <u>C25.FR.41</u>
System under test	CSMS
Description	In order to test that CSMS supports QR codes
Purpose	To verify if the CSMS is able to respond correctly when a QR code is scanned on Charging Station
Prerequisite(s)	- CSMS configured for central cost calculation - CSMS supports payments using dynamic QR codes - CSMS supports url templates for QR code payments containing at least placeholders {chargingstationid} and {totp} Test System has: - WebPaymentsCtrlr.Enabled is <i>true</i> - WebPaymentsCtrlr.URLTemplate is <i><base-url>/{chargingstationid}/{evse}/{totp}/{version}</i> - WebPaymentsCtrlr.ValidityTime is <i>30</i> - WebPaymentsCtrlr.SharedSecret is <i><Configured sharedsecret></i> - WebPaymentsCtrlr.Length is <i>8</i> - WebPaymentsCtrlr.TOTPVersion is <i>1</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger CSMS to perform a payment and start a transaction via QR code webpage

maxenergy is 20000

qrUrl is *<Configured QR base url>* extended with:

{chargingstationId} is *<configured charging station id>*

{totp} is *<valid totp>*

{evseId} is *<configured evse>*

{version} is 1

2. The Test System responds with
RequestStartTransactionResponse with
status *Accepted*

1. The CSMS sends a **RequestStartTransactionRequest**

Main (Test scenario)	
3. The Test System sends a TransactionEventRequest with eventType <i>Started</i> triggerReason <i>RemoteStart</i> idToken.idToken <i><idToken.idToken from step 1></i> idToken.type <i>DirectPayment</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.remoteStartId <i><remoteStartId></i> meterValue[0].timestamp <i><not omitted></i> meterValue[0].sampledValue[0].value <i>10000</i> meterValue[0].sampledValue[0].context <i>Transaction.Begin</i>	4. The CSMS responds with a TransactionEventResponse
Tool validations	
* Step 1: Message RequestStartTransactionRequest idToken.idToken must NOT be omitted idToken.type must be <i>DirectPayment</i> * Step 4: Message TransactionEventResponse idTokenInfo.status must be <i>Accepted</i> transactionLimit.maxEnergy must be <i>20000</i>	
Post scenario validations: - N/a	

TC_C_132_CSMS: Ad hoc payment via static or dynamic QR code - invalid URL parameters

Test case name	Ad hoc payment via static or dynamic QR code - invalid URL parameters
Test case Id	TC_C_132_CSMS
Use case Id(s)	C25
Requirement(s)	C25.FR.09
System under test	CSMS
Description	In order to test that CSMS supports QR codes
Purpose	To verify if the CSMS is able to respond correctly when receiving invalid parameters.
Prerequisite(s)	- CSMS configured for central cost calculation - CSMS supports payments using dynamic QR codes Test System has: - WebPaymentsCtrlr.Enabled is <i>true</i> - WebPaymentsCtrlr.URLTemplate is <i><base-url>/{chargingstationid}/{evse}/{totp}/{version}</i> - WebPaymentsCtrlr.ValidityTime is <i>30</i> - WebPaymentsCtrlr.SharedSecret is <i><Configured sharedsecret></i> - WebPaymentsCtrlr.Length is <i>8</i> - WebPaymentsCtrlr.TOTPVersion is <i>1</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger CSMS to perform a payment and start a transaction via QR code webpage**maxenergy** is *20000***qrUrl** is *<Configured QR base url>* extended with:{chargingstationId} is *<omitted>*{totp} is *<valid totp>*{evseId} is *<configured evse>*{version} is *1*

1. The CSMS does NOT send a
RequestStartTransactionRequest

Tool validations

N/a

Tool validations**Post scenario validations:**

- CSMS detects that not all path parameters from **URLTemplate** are present and shows a web page that QR code is not valid.
- CSMS doesn't forward EV Driver to a web page of PSP to enter payment credentials and SHALL NOT authorize EV Driver to charge due missing *chargingstationid* URL parameter.

TC_C_133_CSMS: Ad hoc payment via static or dynamic QR code - invalid totp

Test case name	Ad hoc payment via static or dynamic QR code - invalid totp
Test case Id	TC_C_133_CSMS
Use case Id(s)	<u>C25</u>
Requirement(s)	<u>C25.FR.07</u> , <u>C25.FR.08</u>
System under test	CSMS
Description	In order to test that Charging Station supports QR codes.
Purpose	To verify if the CSMS is able to respond correctly when receiving invalid TOTP.
Prerequisite(s)	- CSMS configured for central cost calculation - CSMS supports payments using dynamic QR codes Test System has: - WebPaymentsCtrlr.Enabled is <i>true</i> - WebPaymentsCtrlr.URLTemplate is <i><base-url>/{chargingstationid}/{evse}/{totp}/{version}</i> - WebPaymentsCtrlr.ValidityTime is <i>30</i> - WebPaymentsCtrlr.SharedSecret is <i><Configured sharedsecret></i> - WebPaymentsCtrlr.Length is <i>8</i> - WebPaymentsCtrlr.TOTPVersion is <i>1</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger CSMS to perform a payment and start a transaction via QR code webpage**maxenergy** is *20000***qrUrl** is *<Configured QR base url>* extended with:{chargingstationId} is *<configured charging station id>*{totp} is *<invalid totp>*{evseId} is *<configured evse>*{version} is *1*

1. The CSMS does NOT send a
RequestStartTransactionRequest

Tool validations

N/a

Tool validations**Post scenario validations:**

- CSMS detects that TOTP is not valid and shows a web page that QR code is not valid.
- CSMS doesn't forward EV Driver to a web page of PSP to enter payment credentials and SHALL NOT authorize EV Driver to charge due missing *chargingstationid* URL parameter.

5. D Local Authorization List Management

TC_D_01_CSMS: Send Local Authorization List - Full

Test case name	Send Local Authorization List - Full
Test case Id	TC_D_01_CSMS
Use case Id(s)	<u>D01</u>
Requirement(s)	<u>D01.FR.01</u> , <u>D01.FR.06</u> , <u>D01.FR.18</u>
System under test	CSMS
Description	The CSMS sends a Local Authorization List which a Charging Station can use for the authorization of idTokens. The list MAY be either a full list to replace the current list in the Charging Station or it MAY be a differential list with updates to be applied to the current list in the Charging Station.
Purpose	To verify if the CSMS is able to send a Full Local Authorization List according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

2 The Test System responds with a **GetLocalListVersionResponse** with **versionNumber 1**

Note(s): This step is optional

4 The Test System responds with a **SendLocalListResponse** with **status Accepted**

Note(s): If the Local Authorization List is too big for one message, step 1 and 2 will be repeated

CSMS

1. The CSMS sends a **GetLocalListVersionRequest**

3. The CSMS sends a **SendLocalListRequest**

Tool validations

* Step 1:

Message **SendLocalListRequest**

- **updateType** *Full*
- **versionNumber** *<Bigger than 0>*
- **localAuthorizationList** *<Not empty>*
- **localAuthorizationList[n].idTokenInfo** *<Not empty>*

Post scenario validations:

- N/a

TC_D_02_CSMS: Send Local Authorization List - Differential Update

Test case name	Send Local Authorization List - Differential Update
Test case Id	TC_D_02_CSMS
Use case Id(s)	<u>D01</u>
Requirement(s)	<u>D01.FR.01</u> , <u>D01.FR.06</u> , <u>D01.FR.18</u>
System under test	CSMS
Description	The CSMS sends a Local Authorization List which a Charging Station can use for the authorization of idTokens. The list MAY be either a full list to replace the current list in the Charging Station or it MAY be a differential list with updates to be applied to the current list in the Charging Station.
Purpose	To verify if the CSMS is able to send a Differential Local Authorization List according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a Local Authorization list to the Charging Station with type Differential and some idTokens in the message

2 The Test System responds with a **GetLocalListVersionResponse** with **versionNumber 1**

1. The CSMS sends a **GetLocalListVersionRequest**

4 The Test System responds with a **SendLocalListResponse** with **status Accepted**

3. The CSMS sends a **SendLocalListRequest**

Note(s): If the Local Authorization List is too big for one message, step 1 and 2 will be repeated

Tool validations

* Step 1:

Message **SendLocalListRequest**

- **updateType** *Differential*

- **versionNumber** *<Bigger than currently configured in Test System>*

- **localAuthorizationList** *<Not empty>*

Tool validations**Post scenario validations:**

- N/a

TC_D_03_CSMS: Send Local Authorization List - Differential Remove

Test case name	Send Local Authorization List - Differential Remove
Test case Id	TC_D_03_CSMS
Use case Id(s)	<u>D01</u>
Requirement(s)	<u>D01.FR.01</u> , <u>D01.FR.06</u> , <u>D01.FR.18</u> , <u>D01.FR.17</u>
System under test	CSMS
Description	The CSMS sends a Local Authorization List which a Charging Station can use for the authorization of idTokens. The list MAY be either a full list to replace the current list in the Charging Station or it MAY be a differential list with updates to be applied to the current list in the Charging Station.
Purpose	To verify if the CSMS is able to send a Differential Local Authorization List with data without idToken according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a Local Authorization list to the Charging Station with type Differential and AuthorizationData elements without idTokenInfo in the message

2 The Test System responds with a **SendLocalListResponse** with **status Accepted**

1. The CSMS sends a **SendLocalListRequest**

Note(s): If the Local Authorization List is too big for one message, step 1 and 2 will be repeated

Tool validations

* Step 1:

Message **SendLocalListRequest**

- **updateType** *Differential*

- **versionNumber** <Bigger than currently configured in Test System>

- **localAuthorizationList** <AuthorizationData elements without idTokenInfo>

Post scenario validations:

- N/a

TC_D_04_CSMS: Send Local Authorization List - Full with empty list

Test case name	Send Local Authorization List - Full with empty list
Test case Id	TC_D_04_CSMS
Use case Id(s)	<u>D01</u>
Requirement(s)	<u>D01.FR.01</u> , <u>D01.FR.06</u> , <u>D01.FR.18</u>
System under test	CSMS
Description	The CSMS sends a Local Authorization List which a Charging Station can use for the authorization of idTokens. The list MAY be either a full list to replace the current list in the Charging Station or it MAY be a differential list with updates to be applied to the current list in the Charging Station.
Purpose	To verify if the CSMS is able to send a Full Local Authorization List without data according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to send a Local Authorization list to the Charging Station with type full and without AuthorizationData elements in the message

2 The Test System responds with a **SendLocalListResponse** with **status Accepted**

1. The CSMS sends a **SendLocalListRequest**

Note(s): If the Local Authorization List is too big for one message, step 1 and 2 will be repeated

Tool validations

* Step 1:

Message **SendLocalListRequest**

- **updateType Full**

- **localAuthorizationList <Empty>**

Post scenario validations:

- N/a

TC_D_08_CSMS: Get Local List Version - Success

Test case name	Get Local List Version - Success
Test case Id	TC_D_08_CSMS
Use case Id(s)	<u>D02</u>
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS can request a Charging Station for the version number of the Local Authorization List by sending a GetLocalListVersionRequest.
Purpose	To verify if the CSMS is able to request the Local Authorization List version according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to get a Local Authorization list version

2 The Test System responds with a **GetLocalListVersionResponse** with **versionNumber** <Configured versionNumber>

1. The CSMS sends a **GetLocalListVersionRequest**

Tool validations

- N/a

Post scenario validations:

- N/a

TC_D_09_CSMS: Get Local List Version - No list available

Test case name	Get Local List Version - No list available
Test case Id	TC_D_09_CSMS
Use case Id(s)	<u>D02</u>
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS can request a Charging Station for the version number of the Local Authorization List by sending a GetLocalListVersionRequest.
Purpose	To verify if the CSMS is able to request the Local Authorization List version according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to get a Local Authorization list version

2 The Test System responds with a **GetLocalListVersionResponse** with **versionNumber 0**

1. The CSMS sends a **GetLocalListVersionRequest**

Tool validations

- N/a

Post scenario validations:

- N/a

6. E Transactions

TC_E_01_CSMS: Start transaction options - PowerPathClosed

Test case name	Start transaction options - PowerPathClosed
Test case Id	TC_E_01_CSMS
Use case Id(s)	<u>E01(S5)</u>
Requirement(s)	<u>E01.FR.05</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the start options.
Purpose	To verify if the CSMS is able to handle a Charging Station that starts a transaction when the power path has been closed.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	2. The CSMS responds with an AuthorizeResponse
3. The Test System notifies the CSMS about the status change of the connector. Message: StatusNotificationRequest - connectorStatus is <i>Occupied</i> Message: NotifyEventRequest - trigger is <i>Delta</i> - actualValue is <i>Occupied</i> - component.name is <i>Connector</i> - variable.name is <i>AvailabilityState</i>	4. The CSMS responds accordingly.

Main (Test scenario)	
5. The Test System sends a TransactionEventRequest With eventType is <i>Started</i> triggerReason is <i>ChargingStateChanged</i> idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type> evse.id is <Configured evseId> evse.connectorId is <Configured connectorId> transactionInfo.chargingState is <i>SuspendedEVSE</i>	6. The CSMS responds with a TransactionEventResponse
7. The Test System sends a TransactionEventRequest With eventType is <i>Updated</i> triggerReason is <i>ChargingStateChanged</i> transactionInfo.chargingState is <i>Charging</i>	8. The CSMS responds with a TransactionEventResponse

Tool validations
* Step 2: Message: AuthorizeResponse - idTokenInfo.status must be <i>Accepted</i> * Step 6: Message: TransactionEventResponse - idTokenInfo.status must be <i>Accepted</i>
Post scenario validations: N/a

TC_E_02_CSMS: Start transaction options - EnergyTransfer

Test case name	Start transaction options - EnergyTransfer
Test case Id	TC_E_02_CSMS
Use case Id(s)	<u>E01</u> (S6)
Requirement(s)	<u>E01.FR.06</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the start options.
Purpose	To verify if the CSMS is able to handle a Charging Station that starts a transaction when the energy transfer starts.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	2. The CSMS responds with an AuthorizeResponse
3. The Test System notifies the CSMS about the status change of the connector. Message: StatusNotificationRequest - connectorStatus is <i>Occupied</i> Message: NotifyEventRequest - trigger is <i>Delta</i> - actualValue is <i>Occupied</i> - component.name is <i>Connector</i> - variable.name is <i>AvailabilityState</i>	4. The CSMS responds accordingly.
5. The Test System sends a TransactionEventRequest With eventType is <i>Started</i> triggerReason is <i>ChargingStateChanged</i> idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type> evse.id is <Configured evseId> evse.connectorId is <Configured connectorId> transactionInfo.chargingState is <i>Charging</i>	6. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 2:

Message: **AuthorizeResponse**

- **idTokenInfo.status** must be *Accepted*

* Step 6:

Message: **TransactionEventResponse**

- **idTokenInfo.status** must be *Accepted*

Post scenario validations:

N/a

TC_E_03_CSMS: Local start transaction - Cable plugin first - Success

Test case name	Local start transaction - Cable plugin first - Success
Test case Id	TC_E_03_CSMS
Use case Id(s)	<u>E02</u>
Requirement(s)	<u>E02.FR.02</u>
System under test	CSMS
Description	OCPP 2.x.x allows an EV driver to either first connect the EV and EVSE OR present a form of identification. Both sequences will result in being able to charge.
Purpose	To verify if the CSMS is able to handle a Charging Station that is able to start a charging session when the EV driver first connects the EV and EVSE, before authorization.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *EVConnectedPreSession***Main (Test scenario)****Charging Station**

CSMS

1. Execute **Reusable State** *Authorized*2. Execute **Reusable State** *EnergyTransferStarted***Tool validations**

N/a

Post scenario validations:

N/a

TC_E_04_CSMS: Local start transaction - Authorization first - Success

Test case name	Local start transaction - Authorization first - Success
Test case Id	TC_E_04_CSMS
Use case Id(s)	<u>E03</u>
Requirement(s)	<u>E03.FR.02</u>
System under test	CSMS
Description	OCPP 2.x.x allows an EV driver to either first connect the EV and EVSE OR present a form of identification. Both sequences will result in being able to charge.
Purpose	To verify if the CSMS is able to handle a Charging Station that is able to start a charging session when the EV driver first presents a form of identification, before connecting the EV and EVSE.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

1. Execute **Reusable State** *Authorized*2. Execute **Reusable State** *EnergyTransferStarted***Tool validations**

N/a

Post scenario validations:

N/a

TC_E_39_CSMS: Stop transaction options - Deauthorized - timeout

Test case name	Stop transaction options - Deauthorized - timeout
Test case Id	TC_E_39_CSMS
Use case Id(s)	<u>E03</u> , <u>E06</u>
Requirement(s)	<u>E03.FR.04</u> , <u>E03.FR.05</u> , <u>E06.FR.04</u>
System under test	CSMS
Description	OCPP 2.x.x allows an EV driver to either first connect the EV and EVSE OR present a form of identification. Both sequences will result in being able to charge.
Purpose	To verify if the CSMS is able to handle a Charging Station that deauthorizes the transaction after the EVConnectionTimeout has expired.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *Authorized***Main (Test scenario)****Charging Station**

1. The Test System sends a **TransactionEventRequest**
 With **triggerReason** is *EVConnectTimeout*
transactionInfo.stoppedReason is *Timeout*
eventType is *Ended*

Note(s):

- This step will be executed after the *<Configured EV connection timeout>* expires.

CSMS

2. The CSMS responds with a **TransactionEventResponse**

Tool validations

N/a

Post scenario validations:

N/a

TC_E_14_CSMS: Stop transaction options - EVDisconnected - Charging Station side

Test case name	Stop transaction options - EVDisconnected - Charging Station side
Test case Id	TC_E_14_CSMS
Use case Id(s)	<u>E06(S2)</u>
Requirement(s)	<u>E06.FR.02</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the stop options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when the EV and EVSE are disconnected at the Charging Station side.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *EVConnectedPostSession***Main (Test scenario)****Charging Station**

CSMS

1. Execute **Reusable State** *EVDisconnected***Tool validations**

N/a

Post scenario validations:

N/a

TC_E_20_CSMS: Stop transaction options - EVDisconnected - EV side (able to charge IEC 61851-1 EV)

Test case name	Stop transaction options - EVDisconnected - EV side (able to charge IEC 61851-1 EV)
Test case Id	TC_E_20_CSMS
Use case Id(s)	<u>E06(S2)</u> , <u>E10</u>
Requirement(s)	<u>E06.FR.02</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the stop options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when the EV and EVSE are disconnected at the EV side.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *EnergyTransferSuspended***Main (Test scenario)****Charging Station**

CSMS

1. Execute **Reusable State** *EVDisconnected***Tool validations**

N/a

Post scenario validations:

N/a

TC_E_15_CSMS: Stop transaction options - StopAuthorized - Local

Test case name	Stop transaction options - StopAuthorized - Local
Test case Id	TC_E_15_CSMS
Use case Id(s)	E06(S3)
Requirement(s)	E06.FR.03
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the stop options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when the EV driver locally stops the transaction.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

1. The Test System sends a **TransactionEventRequest**
 With **triggerReason** is *StopAuthorized*
transactionInfo.stoppedReason is *Local*
eventType is *Ended*

CSMS

2. The CSMS responds with a **TransactionEventResponse**

Tool validations

N/a

Post scenario validations:

N/a

TC_E_21_CSMS: Stop transaction options - StopAuthorized - Remote

Test case name	Stop transaction options - StopAuthorized - Remote
Test case Id	TC_E_21_CSMS
Use case Id(s)	<u>E06(S3)</u> AND <u>F03</u>
Requirement(s)	<u>E06.FR.03</u> , <u>F03.FR.01</u> , <u>F03.FR.09</u> , <u>F03.FR.10</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the stop options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when it receives a RequestStopTransactionRequest.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *EnergyTransferStarted***Main (Test scenario)****Charging Station**

CSMS

Manual Action: Trigger the CSMS to request the Charging Station to stop the ongoing transaction.

2. The Test System responds with a **RequestStopTransactionResponse** with **status** *Accepted*

1. The CSMS sends a **RequestStopTransactionRequest**

3. The Test System sends a **TransactionEventRequest**. with **triggerReason** is *RemoteStop* **transactionInfo.stoppedReason** is *Remote* **eventType** is *Ended*

4. The CSMS responds with a **TransactionEventResponse**.

Tool validations

* Step 1:

Message: **RequestStopTransactionRequest**- **transactionId** must equal *<transactionId provided by the Test System in before state.>***Post scenario validations:**

N/a

TC_E_09_CSMS: Start transaction options - EVConnected

Test case name	Start transaction options - EVConnected
Test case Id	TC_E_09_CSMS
Use case Id(s)	<u>E01</u> (S2)
Requirement(s)	<u>E01.FR.02</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the start options.
Purpose	To verify if the CSMS is able to handle a Charging Station that starts a transaction when the EV and EVSE are connected.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System notifies the CSMS about the status change of the connector. Message: StatusNotificationRequest - connectorStatus is <i>Occupied</i> Message: NotifyEventRequest - trigger is <i>Delta</i> - actualValue is <i>Occupied</i> - component.name is <i>Connector</i> - variable.name is <i>AvailabilityState</i>	2. The CSMS responds accordingly.
3. The Test System sends a TransactionEventRequest With eventType is <i>Started</i> triggerReason is <i>CablePluggedIn</i> evse.id is <i><Configured evseId></i> evse.connectorId is <i><Configured connectorId></i> transactionInfo.chargingState is <i>EVConnected</i>	4. The CSMS responds with a TransactionEventResponse

Tool validations

N/a

Tool validations**Post scenario validations:**

N/a

TC_E_10_CSMS: Start transaction options - Authorized - Local

Test case name	Start transaction options - Authorized - Local
Test case Id	TC_E_10_CSMS
Use case Id(s)	E01(S3)
Requirement(s)	E01.FR.03
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the start options.
Purpose	To verify if the CSMS is able to handle a Charging Station that starts a transaction when the EV and EVSE are connected.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	2. The CSMS responds with an AuthorizeResponse
3. The Test System sends a TransactionEventRequest With eventType is <i>Started</i> triggerReason is <i>Authorized</i> idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	4. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 2:

Message: **AuthorizeResponse**- **idTokenInfo.status** must be *Accepted*

* Step 4:

Message: **TransactionEventResponse**- **idTokenInfo.status** must be *Accepted*

Post scenario validations:

N/a

TC_E_11_CSMS: Start transaction options - DataSigned

Test case name	Start transaction options - DataSigned
Test case Id	TC_E_11_CSMS
Use case Id(s)	<u>E01</u> (S4)
Requirement(s)	<u>E01.FR.04</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the start options.
Purpose	To verify if the CSMS is able to handle a Charging Station that starts a transaction when the signed meter values are received.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	2. The CSMS responds with an AuthorizeResponse
3. The Test System notifies the CSMS about the status change of the connector. Message: StatusNotificationRequest - connectorStatus is <i>Occupied</i> Message: NotifyEventRequest - trigger is <i>Delta</i> - actualValue is <i>Occupied</i> - component.name is <i>Connector</i> - variable.name is <i>AvailabilityState</i>	4. The CSMS responds accordingly.

Main (Test scenario)	
5. The Test System sends a TransactionEventRequest With eventType is <i>Started</i> triggerReason is <i>SignedDataReceived</i> idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type> evse.id is <Configured evseId> evse.connectorId is <Configured connectorId> meterValue is provided with the following values: sampledValue.value is <i>0.0</i> sampledValue.context is <i>Transaction.Begin</i> sampledValue.signedMeterValue is <Generated SignedMeterValueType>	6. The CSMS responds with a TransactionEventResponse
7. The Test System sends a TransactionEventRequest With eventType is <i>Updated</i> triggerReason is <i>ChargingStateChanged</i> transactionInfo.chargingState is <i>Charging</i>	8. The CSMS responds with a TransactionEventResponse

Tool validations
* Step 2: Message: AuthorizeResponse - idTokenInfo.status must be <i>Accepted</i> * Step 6: Message: TransactionEventResponse - idTokenInfo.status must be <i>Accepted</i>
Post scenario validations: N/a

TC_E_12_CSMS: Start transaction options - ParkingBayOccupied

Test case name	Start transaction options - ParkingBayOccupied
Test case Id	TC_E_12_CSMS
Use case Id(s)	<u>E01(S1)</u>
Requirement(s)	<u>E01.FR.01</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the start options.
Purpose	To verify if the CSMS is able to handle a Charging Station that starts a transaction when the EV and EVSE are connected.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

1. The Test System sends a **TransactionEventRequest**
With **eventType** is *Started*
triggerReason is *EVDetected*

CSMS

2. The CSMS responds with a **TransactionEventResponse**

Tool validations

N/a

Post scenario validations:

N/a

TC_E_38_CSMS: Local start transaction - EV not ready

Test case name	Local start transaction - EV not ready
Test case Id	TC_E_38_CSMS
Use case Id(s)	E03
Requirement(s)	N/a
System under test	CSMS
Description	OCPP 2.x.x allows an EV driver to either first connect the EV and EVSE OR present a form of identification. Both sequences will result in being able to charge.
Purpose	To verify if the CSMS is able to handle a Charging Station that reports an EV is not ready to start the energy transfer (yet).
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *Authorized*

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *EVConnectedPreSession*

2. The Test System sends a **TransactionEventRequest** With **triggerReason** is *ChargingStateChanged* **transactionInfo.chargingState** is *SuspendedEV* **eventType** is *Updated*

3. The CSMS responds with a **TransactionEventResponse**

Tool validations

N/a

Post scenario validations:

N/a

TC_E_07_CSMS: Stop transaction options - PowerPathClosed - Local stop

Test case name	Stop transaction options - PowerPathClosed - Local stop
Test case Id	TC_E_07_CSMS
Use case Id(s)	E06(S5)
Requirement(s)	E06.FR.06
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the stop options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when it is locally stopped by an EV driver.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest With triggerReason is <i>StopAuthorized</i> transactionInfo.stoppedReason is <i>Local</i> eventType is <i>Ended</i>	2. The CSMS responds with a TransactionEventResponse

Tool validations

N/a

Post scenario validations:

N/a

TC_E_08_CSMS: Stop transaction options - EnergyTransfer stopped - StopAuthorized

Test case name	Stop transaction options - EnergyTransfer stopped - StopAuthorized
Test case Id	TC_E_08_CSMS
Use case Id(s)	<u>E06(S6)</u>
Requirement(s)	<u>E06.FR.07</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the stop options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when the energy transfer stopped normally.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *StopAuthorized*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest With triggerReason is <i>ChargingStateChanged</i> transactionInfo.chargingState is <i>EVConnected</i> transactionInfo.stoppedReason is <i>Local</i> eventType is <i>Ended</i>	2. The CSMS responds with a TransactionEventResponse

Tool validations

N/a

Post scenario validations:

N/a

TC_E_16_CSMS: Stop transaction options - Deauthorized - Invalid idToken

Test case name	Stop transaction options - Deauthorized - Invalid idToken
Test case Id	TC_E_16_CSMS
Use case Id(s)	<u>E06(S3)</u>
Requirement(s)	<u>E06.FR.04,E01.FR.11,E01.FR.12</u>
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the start options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when the transaction gets deauthorized by the status from the idTokenInfo at a TransactionEventResponse message and it has been configured to do so.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest With triggerReason is <i>Authorized</i> idToken.idToken <Configured invalid_idtoken_idtoken> idToken.type <Configured invalid_idtoken_type> eventType is <i>Started</i>	2. The CSMS responds with a TransactionEventResponse
3. The Test System sends a TransactionEventRequest With eventType <i>Ended</i> triggerReason <i>Deauthorized</i> transactionInfo.stoppedReason <i>DeAuthorized</i>	4. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 2:

Message: **TransactionEventResponse**- **idTokenInfo.status** must be *Invalid* or *Unknown*+**Post scenario validations:**

N/a

TC_E_17_CSMS: Stop transaction options - Deauthorized - EV side disconnect

Test case name	Stop transaction options - Deauthorized - EV side disconnect
Test case Id	TC_E_17_CSMS
Use case Id(s)	E06(S3)
Requirement(s)	E06.FR.04
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the start options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when the transaction gets deauthorized by a connection loss from the EV side and it has been configured to do so.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferSuspended*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest triggerReason must be <i>EVCommunicationLost</i> transactionInfo.chargingState must be <i>Idle</i> transactionInfo.stoppedReason must be <i>EVDisconnected</i> eventType must be <i>Ended</i>	2. The CSMS responds with a TransactionEventResponse

Tool validations

N/a

Post scenario validations:

N/a

TC_E_22_CSMS: Stop transaction options - EnergyTransfer stopped - SuspendedEV

Test case name	Stop transaction options - EnergyTransfer stopped - SuspendedEV
Test case Id	TC_E_22_CSMS
Use case Id(s)	E06(S6)
Requirement(s)	E06.FR.07
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the stop options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when the energy transfer stopped by the EV.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest With triggerReason is <i>ChargingStateChanged</i> transactionInfo.chargingState is <i>SuspendedEV</i> transactionInfo.stoppedReason is <i>StoppedByEV</i> eventType is <i>Ended</i>	2. The CSMS responds with a TransactionEventResponse

Tool validations

N/a

Post scenario validations:

N/a

TC_E_19_CSMS: Stop transaction options - ParkingBayUnoccupied

Test case name	Stop transaction options - ParkingBayUnoccupied
Test case Id	TC_E_19_CSMS
Use case Id(s)	E06(S1)
Requirement(s)	E06.FR.01
System under test	CSMS
Description	OCPP 2.x.x has a flexible transaction mechanism that allows the transaction start and stop points to be configured differently. This test covers one of the stop options.
Purpose	To verify if the CSMS is able to handle a Charging Station that stops a transaction when the EV left the parking bay.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EVDisconnected*

Main (Test scenario)

Charging Station

1. The Test System sends a **TransactionEventRequest**
 With **triggerReason** is *EVDeparted*
transactionInfo.stoppedReason is *Local*
eventType is *Ended*

CSMS

2. The CSMS responds with a **TransactionEventResponse**

Tool validations

N/a

Post scenario validations:

N/a

TC_E_26_CSMS: Disconnect cable on EV-side - Suspend transaction

Test case name	Disconnect cable on EV-side - Suspend transaction
Test case Id	TC_E_26_CSMS
Use case Id(s)	<u>E10</u>
Requirement(s)	<u>E10.FR.01</u>
System under test	CSMS
Description	The Charging Station can behave in several different ways when the cable is disconnected at the EV side, based on its configuration. This test case tests one of the possible configuration settings.
Purpose	To verify if the CSMS can handle a Charging Station that suspends the transaction when the EV and EVSE are disconnected at the EV side AND is able restart the energy transfer after reconnecting the EV and EVSE.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferSuspended*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest With triggerReason is <i>EVCommunicationLost</i> transactionInfo.chargingState is <i>Idle</i> eventType is <i>Updated</i>	2. The CSMS responds with a TransactionEventResponse
3. The Test System notifies the CSMS about the current state of the connector. Message: StatusNotificationRequest - connectorStatus <i>Available</i> - evseId <i><Configured evseId></i> - connectorId <i><Configured connectorId></i> Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Available"</i> - component.name <i>"Connector"</i> - component.evse.id <i><Configured evseId></i> - component.evse.connectorid <i><Configured connectorId></i> - variable.name <i>"AvailabilityState"</i>	4. The CSMS responds accordingly.
5. The Test System sends a TransactionEventRequest With triggerReason is <i>CablePluggedIn</i> transactionInfo.chargingState is <i>EVConnected</i> eventType is <i>Updated</i>	6. The CSMS responds with a TransactionEventResponse

Main (Test scenario)	
7. The Test System sends a TransactionEventRequest With triggerReason is <i>ChargingStateChanged</i> transactionInfo.chargingState is <i>Charging</i> eventType is <i>Updated</i>	8. The CSMS responds with a TransactionEventResponse
Tool validations	
N/a	
Post scenario validations:	
N/a	

TC_E_29_CSMS: Check Transaction status - Transaction with id ongoing - with message in queue

Test case name	Check Transaction status - Transaction with id ongoing - with message in queue
Test case Id	TC_E_29_CSMS
Use case Id(s)	<u>E14</u>
Requirement(s)	<u>E14.FR.02,E14.FR.04</u>
System under test	CSMS
Description	The CSMS is able to request the status of a transaction and to find out whether there are queued transaction-related messages, using the GetTransactionStatusRequest message.
Purpose	To verify if the CSMS is able to request the status of queued TransactionEventRequest messages from a specific transaction by sending a GetTransactionStatusRequest with a transactionId. The Test System will respond that there are message(s) queued belonging to the ongoing transaction with the requested id.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
1. The Test System closes the WebSocket connection.	
2. The Test System waits a number of seconds equal to <i><Configured Transaction Duration></i> , then it will reconnect to the CSMS.	
4. The Test System responds with a GetTransactionStatusResponse With ongoingIndicator is <i>true</i> messagesInQueue is <i>true</i>	3. The CSMS sends a GetTransactionStatusRequest
5. The Test System sends a TransactionEventRequest With eventType is <i>Updated</i> meterValues is present. offline is <i>true</i>	6. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 3:

Message: **GetTransactionStatusRequest**- **transactionId** *<Generated transactionId from Before>*

Tool validations**Post scenario validations:**

N/a

TC_E_30_CSMS: Check Transaction status - Transaction with id ongoing - without message in queue

Test case name	Check Transaction status - Transaction with id ongoing - without message in queue
Test case Id	TC_E_30_CSMS
Use case Id(s)	<u>E14</u>
Requirement(s)	<u>E14.FR.02,E14.FR.05</u>
System under test	CSMS
Description	The CSMS is able to request the status of a transaction and to find out whether there are queued transaction-related messages, using the GetTransactionStatusRequest message.
Purpose	To verify if the CSMS is able to request the status of queued TransactionEventRequest messages from a specific transaction by sending a GetTransactionStatusRequest with a transactionId. The Test System will respond that there is NO message queued belonging to the ongoing transaction with the requested id.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

2. The Test System responds with a **GetTransactionStatusResponse** With **ongoingIndicator** is *true* **messagesInQueue** is *false*

CSMS

1. The CSMS sends a **GetTransactionStatusRequest**

Tool validations

* Step 1:

Message: **GetTransactionStatusRequest**- **transactionId** must be *<Generated transactionId from Before>*

Post scenario validations:

N/a

TC_E_31_CSMS: Check Transaction status - Transaction with id ended - with message in queue

Test case name	Check Transaction status - Transaction with id ended - with message in queue
Test case Id	TC_E_31_CSMS
Use case Id(s)	<u>E14</u>
Requirement(s)	<u>E14.FR.03,E14.FR.04</u>
System under test	CSMS
Description	The CSMS is able to request the status of a transaction and to find out whether there are queued transaction-related messages, using the GetTransactionStatusRequest message.
Purpose	To verify if the CSMS is able to request the status of queued TransactionEventRequest messages from a specific transaction by sending a GetTransactionStatusRequest with a transactionId. The Test System will respond that there are message(s) queued belonging to an ended transaction with the requested id.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
1. The Test System closes the WebSocket connection.	
2. The Test System waits a number of seconds equal to _<Configured Transaction duration>, then it will reconnect to the CSMS._	
3. The Test System sends a StatusNotificationRequest With evseId is <Configured evseId> connectorId is <Configured connectorId> connectorStatus is Available	4. The CSMS responds with a StatusNotificationResponse
5. The Test System sends a TransactionEventRequest With eventType is Ended offline is true triggerReason is EVCommunicationLost transactionInfo.chargingState is Idle seqNo <Skips two sequence number values>	6. The CSMS responds with a TransactionEventResponse

Main (Test scenario)	
8. The Test System responds with a GetTransactionStatusResponse With ongoingIndicator is <i>false</i> messagesInQueue is <i>true</i>	7. The CSMS sends a GetTransactionStatusRequest
9. The Test System sends a TransactionEventRequest With triggerReason is <i>StopAuthorized</i> eventType is <i>Updated</i> offline is <i>true</i> seqNo <This is the first of the two skipped values>	10. The CSMS responds with a TransactionEventResponse
11. The Test System sends a TransactionEventRequest With triggerReason is <i>ChargingStateChanged</i> transactionInfo.chargingState is <i>EVConnected</i> eventType is <i>Updated</i> offline is <i>true</i> seqNo <This is the second of the two skipped values>	12. The CSMS responds with a TransactionEventResponse

Tool validations
* Step 5: Message: GetTransactionStatusRequest - transactionId <Generated transactionId from Before>
Post scenario validations: N/a

TC_E_33_CSMS: Check Transaction status - Without transactionId - with message in queue

Test case name	Check Transaction status - Without transactionId - with message in queue
Test case Id	TC_E_33_CSMS
Use case Id(s)	<u>E14</u>
Requirement(s)	<u>E14.FR.06,E14.FR.07</u>
System under test	CSMS
Description	The CSMS is able to request the status of a transaction and to find out whether there are queued transaction-related messages, using the GetTransactionStatusRequest message.
Purpose	To verify if the CSMS is able to request the status of queued TransactionEventRequest messages by sending a GetTransactionStatusRequest without a transactionId. The Test System will respond that there are message(s) queued.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
1. The Test System closes the WebSocket connection.	
2. The Test System waits a number of seconds equal to <i><Configured Transaction Duration></i> , then it will reconnect to the CSMS.	
4. The Test System responds with a GetTransactionStatusResponse With ongoingIndicator is omitted. messagesInQueue is <i>true</i>	3. The CSMS sends a GetTransactionStatusRequest
5. The Test System sends a TransactionEventRequest With eventType is <i>Updated</i> meterValues is present. offline is <i>true</i>	6. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 3:

Message: **GetTransactionStatusRequest**- **transactionId** must be omitted.

Tool validations**Post scenario validations:**

N/a

TC_E_34_CSMS: Check Transaction status - Without transactionId - without message in queue

Test case name	Check Transaction status - Without transactionId - without message in queue
Test case Id	TC_E_34_CSMS
Use case Id(s)	<u>E14</u>
Requirement(s)	<u>E14.FR.06,E14.FR.08</u>
System under test	CSMS
Description	The CSMS is able to request the status of a transaction and to find out whether there are queued transaction-related messages, using the GetTransactionStatusRequest message.
Purpose	To verify if the CSMS is able to request the status of queued TransactionEventRequest messages by sending a GetTransactionStatusRequest without a transactionId. The Test System will respond that there are NO message(s) queued.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetTransactionStatusResponse With ongoingIndicator is omitted. messagesInQueue is <i>false</i>	1. The CSMS sends a GetTransactionStatusRequest

Tool validations

* Step 1:

Message: **GetTransactionStatusRequest**- **transactionId** must be omitted.

Post scenario validations:

N/a

TC_E_53_CSMS: Reset Sequence Number - CSMS accepting seqNo = 0 at start of transaction

Test case name	Reset Sequence Number - CSMS accepting seqNo = 0 at start of transaction
Test case Id	TC_E_53_CSMS
Use case Id(s)	E01
Requirement(s)	E01.FR.07
System under test	CSMS
Description	OCPP 2.0.1 Edition 2 recommends that seqNo starts at 0 for every transaction. CSMS must therefore be robust to a seqNo that is not continuously increasing, but that restarts for new transactions. Since a TransactionEventRequest cannot be rejected, this can only be detected by either the complete absence of a TransactionEventResponse from CSMS or an otherwise misbehaving CSMS.
Purpose	To verify if the CSMS accepts that a new transactions starts with a seqNo = 0.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute Reusable State EnergyTransferStarted

Note(s): New transaction will use seqNo 0 for the first TransactionEventRequest.

2. Execute Reusable State EVDisconnected

3. Execute Reusable State EnergyTransferStarted

Note(s): New transaction will use seqNo 0 for the first TransactionEventRequest.

4. Execute Reusable State EVDisconnected

Tool validations

* Step 1:

CSMS accepts the message TransactionEventRequest with eventType = Started and seqNo = 0 and answers with a TransactionEventResponse message.

* Step 3:

CSMS accepts the message TransactionEventRequest with eventType = Started and seqNo = 0 and answers with a TransactionEventResponse message.

Post scenario validations:

N/a

TC_E_102_CSMS: Transactions with fixed cost, energy or time - CSMS and CS both specify limits

Test case name	Transactions with fixed cost, energy or time - CSMS and CS both specify limits
Test case Id	TC_E_102_CSMS
Use case Id(s)	<u>E16</u>
Requirement(s)	<u>E16.FR.01</u> , <u>E16.FR.02</u> , <u>E16.FR.03</u>
System under test	CSMS
Description	EV Driver or CSMS specifies a limit in cost, energy, state of charge or time for transaction.
Purpose	To verify whether the CSMS correctly handles transactions where both the CSMS and Charging Station specify limits.
Prerequisite(s)	- CSMS allows specifying an energy limit. - Test System has TxCtrlr.SupportedLimits contains <i>maxEnergy</i> .

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with eventType <i>Updated</i> triggerReason <i>LimitSet</i> transactionInfo.transactionLimit.maxEnergy 6000	2. The CSMS responds with a TransactionEventResponse
<u>Manual action:</u> Configure the CSMS to override the max energy limit to 10000 (Wh).	
3. The Test System sends a TransactionEventRequest with eventType <i>Updated</i>	4. The CSMS responds with a TransactionEventResponse
5. The Test System sends a TransactionEventRequest with eventType <i>Updated</i> triggerReason <i>LimitSet</i> transactionInfo.transactionLimit.maxEnergy is 10000	6. The CSMS responds with a TransactionEventResponse
7. The Test System sends a TransactionEventRequest with eventType <i>Ended</i> triggerReason <i>EnergyLimitReached</i>	8. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 4:

Message: **TransactionEventResponse**

- **transactionLimit.maxEnergy** must be *10000*

* Step 6:

Message: **TransactionEventResponse**

- **transactionLimit** is *<omitted>*

Post scenario validations:

N/a

TC_E_106_CSMS: Transactions with fixed cost, energy or time - CS specifies energy limit

Test case name	Transactions with fixed cost, energy or time - CS specifies energy limit
Test case Id	TC_E_106_CSMS
Use case Id(s)	E16
Requirement(s)	...
System under test	CSMS
Description	EV Driver or CSMS specifies a limit in cost, energy, state of charge or time for transaction.
Purpose	To verify whether the CSMS correctly handles transactions where the Charging Station specifies an energy limit.
Prerequisite(s)	Test System has TxCtrlr.SupportedLimits contains <i>maxEnergy</i> .

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a TransactionEventRequest with eventType <i>Updated</i> triggerReason <i>LimitSet</i> transactionInfo.transactionLimit.maxEnergy <i>6000</i>	2. The CSMS responds with a TransactionEventResponse
3. The Test System sends a TransactionEventRequest with eventType <i>Ended</i> triggerReason <i>EnergyLimitReached</i>	4. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 2:

Message **TransactionEventResponse** with: * **transactionLimit** is *<omitted>*

Post scenario validations:

N/a

TC_E_107_CSMS: Transactions with fixed cost, energy or time - CS specifies time limit

Test case name	Transactions with fixed cost, energy or time - CS specifies time limit
Test case Id	TC_E_107_CSMS
Use case Id(s)	E16
Requirement(s)	...
System under test	CSMS
Description	EV Driver or CSMS specifies a limit in cost, energy, state of charge or time for transaction.
Purpose	To verify whether the CSMS correctly handles transactions where the Charging Station specifies a time limit.
Prerequisite(s)	Test System has TxCtrl.SupportedLimits contains <i>maxTime</i> .

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

1. The Test System sends a **TransactionEventRequest** with
eventType *Updated*
triggerReason *LimitSet*
transactionInfo.transactionLimit.maxTime *120*

2. The CSMS responds with a **TransactionEventResponse**

Note: The test system waits for the *maxTime* *transactionLimit* to be reached.

3. The Test System sends a **TransactionEventRequest** with
eventType *Ended*
triggerReason *TimeLimitReached*

4. The CSMS responds with a **TransactionEventResponse**

Tool validations

* Step 2:

Message **TransactionEventResponse** with:
transactionLimit is *<omitted>*

Post scenario validations:

N/a

TC_E_108_CSMS: Transactions with fixed cost, energy or time - CS calculates costs and specifies limit

Test case name	Transactions with fixed cost, energy or time - CS calculates costs and specifies limit
Test case Id	TC_E_108_CSMS
Use case Id(s)	E16
Requirement(s)	...
System under test	CSMS
Description	EV Driver or CSMS specifies a limit in cost, energy, state of charge or time for transaction.
Purpose	To verify whether the CSMS correctly handles transactions where the Charging Station uses local cost calculation and specifies a cost limit.
Prerequisite(s)	- Test System has TariffCostCtrlr.Enabled[Cost] is <i>true</i> - Test System has TariffCostCtrlr.Enabled[RunningCost] is <i>true</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger CSMS to configure a tariff with 1 EUR / minute for charging time2. The Test System responds with a **SetDefaultTariffResponse**

1. The CSMS sends a **SetDefaultTariffRequest** with
evseId 0
tariff.tariffId <not omitted>
tariff.currency EUR
tariff.chargingTime.prices[0].priceMinute 1.00

3. Execute **Reusable State** *EnergyTransferStarted*

4. The Test System sends a **TransactionEventRequest** with
eventType *Updated*
triggerReason *LimitSet*
transactionInfo.transactionLimit.maxCost 2.00
costDetails.totalCost.currency EUR
costDetails.totalCost.typeOfCost *NormalCost*
costDetails.totalCost.chargingTime.inclTax 1.00
costDetails.totalCost.total.inclTax 1.00
costDetails.totalUsage.chargingTime 60
costDetails.totalUsage.idleTime <not omitted>

5. The CSMS responds with a **TransactionEventResponse**

Main (Test scenario)	
6. The Test System sends a TransactionEventRequest with eventType <i>Updated</i> triggerReason <i>RunningCost</i> costDetails.totalCost.currency <i>EUR</i> costDetails.totalCost.typeOfCost <i>NormalCost</i> costDetails.totalCost.chargingTime.inclTax <i>1.50</i> costDetails.totalCost.total.inclTax <i>1.50</i> costDetails.totalUsage.chargingTime <i>90</i> costDetails.totalUsage.idleTime <i><not omitted></i>	7. The CSMS responds with a TransactionEventResponse
8. The Test System sends a TransactionEventRequest with eventType <i>Ended</i> triggerReason <i>CostLimitReached</i> costDetails.totalCost.currency <i>EUR</i> costDetails.totalCost.typeOfCost <i>NormalCost</i> costDetails.totalCost.chargingTime.inclTax <i>2.00</i> costDetails.totalCost.total.inclTax <i>2.00</i> costDetails.totalUsage.chargingTime <i>120</i> costDetails.totalUsage.idleTime <i><not omitted></i>	9. The CSMS responds with a TransactionEventResponse
Tool validations	
* Step 1: Message: SetDefaultTariffRequest - evseId must be <i><evse id></i> - tariff.tariffId must be <i><not omitted></i> - tariff.currency must be <i>EUR</i> - tariff.chargingTime.prices[0].priceMinute must be <i>1.00</i> * Step 5: Message: TransactionEventResponse - totalCost must be <i><omitted></i> - transactionLimit must be <i><omitted></i> * Step 7: Message: TransactionEventResponse - totalCost must be <i><omitted></i> - transactionLimit must be <i><omitted></i> * Step 9: Message: TransactionEventResponse - totalCost must be <i><omitted></i> - transactionLimit must be <i><omitted></i>	
Post scenario validations: N/a	

TC_E_109_CSMS: Transactions with fixed cost, energy or time - CSMS calculates costs and specifies cost limit

Test case name	Transactions with fixed cost, energy or time - CSMS calculates costs and specifies cost limit
Test case Id	TC_E_109_CSMS
Use case Id(s)	<u>E16</u>
Requirement(s)	<u>E16.FR.02</u> , <u>E16.FR.11</u>
System under test	CSMS
Description	CSMS will set a limit the transaction for the specified cost. CS will use central cost calculation.
Purpose	To verify whether the CSMS correctly sends cost when central cost calculation is used.
Prerequisite(s)	- CSMS supports setting cost limits - Test System has TxCtrlr.SupportedLimits containing <i>maxCost</i> - Test System has TariffCostCtrlr.Enabled[Cost] is <i>false</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	2. The CSMS responds with a AuthorizeResponse
3. The Test System sends a TransactionEventRequest with With eventType is <i>Started</i> triggerReason is <i>ChargingStateChanged</i> idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type> evse.id is <Configured evseId> evse.connectorId is <Configured connectorId> transactionInfo.transactionId is <transactionId> transactionInfo.chargingState is <i>Charging</i>	4. The CSMS responds with a TransactionEventResponse
5 The Test System sends a TransactionEventRequest with triggerReason is <i>LimitSet</i> transactionInfo.transactionLimit.maxCost is <i>10</i>	6 The CSMS responds with TransactionEventResponse
8. The Test System responds with a CostUpdatedResponse	7. The CSMS sends a CostUpdatedRequest
<u>Note:</u> Sending a CostUpdatedRequest is not required, but if CSMS does send it, the validation applies.	

Tool validations

* Step 2:

Message: **AuthorizeResponse**

idTokenInfo.status must be *Accepted*

* Step 4:

Message: **TransactionEventResponse**

totalCost must be *<not omitted>*

transactionInfo.transactionLimit.maxEnergy must be *<omitted>*

transactionInfo.transactionLimit.maxTime must be *<omitted>*

transactionInfo.transactionLimit.maxCost must be *10*

* Step 6:

Message: **TransactionEventResponse**

totalCost must be *<not omitted>*

transactionInfo.transactionLimit must be *<omitted>*

* Step 8:

Message: **CostUpdatedRequest**

transactionId must be *<transactionId>*

totalCost must be *<not omitted>*

Post scenario validations:

N/a

TC_E_110_CSMS: Transactions with fixed cost, energy or time - CSMS specifies energy limit

Test case name	Transactions with fixed cost, energy or time - CSMS specifies energy limit
Test case Id	TC_E_110_CSMS
Use case Id(s)	<u>E16</u>
Requirement(s)	<u>E16.FR.02</u>
System under test	CSMS
Description	CSMS will set an energy limit on the transaction.
Purpose	To verify whether the CSMS is able to set an energy limit.
Prerequisite(s)	- CSMS supports setting energy limits - Test System has TxCtrlr.SupportedLimits containing <i>maxEnergy</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	2. The CSMS responds with an AuthorizeResponse
<u>Manual action:</u> In CSMS, configure energy limit of 10000 (Wh) for this transaction	
3. The Test System sends a TransactionEventRequest with eventType is <i>Started</i> triggerReason is <i>ChargingStateChanged</i> idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type> evse.id is <Configured evseId> evse.connectorId is <Configured connectorId> transactionInfo.transactionId is <transactionId> transactionInfo.chargingState is <i>Charging</i> transactionInfo.transactionLimit <omitted>	4. The CSMS responds with a TransactionEventResponse
5. The Test System sends a TransactionEventRequest with triggerReason is <i>LimitSet</i> transactionInfo.transactionLimit.maxEnergy is 10000	6 The CSMS responds with TransactionEventResponse

Main (Test scenario)

7. The Test System sends a **TransactionEventRequest** with
eventType is *Updated*
triggerReason is *EnergyLimitReached*
evse.id is *<Configured evseId>*
evse.connectorId is *<Configured connectorId>*
transactionInfo.transactionId is *<transactionId>*
transactionInfo.transactionLimit.maxEnergy *10000*
transactionInfo.transactionLimit.maxTime *<omitted>*
transactionInfo.transactionLimit.maxCost *<omitted>*
transactionInfo.chargingState *SuspendedEVSE*

8. The CSMS responds with a **TransactionEventResponse**

Tool validations

* Step 4:

Message: **TransactionEventResponse**

idTokenInfo.status must be *Accepted*

transactionLimit.maxEnergy must be *10000*

transactionLimit.maxTime must be *<omitted>*

transactionLimit.maxCost must be *<omitted>*

* Step 6:

Message: **TransactionEventResponse**

transactionLimit must be *<omitted>*

Post scenario validations:

N/a

TC_E_111_CSMS: Transactions with fixed cost, energy or time - CSMS specifies time limit

Test case name	Transactions with fixed cost, energy or time - CSMS specifies time limit
Test case Id	TC_E_111_CSMS
Use case Id(s)	<u>E16</u>
Requirement(s)	<u>E16.FR.02</u>
System under test	CSMS
Description	CSMS will set a time limit on the transaction.
Purpose	To verify whether the CSMS is able to set a time limit.
Prerequisite(s)	- CSMS supports setting energy limits - Test System has TxCtrlr.SupportedLimits containing <i>maxTime</i>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest With idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	2. The CSMS responds with an AuthorizeResponse
<u>Manual action:</u> In CSMS, configure time limit of duration 120 for this transaction	
3. The Test System sends a TransactionEventRequest with With eventType is <i>Started</i> triggerReason is <i>ChargingStateChanged</i> idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type> evse.id is <Configured evseId> evse.connectorId is <Configured connectorId> transactionInfo.transactionId is <transactionId> transactionInfo.chargingState is <i>Charging</i> transactionInfo.transactionLimit <omitted>	4. The CSMS responds with a TransactionEventResponse

Main (Test scenario)

5. The Test System sends a **TransactionEventRequest** with
eventType is *Updated*
triggerReason is *LimitSet*
evse.id is *<Configured evseId>*
evse.connectorId is *<Configured connectorId>*
transactionInfo.transactionId is *<transactionId>*
transactionInfo.transactionLimit.maxEnergy *<omitted>*
transactionInfo.transactionLimit.maxTime *120*
transactionInfo.transactionLimit.maxCost *<omitted>*

6. The CSMS responds with a **TransactionEventResponse**

Tool validations

* Step 4:

Message: **TransactionEventResponse**
idTokenInfo.status must be *Accepted*
transactionLimit.maxEnergy must be *<omitted>*
transactionLimit.maxTime must be *120*
transactionLimit.maxCost must be *<omitted>*

* Step 6:

Message: **TransactionEventResponse**
transactionLimit must be *<omitted>*

Post scenario validations:

N/a

TC_E_113_CSMS: Resuming transaction after interruption - TxResumptionTimeout not expired - TxAllowEnergyTransferResumption is true

Test case name	Resuming transaction after interruption - TxResumptionTimeout not expired - TxAllowEnergyTransferResumption is true
Test case Id	TC_E_113_CSMS
Use case Id(s)	<u>E17</u>
Requirement(s)	<u>E17.FR.15</u>
System under test	CSMS
Description	CSMS restores TxProfile after Charging Station resumes transactions which had a Charging profile with purpose TxProfile configured.
Purpose	To verify the CSMS restores the charging profile for a transaction after resuming transactions by the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

The Test System disconnects the connection.

The Test System waits the <Configured transaction duration>

The Test System restores the connection.

1. The Test System sends a **BootNotificationRequest**

2. The CSMS responds with a **BootNotificationResponse**

3. The Test System notifies the CSMS about the current state of all connectors.

Message: **NotifyEventRequest**

- **eventData[0].trigger** *Delta*

- **eventData[0].component.name** *"Connector"*

- **eventData[0].variable.name** *"AvailabilityState"*

Configured EVSE.Connector:

- **eventData[0].actualValue** *"Occupied"*

Other connectors:

- **eventData[0].actualValue** *"Available"*

4. The CSMS responds accordingly.

5 The Test System sends a **SecurityEventNotificationRequest** - **type** must be *StartupOfTheDevice* OR *ResetOrReboot*

6 The CSMS responds with a **SecurityEventNotificationResponse**

Main (Test scenario)	
7 The Test System sends a NotifyEventRequest eventData.trigger = <i>Delta</i> eventData.transactionId = <i><transactionId from this transaction></i> eventData.eventNotificationType = <i>HardWiredNotification</i> eventData.severity = 3 eventData.actualValue = <i>true</i> eventData.component.name = <i>ElectricalFeed</i> eventData.variable.name = <i>Problem</i>	8 The CSMS responds with a NotifyEventResponse
9 The Test System sends a NotifyEventRequest Same content as step 7, but with actualValue = false: eventData.actualValue = <i>false</i>	10 The CSMS responds with a NotifyEventResponse
11. The Test System sends a TransactionEventRequest eventType <i>Updated</i> triggerReason <i>TxResumed</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.chargingState <i>Charging</i>	12. The CSMS responds with a TransactionEventResponse

Tool validations
* Step 2: Message: BootNotificationResponse - status <i>Accepted</i>
Post scenario validations: N/a

TC_E_117_CSMS: Set Charging Profile - Resuming transaction after interruption - TxResumptionTimeout not expired - TxAllowEnergyTransferResumption is true

Test case name	Set Charging Profile - Resuming transaction after interruption - TxResumptionTimeout not expired - TxAllowEnergyTransferResumption is true
Test case Id	TC_E_117_CSMS
Use case Id(s)	<u>E17</u>
Requirement(s)	<u>E17.FR.15</u>
System under test	CSMS
Description	CSMS restores TxProfile after Charging Station resumes transactions which had a Charging profile with purpose TxProfile configured.
Purpose	To verify the CSMS restores the charging profile for a transaction after resuming transactions by the Charging Station.
Prerequisite(s)	SmartChargingCtrlr.ChargingProfilePersistence is <absent> TxCtrlr.TxResumptionTimeout is 999 TxCtrlr.TxAllowEnergyTransferResumption is true

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is EnergyTransferStarted

Main (Test scenario)

Charging Station

CSMS

Manual action: trigger CSMS to set a Charging Profile of type TxProfile on <transactionId>

2. The Test System responds with a **SetChargingProfileResponse** with status *Accepted*

1. The CSMS sends a **SetChargingProfileRequest**

The Test System disconnects the connection.

The Test System waits the <Configured transaction duration>

The Test System restores the connection.

3. The Test System sends a **BootNotificationRequest**

4. The CSMS responds with a **BootNotificationResponse**

Main (Test scenario)	
5. The Test System notifies the CSMS about the current state of all connectors. Message: NotifyEventRequest - eventData[0].trigger <i>Delta</i> - eventData[0].component.name <i>"Connector"</i> - eventData[0].variable.name <i>"AvailabilityState"</i> Configured EVSE.Connector: - eventData[0].actualValue <i>"Occupied"</i> Other connectors: - eventData[0].actualValue <i>"Available"</i>	6. The CSMS responds accordingly.
7. The Test System sends a SecurityEventNotificationRequest - type must be <i>ResetOrReboot</i>	8. The CSMS responds with a SecurityEventNotificationResponse
9. The Test System sends a NotifyEventRequest eventData.trigger = <i>Delta</i> eventData.transactionId = <i><transactionId from this transaction></i> eventData.eventNotificationType = <i>HardWiredNotification</i> eventData.severity = <i>3</i> eventData.actualValue = <i>true</i> eventData.component.name = <i>ElectricalFeed</i> eventData.variable.name = <i>Problem</i>	10 The CSMS responds with a NotifyEventResponse
11 The Test System sends a NotifyEventRequest Same content as step 9, but with actualValue = false: eventData.actualValue = <i>false</i>	12 The CSMS responds with a NotifyEventResponse
13. The Test System sends a TransactionEventRequest eventType <i>Updated</i> triggerReason <i>TxResumed</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.chargingState <i>Charging</i>	14. The CSMS responds with a TransactionEventResponse
16. The Test System responds with a SetChargingProfileResponse with status <i>Accepted</i>	15. The CSMS sends a SetChargingProfileRequest

Tool validations
* Step 1: Message SetChargingProfileRequest - chargingProfile.chargingProfilePurpose must be <i>TxProfile</i> * Step 4: Message: BootNotificationResponse - status must be <i>Accepted</i> * Step 15: Message: SetChargingProfileRequest - chargingProfile.chargingProfilePurpose must be <i>TxProfile</i>
Post scenario validations: N/a

7. F Remote Control

TC_F_01_CSMS: Remote start transaction - Cable plugin first

Test case name	Remote start transaction - Cable plugin first
Test case Id	TC_F_01_CSMS
Use case Id(s)	<u>F01</u>
Requirement(s)	N/a
System under test	CSMS
Description	OCPP 2.x.x allows an EV driver to either first connect the EV and EVSE OR wait for/trigger a RequestStartTransactionRequest. Both sequences will result in being able to charge.
Purpose	To verify if the CSMS is able to handle a Charging Station that is starts a charging session when the EV driver first connects the EV and EVSE, before receiving a RequestStartTransactionRequest message.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to request the Charging Station to start a transaction.

2. The Test System responds with a **RequestStartTransactionResponse** with **status** *Accepted*
transactionId is *<Generated transactionId>*

1. The CSMS sends a **RequestStartTransactionRequest**

3. The Test System sends a **TransactionEventRequest**. with **triggerReason** is *RemoteStart*
transactionInfo.remoteStartId is *<By CSMS provided remoteStartID>*
eventType is *Updated*

4. The CSMS responds with a **TransactionEventResponse**.

5. Execute **Reusable State** *EnergyTransferStarted* (**State** is *Authorized* and *_EVConnected* = true)

Tool validations

* Step 1:

Message: **RequestStartTransactionRequest**

- **idToken.idToken** <Configured valid_idtoken_idtoken>

- **idToken.type** <Configured valid_idtoken_type>

Post scenario validations:

N/a

TC_F_02_CSMS: Remote start transaction - Remote start first - AuthorizeRemoteStart is true

Test case name	Remote start transaction - Remote start first - AuthorizeRemoteStart is true
Test case Id	TC_F_02_CSMS
Use case Id(s)	<u>F02</u>
Requirement(s)	<u>F02.FR.01</u> , <u>F01.FR.01</u>
System under test	CSMS
Description	OCPP 2.x.x allows an EV driver to either first wait for/trigger a RequestStartTransactionRequest OR connect the EV and EVSE. Both sequences will result in being able to charge.
Purpose	To verify if the CSMS is able to handle a Charging Station that starts a charging session when the Charging Stations receives a RequestStartTransactionRequest message (while AuthorizeRemoteStart is true), before the EV driver connects the EV and EVSE (within the connectionTimeout). The Charging Station has to authorize beforehand like a local action to start a transaction.
Prerequisite(s)	AuthEnabled is NOT implemented with mutability ReadOnly and the value set to false

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to request the Charging Station to start a transaction.

2. The Test System responds with a **RequestStartTransactionResponse** with **status** *Accepted* **transactionId** is omitted.

1. The CSMS sends a **RequestStartTransactionRequest**

3. The Test System sends a **AuthorizeRequest** with **idToken.idToken** <Configured valid_idtoken_idtoken> **idToken.type** <Configured valid_idtoken_type>

4. The CSMS responds with a **AuthorizeResponse**.

5. The Test System sends a **TransactionEventRequest** with **triggerReason** is *RemoteStart* **transactionInfo.remoteStartId** is <By Test System generated remoteStartID> **eventType** is *Started*

6. The CSMS responds with a **TransactionEventResponse**.

7. Execute **Reusable State** *EnergyTransferStarted* (**State** is *Authorized* and *_EVConnected* = false)

Tool validations

* Step 1:

Message: **RequestStartTransactionRequest**

- **idToken.idToken** <Configured valid_idtoken_idtoken>

- **idToken.type** <Configured valid_idtoken_type>

* Step 4:

Message: **AuthorizeResponse**

- **idTokenInfo.status** must be *Accepted*

Post scenario validations:

N/a

TC_F_03_CSMS: Remote start transaction - Remote start first - AuthorizeRemoteStart is false

Test case name	Remote start transaction - Remote start first - AuthorizeRemoteStart is false
Test case Id	TC_F_03_CSMS
Use case Id(s)	<u>F02</u>
Requirement(s)	<u>F02.FR.01</u> , <u>F01.FR.02</u>
System under test	CSMS
Description	OCPP 2.x.x allows an EV driver to either first wait for/trigger a RequestStartTransactionRequest OR connect the EV and EVSE. Both sequences will result in being able to charge.
Purpose	To verify if the CSMS is able to handle a Charging Station that starts a charging session when the Charging Stations receives a RequestStartTransactionRequest message (while AuthorizeRemoteStart is false), before the EV driver connects the EV and EVSE (within the connectionTimeout). The Charging station does NOT have to authorize beforehand like a local action to start a transaction.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Trigger the CSMS to request the Charging Station to start a transaction.

2. The Test System responds with a **RequestStartTransactionResponse** with **status** *Accepted* **transactionId** is omitted.

1. The CSMS sends a **RequestStartTransactionRequest**

3. The Test System sends a **TransactionEventRequest** with **triggerReason** is *RemoteStart* **transactionInfo.remoteStartId** is *<By Test System generated remoteStartID>* **eventType** is *Started*

4. The CSMS responds with a **TransactionEventResponse**.

5. Execute **Reusable State** *EnergyTransferStarted* (**State** is *Authorized* and *_EVConnected* = false)

Tool validations
* Step 1: Message: RequestStartTransactionRequest - idToken.idToken <Configured valid_idtoken_idtoken> - idToken.type <Configured valid_idtoken_type>
Post scenario validations: N/a

TC_F_04_CSMS: Remote start transaction - Remote start first - Cable plugin timeout

Test case name	Remote start transaction - Remote start first - Cable plugin timeout
Test case Id	TC_F_04_CSMS
Use case Id(s)	<u>F02</u> , <u>E03</u>
Requirement(s)	<u>E03.FR.04</u> , <u>E03.FR.05</u>
System under test	CSMS
Description	OCPP 2.x.x allows an EV driver to either first wait for/trigger a RequestStartTransactionRequest OR connect the EV and EVSE. Both sequences will result in being able to charge.
Purpose	To verify if the CSMS is able to handle a Charging Station that deauthorizes the transaction after the EVConnectionTimeout has been reached.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to request the Charging Station to start a transaction.

2. The Test System responds with a **RequestStartTransactionResponse** with **status** *Accepted* **transactionId** is omitted.

1. The CSMS sends a **RequestStartTransactionRequest**

3. The Test System sends a **TransactionEventRequest**. with **triggerReason** is *RemoteStart* **transactionInfo.remoteStartId** is *<By Test System generated remoteStartID>* **eventType** is *Started*

4. The CSMS responds with a **TransactionEventResponse**.

5. The Test System sends a **TransactionEventRequest**. with **triggerReason** is *EVConnectTimeout* **eventType** is *Updated*

6. The CSMS responds with a **TransactionEventResponse**.

Note(s):

- This step will be executed after the *<Configured Transaction Duration>* has been reached.

Tool validations

* Step 1:

Message: **RequestStartTransactionRequest**

- **idToken.idToken** <Configured valid_idtoken_idtoken>

- **idToken.type** <Configured valid_idtoken_type>

Post scenario validations:

N/a

TC_F_06_CSMS: Remote unlock Connector - Without ongoing transaction - Accepted

Test case name	Remote unlock Connector - Without ongoing transaction - Accepted
Test case Id	TC_F_06_CSMS
Use case Id(s)	F05
Requirement(s)	n/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to sent an UnlockConnectorRequest to the charging station. It sometimes happens that a connector of a Charging Station socket does not unlock correctly. This happens most of the time when there is tension on the charging cable. This means the driver cannot unplug his charging cable from the Charging Station. To help a driver, the CSO can send a UnlockConnectorRequest to the Charging Station. The Charging Station will then try to unlock the connector again.
Purpose	To verify if the CSMS is able to perform the remote unlock connector mechanism as described at the OCPP specification.
Prerequisite(s)	

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a UnlockConnectorResponse with status <i>Unlocked</i>	1. The CSMS sends a UnlockConnectorRequest

Tool validations

* Step 1:

Message **UnlockConnectorRequest**- **evseId** <Configured evseId>- **connectorId** <Configured connectorId>

Post scenario validations:

- N/a

TC_F_11_CSMS: Trigger message - MeterValues - Specific EVSE

Test case name	Trigger message - MeterValues - Specific EVSE
Test case Id	TC_F_11_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.01,F06.FR.02</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a MeterValuesRequest for a specific EVSE, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	1. The CSMS sends a TriggerMessageRequest
3. The Test System sends a MeterValuesRequest With evseId <Configured evseId> meterValue[0].sampledValue.context <i>Trigger</i>	4. The CSMS responds with a MeterValuesResponse

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** must be *MeterValues*- **evse.id** must be <Configured evseId>

Post scenario validations:

N/a

TC_F_12_CSMS: Trigger message - MeterValues - All EVSE

Test case name	Trigger message - MeterValues - All EVSE
Test case Id	TC_F_12_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.01</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a MeterValuesRequest for all EVSE, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	1. The CSMS sends a TriggerMessageRequest
3. The Test System sends a MeterValuesRequest With evseId omitted meterValue[0].sampledValue.context <i>Trigger</i> <u>Note(s):</u> <i>- This step will be executed for every EVSE.</i>	4. The CSMS responds with a MeterValuesResponse

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** must be *MeterValues***Post scenario validations:**

N/a

TC_F_13_CSMS: Trigger message - TransactionEvent - Specific EVSE

Test case name	Trigger message - TransactionEvent - Specific EVSE
Test case Id	TC_F_13_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.01,F06.FR.02</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a TransactionEventRequest for a specific EVSE, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	1. The CSMS sends a TriggerMessageRequest
3. The Test System sends a TransactionEventRequest With evse.id <i><Configured evseId></i> triggerReason <i>Trigger</i> transactionInfo.chargingState <i>Charging</i> meterValue is present meterValue[0].sampledValue.context <i>Trigger</i>	4. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** must be *TransactionEvent*- **evse.id** must be *<Configured evseId>*

Post scenario validations:

N/a

TC_F_14_CSMS: Trigger message - TransactionEvent - All EVSE

Test case name	Trigger message - TransactionEvent - All EVSE
Test case Id	TC_F_14_CSMS
Use case Id(s)	F06
Requirement(s)	F06.FR.01
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a TransactionEventRequest for all EVSE, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	1. The CSMS sends a TriggerMessageRequest
3. The Test System sends a TransactionEventRequest With evse.id omitted triggerReason <i>Trigger</i> transactionInfo.chargingState <i>Charging</i> meterValue is present meterValue[0].sampledValue.context <i>Trigger</i> <u>Note(s):</u> - This step will be executed for every EVSE.	4. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** must be *TransactionEvent*

Post scenario validations:

N/a

TC_F_15_CSMS: Trigger message - LogStatusNotification - Idle

Test case name	Trigger message - LogStatusNotification - Idle
Test case Id	TC_F_15_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.01</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a LogStatusNotificationRequest, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	1. The CSMS sends a TriggerMessageRequest
3. The Test System sends a LogStatusNotificationRequest with status <i>Idle</i>	4. The CSMS responds with a LogStatusNotificationResponse

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** must be *LogStatusNotification***Post scenario validations:**

N/a

TC_F_18_CSMS: Trigger message - FirmwareStatusNotification - Idle

Test case name	Trigger message - FirmwareStatusNotification - Idle
Test case Id	TC_F_18_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.01</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a FirmwareStatusNotificationRequest, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	1. The CSMS sends a TriggerMessageRequest
3. The Test System sends a FirmwareStatusNotificationRequest with status <i>Idle</i>	4. The CSMS responds with a FirmwareStatusNotificationResponse

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** must be *FirmwareStatusNotification***Post scenario validations:**

N/a

TC_F_20_CSMS: Trigger message - Heartbeat

Test case name	Trigger message - Heartbeat
Test case Id	TC_F_20_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.01</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a HeartbeatRequest, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	1. The CSMS sends a TriggerMessageRequest
3. The Test System sends a HeartbeatRequest	4. The CSMS responds with a HeartbeatResponse

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** must be *Heartbeat***Post scenario validations:**

N/a

TC_F_23_CSMS: Trigger message - StatusNotification - Specific EVSE - Available

Test case name	Trigger message - StatusNotification - Specific EVSE - Available
Test case Id	TC_F_23_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.01,F06.FR.02,F06.FR.13</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a StatusNotificationRequest for a specific available EVSE, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	1. The CSMS sends a TriggerMessageRequest
3. The Test System notifies the CSMS about the current state of the connector. Message: StatusNotificationRequest - connectorStatus <i>Available</i> - evseId <i><Configured evseId></i> - connectorId <i><Configured connectorId></i> Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Available"</i> - component.name <i>"Connector"</i> - component.evse.id <i><Configured evseId></i> - component.evse.connectorid <i><Configured connectorId></i> - variable.name <i>"AvailabilityState"</i>	4. The CSMS responds accordingly.

Tool validations

* Step 1:

Message: **TriggerMessageRequest**

- **requestedMessage** must be *StatusNotification*

- **evse.id** must be *<Configured evseId>*

Post scenario validations:

N/a

TC_F_24_CSMS: Trigger message - StatusNotification - Specific EVSE - Occupied

Test case name	Trigger message - StatusNotification - Specific EVSE - Occupied
Test case Id	TC_F_24_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.01,F06.FR.02,F06.FR.13</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger the Charging Station to send a StatusNotificationRequest for a specific occupied EVSE, using a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System notifies the CSMS about the current state of the connector. Message: StatusNotificationRequest - connectorStatus <i>Occupied</i> - evseId <i><Configured evseId></i> - connectorId <i><Configured connectorId></i> Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Occupied"</i> - component.name <i>"Connector"</i> - component.evse.id <i><Configured evseId></i> - component.evse.connectorid <i><Configured connectorId></i> - variable.name <i>"AvailabilityState"</i>	2. The CSMS responds accordingly.
4. The Test System responds with a TriggerMessageResponse with status <i>Accepted</i>	3. The CSMS sends a TriggerMessageRequest

Main (Test scenario)

5. The Test System notifies the CSMS about the current state of the connector.

Message: **StatusNotificationRequest**

- **connectorStatus** *Occupied*

- **evseId** *<Configured evseId>*

- **connectorId** *<Configured connectorId>*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Occupied"*

- **component.name** *"Connector"*

- **component.evse.id** *<Configured evseId>*

- **component.evse.connectorid** *<Configured connectorId>*

- **variable.name** *"AvailabilityState"*

6. The CSMS responds accordingly.

Tool validations

* Step 1:

Message: **TriggerMessageRequest**

- **requestedMessage** must be *StatusNotification*

- **evse.id** must be *<Configured evseId>*

Post scenario validations:

N/a

TC_F_27_CSMS: Trigger message - NotImplemented

Test case name	Trigger message - NotImplemented
Test case Id	TC_F_27_CSMS
Use case Id(s)	<u>F06</u>
Requirement(s)	<u>F06.FR.08</u>
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to handle a Charging Station that does not support the requested message value from a TriggerMessageRequest.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a TriggerMessageResponse with status <i>NotImplemented</i>	1. The CSMS sends a TriggerMessageRequest

Tool validations

N/a

Post scenario validations:

N/a

TC_F_100_CSMS: Trigger message - CustomTrigger

Test case name	Trigger message - CustomTrigger
Test case Id	TC_F_100_CSMS
Use case Id(s)	F06
Requirement(s)	...
System under test	CSMS
Description	The CSMS can request a Charging Station to send Charging Station-initiated messages. In the request the CSMS indicates which message it wishes to receive.
Purpose	To verify if the CSMS is able to trigger messages using a CustomTrigger.
Prerequisite(s)	The Test System reports support for the following: component.name CustomizationCtrlr variable.name CustomTriggers variableAttribute.value [Custom1, Custom2, Custom3]

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual action: Use CSMS to trigger custom trigger Custom2 for <Configured evseId>

2. The Test System responds with a **TriggerMessageResponse** with
status *Accepted*

1. The CSMS sends a **TriggerMessageRequest**

Tool validations

* Step 1:

Message: **TriggerMessageRequest**- **requestedMessage** CustomTrigger- **customTrigger** Custom2- **evse.id** <Configured evseId>

Post scenario validations:

N/a

8. G Availability

TC_G_03_CSMS: Change Availability EVSE - Operative to inoperative

Test case name	Change Availability EVSE - Operative to inoperative
Test case Id	TC_G_03_CSMS
Use case Id(s)	<u>G03</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS requests the Charging Station to change the availability of one of the EVSEs from Operative to Inoperative. An EVSE is considered Operative in any status other than Faulted and Unavailable.
Purpose	To verify if the CSMS is able to perform the change availability mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *Unavailable* for <Configured evseId>

Tool validations

N/a

Post scenario validations:

- N/a

TC_G_04_CSMS: Change Availability EVSE - Inoperative to operative

Test case name	Change Availability EVSE - Inoperative to operative
Test case Id	TC_G_04_CSMS
Use case Id(s)	G03
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS requests the Charging Station to change the availability of one of the EVSEs from Inoperative to Operative. An EVSE is considered Operative in any status other than Faulted and Unavailable.
Purpose	To verify if the CSMS is able to perform the change availability mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

Unavailable for <Configured evseId>

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**Manual Action: Request the CSMS to change the availability of an EVSE to Operative.

2. The Test System responds with a **ChangeAvailabilityResponse** with **status** *Accepted*

1. The CSMS sends a **ChangeAvailabilityRequest**

3. The Test System notifies the CSMS about the current state of all connectors belonging to the specified EVSE (and optionally also from the EVSE itself).

Message: **StatusNotificationRequest**

- **connectorStatus** *Available*

- **evseId** <Configured evseId>

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Available"*

- **component.name** *"EVSE" / Connector*

- **component.evse.id** <Configured evseId>

- **variable.name** *"AvailabilityState"*

4. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ChangeAvailabilityRequest**

- **operationalStatus** *Operative*
- **evse.id** *<Configured evseId>*
- **connectorId** *omit*

Post scenario validations:

- N/a

TC_G_05_CSMS: Change Availability Charging Station - Operative to inoperative

Test case name	Change Availability Charging Station - Operative to inoperative
Test case Id	TC_G_05_CSMS
Use case Id(s)	<u>G04</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS requests the Charging Station to change the availability from operative to inoperative. A Charging Station is considered Operative when it is charging or ready for charging. A Charging Station is considered Inoperative when it does not allow any charging.
Purpose	To verify if the CSMS is able to perform the change availability mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**Manual Action: Request the CSMS to change the availability of the Charging Station to Inoperative.

2. The Test System responds with a **ChangeAvailabilityResponse** with **status Accepted**

1. The CSMS sends a **ChangeAvailabilityRequest**

3. The Test System notifies the CSMS about the current state of all connectors

Message: **StatusNotificationRequest**

- **connectorStatus** *Unavailable*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Unavailable"*

- **component.name** *"Connector"*

- **variable.name** *"AvailabilityState"*

4. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ChangeAvailabilityRequest**

- **operationalStatus** *Inoperative*

- **evseId** *omit*

- **connectorId** *omit*

Post scenario validations:

- N/a

TC_G_06_CSMS: Change Availability Charging Station - Inoperative to operative

Test case name	Change Availability Charging Station - Inoperative to operative
Test case Id	TC_G_06_CSMS
Use case Id(s)	<u>G04</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS requests the Charging Station to change the availability from inoperative to operative. A Charging Station is considered Operative when it is charging or ready for charging. A Charging Station is considered Inoperative when it does not allow any charging.
Purpose	To verify if the CSMS is able to perform the change availability mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):Charging Station set to *Unavailable* (Original status was Available)**Main (Test scenario)****Charging Station****CSMS**Manual Action: Request the CSMS to change the availability of the Charging Station to Inoperative.

2. The Test System responds with a **ChangeAvailabilityResponse** with **status Accepted**

1. The CSMS sends a **ChangeAvailabilityRequest**

3. The Test System notifies the CSMS about the current state of all connectors.

Message: **StatusNotificationRequest**

- **connectorStatus** Available

Message: **NotifyEventRequest**

- **trigger** Delta

- **actualValue** "Available"

- **component.name** "Connector"

- **variable.name** "AvailabilityState"

4. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ChangeAvailabilityRequest**

- **operationalStatus** *Operative*

- **evseId** *omit*

- **connectorId** *omit*

Post scenario validations:

- N/a

TC_G_07_CSMS: Change Availability Connector - Operative to inoperative

Test case name	Change Availability Connector - Operative to inoperative
Test case Id	TC_G_07_CSMS
Use case Id(s)	G03
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS requests the Charging Station to change the availability of one of the Connectors of one EVSE from Operative to Inoperative. A Connector is considered Operative in any status other than Faulted and Unavailable.
Purpose	To verify if the CSMS is able to perform the change availability mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to change the availability of a Connector to Inoperative.

2. The Test System responds with a **ChangeAvailabilityResponse** with **status Accepted**

1. The CSMS sends a **ChangeAvailabilityRequest**

3. The Test System notifies the CSMS about the current state of the connector.

Message: **StatusNotificationRequest**

- **connectorStatus** *Unavailable*

- **evseId** *<Configured evseId>*

- **connectorId** *<Configured connectorId>*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Unavailable"*

- **component.name** *"Connector"*

- **component.evse.id** *<Configured evseId>*

- **component.evse.connectorid** *<Configured connectorId>*

- **variable.name** *"AvailabilityState"*

4. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ChangeAvailabilityRequest**

- **operationalStatus** *Inoperative*

- **evse.id** *<Configured evseId>*

- **evse.connectorId** *<Configured connectorId>*

Post scenario validations:

N/a

TC_G_08_CSMS: Change Availability Connector - Inoperative to operative

Test case name	Change Availability Connector - Inoperative to operative
Test case Id	TC_G_08_CSMS
Use case Id(s)	G03
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS requests the Charging Station to change the availability of one of the Connectors from one EVSE from Inoperative to Operative. A Connector is considered Operative in any status other than Faulted and Unavailable.
Purpose	To verify if the CSMS is able to perform the change availability mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

Unavailable for <Configured connectorId>

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to change the availability of a Connector to Operative.

2. The Test System responds with a **ChangeAvailabilityResponse** with **status Accepted**

1. The CSMS sends a **ChangeAvailabilityRequest**

3. The Test System notifies the CSMS about the current state of the connector.

Message: **StatusNotificationRequest**

- **connectorStatus** Available

- **evseId** <Configured evseId>

- **connectorId** <Configured connectorId>

Message: **NotifyEventRequest**

- **trigger** Delta

- **actualValue** "Available"

- **component.name** "Connector"

- **component.evse.id** <Configured evseId>

- **component.evse.connectorid** <Configured connectorId>

- **variable.name** "AvailabilityState"

4. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ChangeAvailabilityRequest**

- **operationalStatus** *Operative*

- **evse.id** *<Configured evseId>*

- **evse.connectorId** *<Configured connectorId>*

Post scenario validations:

N/a

TC_G_11_CSMS: Change Availability EVSE - With ongoing transaction

Test case name	Change Availability EVSE - With ongoing transaction
Test case Id	TC_G_11_CSMS
Use case Id(s)	G03
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS requests the Charging Station to change the availability of one of the EVSEs from Operative to Inoperative. An EVSE is considered Operative in any status other than Faulted and Unavailable.
Purpose	To verify if the CSMS is able to send a change availability request during a transaction according to the mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

Note(s): Request the CSMS to change the availability to inoperative

2. The Test System responds with a **ChangeAvailabilityResponse** with **status Scheduled**

1. The CSMS sends a **ChangeAvailabilityRequest**

Note(s): Wait for <Configured Transaction Duration>3. Execute **Reusable State StopAuthorized**4. Execute **Reusable State EVConnectedPostSession**5. Execute **Reusable State EVDisconnected**

Main (Test scenario)	
6. The Test System notifies the CSMS about the current state of all connectors with Message: StatusNotificationRequest - connectorStatus <i>Unavailable</i> - evseId <i><Configured evseId></i> OR Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Unavailable"</i> - component.name <i>"Connector"</i> - component.evse.id <i><Configured evseId></i> - variable.name <i>"AvailabilityState"</i>	7. The CSMS responds accordingly.

Tool validations
* Step 1: Message ChangeAvailabilityRequest - operationalStatus <i>Inoperative</i> - evse.id <i><Configured evseId></i> - connectorId <i>omit</i>
Post scenario validations: - A respond to report the state of a connector has been received for all connectors.

TC_G_14_CSMS: Change Availability Charging Station - With ongoing transaction

Test case name	Change Availability Charging Station - With ongoing transaction
Test case Id	TC_G_14_CSMS
Use case Id(s)	G04
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS requests the Charging Station to change the availability of one of the EVSEs from Operative to Inoperative. An EVSE is considered Operative in any status other than Faulted and Unavailable.
Purpose	To verify if the CSMS is able to send a change availability request during a transaction according to the mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

Note(s): Request the CSMS to change the availability of the station to inoperative

2. The Test System responds with a **ChangeAvailabilityResponse** with **status Scheduled**

1. The CSMS sends a **ChangeAvailabilityRequest**

3. The Test System notifies the CSMS about the current state of all unoccupied connectors with Message: **StatusNotificationRequest** - **connectorStatus Unavailable**

4. The CSMS responds accordingly.

Note(s): Wait for *<Configured Transaction Duration>*

5. Execute **Reusable State StopAuthorized**

6. Execute **Reusable State EVConnectedPostSession**

7. Execute **Reusable State EVDisconnected**

Main (Test scenario)

8. The Test System notifies the CSMS about the current state of the configured connector with

Message: **StatusNotificationRequest**

- **connectorStatus** *Unavailable*

9. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ChangeAvailabilityRequest**

- **operationalStatus** *Inoperative*

- **evseId** *omit*

- **connectorId** *omit*

Post scenario validations:

- A respond to report the state of a connector has been received for all connectors.

TC_G_17_CSMS: Change Availability Connector - With ongoing transaction

Test case name	Change Availability Connector - With ongoing transaction
Test case Id	TC_G_17_CSMS
Use case Id(s)	G03
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers how the CSMS requests the Charging Station to change the availability of one of the EVSEs from Operative to Inoperative. An EVSE is considered Operative in any status other than Faulted and Unavailable.
Purpose	To verify if the CSMS is able to send a change availability request during a transaction according to the mechanism as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

Note(s): Request the CSMS to change the availability of one connector to inoperative

2. The Test System responds with a **ChangeAvailabilityResponse** with **status Scheduled**

1. The CSMS sends a **ChangeAvailabilityRequest**

Note(s): Wait for *<Configured Transaction Duration>*

3. Execute **Reusable State StopAuthorized**

4. Execute **Reusable State EVConnectedPostSession**

5. Execute **Reusable State EVDisconnected**

6. The Test System notifies the CSMS about the current state of all connectors with
 Message: **StatusNotificationRequest**
 - **connectorStatus** *Unavailable*
 - **evseId** *<Configured evseId>*
 - **connectorId** *<Configured connectorId>*

7. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ChangeAvailabilityRequest**

- **operationalStatus** *Inoperative*
- **evse.id** *<Configured evseId>*
- **evse.connectorId** *<Configured connectorId>*

Post scenario validations:

- A respond to report the state of a connector has been received for all connectors.

TC_G_20_CSMS: Connector status Notification - Lock Failure

Test case name	Connector status Notification - Lock Failure
Test case Id	TC_G_20_CSMS
Use case Id(s)	G05
Requirement(s)	G05.FR.03
System under test	CSMS
Description	This test case describes how the EV Driver is prevented from starting a charge session at the Charging Station while the Connector is not locked properly.
Purpose	To verify if the CSMS responds on a <code>notifyEventrequest</code> as described at the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
- The Test System sends a NotifyEventRequest with - <code>eventData.trigger</code> <i>Delta</i> - <code>eventData.component.name</code> <i>"ConnectorPlugRetentionLock"</i> - <code>eventData.variable.name</code> <i>"Problem"</i> - <code>eventData.actualValue</code> <i>"true"</i>	The CSMS responds with a NotifyEventResponse

Tool validations

N/a

Post scenario validations:

- N/a

9. H Reservation

TC_H_01_CSMS: Reserve a specific EVSE - Accepted - Valid idToken

Test case name	Reserve a specific EVSE - Accepted - Valid idToken
Test case Id	TC_H_01_CSMS
Use case Id(s)	<u>H01</u> (S2), <u>H03</u>
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS is able to reserve a specific EVSE for a specific IdToken by sending a ReserveNowRequest containing an evseId.
Purpose	To verify if the CSMS is able to request the Charging Station to reserve a specific EVSE, until the EV Driver with the specified IdToken arrives.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State Reserved** for <Configured evseId>

Tool validations

N/a

Post scenario validations:

N/a

TC_H_07_CSMS: Reserve a specific EVSE - Reservation Ended / not used

Test case name	Reserve a specific EVSE - Reservation Ended / not used
Test case Id	TC_H_07_CSMS
Use case Id(s)	<u>H01</u> (S2), <u>H04</u>
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS is able to reserve a specific EVSE for a specific IdToken by sending a ReserveNowRequest containing an evseId.
Purpose	To verify if the CSMS is able to handle a reservation that is canceled by the Charging Station, because the EV driver did not arrive before the set expiryDateTime was reached.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to send a ReserveNowRequest for a specific EVSE.

2. The Test System responds with a **ReserveNowResponse**
With status *Accepted*

1. The CSMS sends a **ReserveNowRequest** with
expiryDateTime *current time + <configured transaction duration>*

3. The Test System notifies the CSMS about the current state of the connector(s) of the configured EVSE

4. The CSMS responds accordingly.

Message: **StatusNotificationRequest**
with **connectorStatus** *Reserved*
Message: **NotifyEventRequest**
with **trigger** *Delta*
actualValue *"Reserved"*
component.name *"Connector"*
variable.name *"AvailabilityState"*

Main (Test scenario)	
5. The Test System notifies the CSMS about the current state of the connector(s) of the configured EVSE Message: StatusNotificationRequest with connectorStatus <i>Available</i> Message: NotifyEventRequest with trigger <i>Delta</i> actualValue <i>"Available"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i> <u>Note(s):</u> - The Test System waits until the provided <i>expiryDateTime</i> from step 1 expires before executing this step.	6. The CSMS responds accordingly.
7. The Test System sends a ReservationStatusUpdateRequest With reservationUpdateStatus <i>Expired</i> reservationId <i><id received at step 1></i>	8. The CSMS responds with a ReservationStatusUpdateResponse

Tool validations
* Step 1: Message: ReserveNowRequest - evseId must be <i><Configured evseId></i> - connectorType must be omitted - idToken.idToken <i><Configured valid_idtoken_idtoken></i> - idToken.type <i><Configured valid_idtoken_type></i>
Post scenario validations: N/a

TC_H_08_CSMS: Reserve an unspecified EVSE - Accepted

Test case name	Reserve an unspecified EVSE - Accepted
Test case Id	TC_H_08_CSMS
Use case Id(s)	<u>H01</u> (S1), <u>H03</u>
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS is able to reserve an unspecified EVSE for a specific IdToken by sending a ReserveNowRequest without an evseId.
Purpose	To verify if the CSMS is able to request the Charging Station to reserve an unspecified EVSE, until the EV Driver with the specified IdToken arrives.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**

Manual Action: Trigger the CSMS to send a ReserveNowRequest for an unspecified EVSE.

2. The Test System responds with a **ReserveNowResponse** with **status Accepted**

1. The CSMS sends a **ReserveNowRequest**

3. The Test System notifies the CSMS about the current state of the connector(s) of the configured EVSE

4. The CSMS responds accordingly.

Message: **StatusNotificationRequest**

with **connectorStatus Reserved**

Message: **NotifyEventRequest**

with **trigger Delta**

actualValue "Reserved"

component.name "Connector"

variable.name "AvailabilityState"

Note(s):

- The Test System will execute this step, if it is configured with only one EVSE.

Tool validations

* Step 1:

Message: **ReserveNowRequest**

- **evseId** must be omitted
- **connectorType** must be omitted
- **idToken.idToken** <Configured valid_idtoken_idtoken>
- **idToken.type** <Configured valid_idtoken_type>

Post scenario validations:

N/a

TC_H_14_CSMS: Reserve an unspecified EVSE - Amount of EVSEs available equals the amount of reservations

Test case name	Reserve an unspecified EVSE - Amount of EVSEs available equals the amount of reservations
Test case Id	TC_H_14_CSMS
Use case Id(s)	<u>H01</u> (S1)
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS is able to reserve an unspecified EVSE for a specific IdToken by sending a ReserveNowRequest without an evseId.
Purpose	To verify if the CSMS is able to handle that the Charging Station sets all available EVSE to reserved, when the amount of EVSEs available equals the amount of reservations.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**Manual Action: Trigger the CSMS to send a ReserveNowRequest for an unspecified EVSE.

2. The Test System responds with a **ReserveNowResponse** with **status Accepted**

1. The CSMS sends a **ReserveNowRequest**

Note(s):

- This step needs to executed time the amount of EVSE configured for the Test System.

3. The Test System notifies the CSMS about the current state of all connectors

4. The CSMS responds accordingly.

Message: **StatusNotificationRequest**

with **connectorStatus Reserved**

Message: **NotifyEventRequest**

with **trigger Delta**

actualValue "Reserved"

component.name "Connector"

variable.name "AvailabilityState"

Note(s):

- This step will be executed after the last ReserveNowRequest has been sent from step 1.

Tool validations

* Step 1:

Message: **ReserveNowRequest**

- **evseId** must be omitted
- **connectorType** must be omitted
- **idToken.idToken** <Configured valid_idtoken_idtoken>
- **idToken.type** <Configured valid_idtoken_type>

Post scenario validations:

N/a

TC_H_15_CSMS: Reserve a connector with a specific type - Success

Test case name	Reserve a connector with a specific type - Success
Test case Id	TC_H_15_CSMS
Use case Id(s)	<u>H01(S3)</u> , <u>H03</u>
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS is able to reserve an EVSE with a connector with a specific type for a specific IdToken by sending a ReserveNowRequest with a connectorType.
Purpose	To verify if the CSMS is able to request the Charging Station to reserve an EVSE with a connector with a specific type, until the EV Driver with the specified IdToken arrives.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**Manual Action: Trigger the CSMS to send a ReserveNowRequest for a specific ConnectorType.

2. The Test System responds with a **ReserveNowResponse**
With status *Accepted*

1. The CSMS sends a **ReserveNowRequest**

3. The Test System notifies the CSMS about the current state of the connector

4. The CSMS responds accordingly.

Message: **StatusNotificationRequest**
with **connectorStatus** *Reserved*
Message: **NotifyEventRequest**
with **trigger** *Delta*
actualValue *"Reserved"*
component.name *"Connector"*
variable.name *"AvailabilityState"*

Tool validations

* Step 1:

Message: **ReserveNowRequest**

- **evseId** must be omitted
- **connectorType** must be *<Configured connectorType>*
- **idToken.idToken** *<Configured valid_idtoken_idtoken>*
- **idToken.type** *<Configured valid_idtoken_type>*

Post scenario validations:

N/a

TC_H_17_CSMS: Cancel reservation of an EVSE - Success

Test case name	Cancel reservation of an EVSE - Success
Test case Id	TC_H_17_CSMS
Use case Id(s)	H02
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS is able to cancel a reservation by sending a CancelReservationRequest to the Charging Station.
Purpose	To verify if the CSMS is able to request the Charging Station to cancel a reservation, by sending a CancelReservationRequest
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to send a **ReserveNowRequest** for a specific EVSE.

2. The Test System responds with a **ReserveNowResponse** with **status** *Accepted*

1. The CSMS sends a **ReserveNowRequest**

3. The Test System notifies the CSMS about the current state of the connector(s) of the configured EVSE

4. The CSMS responds accordingly.

Message: **StatusNotificationRequest**

with **connectorStatus** *Reserved*

Message: **NotifyEventRequest**

with **trigger** *Delta*

actualValue *"Reserved"*

component.name *"Connector"*

variable.name *"AvailabilityState"*

Manual Action: Trigger the CSMS to send a **CancelReservationRequest** for the reservation created at step 1.

6. The Test System responds with a **CancelReservationResponse**
With status *Accepted*

5. The CSMS sends a **CancelReservationRequest**

Main (Test scenario)

7. The Test System notifies the CSMS about the current state of the connector(s) of the configured EVSE

Message: **StatusNotificationRequest**
with **connectorStatus** *Available*
Message: **NotifyEventRequest**
with **trigger** *Delta*
actualValue *"Available"*
component.name *"Connector"*
variable.name *"AvailabilityState"*

8. The CSMS responds accordingly.

Tool validations

* Step 1:

Message: **ReserveNowRequest**

- **evseId** must be *<Configured evseId>*
- **connectorType** must be omitted
- **idToken.idToken** *<Configured valid_idtoken_idtoken>*
- **idToken.type** *<Configured valid_idtoken_type>*

* Step 5:

Message: **CancelReservationRequest**

- **reservationId** must be equal to the **id** provided at step 1

Post scenario validations:

N/a

TC_H_19_CSMS: Reserve a specific EVSE - Use a reserved EVSE with GroupId

Test case name	Reserve a specific EVSE - Use a reserved EVSE with GroupId
Test case Id	TC_H_19_CSMS
Use case Id(s)	<u>H01</u> , <u>H03</u>
Requirement(s)	N/a
System under test	CSMS
Description	The CSMS is able to reserve an EVSE for a specific group by sending a ReserveNowRequest containing a groupIdToken .
Purpose	To verify if the CSMS is able to request the Charging Station create a reservation for a specific group, by sending a ReserveNowRequest with a groupIdToken
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to send a *ReserveNowRequest* with a *groupIdToken* for a specific EVSE.

2. The Test System responds with a **ReserveNowResponse**
With status *Accepted*

1. The CSMS sends a **ReserveNowRequest**

3. The Test System notifies the CSMS about the current state of the connector(s) of the configured EVSE

4. The CSMS responds accordingly.

Message: **StatusNotificationRequest**
with **connectorStatus** *Reserved*
Message: **NotifyEventRequest**
with **trigger** *Delta*
actualValue *"Reserved"*
component.name *"Connector"*
variable.name *"AvailabilityState"*

Tool validations

* Step 1:

Message: **ReserveNowRequest**

- **evseId** must be *<Configured evseId>*
- **connectorType** must be omitted
- **groupIdToken** must be provided
- **idToken.idToken** *<Configured valid_idtoken_idtoken>*
- **idToken.type** *<Configured valid_idtoken_type>*

Post scenario validations:

N/a

TC_H_20_CSMS: Charging Station cancels reservation when Faulted

Test case name	Charging Station cancels reservation when Faulted
Test case Id	TC_H_20_CSMS
Use case Id(s)	<u>H01</u>
Requirement(s)	N/a
System under test	CSMS
Description	The Charging Station will cancel reservations, when the EVSE specified for a reservation is set to an inoperative state.
Purpose	To verify if the CSMS is able to handle it when the reservation is canceled when the availability state of the EVSE specified for the reservation is set to Faulted by the Test System.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to send a *ReserveNowRequest* for a specific EVSE.

2. The Test System responds with a **ReserveNowResponse**
With status *Accepted*

1. The CSMS sends a **ReserveNowRequest**

3. The Test System notifies the CSMS about the current state of the connector(s) of the configured EVSE

4. The CSMS responds accordingly.

Message: **StatusNotificationRequest**

with **connectorStatus** *Reserved*

Message: **NotifyEventRequest**

with **trigger** *Delta*

actualValue *"Reserved"*

component.name *"Connector"*

variable.name *"AvailabilityState"*

Main (Test scenario)	
5. The Test System notifies the CSMS about the current state of the connector(s) of the configured EVSE Message: StatusNotificationRequest with connectorStatus <i>Faulted</i> Message: NotifyEventRequest with trigger <i>Delta</i> actualValue <i>"Faulted"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	6. The CSMS responds accordingly.
7. The Test System sends a ReservationStatusUpdateRequest With reservationUpdateStatus <i>Removed</i> reservationId <i><id received at step 1></i>	8. The CSMS responds with a ReservationStatusUpdateResponse
Tool validations	
* Step 1: Message: ReserveNowRequest - evseId must be <i><Configured evseId></i> - connectorType must be omitted - idToken.idToken <i><Configured valid_idtoken_idtoken></i> - idToken.type <i><Configured valid_idtoken_type></i>	
Post scenario validations: N/a	

TC_H_22_CSMS: Reserve a specific EVSE - Configured to Reject

Test case name	Reserve a specific EVSE - Configured to Reject
Test case Id	TC_H_22_CSMS
Use case Id(s)	<u>H01</u>
Requirement(s)	
System under test	CSMS
Description	The CSMS is able to reserve a specific EVSE for a specific IdToken by sending a ReserveNowRequest containing an evseId.
Purpose	To verify if the CSMS is able to correctly read the respond from a charging station when it is configured not to accept reservations.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

2. The Test System responds with a **ReserveNowResponse** with
- status *Rejected*

CSMS

1. The CSMS sends a **ReserveNowRequest**

Tool validations

N/a

Post scenario validations:

N/a

10. I Tariff and Cost

TC_I_01_CSMS: Show EV Driver running total cost during charging - costUpdatedRequest

Test case name	Show EV Driver running total cost during charging - costUpdatedRequest
Test case Id	TC_I_01_CSMS
Use case Id(s)	<u>I02</u>
Requirement(s)	<u>I02.FR.01</u>
System under test	CSMS
Description	While a transaction is ongoing, the driver wants to know how much the running total cost is, updated at a relevant interval.
Purpose	To verify if the CSMS is able to correctly send the running total cost as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest with idToken.idToken <Configured valid_idtoken_idtoken> idToken.type <Configured valid_idtoken_type>	2. The CSMS responds with an AuthorizeResponse
3. The Test System sends a TransactionEventRequest with - triggerReason <i>Authorized</i> - idToken.idToken <Configured valid_idtoken_idtoken> - idToken.type <Configured valid_idtoken_type> - eventType <i>Updated</i>	4. The CSMS responds with a TransactionEventResponse
5. Execute Reusable State <i>EVConnectedPreSession</i>	
6. Execute Reusable State <i>EnergyTransferStarted</i>	

Main (Test scenario)	
7. The Test System sends a TransactionEventRequest With triggerReason is <i>MeterValuePeriodic</i> eventType is <i>Updated</i> timestamp <The intervals between the timestamps of the received Meter Value messages equals the configured sampled Meter Values interval>. sampledValue.context is <i>Sample.Periodic</i> <u>Note(s):</u> - This step will be executed every _ <Configured sampled Meter Values interval> - The Test System will end the testcase after two MeterValues.	8. The CSMS responds with a TransactionEventResponse
10. The Test System responds with a CostUpdatedResponse	9. The CSMS sends a CostUpdatedRequest <u>Note(s):</u> - This step will be executed after every TransactionEventResponse, if the message did not contain a totalCost.

Tool validations
* Step 2: Message AuthorizeResponse - idTokenInfo.status <i>Accepted</i> * Step 4: Message TransactionEventResponse - idTokenInfo.status <i>Accepted</i> - totalCost <Optional> * Step 7: Message (Optional) CostUpdatedRequest - transactionId <Generated TransactionId>
Post scenario validations: - N/a

TC_I_02_CSMS: Show EV Driver Final Total Cost After Charging

Test case name	Show EV Driver Final Total Cost After Charging
Test case Id	TC_I_02_CSMS
Use case Id(s)	<u>I03</u>
Requirement(s)	<u>I03.FR.02</u>
System under test	CSMS
Description	While a transaction is ongoing, the driver wants to know how much the running total cost is, updated at a relevant interval.
Purpose	To verify if the CSMS is able to correctly send the total cost as described in the OCPP specification.
Prerequisite(s)	- N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

- CSMS is configured with a tariff which is based on energy consumed.

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. Execute Reusable State <i>EVConnectedPreSession</i> - The TransactionEventRequest contains the MeterValue field. - sampledValue[0].value <i>1000</i> - sampledValue[0].context <i>Transaction.Begin</i>	
2. Execute Reusable State <i>Authorized</i>	
3. Execute Reusable State <i>EnergyTransferStarted</i>	
4. Execute Reusable State <i>StopAuthorized</i>	
5. Execute Reusable State <i>EVConnectedPostSession</i>	
6. The Test System notifies the CSMS about the current state of the configured connector. Message: StatusNotificationRequest - connectorStatus <i>Available</i> Message: NotifyEventRequest - trigger <i>Delta</i> - actualValue <i>"Available"</i> - component.name <i>"Connector"</i> - variable.name <i>"AvailabilityState"</i>	7. The CSMS responds accordingly.

Main (Test scenario)	
8. The Test System sends a TransactionEventRequest with - triggerReason <i>EVCommunicationLost</i> - eventType <i>Ended</i> - transactionInfo.chargingState <i>Idle</i> - transactionInfo.stoppedReason <i>EVDisconnected</i> - meterValue[0].sampledValue[0].value <i>6000</i> - meterValue[0].sampledValue[0].context <i>Transaction.End</i>	9. The CSMS responds with a TransactionEventResponse
Tool validations	
* Step 9: Message TransactionEventResponse - totalCost <i><Not omitted></i>	
Post scenario validations: - N/a	

TC_I_101_CSMS: Set Default Tariff - startTimeOfDay, endTimeOfDay

Test case name	Set Default Tariff - startTimeOfDay, endTimeOfDay
Test case Id	TC_I_101_CSMS
Use case Id(s)	<u>I07</u> , <u>I12</u>
Requirement(s)	...
System under test	CSMS
Description	To set the default tariff on the charging station, for example for ad hoc charging, or when there is no driver-specific tariff.
Purpose	To verify if the CSMS is able to set a tariff with conditions.
Prerequisite(s)	- The CSMS supports local cost calculation by CS - The CSMS supports conditions in Tariffs - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0 - CS has TariffCtrlr.ConditionsSupported[Tariff] = true

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to set a default tariff on EVSE 0 with

- validFrom

- Price for energy with condition on startTimeOfDay and endTimeOfDay

2. The Test System responds with **SetDefaultTariffResponse** with **status Accepted**

1. The CSMS sends a **SetDefaultTariffRequest**

Tool validations

* Step 1:

Message **SetDefaultTariffRequest**

- evseId 0

- tariff.validFrom must be <not omitted>

- tariff.energy.prices[0].priceKwh must be <not omitted>

- tariff.energy.prices[0].conditions.startTimeOfDay must be in format as per RFC 3339: time-hour ":" time-minute

- tariff.energy.prices[0].conditions.endTimeOfDay must be in format as per RFC 3339: time-hour ":" time-minute

Post scenario validations:

N/a

DRAFT

TC_I_102_CSMS: Set Default Tariff - TariffMaxElements

Test case name	Set Default Tariff - TariffMaxElements
Test case Id	TC_I_102_CSMS
Use case Id(s)	<u>I07</u> , <u>I11</u>
Requirement(s)	...
System under test	CSMS
Description	To set the default tariff on the charging station, for example for ad hoc charging, or when there is no driver-specific tariff.
Purpose	To verify if the CSMS is able to process a response indicating TooManyElements.
Prerequisite(s)	- The CSMS supports local cost calculation by CS - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0 - CS has TariffCtrlr.ConditionsSupported[Tariff] = true

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: _Request the CSMS to set a default tariff on EVSE 0 with

- validFrom
- Price for energy in peak hours
- Price for energy other hours

2. The Test System responds with **SetDefaultTariffResponse** with **status TooManyElements**

1. The CSMS sends a **SetDefaultTariffRequest**

Tool validations

* Step 1:

Message **SetDefaultTariffRequest**

- **evseId** 0
- **tariff.validFrom** must be *<not omitted>*
- **tariff.energy.prices[0].priceKwh** must be *<not omitted>*
- **tariff.energy.prices[0].conditions.startTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*
- **tariff.energy.prices[0].conditions.endTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*
- **tariff.energy.prices[1].priceKwh** must be *<not omitted>*
- **tariff.energy.prices[1].conditions.startTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*
- **tariff.energy.prices[1].conditions.endTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*

Post scenario validations:

N/a

TC_I_105_CSMS: Set Default Tariff - TariffConditionsSupported is false

Test case name	Set Default Tariff - TariffConditionsSupported is false
Test case Id	TC_I_105_CSMS
Use case Id(s)	<u>I07</u>
Requirement(s)	...
System under test	CSMS
Description	To set the default tariff on the charging station, for example for ad hoc charging, or when there is no driver-specific tariff.
Purpose	To verify if the CSMS is able to process a response indicating ConditionNotSupported.
Prerequisite(s)	- The CSMS supports local cost calculation by CS - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0 - CS has TariffCtrlr.ConditionsSupported[Tariff] = true

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: _Request the CSMS to set a default tariff on EVSE 0 with

- validFrom
- Price for energy in peak hours
- Price for energy other hours

2. The Test System responds with **SetDefaultTariffResponse** with **status** *ConditionsNotSupported*

1. The CSMS sends a **SetDefaultTariffRequest**

Tool validations

* Step 1:

Message **SetDefaultTariffRequest**

- **evseId** 0
- **tariff.validFrom** must be *<not omitted>*
- **tariff.energy.prices[0].priceKwh** must be *<not omitted>*
- **tariff.energy.prices[0].conditions.startTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*
- **tariff.energy.prices[0].conditions.endTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*
- **tariff.energy.prices[1].priceKwh** must be *<not omitted>*
- **tariff.energy.prices[1].conditions.startTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*
- **tariff.energy.prices[1].conditions.endTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*

Post scenario validations:

N/a

TC_I_106_CSMS: Set Default Tariff - validations

Test case name	Set Default Tariff - validations
Test case Id	TC_I_106_CSMS
Use case Id(s)	<u>I07</u> , <u>I09</u>
Requirement(s)	...
System under test	CSMS
Description	To set the default tariff on the charging station, for example for ad hoc charging, or when there is no driver-specific tariff.
Purpose	To verify if the CSMS supports configuring setting local cost calculation.
Prerequisite(s)	- The CSMS supports local cost calculation by CS - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0 - CS has TariffCtrlr.ConditionsSupported[Tariff] = true

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to set a default tariff on EVSE 0 with

- validFrom
- Price for energy with a taxRate
- Price for chargingTime with a taxRate
- Price for idleTime with a taxRate
- Price for fixedFee with a taxRate
- Price for minCost with a taxRate
- Price for maxCost with a taxRate
- Price for reservationTime with a taxRate

2. The Test System responds with **SetDefaultTariffResponse** with **status Accepted**

1. The CSMS sends a **SetDefaultTariffRequest**

Main (Test scenario)	
<u>Manual Action:</u> Request the CSMS to set a default tariff on EVSE 1 with - validFrom - Price for energy with a taxRate - Price for chargingTime with a taxRate - Price for idleTime with a taxRate - Price for fixedFee with a taxRate - Price for minCost with a taxRate - Price for maxCost with a taxRate - Price for reservationFixed with a taxRate	
4. The Test System responds with SetDefaultTariffResponse with status Accepted	3. The CSMS sends a SetDefaultTariffRequest with
<u>Manual Action:</u> Request the CSMS to get the tariffs for EVSE 0	
6. The Test System responds with GetTariffsResponse with status Accepted tariffAssignments[0].tariffId <value of tariffId from step 1> tariffAssignments[0].tariffKind DefaultTariff tariffAssignments[0].evseIds [2] tariffAssignments[0].validFrom must be <not omitted> tariffAssignments[1].tariffId <value of tariffId from step 3> tariffAssignments[1].tariffKind DefaultTariff tariffAssignments[1].evseIds [1] tariffAssignments[1].validFrom must be <not omitted>	5. The CSMS sends a GetTariffsRequest

Tool validations

* Step 1:

Message **SetDefaultTariffRequest**

- **evseId** 0
- **tariff.tariffId** must be *<not omitted>*
- **tariff.validFrom** must be *<not omitted>*
- **tariff.energy** must be *<not omitted>*
- **tariff.energy.taxRates** must be *<not omitted>*
- **tariff.chargingTime** must be *<not omitted>*
- **tariff.chargingTime.taxRates** must be *<not omitted>*
- **tariff.idleTime** must be *<not omitted>*
- **tariff.idleTime.taxRates** must be *<not omitted>*
- **tariff.fixedFee** must be *<not omitted>*
- **tariff.fixedFee.taxRates** must be *<not omitted>*
- **tariff.minCost** must be *<not omitted>*
- **tariff.minCost.exclTax** or **tariff.minCost.inclTax** must be *<not omitted>* (at least one must exist)
- **tariff.minCost.taxRates** must be *<not omitted>*
- **tariff.maxCost** must be *<not omitted>*
- **tariff.maxCost.exclTax** or **tariff.maxCost.inclTax** must be *<not omitted>* (at least one must exist)
- **tariff.maxCost.taxRates** must be *<not omitted>*
- **tariff.reservationTime** must be *<not omitted>*
- **tariff.reservationTime.taxRates** must be *<not omitted>*

Tool validations

* Step 3:

Message **SetDefaultTariffRequest**

- **evseId** 1

- **tariff.tariffId** must be *<Different from tariffId step 1>*

- **tariff.validFrom** must be *<not omitted>*

- **tariff.energy** must be *<not omitted>*

- **tariff.energy.taxRates** must be *<not omitted>*

- **tariff.chargingTime** must be *<not omitted>*

- **tariff.chargingTime.taxRates** must be *<not omitted>*

- **tariff.idleTime** must be *<not omitted>*

- **tariff.idleTime.taxRates** must be *<not omitted>*

- **tariff.fixedFee** must be *<not omitted>*

- **tariff.fixedFee.taxRates** must be *<not omitted>*

- **tariff.minCost** must be *<not omitted>*

- **tariff.minCost.exclTax** must be *<not omitted>*

- **tariff.minCost.inclTax** must be *<not omitted>*

- **tariff.minCost.taxRates** must be *<not omitted>*

- **tariff.maxCost** must be *<not omitted>*

- **tariff.maxCost.exclTax** must be *<not omitted>*

- **tariff.maxCost.inclTax** must be *<not omitted>*

- **tariff.maxCost.taxRates** must be *<not omitted>*

- **tariff.reservationFixed** must be *<not omitted>*

- **tariff.reservationFixed.taxRates** must be *<not omitted>*

* Step 5:

Message **GetTariffsRequest**

- **evseId** must be 0

Post scenario validations:

N/a

TC_I_109_CSMS: Receive Driver Tariff - Goodflow

Test case name	Receive Driver Tariff - Goodflow
Test case Id	TC_I_109_CSMS
Use case Id(s)	<u>I08</u>
Requirement(s)	<u>I08.FR.01</u>
System under test	CSMS
Description	To support receiving the driver-specific tariff to enable local cost calculation based on a tariff for this driver.
Purpose	To verify if the CSMS supports driver tariffs.
Prerequisite(s)	- The CSMS supports local cost calculation by CS - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0

Before (Preparations)

Configuration State:

CSMS is configured to provide a driver specific tariff for <Configured valid_idtoken_idtoken> that specifies:

- currency: <not omitted>
- energy price: 0.25 per kWh (ex VAT), vat 20%
- idle price: 0.10 per minute (ex VAT), vat 20%
- fixed fee: 0.50 (ex VAT), vat 20%

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest with idToken.idToken <Configured valid_idtoken_idtoken>	2. The CSMS responds with AuthorizeResponse

Tool validations

* Step 2:

Message **AuthorizeResponse**

- **idTokenInfo.status** must be *Accepted*
- **tariff.tariffId** must be *<not omitted>*
- **tariff.currency** must be *<not omitted>*

- **tariff.energy.prices[0].priceKwh** must be *0.25*
- **tariff.energy.taxRates[0].tax** must be *20*
- **tariff.energy.taxRates[0].type** must be *VAT*

- **tariff.idleTime.prices[0].priceMinute** must be *0.10*
- **tariff.idleTime.taxRates[0].tax** must be *20*
- **tariff.idleTime.taxRates[0].type** must be *VAT*

- **tariff.fixedFee.prices[0].priceFixed** must be *0.50*
- **tariff.fixedFee.taxRates[0].tax** must be *20*
- **tariff.fixedFee.taxRates[0].type** must be *VAT*

Post scenario validations:

N/a

TC_I_110_CSMS: Clear Tariffs - DefaultTariff

Test case name	Clear Tariffs - DefaultTariff
Test case Id	TC_I_110_CSMS
Use case Id(s)	I10
Requirement(s)	...
System under test	Charging Station
Description	To clear previously set tariffs on the charging station.
Purpose	To verify if the CSMS is able to clear tariffs.
Prerequisite(s)	- The CSMS supports local cost calculation by CS - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to set a default tariff on EVSE 0.

2. The Test System responds with **SetDefaultTariffResponse** with **status Accepted**

1. The CSMS sends a **SetDefaultTariffRequest**

Manual Action: Request the CSMS to clear only the tariff set in step 1.

4. The Test System responds with **ClearTariffsResponse** with **clearTariffsResult[0].tariffId** <SetDefaultTariffRequest.tariff.tariffId of step 1>
clearTariffsResult[0].status Accepted

3. The CSMS sends a **ClearTariffsRequest**

Manual Action: Request the CSMS to clear all tariffs.

6. The Test System responds with **ClearTariffsResponse** with **clearTariffsResult[0].tariffId** <omitted>
clearTariffsResult[0].status NoTariff

5. The CSMS sends a **ClearTariffsRequest**

Tool validations

* Step 1:

Message **SetDefaultTariffRequest**

- **evseId** 0

- **tariff.tariffId** must be *<not omitted>*

* Step 3:

Message **ClearTariffsRequest**

- **tariffIds[0]** must be *_<SetDefaultTariffRequest.tariff.tariffId of step 1>*

* Step 5:

Message **ClearTariffsRequest**

- **tariffIds** must be *<omitted>*

Post scenario validations:

N/a

TC_I_113_CSMS: Local Cost Calculation - Change transaction tariff - TariffMaxElements

Test case name	Local Cost Calculation - Change transaction tariff - TariffMaxElements
Test case Id	TC_I_113_CSMS
Use case Id(s)	I11
Requirement(s)	...
System under test	CSMS
Description	CSMS changes the tariff that is associated with a transaction. This may be needed when dealing with unexpected price changes.
Purpose	To verify if the CSMS is able to process a response indicating TooManyElements.
Prerequisite(s)	- The CSMS supports local cost calculation by CS. - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *EnergyTransferStarted*Manual Action: Request the CSMS to change the tariff of the transaction

3. The Test System responds with **ChangeTransactionTariffResponse** with status *TooManyElements*

2. The CSMS sends a **ChangeTransactionTariffRequest** with

Tool validations

* Step 2:

Message **ChangeTransactionTariffRequest**- **transactionId** must be *<transaction id>*

Post scenario validations:

N/a

TC_I_114_CSMS: Local Cost Calculation - Change transaction tariff - ConditionNotSupported

Test case name	Local Cost Calculation - Change transaction tariff - TariffConditionsSupported is false
Test case Id	TC_I_114_CSMS
Use case Id(s)	I11
Requirement(s)	...
System under test	CSMS
Description	CSMS changes the tariff that is associated with a transaction. This may be needed when dealing with unexpected price changes.
Purpose	To verify if the CSMS is able to process a response indicating ConditionNotSupported.
Prerequisite(s)	- The CSMS supports local cost calculation by CS. - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0 - CS has TariffCtrlr.ConditionsSupported[Tariff] = true

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute Reusable State *EnergyTransferStarted*Manual Action: Request the CSMS to change the tariff of the transaction with

- validFrom

- Price for energy with condition on startTimeOfDay and endTimeOfDay

3. The Test System responds with **ChangeTransactionTariffResponse** with **status** *ConditionNotSupported*

2. The CSMS sends a **ChangeTransactionTariffRequest** with

Tool validations

* Step 2:

Message **ChangeTransactionTariffRequest**- **transactionId** must be <transaction id>- **tariff.validFrom** must be <not omitted>- **tariff.energy.prices[0].priceKwh** must be <not omitted>- **tariff.energy.prices[0].conditions.startTimeOfDay** must be in format as per RFC 3339: time-hour ":" time-minute- **tariff.energy.prices[0].conditions.endTimeOfDay** must be in format as per RFC 3339: time-hour ":" time-minute

Tool validations**Post scenario validations:**

N/a

TC_I_115_CSMS: Local Cost Calculation - Change transaction tariff - Good flow

Test case name	Local Cost Calculation - Change transaction tariff - TariffConditionsSupported is true
Test case Id	TC_I_115_CSMS
Use case Id(s)	I11
Requirement(s)	...
System under test	CSMS
Description	CSMS changes the tariff that is associated with a transaction. This may be needed when dealing with unexpected price changes.
Purpose	To verify if the CSMS is able to change the tariff of a transaction with a tariff with conditions.
Prerequisite(s)	- The CSMS supports local cost calculation by CS. - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.MaxElements[Tariff] > 0 - CS has TariffCtrlr.ConditionsSupported[Tariff] = true

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *EnergyTransferStarted*Manual Action: Request the CSMS to change the tariff of the transaction with- *validFrom*- *Price for energy with condition on startTimeOfDay and endTimeOfDay*

3. The Test System responds with **ChangeTransactionTariffResponse** with **status Accepted**

2. The CSMS sends a **ChangeTransactionTariffRequest** with

4. The Test System sends a **TransactionEventRequest** with **eventType Updated**
triggerReason TariffChanged
transactionInfo.tariffId <tariff.tariffId from step 2>

5. The CSMS responds with a **TransactionEventResponse**

Tool validations

* Step 2:

Message **ChangeTransactionTariffRequest**

- **transactionId** must be *<transaction id>*
- **tariff.validFrom** must be *<not omitted>*
- **tariff.energy.prices[0].priceKwh** must be *<not omitted>*
- **tariff.energy.prices[0].conditions.startTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*
- **tariff.energy.prices[0].conditions.endTimeOfDay** must be in *format as per RFC 3339: time-hour ":" time-minute*

Post scenario validations:

N/a

TC_I_122_CSMS: Local Cost Calculation - Cost Details of Transaction

Test case name	Local Cost Calculation - Cost Details of Transaction
Test case Id	TC_I_122_CSMS
Use case Id(s)	I12
Requirement(s)	...
System under test	CSMS
Description	Charging Station calculates cost of the transaction locally and returns a break-down of the cost at end of the transaction for every charging period for which a different tariff element was active. CSMS can use this to generate invoices or Charge Detail Records for EMSPs.
Purpose	To verify if the CSMS is able to process the cost details sent by the Charging Station.
Prerequisite(s)	- The CSMS supports local cost calculation by CS. - CS has TariffCtrlr.Enabled[Tariff] = true - CS has TariffCtrlr.Enabled[Cost] = true - CS has TariffCtrlr.Enabled[RunningCost] = true - CS has TariffCtrlr.Interval[Cost] = 30 - CS has TariffCtrlr.MaxElements[Tariff] > 0 - CS has TariffCtrlr.ConditionsSupported[Tariff] = true

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to set a default tariff on EVSE 0 with

- Price for energy with a taxRate
- Price for chargingTime with a taxRate
- Price for idleTime with a taxRate
- Price for fixedFee with a taxRate
- Price for minCost with a taxRate
- Price for maxCost with a taxRate

2. The Test System responds with **SetDefaultTariffResponse** with **status Accepted**

1. The CSMS sends a **SetDefaultTariffRequest**

3. Execute **Reusable State** *EnergyTransferStarted*

Main (Test scenario)

4. The Test System sends a **TransactionEventRequest** with
eventType *Updated*

triggerReason *RunningCost*

costDetails.failureToCalculate *<omitted>*

costDetails.failureReason *<omitted>*

costDetails.totalCost.currency *EUR*

costDetails.totalCost.typeOfCost *NormalCost*

costDetails.totalCost.reservation *<omitted>*

costDetails.totalCost.fixed.exclTax *15*

costDetails.totalCost.fixed.inclTax *15.75*

costDetails.totalCost.fixed.taxRates[0].type *t5_0*

costDetails.totalCost.fixed.taxRates[0].tax *5.0*

costDetails.chargingPeriods[0].tariffId

<SetDefaultTariffRequest.tariff.tariffId of step 1>

costDetails.chargingPeriods[0].startPeriod *<txStartDateTime>*

costDetails.chargingPeriods[0].dimensions[0].type

ChargingTime

costDetails.chargingPeriods[0].dimensions[0].volume *15*

costDetails.chargingPeriods[0].dimensions[1].type *Energy*

costDetails.chargingPeriods[0].dimensions[1].volume *12*

5. The CSMS responds with a **TransactionEventResponse**

Tool validations

* Step 1:

Message **SetDefaultTariffRequest**

- **evseId** *0*

- **tariff.energy** must be *<not omitted>*

- **tariff.energy.taxRates** must be *<not omitted>*

- **tariff.chargingTime** must be *<not omitted>*

- **tariff.chargingTime.taxRates** must be *<not omitted>*

- **tariff.idleTime** must be *<not omitted>*

- **tariff.idleTime.taxRates** must be *<not omitted>*

- **tariff.fixedFee** must be *<not omitted>*

- **tariff.fixedFee.taxRates** must be *<not omitted>*

- **tariff.minCost** must be *<not omitted>*

- **tariff.minCost.exclTax** must be *<not omitted>*

- **tariff.minCost.inclTax** must be *<not omitted>*

- **tariff.minCost.taxRates** must be *<not omitted>*

- **tariff.maxCost** must be *<not omitted>*

- **tariff.maxCost.exclTax** must be *<not omitted>*

- **tariff.maxCost.inclTax** must be *<not omitted>*

- **tariff.maxCost.taxRates** must be *<not omitted>*

Post scenario validations:

N/a

11.J Meter Values

TC_J_01_CSMS: Clock-aligned Meter Values - No transaction ongoing

Test case name	Clock-aligned Meter Values - No transaction ongoing
Test case Id	TC_J_01_CSMS
Use case Id(s)	J01
Requirement(s)	J01.FR.18
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending clock-aligned Meter Values, when there is no ongoing transaction.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
-------------------------	-------------

Main (Test scenario)

1. The Test System notifies the CSMS about its measured Meter Values.

Message: **MeterValuesRequest**

- **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the _<Configured clock-aligned Meter Values interval>.

- **sampledValue.context** is *Sample.Clock*

Message: **NotifyEventRequest**

- **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the _<Configured clock-aligned Meter Values interval>.

- **trigger** is *Periodic*

- **component.name** is *FiscalMetering*

Note(s):

- This step will be executed every _<Configured clock-aligned Meter Values interval>

- This step will be executed for evseId=0 and all configured EVSE.

- The Test System will end the testcase after it has send three Meter Value messages.

2. The CSMS responds accordingly.

Tool validations

N/a

Post scenario validations:

N/a

TC_J_02_CSMS: Clock-aligned Meter Values - Transaction ongoing

Test case name	Clock-aligned Meter Values - Transaction ongoing
Test case Id	TC_J_02_CSMS
Use case Id(s)	J01
Requirement(s)	J01.FR.18
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending clock-aligned Meter Values, when there is an ongoing transaction.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted* for <Configured evseId>

Main (Test scenario)

Charging Station

1. The Test System notifies the CSMS about its measured Meter Values.

Message: **MeterValuesRequest**

- **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the configured clock-aligned Meter Values interval>.

- **sampledValue.context** is *Sample.Clock*

Message: **NotifyEventRequest**

- **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the configured clock-aligned Meter Values interval>.

- **trigger** is *Periodic*

- **component.name** is *FiscalMetering*

Note(s):

- This step will be executed every <Configured clock-aligned Meter Values interval>

- This step will be executed for evseId=0 and all configured idle EVSE.

CSMS

2. The CSMS responds accordingly.

Main (Test scenario)	
3. The Test System sends a TransactionEventRequest With triggerReason is <i>MeterValueClock</i> eventType is <i>Updated</i> timestamp <i><The intervals between the timestamps of the received Meter Value messages equals the configured clock-aligned Meter Values interval></i>. sampledValue.context is <i>Sample.Clock</i> <u>Note(s):</u> - This step will be executed every _<i><Configured clock-aligned Meter Values interval></i> - The Test System will end the testcase after the _<i><Configured transaction duration></i> is reached._	4. The CSMS responds with a TransactionEventResponse
Tool validations	
N/a	
Post scenario validations:	
N/a	

TC_J_03_CSMS: Clock-aligned Meter Values - EventType Ended

Test case name	Clock-aligned Meter Values - EventType Ended
Test case Id	TC_J_03_CSMS
Use case Id(s)	J01
Requirement(s)	J01.FR.18
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending clock-aligned Meter Values, when a transaction ends.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

1. Execute Reusable State *EVDIsconnected*

- The **TransactionEventRequest** containing *eventType Ended* contains the *MeterValue* field.
- **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the configured value at configured clock-aligned Tx ended Meter Values interval>.
- **sampledValue.context** is *Sample.Clock* AND the last one has *Transaction.End*

Note(s):

- This step will be executed after the _<Configured transaction duration> is reached._
- This causes the transaction to stop.

Tool validations

N/a

Post scenario validations:

N/a

TC_J_04_CSMS: Clock-aligned Meter Values - Signed

Test case name	Clock-aligned Meter Values - Signed
Test case Id	TC_J_04_CSMS
Use case Id(s)	J01
Requirement(s)	J01.FR.21
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending clock-aligned Meter Values, when a transaction ends.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

1. Execute Reusable State *EVDIsconnected*

- The **TransactionEventRequest** containing eventType *Ended* contains the MeterValue field.
- **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the configured value at configured clock-aligned Tx ended Meter Values interval>.
- **sampledValue.context** is *Sample.Clock* AND the last one has *Transaction.End*
- **sampledValue.signedMeterValue** is <Generated SignedMeterValueType>

Note(s):

- This step will be executed after the _<Configured transaction duration> is reached._
- This causes the transaction to stop.

Tool validations

N/a

Post scenario validations:

N/a

TC_J_07_CSMS: Sampled Meter Values - EventType Started - EVSE known

Test case name	Sampled Meter Values - EventType Started - EVSE known
Test case Id	TC_J_07_CSMS
Use case Id(s)	J02
Requirement(s)	J02.FR.19
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending start sampled Meter Values, when a transaction starts.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute Reusable State *EVConnectedPreSession*

- The **TransactionEventRequest** contains the MeterValue field.
- **sampledValue.context** is *Transaction.Begin*

Tool validations

N/a

Post scenario validations:

N/a

TC_J_08_CSMS: Sampled Meter Values - Context Transaction.Begin - EVSE not known

Test case name	Sampled Meter Values - Context Transaction.Begin - EVSE not known
Test case Id	TC_J_08_CSMS
Use case Id(s)	J02
Requirement(s)	J02.FR.19
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending start sampled Meter Values, when a transaction starts.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute Reusable State *Authorized*2. Execute Reusable State *EVConnectedPreSession*- The **TransactionEventRequest** contains the MeterValue field.- **sampledValue.context** is *Transaction.Begin*3. Execute Reusable State *EnergyTransferStarted*

Tool validations

N/a

Post scenario validations:

N/a

TC_J_09_CSMS: Sampled Meter Values - EventType Updated

Test case name	Sampled Meter Values - EventType Updated
Test case Id	TC_J_09_CSMS
Use case Id(s)	J02
Requirement(s)	J02.FR.19
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending sampled Meter Values, when there is an ongoing transaction.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

1. The Test System sends a **TransactionEventRequest** With **triggerReason** is *MeterValuePeriodic* **eventType** is *Updated* **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the configured sampled Meter Values interval>. **sampledValue.context** is *Sample.Periodic*

Note(s):

- This step will be executed every _<Configured sampled Meter Values interval>
- The Test System will end the testcase after three MeterValues.

CSMS

2. The CSMS responds with a **TransactionEventResponse**

Tool validations

N/a

Post scenario validations:

N/a

TC_J_10_CSMS: Sampled Meter Values - EventType Ended

Test case name	Sampled Meter Values - EventType Ended
Test case Id	TC_J_10_CSMS
Use case Id(s)	J02
Requirement(s)	J02.FR.19
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending sampled Meter Values, when a transaction ends.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

1. Execute Reusable State *EVDisconnected*

- The **TransactionEventRequest** containing eventType *Ended* contains the MeterValue field.
- **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the configured value at configured clock-aligned Tx ended Meter Values interval>.
- **sampledValue.context** is *Sample.Periodic* AND the last one has *Transaction.End*

Note(s):

- This step will be executed after the _<Configured transaction duration> is reached._
- This causes the transaction to stop.

Tool validations

N/a

Post scenario validations:

N/a

TC_J_11_CSMS: Sampled Meter Values - Signed

Test case name	Sampled Meter Values - Signed
Test case Id	TC_J_11_CSMS
Use case Id(s)	J02
Requirement(s)	J02.FR.21
System under test	CSMS
Description	The Charging Station samples the electrical meter or other sensor/transducer hardware to provide information about its Meter Values. Depending on configuration settings, the Charging Station will send Meter Values.
Purpose	To verify if the CSMS is able to handle a Charging Station sending sampled Meter Values, when a transaction ends.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

1. Execute Reusable State *EVDDisconnected*

- The **TransactionEventRequest** containing eventType *Ended* contains the MeterValue field.
- **timestamp** <The intervals between the timestamps of the received Meter Value messages equals the configured value at configured clock-aligned Tx ended Meter Values interval>.
- **sampledValue.context** is *Sample.Periodic* AND the last one has *Transaction.End*
- **sampledValue.signedMeterValue** is <Generated SignedMeterValueType>

Note(s):

- This step will be executed after the _<Configured transaction duration> is reached._
- This causes the transaction to stop.

Tool validations

N/a

Post scenario validations:

N/a

12. K Smart Charging

TC_K_01_CSMS: Set Charging Profile - TxDefaultProfile - Specific EVSE

Test case name	Set Charging Profile - TxDefaultProfile - Specific EVSE
Test case Id	TC_K_01_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	<u>K01.FR.31</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to send a TxDefaultProfile charging profile for a specific EVSE as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse with status Accepted	1. The CSMS sends a SetChargingProfileRequest with - chargingProfile.id <Configured chargingProfileId>

Tool validations

* Step 1:

Message **SetChargingProfileRequest**

evseId <Configured evseId> AND

chargingProfile.stackLevel <Configured stackLevel> AND

chargingProfile.chargingProfilePurpose TxDefaultProfile AND

chargingProfile.chargingProfileKind Absolute AND

chargingProfile.validFrom now AND

chargingProfile.validTo now + <Configured Charging Schedule Duration> AND

chargingProfile.chargingSchedule.startSchedule now AND

chargingProfile.chargingSchedule.chargingRateUnit <Configured chargingRateUnit> AND

chargingProfile.chargingSchedule.duration <Configured duration> AND

chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod 0 AND

chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 6.0 or 6000.0 AND

chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases> where <Configured numberPhases> not 3 OR

chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases> or <omit> where <Configured numberPhases> 3

Post scenario validations:

- N/a

TC_K_02_CSMS: Set Charging Profile - TxProfile without ongoing transaction on the specified EVSE

Test case name	Set Charging Profile - TxProfile without ongoing transaction on the specified EVSE
Test case Id	TC_K_02_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	N/a
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to send a TxProfile and read the charger's feedback while no transaction is ongoing for a specific EVSE as described at the OCPP specification.
Prerequisite(s)	If the CSMS supports sending a TxProfile while there is no transaction ongoing.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse with status Rejected	1. The CSMS sends a SetChargingProfileRequest - chargingProfile.id <Configured chargingProfileId>

Tool validations

* Step 1:

Message **SetChargingProfileRequest**

- **evseId** <Configured evseId> AND
- **chargingProfile.chargingProfilePurpose** TxProfile AND
- **chargingProfile.stackLevel** <Configured stackLevel> AND
- **chargingProfile.chargingProfileKind** Relative AND
- **chargingProfile.chargingSchedule.startSchedule** must be omitted AND
- **chargingProfile.chargingSchedule.chargingRateUnit** <Configured chargingRateUnit> AND
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod** 0 AND
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.limit** 7.0 or 7000.0 AND
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases** <Configured numberPhases> where <Configured numberPhases> not 3 OR
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases** <Configured numberPhases> or <omit> where <Configured numberPhases> 3

Post scenario validations:

- N/a

TC_K_03_CSMS: Set Charging Profile - ChargingStationMaxProfile

Test case name	Set Charging Profile - ChargingStationMaxProfile
Test case Id	TC_K_03_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	<u>K01.FR.31</u> , <u>K01.FR.38</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to send a ChargingStationMaxProfile charging profile as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse with status Accepted	1. The CSMS sends a SetChargingProfileRequest - chargingProfile.id <Configured chargingProfileId>

Tool validations
* Step 1: Message SetChargingProfileRequest evseId 0 AND chargingProfile.stackLevel <Configured stackLevel> AND chargingProfile.chargingProfilePurpose <i>ChargingStationMaxProfile</i> AND chargingProfile.chargingProfileKind <i>Absolute</i> chargingProfile.chargingSchedule.chargingRateUnit <Configured ChargingRateUnit> chargingProfile.chargingSchedule.duration <Configured duration> chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod 0 chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 8.0 or 8000.0 chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases> where <Configured numberPhases> not 3 OR chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases> or <omit> where <Configured numberPhases> 3 chargingProfile.chargingSchedule.startSchedule <Not omitted>
Post scenario validations: - N/a

TC_K_04_CSMS: Replace charging profile - With chargingProfileId

Test case name	Replace charging profile - With chargingProfileId
Test case Id	TC_K_04_CSMS
Use case Id(s)	n/a
Requirement(s)	n/a
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to replace a charging profile with the same ProfileKind, Purpose, and stackLevel, but a different limit.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse with status Accepted	1. The CSMS sends a SetChargingProfileRequest with chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 8.0 or 8000.0
4. The Test System responds with a SetChargingProfileResponse with status Accepted	3. The CSMS sends a SetChargingProfileRequest with chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 6.0 or 6000.0

Tool validations

* Step 1:

Message **SetChargingProfileRequest**

chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod 0 AND

chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 8.0 (A) or 8000.0 (W)

The **chargingSchedule** contains only one **chargingSchedulePeriod**

* Step 3:

Message **SetChargingProfileRequest**

chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod 0 AND

chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 6.0 (A) or 6000.0 (W)

The **chargingSchedule** contains only one **chargingSchedulePeriod**

* Step 1/3:

Message **SetChargingProfileRequest**

chargingProfile.id <Same id for both chargingProfiles>

chargingProfile.chargingSchedule.startSchedule must NOT be omitted.

chargingProfile.stackLevel <Configured stackLevel> AND

chargingProfile.chargingProfilePurpose <Equal value for both profiles> AND (TxDefaultProfile OR ChargingStationMaxProfile)

chargingProfile.chargingProfileKind <Equal value for both profiles> AND

If **chargingProfile.chargingProfilePurpose** is TxDefaultProfile then **chargingProfile.chargingProfileKind** must be Absolute OR Recurring

If **chargingProfile.chargingProfilePurpose** is ChargingStationMaxProfile then **chargingProfile.chargingProfileKind** must be Absolute

If **chargingProfile.chargingProfileKind** is Recurring then **chargingProfile.recurrencyKind** must NOT be omitted, else omitted

The received Charging Profiles must comply with the requirements defined at part 2 specification.

Post scenario validations:

- N/a

TC_K_05_CSMS: Clear Charging Profile - With chargingProfileId

Test case name	Clear Charging Profile - With chargingProfileId
Test case Id	TC_K_05_CSMS
Use case Id(s)	<u>K10</u>
Requirement(s)	<u>K10.FR.02</u>
System under test	CSMS
Description	If the CSMS wishes to clear some or all of the charging profiles that were previously sent to the Charging Station, then the CSMS sends a ClearChargingProfileRequest to the Charging Station.
Purpose	To verify if the CSMS is able to request the charging station to clear a specific charging profile (not TxDefault) with only a chargingProfileId as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

CSMS sends a **GetChargingProfilesRequest**Test System responds with a **GetChargingProfilesResponse** with status *Accepted*Test System sends a **ReportChargingProfilesRequest**CSMS responds with a **ReportChargingProfilesResponse**

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a ClearChargingProfileResponse with status <i>Accepted</i>	1. The CSMS sends a ClearChargingProfileRequest with chargingProfileId <Generated chargingProfileId> AND chargingProfileCriteria omit

Tool validations

* Step 1:

Message **ClearChargingProfileRequest****chargingProfileId** <Generated chargingProfileId> AND **chargingProfileCriteria** omit

Post scenario validations:

- N/a

TC_K_06_CSMS: Clear Charging Profile - With stackLevel/purpose combination for one profile

Test case name	Clear Charging Profile - With stackLevel/purpose combination for one profile
Test case Id	TC_K_06_CSMS
Use case Id(s)	<u>K10</u>
Requirement(s)	<u>K10.FR.02</u>
System under test	CSMS
Description	If the CSMS wishes to clear some or all of the charging profiles that were previously sent to the Charging Station, then the CSMS sends a ClearChargingProfileRequest to the Charging Station.
Purpose	To verify if the CSMS is able to request the charging station to clear a specific charging profile with a stackLevel/purpose combination for a chargingProfileId as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a ClearChargingProfileResponse with status <i>Accepted</i>	1. The CSMS sends a ClearChargingProfileRequest with chargingProfilePurpose <i>TxDefaultProfile</i> AND evseId <i><Configured evseId></i> AND stackLevel <i><Configured stackLevel></i>

Tool validations

* Step 1:

Message **ClearChargingProfileRequest**
chargingProfileCriteria.chargingProfilePurpose *TxDefaultProfile* AND
chargingProfileCriteria.stackLevel *<Configured stackLevel>* AND
chargingProfileCriteria.evseId *<Configured evseId>* AND
chargingProfileId must be omitted.

Post scenario validations:

- N/a

TC_K_08_CSMS: Clear Charging Profile - Without previous charging profile

Test case name	Clear Charging Profile - Without previous charging profile
Test case Id	TC_K_08_CSMS
Use case Id(s)	<u>K10</u>
Requirement(s)	N/a
System under test	CSMS
Description	If the CSMS wishes to clear some or all of the charging profiles that were previously sent to the Charging Station, then the CSMS sends a ClearChargingProfileRequest to the Charging Station.
Purpose	To verify if the CSMS is able to request the charging station to clear a specific charging profile with a chargingProfileId and stackLevel /purpose combination while the Charging stations does not accept as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a ClearChargingProfileResponse with status <i>Unknown</i>	1. The CSMS sends a ClearChargingProfileRequest with chargingProfilePurpose <i>TxDefaultProfile</i> AND evseId <i><Configured evseId></i> AND stackLevel <i><Configured stackLevel></i>

Tool validations

* Step 1:

Message **ClearChargingProfileRequest**
chargingProfileCriteria.chargingProfilePurpose *TxDefaultProfile* AND
chargingProfileCriteria.stackLevel *<Configured stackLevel>* AND
chargingProfileCriteria.evseId *<Configured evseId>* AND
chargingProfileId must be omitted.

Post scenario validations:

- N/a

TC_K_10_CSMS: Set Charging Profile - TxDefaultProfile - All EVSE

Test case name	Set Charging Profile - TxDefaultProfile - All EVSE
Test case Id	TC_K_10_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	<u>K01.FR.31</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to send a TxDefaultProfile charging profile for all EVSE as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse with status Accepted	1. The CSMS sends a SetChargingProfileRequest with - chargingProfile.id <Configured chargingProfileId>

Tool validations
* Step 1: Message SetChargingProfileRequest evseId 0 AND chargingProfile.stackLevel <Configured stackLevel> AND chargingProfile.chargingProfilePurpose TxDefaultProfile AND chargingProfile.chargingProfileKind Absolute AND chargingProfile.chargingSchedule.startSchedule <Not omitted> AND chargingProfile.chargingSchedule.chargingRateUnit <Configured ChargingRateUnit> AND chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod 0 AND chargingProfile.chargingSchedule.duration <Configured duration> chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 6.0 or 6000.0 AND chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases> where <Configured numberPhases> not 3 OR chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases> or <omit> where <Configured numberPhases> 3
Post scenario validations: - N/a

TC_K_15_CSMS: Set Charging Profile - Not Supported

Test case name	Set Charging Profile - Not Supported
Test case Id	TC_K_15_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	N/a
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to send a Profile, while the charging station does not support chargingprofiles, and read the response as described at the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with RPC Framework: CALLERROR: NotSupported.	1. The CSMS sends a SetChargingProfileRequest with: evseId <Configured evseId> AND chargingProfile.stackLevel <Configured stackLevel> AND chargingProfile.chargingProfilePurpose TxDefaultProfile AND chargingProfile.chargingProfileKind Absolute AND chargingProfile.chargingSchedule.startSchedule <Not omitted> AND chargingProfile.chargingSchedule.chargingRateUnit <Configured ChargingRateUnit> AND chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod 0 AND chargingProfile.chargingSchedule.duration <Configured duration> chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 6.0 or 6000.0 AND chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases> where <Configured numberPhases> not 3 OR chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases> or <omit> where <Configured numberPhases> 3

Tool validations
* Step 1: Message SetChargingProfileRequest evseId <Configured evseId> AND chargingProfile.stackLevel <Configured stackLevel> AND chargingProfile.chargingProfilePurpose <i>TxDefaultProfile</i> AND chargingProfile.chargingProfileKind <i>Absolute</i> AND chargingProfile.chargingSchedule.startSchedule <Not omitted> AND chargingProfile.chargingSchedule.chargingRateUnit <Configured ChargingRateUnit> AND chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod 0 AND chargingProfile.chargingSchedule.duration <Configured duration> chargingProfile.chargingSchedule.chargingSchedulePeriod.limit 6.0 or 6000.0 AND chargingProfile.chargingSchedule.chargingSchedulePeriod.numberPhases <Configured numberPhases>
Post scenario validations: - N/a

TC_K_19_CSMS: Set Charging Profile - ChargingProfileKind is Recurring

Test case name	Set Charging Profile - ChargingProfileKind is Recurring
Test case Id	TC_K_19_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	N/a
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to send a Profile with a recurrencyKind specified as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse with - status <i>Accepted</i>	1. The CSMS sends a SetChargingProfileRequest

Tool validations*** Step 1:**Message **SetChargingProfileRequest**

- **evseId** <Configured evseId> AND
- **chargingProfile.stackLevel** <Configured stackLevel> AND
- **chargingProfile.chargingProfilePurpose** *TxDefaultProfile* AND
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.startPeriod** 0 AND
- **chargingProfile.chargingProfileKind** *Recurring* AND
- **chargingProfile.recurrencyKind** <Configured recurrencyKind>

Post scenario validations:

- N/a

TC_K_29_CSMS: Get Charging Profile - EvseId 0

Test case name	Get Charging Profile - EvseId 0
Test case Id	TC_K_29_CSMS
Use case Id(s)	<u>K09</u>
Requirement(s)	<u>K09.FR.03</u>
System under test	CSMS
Description	With the GetChargingProfilesRequest message the CSMS can ask a Charging Station to report all, or a subset of all the install Charging Profiles from the different possible sources. This can be used for some automatic smart charging control system, or for debug purposes by a CSO.
Purpose	To verify if the CSMS is able to request charging profiles installed on the charging station itself and read in the reports as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:*EnergyTransferStarted***Main (Test scenario)**

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest with - evseId 0
3. The Test System sends a ReportChargingProfilesRequest with - requestId <Received requestId>	4. The CSMS responds with a ReportChargingProfilesResponse

Tool validations

* Step 1:

Message **GetChargingProfilesRequest**- **evseId** 0 AND- **chargingProfile.chargingProfilePurpose** <Configured chargingProfilePurpose> ANDNote: chargingProfilePurpose is included, because the chargingProfile field is required and may not be left empty.- **chargingProfile.stackLevel** must be omitted AND- **chargingProfile.chargingLimitSource** must be omitted AND- **chargingProfile.chargingProfileId** must be omitted**Post scenario validations:**

- N/a

DRAFT

TC_K_30_CSMS: Get Charging Profile - EvseId > 0

Test case name	Get Charging Profile - EvseId > 0
Test case Id	TC_K_30_CSMS
Use case Id(s)	<u>K09</u>
Requirement(s)	<u>K09.FR.03</u>
System under test	CSMS
Description	With the GetChargingProfilesRequest message the CSMS can ask a Charging Station to report all, or a subset of all the install Charging Profiles from the different possible sources. This can be used for some automatic smart charging control system, or for debug purposes by a CSO.
Purpose	To verify if the CSMS is able to request charging profiles installed on a specific EVSE and read in the reports as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest
3. The Test System sends a ReportChargingProfilesRequest with - requestId <i><Received requestId></i>	4. The CSMS responds with a ReportChargingProfilesResponse

Tool validations

* Step 1:

Message **GetChargingProfilesRequest**- **evseId** *<Configured evseId>* AND- **chargingProfile.chargingProfilePurpose** *<Configured chargingProfilePurpose>* ANDNote: *chargingProfilePurpose* is included, because the *chargingProfile* field is required and may not be left empty.- **chargingProfile.stackLevel** must be omitted AND- **chargingProfile.chargingLimitSource** must be omitted AND- **chargingProfile.chargingProfileId** must be omitted

Post scenario validations:

- N/a

DRAFT

TC_K_31_CSMS: Get Charging Profile - No EvseId

Test case name	Get Charging Profile - No EvseId
Test case Id	TC_K_31_CSMS
Use case Id(s)	<u>K09</u>
Requirement(s)	<u>K09.FR.03</u>
System under test	CSMS
Description	With the GetChargingProfilesRequest message the CSMS can ask a Charging Station to report all, or a subset of all the install Charging Profiles from the different possible sources. This can be used for some automatic smart charging control system, or for debug purposes by a CSO.
Purpose	To verify if the CSMS is able to request all charging profiles installed on a charger and read in the reports as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest with - requestId <i><Received requestId></i>
3. The Test System sends a ReportChargingProfilesRequest with - requestId <i><Received requestId></i> AND - tbtc <i>true</i> AND - evseId <i>i</i>	4. The CSMS responds with a ReportChargingProfilesResponse
Note(s): - Step 3 and 4 are repeated for every evse	

Tool validations

* Step 1:

Message **GetChargingProfilesRequest**

- **evseId** must be omitted AND
- **chargingProfile.chargingProfilePurpose** <Configured chargingProfilePurpose> AND

Note: *chargingProfilePurpose* is included, because the *chargingProfile* field is required and may not be left empty.

- **chargingProfile.stackLevel** must be omitted AND
- **chargingProfile.chargingLimitSource** must be omitted AND
- **chargingProfile.chargingProfileId** must be omitted

Post scenario validations:

- N/a

TC_K_32_CSMS: Get Charging Profile - chargingProfileId

Test case name	Get Charging Profile - chargingProfileId
Test case Id	TC_K_32_CSMS
Use case Id(s)	<u>K09</u>
Requirement(s)	<u>K09.FR.03</u>
System under test	CSMS
Description	With the GetChargingProfilesRequest message the CSMS can ask a Charging Station to report all, or a subset of all the install Charging Profiles from the different possible sources. This can be used for some automatic smart charging control system, or for debug purposes by a CSO.
Purpose	To verify if the CSMS is able to request a specific charging profile and read in the reports as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest - chargingProfileId <i><Received chargingProfileId></i>
3. The Test System sends a ReportChargingProfilesRequest with - requestId <i>Generated Id</i>	4. The CSMS responds with a ReportChargingProfilesResponse

Tool validations

* Step 1:

Message **GetChargingProfilesRequest**

- **evseId** must be omitted AND
- **chargingProfile.chargingProfilePurpose** must be omitted AND
- **chargingProfile.stackLevel** must be omitted AND
- **chargingProfile.chargingLimitSource** must be omitted AND
- **chargingProfile.chargingProfileId** *<received chargingProfileId>*

Post scenario validations:

- N/a

TC_K_33_CSMS: Get Charging Profile - EvseId > 0 + stackLevel

Test case name	Get Charging Profile - EvseId > 0 + stackLevel
Test case Id	TC_K_33_CSMS
Use case Id(s)	<u>K09</u>
Requirement(s)	<u>K09.FR.03</u>
System under test	CSMS
Description	With the GetChargingProfilesRequest message the CSMS can ask a Charging Station to report all, or a subset of all the install Charging Profiles from the different possible sources. This can be used for some automatic smart charging control system, or for debug purposes by a CSO.
Purpose	To verify if the CSMS is able to request charging profiles with a specific stackLevel installed on a specific EVSE and read in the reports as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest
3. The Test System sends a ReportChargingProfilesRequest with - requestId <i>Generated Id</i>	4. The CSMS responds with a ReportChargingProfilesResponse

Tool validations

* Step 1:

Message **GetChargingProfilesRequest**

- **evseId** <Configured evseId> AND
- **chargingProfile.stackLevel** <Configured stackLevel> AND
- **chargingProfile.chargingProfilePurpose** must be omitted AND
- **chargingProfile.chargingLimitSource** must be omitted AND
- **chargingProfile.chargingProfileId** must be omitted

Post scenario validations:

- N/a

TC_K_34_CSMS: Get Charging Profile - EvseId > 0 + chargingLimitSource

Test case name	Get Charging Profile - EvseId > 0 + chargingLimitSource
Test case Id	TC_K_34_CSMS
Use case Id(s)	<u>K09</u>
Requirement(s)	<u>K09.FR.03</u>
System under test	CSMS
Description	With the GetChargingProfilesRequest message the CSMS can ask a Charging Station to report all, or a subset of all the install Charging Profiles from the different possible sources. This can be used for some automatic smart charging control system, or for debug purposes by a CSO.
Purpose	To verify if the CSMS is able to request charging profiles with a specific chargingLimitSource installed on a specific EVSE and read in the reports as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest
3. The Test System sends a ReportChargingProfilesRequest with - requestId <i>Generated Id</i>	4. The CSMS responds with a ReportChargingProfilesResponse

Tool validations

* Step 1:

Message **GetChargingProfilesRequest**- **evseId** <Configured evseId> AND- **chargingProfile.chargingLimitSource** <Configured chargingLimitSource> AND- **chargingProfile.stackLevel** must be omitted AND- **chargingProfile.chargingProfilePurpose** must be omitted AND- **chargingProfile.chargingProfileId** must be omitted

Post scenario validations:

- N/a

TC_K_35_CSMS: Get Charging Profile - EvseId > 0 + chargingProfilePurpose

Test case name	Get Charging Profile - EvseId > 0 + chargingProfilePurpose
Test case Id	TC_K_35_CSMS
Use case Id(s)	<u>K09</u>
Requirement(s)	<u>K09.FR.03</u>
System under test	CSMS
Description	With the GetChargingProfilesRequest message the CSMS can ask a Charging Station to report all, or a subset of all the install Charging Profiles from the different possible sources. This can be used for some automatic smart charging control system, or for debug purposes by a CSO.
Purpose	To verify if the CSMS is able to request charging profiles with a specific chargingProfilePurpose installed on a specific EVSE and read in the reports as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest
3. The Test System sends a ReportChargingProfilesRequest with - requestId <i>Generated Id</i>	4. The CSMS responds with a ReportChargingProfilesResponse

Tool validations

* Step 1:

Message **GetChargingProfilesRequest**

- **evseId** <Configured evseId> AND
- **chargingProfile.chargingProfilePurpose** <Configured chargingProfilePurpose> AND
- **chargingProfile.stackLevel** must be omitted AND
- **chargingProfile.chargingLimitSource** must be omitted AND
- **chargingProfile.chargingProfileId** must be omitted

Post scenario validations:

- N/a

TC_K_36_CSMS: Get Charging Profile - EvseId > 0 + chargingProfilePurpose + stackLevel

Test case name	Get Charging Profile - EvseId > 0 + chargingProfilePurpose + stackLevel
Test case Id	TC_K_36_CSMS
Use case Id(s)	<u>K09</u>
Requirement(s)	<u>K09.FR.03</u>
System under test	CSMS
Description	With the GetChargingProfilesRequest message the CSMS can ask a Charging Station to report all, or a subset of all the install Charging Profiles from the different possible sources. This can be used for some automatic smart charging control system, or for debug purposes by a CSO.
Purpose	To verify if the CSMS is able to request charging profiles with a specific chargingProfilePurpose AND stackLevel installed on a specific EVSE and read in the reports as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest
3. The Test System sends a ReportChargingProfilesRequest with - requestId <i>Generated Id</i>	4. The CSMS responds with a ReportChargingProfilesResponse

Tool validations

* Step 1:

Message **GetChargingProfilesRequest**

- **evseId** <Configured evseId> AND
- **chargingProfile.chargingProfilePurpose** <Configured chargingProfilePurpose> AND
- **chargingProfile.chargingLimitSource** must be omitted AND
- **chargingProfile.stackLevel** <Configured stackLevel> AND
- **chargingProfile.chargingProfileId** must be omitted

Post scenario validations:

- N/a

TC_K_60_CSMS: Set Charging Profile - TxProfile with ongoing transaction on the specified EVSE

Test case name	Set Charging Profile - TxProfile with ongoing transaction on the specified EVSE
Test case Id	TC_K_60_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	<u>K01.FR.03</u> , <u>K01.FR.31</u>
System under test	CSMS
Description	The CSMS sets a TxProfile on a specific EVSE for a currently ongoing transaction.
Purpose	To verify if the CSMS is able to exchange messages to set a TxProfile on a specific EVSE for a currently ongoing transaction.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse With status is <i>Accepted</i>	1. The CSMS sends a SetChargingProfileRequest

Tool validations

* Step 1:

(Message: **SetChargingProfileRequest**)**chargingProfile.chargingProfilePurpose** is *TxProfile* AND**chargingProfile.evseId** is *<Configured evseId>* AND**chargingProfile.transactionId** *<Generated transactionId>* AND**chargingProfile.chargingProfileKind** is *Relative* OR *Absolute*If **chargingProfileKind** is *Relative* then **chargingSchedule.startSchedule** must be omitted.If **chargingProfileKind** is *Absolute* then **chargingSchedule.startSchedule** must NOT be omitted.

The received Charging Profile must comply with the requirements defined at part 2 specification.

Post scenario validations:

N/a

TC_K_37_CSMS: Remote start transaction with charging profile - Success

Test case name	Remote start transaction with charging profile - Success
Test case Id	TC_K_37_CSMS
Use case Id(s)	<u>K05,F01</u>
Requirement(s)	<u>K05.FR.02,F01.FR.08,F01.FR.09,F01.FR.11</u>
System under test	CSMS
Description	The CSMS sets a TxProfile on a specific EVSE inside a RequestStartTransactionRequest message.
Purpose	To verify if the CSMS is able to set a TxProfile on a specific EVSE in a RequestStartTransactionRequest message.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a RequestStartTransactionResponse With <i>status Accepted</i>	1. The CSMS sends a RequestStartTransactionRequest
3. The Test System sends a TransactionEventRequest With <i>triggerReason RemoteStart</i> transactionInfo.remoteStartId is present.	4. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 1:

Message: **RequestStartTransactionRequest****idToken.idToken** <Configured valid idToken>**idToken.type** <Configured valid idToken type>**evseId** <Configured evseId>**chargingProfile** contains:**chargingProfile.chargingProfilePurpose** is *TxProfile***chargingProfile.transactionId** is omitted**chargingProfile.chargingProfileKind** is *Relative* OR *Absolute*If **chargingProfileKind** is *Relative* then **chargingSchedule.startSchedule** must be omitted.If **chargingProfileKind** is *Absolute* then **chargingSchedule.startSchedule** must NOT be omitted.

The received Charging Profile must comply with the requirements defined at part 2 specification.

Tool validations**Post scenario validations:**

N/a

TC_K_43_CSMS: Get Composite Schedule - Specific EVSE

Test case name	Get Composite Schedule - Specific EVSE
Test case Id	TC_K_43_CSMS
Use case Id(s)	<u>K08</u>
Requirement(s)	<u>K08.FR.01</u>
System under test	CSMS
Description	The CSMS requests a composite schedule which is a combination of local limits and the prevailing Charging Profiles of the different chargingProfilePurposes and stack levels.
Purpose	To verify if the CSMS is able to calculate request a composite schedule from the Charging Station for a specific EVSE.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetCompositeScheduleResponse With status Accepted schedule.evseId 1 schedule.duration is 300 schedule.chargingRateUnit <Specified chargingRateUnit from step 1> schedule.chargingSchedulePeriod[0].startPeriod 0 Note: Multiply limit by 1000 if chargingRateUnit is W schedule.chargingSchedulePeriod[0].limit 10	1. The CSMS sends a GetCompositeScheduleRequest

Tool validations

* Step 1:

(Message: **GetCompositeScheduleRequest**)**evseId 1****duration** is <Configured duration>**chargingRateUnit** <Configured chargingRateUnit>

Post scenario validations:

N/a

TC_K_44_CSMS: Get Composite Schedule - Charging Station

Test case name	Get Composite Schedule - Charging Station
Test case Id	TC_K_44_CSMS
Use case Id(s)	<u>K08</u>
Requirement(s)	<u>K08.FR.01</u>
System under test	CSMS
Description	The CSMS requests a composite schedule which is a combination of local limits and the prevailing Charging Profiles of the different chargingProfilePurposes and stack levels.
Purpose	To verify if the CSMS is able to calculate request a composite schedule from the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetCompositeScheduleResponse With status <i>Accepted</i> schedule.evseId 0 schedule.duration is 300 schedule.chargingRateUnit <Specified chargingRateUnit from step 1> schedule.chargingSchedulePeriod[0].startPeriod 0 <i>Note: Multiply limit by 1000 if chargingRateUnit is W</i> schedule.chargingSchedulePeriod[0].limit 10	1. The CSMS sends a GetCompositeScheduleRequest

Tool validations

* Step 1:
 (Message: **GetCompositeScheduleRequest**)
evseId 0
duration is <Configured duration>
chargingRateUnit <Configured chargingRateUnit>

Post scenario validations:

N/a

TC_K_48_CSMS: EMS Control - Set / Update External Charging Limit (not on a transaction)

Test case name	EMS Control - Set / Update External Charging Limit (not on a transaction)
Test case Id	TC_K_48_CSMS
Use case Id(s)	<u>K12</u>
Requirement(s)	N/a
System under test	CSMS
Description	A charging schedule or charging limit can be imposed by an external system on the Charging Station for new transactions or on the grid connection. An External Control System sends a charging limit to a Charging Station. This limit is then sent to the CSMS.
Purpose	To verify if the CSMS is able to receive the request from a charging station and respond correctly as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

1. The Test System sends a **NotifyChargingLimitRequest** with
- **chargingLimit.chargingLimitSource** EMS

CSMS

2. The CSMS responds with a **NotifyChargingLimitResponse**

Tool validations

- N/a

Post scenario validations:

- N/a

TC_K_50_CSMS: EMS Control - Reset / release external charging limit - Without ongoing transaction

Test case name	EMS Control - Reset / release external charging limit - Without ongoing transaction
Test case Id	TC_K_50_CSMS
Use case Id(s)	<u>K13</u>
Requirement(s)	N/a
System under test	CSMS
Description	A charging schedule or charging limit can be removed by an external system on the Charging Station. An external control system sends a signal to release a previously imposed charging limit to a Charging Station. The Charging Station notifies the CSMS about this.
Purpose	To verify if the CSMS is able to receive the notify from a charging station and respond correctly as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State: N/a

Main (Test scenario)	
Charging Station	CSMS
1. The Test System sends a ClearedChargingLimitRequest with - chargingLimitSource <i>EMS</i>	2. The CSMS responds with a ClearedChargingLimitResponse

Tool validations
- N/a
Post scenario validations: - N/a

TC_K_51_CSMS: EMS Control - Reset / release external charging limit - With ongoing transaction

Test case name	EMS Control - Reset / release external charging limit - With ongoing transaction
Test case Id	TC_K_51_CSMS
Use case Id(s)	<u>K13</u>
Requirement(s)	N/a
System under test	CSMS
Description	A charging schedule or charging limit can be removed by an external system on the Charging Station. An external control system sends a signal to release a previously imposed charging limit to a Charging Station. The Charging Station notifies the CSMS about this.
Purpose	To verify if the CSMS is able to receive the notify from a charging station and respond correctly as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a ClearedChargingLimitRequest with - chargingLimitSource <i>EMS</i>	2. The CSMS responds with a ClearedChargingLimitResponse
3. The Test System sends a TransactionEventRequest with - eventType <i>Updated</i> - triggerReason <i>ChargingRateChanged</i>	4. The CSMS responds with a TransactionEventResponse

Tool validations

- N/a

Post scenario validations:

- N/a

TC_K_52_CSMS: EMS Control - Set / Update External Charging Limit (not on a transaction) - ChargingStationExternalConstraints in report

Test case name	EMS Control - Set / Update External Charging Limit (not on a transaction) - ChargingStationExternalConstraints in report
Test case Id	TC_K_52_CSMS
Use case Id(s)	<u>K12</u>
Requirement(s)	N/a
System under test	CSMS
Description	A charging schedule or charging limit can be removed by an external system on the Charging Station. An external control system sends a signal to release a previously imposed charging limit to a Charging Station. The Charging Station notifies the CSMS about this.
Purpose	To verify if the CSMS is able to correctly receive the report when a charging limit has been externally changed in a charging station as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetChargingProfilesResponse with - status <i>Accepted</i>	1. The CSMS sends a GetChargingProfilesRequest
3. The Test System sends a ReportChargingProfilesRequest with - requestId <i>Generated Id</i> - chargingProfile.chargingProfilePurpose <i>ChargingStationExternalConstraints</i>	4. The CSMS responds with a ReportChargingProfilesResponse

Tool validations

N/a

Post scenario validations:

- N/a

TC_K_53_CSMS: Charging with load leveling based on High Level Communication - Success

Test case name	Charging with load leveling based on High Level Communication - Success
Test case Id	TC_K_53_CSMS
Use case Id(s)	<u>K15</u>
Requirement(s)	<u>K15.FR.02,K15.FR.03,K15.FR.05,K15.FR.07,K15.FR.11</u>
System under test	CSMS
Description	ISO15118-1 E1 AC Charging with load leveling based on High Level Communication, and E4 DC charging with load leveling based on High Level Communication.
Purpose	To verify if the CSMS is able to perform load leveling when it receives the EV charging needs from the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *Authorized*State is *EVConnectedPreSession*

Main (Test scenario)

Charging Station

CSMS

1. Execute reusable state *ISO15118SmartCharging*2. The CSMS does NOT send a **SetChargingProfileRequest**Note(s):

- The CSMS must NOT initiate a renegotiate after starting the transaction, without cause. For example; a smart charging algorithm or an external trigger, etc.

Tool validations

- N/a

Post scenario validations:

N/a

TC_K_55_CSMS: Charging with load leveling based on High Level Communication - EV charging profile exceeds limits

Test case name	Charging with load leveling based on High Level Communication - EV charging profile exceeds limits
Test case Id	TC_K_55_CSMS
Use case Id(s)	<u>K15,K16,K17</u>
Requirement(s)	<u>K15.FR.12,K15.FR.13,K16.FR.07,K16.FR.08,K17.FR.12,K17.FR.13</u>
System under test	CSMS
Description	ISO15118-1 E1 AC Charging with load leveling based on High Level Communication, and E4 DC charging with load leveling based on High Level Communication.
Purpose	To verify if the CSMS is able to renegotiate when it receives the EV charging schedule which exceeds the profile limits.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *Authorized*State is *EVConnectedPreSession***Main (Test scenario)**

Charging Station	CSMS
1. The Test System sends a NotifyEVChargingNeedsRequest . With evseId <Configured evseId> maxScheduleTuples & chargingNeeds <Configured values from mock EV>	2. The CSMS responds with a NotifyEVChargingNeedsResponse .
4. The Test System responds with a SetChargingProfileResponse With status <i>Accepted</i>	3. The CSMS sends a SetChargingProfileRequest
5. The Test System sends a NotifyEVChargingScheduleRequest . With evseId <Configured evseId> chargingSchedule <ChargingSchedule that exceeds the limits of the chargingSchedule provided at step 3.>	6. The CSMS responds with a NotifyEVChargingScheduleResponse .
7. The Test System sends a TransactionEventRequest . With triggerReason <i>ChargingStateChanged</i> transactionInfo.chargingState <i>Charging</i>	8. The CSMS responds with a TransactionEventResponse .

Main (Test scenario)	
10. The Test System responds with a SetChargingProfileResponse With status <i>Accepted</i>	9. The CSMS sends a SetChargingProfileRequest
11. The Test System sends a NotifyEVChargingScheduleRequest . With evseId <i><Configured evseId></i> chargingSchedule <i><ChargingSchedule provided at step 9></i>	12. The CSMS responds with a NotifyEVChargingScheduleResponse .

Tool validations
* Step 2: (Message: NotifyEVChargingNeedsResponse) status <i>Accepted</i> * Step 3: (Message: SetChargingProfileRequest) evseId <i><Configured evseId></i> chargingProfilePurpose <i>TxProfile</i> transactionId <i><Provided transactionId from before></i> * Step 6: (Message: NotifyEVChargingScheduleResponse) status <i>Rejected</i> * Step 9: (Message: SetChargingProfileRequest) evseId <i><Configured evseId></i> chargingProfilePurpose <i>TxProfile</i> transactionId <i><Provided transactionId from before></i> * Step 12: (Message: NotifyEVChargingScheduleResponse) status <i>Accepted</i> Post scenario validations: N/a

TC_K_57_CSMS: Renegotiating a Charging Schedule ISO 15118-20 - Initiated by EV

Test case name	Renegotiating a Charging Schedule ISO 15118-20 - Initiated by EV
Test case Id	TC_K_57_CSMS
Use case Id(s)	<u>K17</u>
Requirement(s)	<u>K17.FR.02,K17.FR.03,K17.FR.05,K17.FR.07,K17.FR.11</u>
System under test	CSMS
Description	The EV signals the Charging Station that it wants to renegotiate and it provides new charging needs, which the Charging Station sends to the CSMS. Based on this and other parameters, the CSMS calculates a new charging schedule and sends it via SetChargingProfileRequest to Charging Station, which communicates it to the EV.
Purpose	To verify if the CSMS is able to renegotiate when the EV signals the Charging Station that it wants to renegotiate and it provides new charging needs, which the Charging Station sends to the CSMS.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is

Authorized AND
EVConnectedPreSession AND
ISO15118SmartCharging

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a NotifyEVChargingNeedsRequest . With evseId <Configured evseId> maxScheduleTuples & chargingNeeds <Configured values from mock EV>	2. The CSMS responds with a NotifyEVChargingNeedsResponse .
4. The Test System responds with a SetChargingProfileResponse With status Accepted	3. The CSMS sends a SetChargingProfileRequest <u>Note(s)</u> : - If NotifyEVChargingNeedsResponseStatus was Processing, the Test System will wait 60 seconds for the request
5. The Test System sends a NotifyEVChargingScheduleRequest . With evseId <Configured evseId> chargingSchedule <ChargingSchedule provided at step 3>	6. The CSMS responds with a NotifyEVChargingScheduleResponse .

Tool validations
* Step 2: (Message: NotifyEVChargingNeedsResponse) status <i>Accepted</i> or <i>Processing</i> * Step 3: (Message: SetChargingProfileRequest) evseId <i><Configured evseId></i> chargingProfilePurpose <i>TxProfile</i> transactionId <i><Provided transactionId from before></i> * Step 6: (Message: NotifyEVChargingScheduleResponse) status <i>Accepted</i>
Post scenario validations: N/a

TC_K_58_CSMS: Renegotiating a Charging Schedule ISO 15118-20 - Initiated by CSMS

Test case name	Renegotiating a Charging Schedule ISO 15118-2 - Initiated by CSMS
Test case Id	TC_K_58_CSMS
Use case Id(s)	<u>K16</u>
Requirement(s)	<u>K16.FR.06</u>
System under test	CSMS
Description	The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by the EV. The CSMS calculates a ChargingSchedule to stay within limits which MAY be imposed by an external system.
Purpose	To verify if the CSMS is able to renegotiate power/current drawn by the EV.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

State is

Authorized AND*EVConnectedPreSession* AND*ISO15118SmartCharging***Main (Test scenario)**

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse With status <i>Accepted</i>	1. The CSMS sends a SetChargingProfileRequest
3. The Test System sends a NotifyEVChargingScheduleRequest . With evseId <i><Configured evseId></i> chargingSchedule <i><ChargingSchedule provided at step 1></i>	4. The CSMS responds with a NotifyEVChargingScheduleResponse .

Tool validations

* Step 1:

(Message: **SetChargingProfileRequest**)**evseId** *<Configured evseId>***chargingProfilePurpose** *TxProfile***transactionId** *<Provided transactionId from before>*

* Step 4:

(Message: **NotifyEVChargingScheduleResponse**)**status** *Accepted*

Tool validations**Post scenario validations:**

N/a

TC_K_59_CSMS: Renegotiating a Charging Schedule ISO 15118-20 - Initiated by CSMS - Send NotifyEVChargingNeeds

Test case name	Renegotiating a Charging Schedule ISO 15118-2 - Initiated by CSMS - Send NotifyEVChargingNeeds
Test case Id	TC_K_59_CSMS
Use case Id(s)	<u>K16</u>
Requirement(s)	<u>K16.FR.12</u>
System under test	CSMS
Description	The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by the EV. The CSMS calculates a ChargingSchedule to stay within limits which MAY be imposed by an external system.
Purpose	To verify if the CSMS is able to handle a Charging Stations resending the charging needs of the EV.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is

Authorized AND

EVConnectedPreSession AND

ISO15118SmartCharging

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger SetChargingProfile

evseId <Configured evseId>

chargingProfilePurpose TxProfile

transactionId <Provided transactionId from before>

2. The Test System responds with a

SetChargingProfileResponseWith **status Accepted**1. The CSMS sends a **SetChargingProfileRequest**3. The Test System sends a **NotifyEVChargingNeedsRequest**.With **evseId** <Configured evseId>**maxScheduleTuples** & **chargingNeeds** <Configured values from mock EV>4. The CSMS responds with a
NotifyEVChargingNeedsResponse.

6. The Test System responds with a

SetChargingProfileResponseWith **status Accepted**5. The CSMS sends a **SetChargingProfileRequest** Note(s):
- If **NotifyEVChargingNeedsResponseStatus** was Processing,
the Test System will wait 60 seconds for the request

Main (Test scenario)	
7. The Test System sends a NotifyEVChargingScheduleRequest . With evseId <Configured evseId> chargingSchedule <ChargingSchedule provided at step 3>	8. The CSMS responds with a NotifyEVChargingScheduleResponse .

Tool validations
* Step 1: (Message: SetChargingProfileRequest) evseId <Configured evseId> chargingProfilePurpose TxProfile transactionId <Provided transactionId from before> * Step 4: (Message: NotifyEVChargingNeedsResponse) status Accepted or Processing * Step 5: (Message: SetChargingProfileRequest) evseId <Configured evseId> chargingProfilePurpose TxProfile transactionId <Provided transactionId from before> * Step 8: (Message: NotifyEVChargingScheduleResponse) status Accepted
Post scenario validations: N/a

TC_K_70_CSMS: Set Charging Profile - Multiple Profiles

Test case name	Set Charging Profile - Multiple Profiles
Test case Id	TC_K_70_CSMS
Use case Id(s)	n/a
Requirement(s)	n/a
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to set multiple Charging Profiles.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:State is *EnergyTransferStarted***Main (Test scenario)**

Charging Station	CSMS
2. The Test System responds with a SetChargingProfileResponse with status Accepted	1. The CSMS sends a SetChargingProfileRequest with chargingProfilePurpose TxDefaultProfile
4. The Test System responds with a SetChargingProfileResponse with status Accepted	3. The CSMS sends a SetChargingProfileRequest with chargingProfilePurpose ChargingStationMaxProfile

Tool validations

* Step 1:

Message **SetChargingProfileRequest**

chargingProfile.chargingProfilePurpose *TxDefaultProfile*

chargingProfile.chargingProfileKind is *Absolute* OR *Recurring*

chargingProfile.chargingSchedule.startSchedule must NOT be omitted.

If **chargingProfile.chargingProfileKind** is *Recurring* then **chargingProfile.recurrencyKind** must NOT be omitted.

* Step 3:

Message **SetChargingProfileRequest**

chargingProfile.id *<different id from chargingProfile at step 1>*

chargingProfile.chargingProfilePurpose *ChargingStationMaxProfile*

chargingProfile.chargingProfileKind is *Absolute*

chargingProfile.chargingSchedule.startSchedule must NOT be omitted.

The received Charging Profiles must comply with the requirements defined at part 2 specification.

Post scenario validations:

- N/a

TC_K_100_CSMS: Set Charging Profile - maxOfflineDuration

Test case name	Set Charging Profile - maxOfflineDuration
Test case Id	TC_K_100_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs.
Purpose	To verify if the CSMS is able to specify a maxOfflineDuration and invalidAfterOfflineDuration the charging profile.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a TxDefaultProfile charging profile with a maxOfflineDuration and invalidAfterOfflineDuration set to true

2. The Test System responds with a **SetChargingProfileResponse** with status *Accepted*

1. The CSMS sends a **SetChargingProfileRequest**

Tool validations

* Step 1:

Message **SetChargingProfileRequest**

- **chargingProfile.chargingProfilePurpose** must be *TxDefaultProfile*

- **chargingProfile.maxOfflineDuration** must be *<not omitted>*

- **chargingProfile.invalidAfterOfflineDuration** must be *true*

Post scenario validations:

N/a

TC_K_101_CSMS: Set Charging Profile - Change operation mode

Test case name	Set Charging Profile - Change operation mode
Test case Id	TC_K_101_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	...
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs.
Purpose	To verify if the CSMS is able to handle the message that the Charging Station sends when a charging profile changes the operationMode of a transaction.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

State is ISO15118-20 transaction started in V2X AC_BPT energy transfer mode

Main (Test scenario)

Charging Station	CSMS
<u>Manual Action:</u> Request the CSMS to send a TxDefaultProfile charging profile with operationMode Idle	
2. The Test System responds with a SetChargingProfileResponse with status Accepted	1. The CSMS sends a SetChargingProfileRequest
3. The Test System sends a TransactionEventRequest with eventType_Updated triggerReason OperationModeChanged transactionInfo.operationMode Idle	4. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 1:

Message SetChargingProfileRequest

- chargingProfile.chargingProfilePurpose must be TxDefaultProfile

- chargingProfile.chargingSchedule.chargingSchedulePeriod[0].operationMode must be Idle

* Step 4:

Message: TransactionEventResponse

- status must be Accepted

Tool validations**Post scenario validations:**

N/a

TC_K_102_CSMS: Set Charging Profile - limitAtSoc

Test case name	Set Charging Profile - limitAtSoc
Test case Id	TC_K_102_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	<u>K01.FR.104</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs.
Purpose	To verify if the CSMS is able to set a SoC limit for the transaction using a Charging Profile.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a TxDefaultProfile charging profile with limitAtSoc specified

2. The Test System responds with a **SetChargingProfileResponse** with status *Accepted*

1. The CSMS sends a **SetChargingProfileRequest**

Tool validations

* Step 1:

Message **SetChargingProfileRequest**

- **chargingProfile.chargingProfilePurpose** must be *TxDefaultProfile*

- **chargingProfile.chargingSchedule.limitAtSoC** must be *<not omitted>*

Post scenario validations:

N/a

TC_K_104_CSMS: Set Charging Profile - PriorityCharging

Test case name	Set Charging Profile - PriorityCharging
Test case Id	TC_K_104_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	<u>K01.FR.72</u> , <u>K01.FR.73</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs.
Purpose	To verify if the CSMS is able to set a PriorityCharging Charging Profile.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a PriorityCharging charging profile

2. The Test System responds with a **SetChargingProfileResponse** with status *Accepted*

1. The CSMS sends a **SetChargingProfileRequest**

Tool validations

* Step 1:

Message **SetChargingProfileRequest**- **chargingProfile.chargingProfilePurpose** must be *PriorityCharging*- **chargingProfile.chargingSchedule.duration** must be *<omitted>*

Post scenario validations:

N/a

TC_K_106_CSMS: Set Charging Profile - randomizedDelay

Test case name	Set Charging Profile - randomizedDelay
Test case Id	TC_K_106_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	<u>K01.FR.96</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs.
Purpose	To verify if the CSMS is able to set a Charging Profile with a randomizedDelay which uses useLocalTime.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a TxDefaultProfile charging profile with useLocalTime true and a randomizedDelay > 0

2. The Test System responds with a **SetChargingProfileResponse** with **status Accepted**

1. The CSMS sends a **SetChargingProfileRequest**

Tool validations

* Step 1:

Message **SetChargingProfileRequest**

- **chargingProfile.chargingProfilePurpose** must be *TxDefaultProfile*
- **chargingProfile.chargingSchedule.useLocalTime** must be *true*
- **chargingProfile.chargingSchedule.randomizedDelay** must be *> 0*

Post scenario validations:

N/a

TC_K_109_CSMS: EMS Control - Set Charging Profile - MaxExternalConstraintsId

Test case name	EMS Control - Set Charging Profile - MaxExternalConstraintsId
Test case Id	TC_K_109_CSMS
Use case Id(s)	<u>K01</u>
Requirement(s)	<u>K01.FR.80</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to send a charging profile using the right id set by MaxExternalConstraintsId as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Configure variable MaxExternalConstraintsId on the CSMS

2. The Test System responds with SetVariablesResponse

1. The CSMS sends SetVariablesRequest with:
 - **variable.name** MaxExternalConstraintsId
 - **component.name** SmartChargingCtrlr
 - **attributeValue** 10

Manual Action: Request the CSMS to send a TxDefaultProfile Charging Profile

4. The Test System responds with a SetChargingProfileResponse with status Accepted

3. The CSMS sends a SetChargingProfileRequest

Manual Action: Request the CSMS to clear the charging profile with <chargingProfileId from step 3>

6. The Test System responds with a ClearChargingProfileResponse with status Accepted

5. The CSMS sends a ClearChargingProfileRequest with chargingProfileId <chargingProfileId from step 3> AND chargingProfileCriteria omit

Manual Action: Configure variable MaxExternalConstraintsId on the CSMS

Main (Test scenario)	
8. The Test System responds with SetVariablesResponse	7. The CSMS sends SetVariablesRequest with: - variable.name <i>MaxExternalConstraintsId</i> - component.name <i>SmartChargingCtrlr</i> - attributeValue <i>2147400000</i>
<u>Manual Action:</u> Request the CSMS to send a <i>TxDefaultProfile</i> Charging Profile	
10. The Test System responds with a SetChargingProfileResponse with status <i>Accepted</i>	9. The CSMS sends a SetChargingProfileRequest

Tool validations
* Step 3: Message SetChargingProfileRequest chargingProfile.id must be greater than 10
* Step 5: Message ClearChargingProfileRequest chargingProfile.id must be <chargingProfileId from step 1>
* Step 9: Message SetChargingProfileRequest chargingProfile.id must be greater than 2147400000
Post scenario validations: - N/a

TC_K_113_CSMS: Renegotiating a Charging Schedule ISO 15118-20 - Initiated by CSMS -

Test case name	Renegotiating a Charging Schedule ISO 15118-20 - Initiated by CSMS
Test case Id	TC_K_113_CSMS
Use case Id(s)	<u>K16</u>
Requirement(s)	<u>K16.FR.06</u>
System under test	CSMS
Description	The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by the EV. The CSMS calculates a ChargingSchedule to stay within limits which MAY be imposed by an external system.
Purpose	To verify if the CSMS is able to renegotiate power/current drawn by the EV.
Prerequisite(s)	CSMS supports ISO 15118-20

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *Authorized*State is *EVConnectedPreSession*State is *ISO15118SmartCharging* (iso15118-20)

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger SetChargingProfile

evseId <Configured evseId>

chargingProfilePurpose TxProfile

transactionId <Provided transactionId from before>

2. The Test System responds with a

SetChargingProfileResponseWith **status Accepted**3. The Test System sends a **NotifyEVChargingScheduleRequest**.With **evseId** <Configured evseId>**chargingSchedule** <ChargingSchedule provided at step 1>1. The CSMS sends a **SetChargingProfileRequest**4. The CSMS responds with a **NotifyEVChargingScheduleResponse**.

Tool validations

* Step 1:

(Message: **SetChargingProfileRequest**)

evseId <Configured evseId>

chargingProfilePurpose TxProfile

transactionId <Provided transactionId from before>

* Step 4:

(Message: **NotifyEVChargingScheduleResponse**)

status Accepted

Post scenario validations:

N/a

TC_K_114_CSMS: Renegotiating a Charging Schedule ISO 15118-20 - Initiated by EV

Test case name	Renegotiating a Charging Schedule ISO 15118-20 - Initiated by EV
Test case Id	TC_K_114_CSMS
Use case Id(s)	<u>K17</u>
Requirement(s)	<u>K17.FR.02</u> , <u>K17.FR.03</u> , <u>K17.FR.05</u> , <u>K17.FR.07</u> , <u>K17.FR.08</u> , <u>K17.FR.11</u>
System under test	CSMS
Description	The EV signals the Charging Station that it wants to renegotiate and it provides new charging needs, which the Charging Station sends to the CSMS. Based on this and other parameters, the CSMS calculates a new charging schedule and sends it via SetChargingProfileRequest to Charging Station, which communicates it to the EV.
Purpose	To verify if the CSMS is able to renegotiate when the EV signals the Charging Station that it wants to renegotiate and it provides new charging needs, which the Charging Station sends to the CSMS.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is EVConnectedPreSession

State is Authorized

State is ISO15118SmartCharging (iso15118-20)

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a NotifyEVChargingNeedsRequest with evseId <Configured evseId> maxScheduleTuples 1 chargingNeeds.requestedEnergyTransfer AC_three_phase chargingNeeds.availableEnergyTransfer [AC_single_phase, AC_three_phase, AC_BPT] chargingNeeds.controlMode ScheduledControl chargingNeeds.v2xChargingParameters.maxChargePower 6000 chargingNeeds.v2xChargingParameters.maxDischargePower 0	2. The CSMS responds with a NotifyEVChargingNeedsResponse .
4. The Test System responds with a SetChargingProfileResponse with status Accepted	3. The CSMS sends a SetChargingProfileRequest <u>Note(s):</u> - If NotifyEVChargingNeedsResponseStatus was Processing, the Test System will wait 60 seconds for the request

Main (Test scenario)	
5. The Test System sends a NotifyEVChargingScheduleRequest with evseId <Configured evseId> chargingSchedule <ChargingSchedule provided at step 3>	6. The CSMS responds with a NotifyEVChargingScheduleResponse .

Tool validations
* Step 2: Message: NotifyEVChargingNeedsResponse - status must be <i>Accepted</i> or <i>Processing</i> * Step 3: Message: SetChargingProfileRequest - evseId must be <Configured evseId> - chargingProfilePurpose must be <i>TxProfile</i> - transactionId must be <Provided transactionId from before> - chargingProfile.chargingSchedule must be size = 1 * Step 6: Message: NotifyEVChargingScheduleResponse - status must be <i>Accepted</i>
Post scenario validations: N/a

TC_K_115_CSMS: ISO 15118-20 Dynamic Control Mode - Success

Test case name	ISO 15118-20 Dynamic Control Mode - Success
Test case Id	TC_K_115_CSMS
Use case Id(s)	<u>K17</u> ,K19
Requirement(s)	<u>K17.FR.02</u> , <u>K17.FR.03</u> , <u>K17.FR.05</u> , <u>K17.FR.07</u> , <u>K17.FR.08</u> , <u>K17.FR.11</u> , <u>K17.FR.19</u> , <u>K19.FR.02</u> , <u>K19.FR.03</u> , <u>K19.FR.05</u> , <u>K19.FR.07</u> , <u>K19.FR.08</u>
System under test	CSMS
Description	The EV signals the Charging Station that it wants to renegotiate and it provides new charging needs, which the Charging Station sends to the CSMS. Based on this and other parameters, the CSMS calculates a new charging schedule and sends it via SetChargingProfileRequest to Charging Station, which communicates it to the EV.
Purpose	To verify if the CSMS is able to renegotiate when the EV signals the Charging Station that it wants to renegotiate for DynamicControl and it provides new charging needs, which the Charging Station sends to the CSMS.
Prerequisite(s)	CSMS supports ISO 15118-20.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is Authorized

State is EVConnectedPreSession

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a NotifyEVChargingNeedsRequest with evseId <Configured evseId> maxScheduleTuples 3 chargingNeeds.requestedEnergyTransfer AC_BPT chargingNeeds.availableEnergyTransfer [AC_single_phase, AC_three_phase, AC_BPT] chargingNeeds.controlMode DynamicControl chargingNeeds.v2xChargingParameters.maxChargePower 5000 chargingNeeds.v2xChargingParameters.maxDischargePower 5000	2. The CSMS responds with a NotifyEVChargingNeedsResponse .
4. The Test System responds with a SetChargingProfileResponse with status Accepted	3. The CSMS sends a SetChargingProfileRequest <u>Note(s):</u> - If NotifyEVChargingNeedsResponseStatus was Processing, the Test System will wait 60 seconds for the request

Main (Test scenario)	
5. The Test System sends a NotifyEVChargingScheduleRequest with evseId <Configured evseId> chargingSchedule <ChargingSchedule provided at step 3>	6. The CSMS responds with a NotifyEVChargingScheduleResponse .

Tool validations
* Step 2: Message: NotifyEVChargingNeedsResponse - status must be <i>Accepted</i> or <i>Processing</i> * Step 3: Message: SetChargingProfileRequest - evseId must be <Configured evseId> - chargingProfilePurpose must be <i>TxProfile</i> - transactionId must be <Provided transactionId from before> - chargingProfile.chargingSchedule[*].chargingSchedulePeriods must be size <= 3 - chargingProfile.chargingSchedule[*].absolutePriceSchedule.priceRuleStacks must be size <= 3 - chargingProfile.chargingSchedule[*].priceLevelSchedule.priceLevelScheduleEntries must be size <= 3 * Step 6: Message: NotifyEVChargingScheduleResponse - status must be <i>Accepted</i>
Post scenario validations: N/a

TC_K_117_CSMS: ISO 15118-20 Dynamic Control Mode - Adjusting charging schedule

Test case name	ISO 15118-20 Dynamic Control Mode - Adjusting charging schedule
Test case Id	TC_K_117_CSMS
Use case Id(s)	<u>K20</u>
Requirement(s)	<u>K20.FR.03</u>
System under test	CSMS
Description	The EV signals the Charging Station that it wants to renegotiate and it provides new charging needs, which the Charging Station sends to the CSMS. Based on this and other parameters, the CSMS calculates a new charging schedule and sends it via SetChargingProfileRequest to Charging Station, which communicates it to the EV.
Purpose	To verify if the CSMS is able to provide a new charging profile when EV provides a changed departure time or target energy amount.
Prerequisite(s)	CSMS supports ISO15118-20

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *Authorized*State is *EVConnectedPreSession*State is *ISO15118SmartCharging* (iso15118-20)

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a NotifyEVChargingNeedsRequest with evseId <Configured evseId> maxScheduleTuples 3 chargingNeeds.requestedEnergyTransfer <i>AC_BPT</i> chargingNeeds.availableEnergyTransfer [<i>AC_single_phase</i>, <i>AC_three_phase</i>, <i>AC_BPT</i>] chargingNeeds.controlMode <i>DynamicControl</i> chargingNeeds.departureTime <Current DateTime + 2 hours> chargingNeeds.v2xChargingParameters.evTargetEnergyRequest 1000 chargingNeeds.v2xChargingParameters.maxChargePower 5000 chargingNeeds.v2xChargingParameters.maxDischargePower 5000	2. The CSMS responds with a NotifyEVChargingNeedsResponse.

Main (Test scenario)	
4. The Test System responds with a SetChargingProfileResponse with status <i>Accepted</i>	3. The CSMS sends a SetChargingProfileRequest <u>Note(s):</u> - If NotifyEVChargingNeedsResponseStatus was <i>Processing</i> , the Test System will wait 60 seconds for the request
5. The Test System sends a NotifyEVChargingScheduleRequest with evseId <i><Configured evseId></i> chargingSchedule <i><ChargingSchedule provided at step 3></i>	6. The CSMS responds with a NotifyEVChargingScheduleResponse .
7. The Test System sends a NotifyEVChargingNeedsRequest with evseId <i><Configured evseId></i> maxScheduleTuples 3 chargingNeeds.requestedEnergyTransfer <i>AC_BPT</i> chargingNeeds.availableEnergyTransfer <i>[AC_single_phase, AC_three_phase, AC_BPT]</i> chargingNeeds.controlMode <i>DynamicControl</i> chargingNeeds.departureTime <i><Current DateTime + 3 hours></i> chargingNeeds.v2xChargingParameters.evTargetEnergyRequest 1000 chargingNeeds.v2xChargingParameters.maxChargePower 5000 chargingNeeds.v2xChargingParameters.maxDischargePower 5000	8. The CSMS responds with a NotifyEVChargingNeedsResponse .
10. The Test System responds with a SetChargingProfileResponse with status <i>Accepted</i>	9. The CSMS sends a SetChargingProfileRequest <u>Note(s):</u> - If NotifyEVChargingNeedsResponseStatus was <i>Processing</i> , the Test System will wait 60 seconds for the request
11. The Test System sends a NotifyEVChargingScheduleRequest with evseId <i><Configured evseId></i> chargingSchedule <i><ChargingSchedule provided at step 5></i>	12. The CSMS responds with a NotifyEVChargingScheduleResponse .
13. The Test System sends a NotifyEVChargingNeedsRequest with evseId <i><Configured evseId></i> maxScheduleTuples 3 chargingNeeds.requestedEnergyTransfer <i>AC_BPT</i> chargingNeeds.availableEnergyTransfer <i>[AC_single_phase, AC_three_phase, AC_BPT]</i> chargingNeeds.controlMode <i>DynamicControl</i> chargingNeeds.departureTime <i><Current DateTime + 3 hours></i> chargingNeeds.v2xChargingParameters.evTargetEnergyRequest 3000 chargingNeeds.v2xChargingParameters.maxChargePower 5000 chargingNeeds.v2xChargingParameters.maxDischargePower 5000	14. The CSMS responds with a NotifyEVChargingNeedsResponse .

Main (Test scenario)	
16. The Test System responds with a SetChargingProfileRequest with status <i>Accepted</i>	15. The CSMS sends a SetChargingProfileRequest <u>Note(s)</u>: - If NotifyEVChargingNeedsResponseStatus was <i>Processing</i>, the Test System will wait 60 seconds for the request
17. The Test System sends a NotifyEVChargingScheduleRequest with evseId <i><Configured evseId></i> chargingSchedule <i><ChargingSchedule provided at step 5></i>	18. The CSMS responds with a NotifyEVChargingScheduleResponse.

Tool validations
* Step 2: Message: NotifyEVChargingNeedsResponse - status must be <i>Accepted</i> or <i>Processing</i> * Step 3: Message: SetChargingProfileRequest - evseId must be <i><Configured evseId></i> - chargingProfilePurpose must be <i>TxProfile</i> - transactionId must be <i><Provided transactionId from before></i> - chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].setpoint must be <i><Not omitted></i> * Step 6: Message: NotifyEVChargingScheduleResponse - status must be <i>Accepted</i> * Step 8: Message: NotifyEVChargingNeedsResponse - status must be <i>Accepted</i> or <i>Processing</i> * Step 9: Message: SetChargingProfileRequest - evseId must be <i><Configured evseId></i> - chargingProfilePurpose must be <i>TxProfile</i> - transactionId must be <i><Provided transactionId from before></i> * Step 12: Message: NotifyEVChargingScheduleResponse - status must be <i>Accepted</i> * Step 14: Message: NotifyEVChargingNeedsResponse - status must be <i>Accepted</i> or <i>Processing</i> * Step 15: Message: SetChargingProfileRequest - evseId must be <i><Configured evseId></i> - chargingProfilePurpose must be <i>TxProfile</i> - transactionId must be <i><Provided transactionId from before></i> * Step 18: Message: NotifyEVChargingScheduleResponse - status must be <i>Accepted</i> Post scenario validations: N/a

TC_K_118_CSMS: Priority Charging - Requesting priority charging remotely

Test case name	Priority Charging - Requesting priority charging remotely
Test case Id	TC_K_118_CSMS
Use case Id(s)	<u>K21</u>
Requirement(s)	<u>K21.FR.09</u> , <u>K21.FR.10</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify the CSMS supports priority charging initiated from CSMS.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
<u>Manual Action:</u> Request the CSMS to use priority charging for the transaction	
2. The Test System responds with a UsePriorityChargingResponse with status <i>Accepted</i>	1. The CSMS sends a UsePriorityChargingRequest
3. The Test System sends a NotifyPriorityChargingRequest with transactionId <transactionId> activated <i>true</i>	4. The CSMS responds with a NotifyPriorityChargingResponse

Tool validations

* Step 1:

Message: **UsePriorityChargingRequest**- **transactionId** must be <transactionId>- **activate** *true*

Post scenario validations:

N/a

TC_K_121_CSMS: Dynamic charging profiles from CSMS - Pull

Test case name	Dynamic charging profiles from CSMS - Pull
Test case Id	TC_K_121_CSMS
Use case Id(s)	<u>K28</u>
Requirement(s)	<u>K28.FR.01</u> , <u>K28.FR.02</u> , <u>K28.FR.07</u> , <u>K28.FR.12</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to support DynamicControl charging profiles.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a dynamic charging profile to the CS

2. The Test System responds with a **SetChargingProfileResponse** with **status Accepted**

1. The CSMS sends a **SetChargingProfileRequest**

3. The Test System sends a **PullDynamicScheduleUpdateRequest** with **chargingProfileId** <unknown chargingProfileId>

4. The CSMS responds with a **PullDynamicScheduleUpdateResponse**

5. The Test System sends a **PullDynamicScheduleUpdateRequest** with **chargingProfileId** must be <chargingProfileId received in step 1>

6. The CSMS responds with a **PullDynamicScheduleUpdateResponse**

Tool validations

* Step 1:

Message: **SetChargingProfileRequest**

- **chargingProfile.chargingProfileKind** must be *Dynamic*
- **chargingProfile.chargingSchedule** must be size = *1*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriods** must be size = *1*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].startPeriod** must be *0*

* Step 4:

Message: **PullDynamicScheduleUpdateResponse**

- **status** must be *Rejected*

* Step 6:

Message: **PullDynamicScheduleUpdateResponse**

- **status** must be *Accepted*
- **scheduleUpdate** must be <not omitted>
- **scheduleUpdate** must at least have a single field set

Post scenario validations:

N/a

TC_K_126_CSMS: ISO 15118-20 Dynamic Control Mode - Sets no charging profile

Test case name	ISO 15118-20 Dynamic Control Mode - Sets no charging profile
Test case Id	TC_K_126_CSMS
Use case Id(s)	<u>K19</u>
Requirement(s)	<u>K19.FR.02</u> , <u>K19.FR.04</u>
System under test	CSMS
Description	Authorization of an EV by the CSMS to start a V2X power transfer loop.
Purpose	To check if the CSMS supports ISO15118-20 Dynamic Control without setting a charging profile.
Prerequisite(s)	CSMS supports ISO15118-20 The CSMS is able to turn off providing a Charging Profile during negotiation.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is ISO15118-20 transaction just started exposing:

- <allowedEnergyTransferModes>

- <transactionId>

Main (Test scenario)

Charging Station

1. The Test System sends a **NotifyEVChargingNeedsRequest** with:

evseId <Configured evseId>

chargingNeeds.requestedEnergyTransfer one of <allowedEnergyTransferModes> or if omitted *AC_three_phase*

chargingNeeds.availableEnergyTransfer [*AC_single_phase*, *AC_two_phase*, *AC_three_phase*, *AC_BPT*, *AC_BPT_DER*, *AC_DER*]

chargingNeeds.controlMode *DynamicControl*

chargingNeeds.v2xChargingParameters.maxChargePower *8000W*

chargingNeeds.v2xChargingParameters.maxDischargePower *0W*

CSMS

2. Then CSMS SHALL respond with a **NotifyEVChargingNeedsResponse**

Tool validations

* Step 2:

Message: **NotifyEVChargingNeedsResponse**- **status** must be *NoChargingProfile*

Post scenario validations:

N/a

13. L Firmware Management

TC_L_01_CSMS: Secure Firmware Update - Installation successful

Test case name	Secure Firmware Update - Installation successful
Test case Id	TC_L_01_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11,L01.FR.15</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an <code>UpdateFirmwareRequest</code> with a <code>signingCertificate</code> .
Purpose	To verify if the CSMS is able to request the Charging Station to securely download and install a new firmware.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a UpdateFirmwareResponse With status <i>Accepted</i>	1. The CSMS sends a UpdateFirmwareRequest
3. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Downloading</i>	4. The CSMS responds with a FirmwareStatusNotificationResponse .
5. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Downloaded</i>	6. The CSMS responds with a FirmwareStatusNotificationResponse .
7. The Test System sends a FirmwareStatusNotificationRequest . With status <i>SignatureVerified</i>	8. The CSMS responds with a FirmwareStatusNotificationResponse .
9. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installing</i>	10. The CSMS responds with a FirmwareStatusNotificationResponse .

Main (Test scenario)	
11. The Test System sends a FirmwareStatusNotificationRequest . With status <i>InstallRebooting</i>	12. The CSMS responds with a FirmwareStatusNotificationResponse .
13. The Test System sends a BootNotificationRequest With reason <i>FirmwareUpdate</i>	14. The CSMS responds with a BootNotificationResponse
15. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest connectorStatus <i>Available</i> Message: NotifyEventRequest trigger <i>Delta</i> actualValue <i>"Available"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	16. The CSMS responds accordingly.
17. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installed</i>	18. The CSMS responds with a FirmwareStatusNotificationResponse .

Tool validations
* Step 1: Message UpdateFirmwareRequest - firmware.signingCertificate <i><Configured signingCertificate></i> - firmware.signature <i><Configured signature></i> * Step 14: Message BootNotificationResponse - status <i>Accepted</i>
Post scenario validations: N/a

TC_L_02_CSMS: Secure Firmware Update - InstallScheduled

Test case name	Secure Firmware Update - InstallScheduled
Test case Id	TC_L_02_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11,L01.FR.15</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an <code>UpdateFirmwareRequest</code> with a <code>signingCertificate</code> .
Purpose	To verify if the CSMS is able to request the Charging Station to securely download a new firmware and install it
Prerequisite(s)	The CSMS configuration <code>firmware installDateTime</code> needs to be set to a future <code>dateTime</code> .

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a <code>UpdateFirmwareRequest</code>
2. The Test System responds with a <code>UpdateFirmwareResponse</code> With status <i>Accepted</i>	
3. The Test System sends a <code>FirmwareStatusNotificationRequest</code>. With status <i>Downloading</i>	4. The CSMS responds with a <code>FirmwareStatusNotificationResponse</code>.
5. The Test System sends a <code>FirmwareStatusNotificationRequest</code>. With status <i>Downloaded</i>	6. The CSMS responds with a <code>FirmwareStatusNotificationResponse</code>.
7. The Test System sends a <code>FirmwareStatusNotificationRequest</code>. With status <i>SignatureVerified</i>	8. The CSMS responds with a <code>FirmwareStatusNotificationResponse</code>.
9. The Test System sends a <code>FirmwareStatusNotificationRequest</code>. With status <i>InstallScheduled</i>	10. The CSMS responds with a <code>FirmwareStatusNotificationResponse</code>.

Main (Test scenario)	
11. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installing</i> <u>Note(s):</u> - This step will be executed after the given <i>installDateTime</i> from step 1 has been reached.	12. The CSMS responds with a FirmwareStatusNotificationResponse .
13. The Test System sends a FirmwareStatusNotificationRequest . With status <i>InstallRebooting</i>	14. The CSMS responds with a FirmwareStatusNotificationResponse .
15. The Test System sends a BootNotificationRequest With reason <i>FirmwareUpdate</i>	16. The CSMS responds with a BootNotificationResponse
17. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest connectorStatus <i>Available</i> Message: NotifyEventRequest trigger <i>Delta</i> actualValue <i>"Available"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	18. The CSMS responds accordingly.
19. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installed</i>	20. The CSMS responds with a FirmwareStatusNotificationResponse .

Tool validations
* Step 1: Message UpdateFirmwareRequest - firmware.installDateTime <i><A dateTime in the future></i> * Step 16: Message BootNotificationResponse - status <i>Accepted</i>
Post scenario validations: N/a

TC_L_03_CSMS: Secure Firmware Update - DownloadScheduled

Test case name	Secure Firmware Update - DownloadScheduled
Test case Id	TC_L_03_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11,L01.FR.15</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to request the Charging Station to schedule securely downloading a new firmware.
Prerequisite(s)	The CSMS configuration firmware retrieveDateTime needs to be set to a future dateTime.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a UpdateFirmwareRequest
2. The Test System responds with a UpdateFirmwareResponse With status Accepted	
3. The Test System sends a FirmwareStatusNotificationRequest. With status DownloadScheduled	4. The CSMS responds with a FirmwareStatusNotificationResponse.
5. The Test System sends a FirmwareStatusNotificationRequest. With status Downloading	6. The CSMS responds with a FirmwareStatusNotificationResponse.
<u>Note(s):</u> - This step will be executed after the given retrieveDateTime from step 1 has been reached.	
7. The Test System sends a FirmwareStatusNotificationRequest. With status Downloaded	8. The CSMS responds with a FirmwareStatusNotificationResponse.
9. The Test System sends a FirmwareStatusNotificationRequest. With status SignatureVerified	10. The CSMS responds with a FirmwareStatusNotificationResponse.

Main (Test scenario)	
11. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installing</i>	12. The CSMS responds with a FirmwareStatusNotificationResponse .
13. The Test System sends a FirmwareStatusNotificationRequest . With status <i>InstallRebooting</i>	14. The CSMS responds with a FirmwareStatusNotificationResponse .
15. The Test System sends a BootNotificationRequest With reason <i>FirmwareUpdate</i>	16. The CSMS responds with a BootNotificationResponse
17. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest connectorStatus <i>Available</i> Message: NotifyEventRequest trigger <i>Delta</i> actualValue <i>"Available"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	18. The CSMS responds accordingly.
19. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installed</i>	20. The CSMS responds with a FirmwareStatusNotificationResponse .

Tool validations
* Step 1: Message UpdateFirmwareRequest - firmware.retrieveDateTime <i><A dateTime in the future></i> * Step 16: Message BootNotificationResponse - status <i>Accepted</i>
Post scenario validations: N/a

TC_L_04_CSMS: Secure Firmware Update - RevokedCertificate

Test case name	Secure Firmware Update - RevokedCertificate
Test case Id	TC_L_04_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to handle a Charging Station reporting the firmware signing certificate is revoked.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a UpdateFirmwareResponse With status <i>RevokedCertificate</i>	1. The CSMS sends a UpdateFirmwareRequest

Tool validations

N/a

Post scenario validations:

N/a

TC_L_05_CSMS: Secure Firmware Update - InvalidCertificate

Test case name	Secure Firmware Update - InvalidCertificate
Test case Id	TC_L_05_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to handle a Charging Station reporting the firmware signing certificate is invalid.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a UpdateFirmwareResponse With status <i>InvalidCertificate</i>	1. The CSMS sends a UpdateFirmwareRequest
3. The Test System sends a SecurityEventNotificationRequest With type is <i>InvalidFirmwareSigningCertificate</i>	4. The CSMS responds with a SecurityEventNotificationResponse

Tool validations

N/a

Post scenario validations:

N/a

TC_L_06_CSMS: Secure Firmware Update - InvalidSignature

Test case name	Secure Firmware Update - InvalidSignature
Test case Id	TC_L_06_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to handle a Charging Station reporting the signature is invalid.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a UpdateFirmwareRequest
2. The Test System responds with a UpdateFirmwareResponse With status <i>Accepted</i>	
3. The Test System sends a FirmwareStatusNotificationRequest. With status <i>Downloading</i>	4. The CSMS responds with a FirmwareStatusNotificationResponse.
5. The Test System sends a FirmwareStatusNotificationRequest. With status <i>Downloaded</i>	6. The CSMS responds with a FirmwareStatusNotificationResponse.
7. The Test System sends a FirmwareStatusNotificationRequest. With status <i>InvalidSignature</i>	8. The CSMS responds with a FirmwareStatusNotificationResponse.
9. The Test System sends a SecurityEventNotificationRequest With type is <i>InvalidFirmwareSignature</i>	10. The CSMS responds with a SecurityEventNotificationResponse

Tool validations

N/a

Tool validations**Post scenario validations:**

N/a

TC_L_07_CSMS: Secure Firmware Update - DownloadFailed

Test case name	Secure Firmware Update - DownloadFailed
Test case Id	TC_L_07_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an <code>UpdateFirmwareRequest</code> with a <code>signingCertificate</code> .
Purpose	To verify if the CSMS is able to handle a Charging Station reporting it failed to download the firmware.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a <code>UpdateFirmwareRequest</code>
2. The Test System responds with a <code>UpdateFirmwareResponse</code> With status <i>Accepted</i>	
3. The Test System sends a <code>FirmwareStatusNotificationRequest</code>. With status <i>Downloading</i>	4. The CSMS responds with a <code>FirmwareStatusNotificationResponse</code>.
5. The Test System sends a <code>FirmwareStatusNotificationRequest</code>. With status <i>DownloadFailed</i>	6. The CSMS responds with a <code>FirmwareStatusNotificationResponse</code>.

Tool validations

N/a

Post scenario validations:

N/a

TC_L_08_CSMS: Secure Firmware Update - InstallVerificationFailed

Test case name	Secure Firmware Update - InstallVerificationFailed
Test case Id	TC_L_08_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to handle a Charging Station reporting the verification of the firmware failed during installation.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a UpdateFirmwareRequest
2. The Test System responds with a UpdateFirmwareResponse With status Accepted	
3. The Test System sends a FirmwareStatusNotificationRequest. With status Downloading	4. The CSMS responds with a FirmwareStatusNotificationResponse.
5. The Test System sends a FirmwareStatusNotificationRequest. With status Downloaded	6. The CSMS responds with a FirmwareStatusNotificationResponse.
7. The Test System sends a FirmwareStatusNotificationRequest. With status SignatureVerified	8. The CSMS responds with a FirmwareStatusNotificationResponse.
9. The Test System sends a FirmwareStatusNotificationRequest. With status Installing	10. The CSMS responds with a FirmwareStatusNotificationResponse.
11. The Test System sends a FirmwareStatusNotificationRequest. With status InstallVerificationFailed	12. The CSMS responds with a FirmwareStatusNotificationResponse.

Tool validations
N/a
Post scenario validations: N/a

TC_L_09_CSMS: Secure Firmware Update - InstallationFailed

Test case name	Secure Firmware Update - InstallationFailed
Test case Id	TC_L_09_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to handle a Charging Station reporting the installation of the firmware failed.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a UpdateFirmwareRequest
2. The Test System responds with a UpdateFirmwareResponse With status Accepted	
3. The Test System sends a FirmwareStatusNotificationRequest . With status Downloading	4. The CSMS responds with a FirmwareStatusNotificationResponse .
5. The Test System sends a FirmwareStatusNotificationRequest . With status Downloaded	6. The CSMS responds with a FirmwareStatusNotificationResponse .
7. The Test System sends a FirmwareStatusNotificationRequest . With status SignatureVerified	8. The CSMS responds with a FirmwareStatusNotificationResponse .
9. The Test System sends a FirmwareStatusNotificationRequest . With status Installing	10. The CSMS responds with a FirmwareStatusNotificationResponse .
11. The Test System sends a FirmwareStatusNotificationRequest . With status InstallRebooting	12. The CSMS responds with a FirmwareStatusNotificationResponse .

Main (Test scenario)	
13. The Test System sends a BootNotificationRequest With reason <i>FirmwareUpdate</i>	14. The CSMS responds with a BootNotificationResponse
15. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest connectorStatus <i>Available</i> Message: NotifyEventRequest trigger <i>Delta</i> actualValue <i>"Available"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	16. The CSMS responds accordingly.
17. The Test System sends a FirmwareStatusNotificationRequest . With status <i>InstallationFailed</i>	18. The CSMS responds with a FirmwareStatusNotificationResponse .
Tool validations	
* Step 14: Message BootNotificationResponse - status <i>Accepted</i>	
Post scenario validations: N/a	

TC_L_10_CSMS: Secure Firmware Update - AcceptedCanceled

Test case name	Secure Firmware Update - AcceptedCanceled
Test case Id	TC_L_10_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11,L01.FR.24</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to handle a Charging Station reporting an ongoing installation of a firmware was canceled and it is now starting the new firmware update.
Prerequisite(s)	The CSMS is able to request a new firmware update, while there is already one ongoing on the Charging Station.

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): N/a

Main (Test scenario)	
Charging Station	CSMS
2. The Test System responds with a UpdateFirmwareResponse With status Accepted	1. The CSMS sends a UpdateFirmwareRequest
3. The Test System sends a FirmwareStatusNotificationRequest . With status Downloading	4. The CSMS responds with a FirmwareStatusNotificationResponse .
6. The Test System responds with a UpdateFirmwareResponse With status AcceptedCanceled	5. The CSMS sends a UpdateFirmwareRequest
7. The Test System sends a FirmwareStatusNotificationRequest . With status Downloading	8. The CSMS responds with a FirmwareStatusNotificationResponse .
9. The Test System sends a FirmwareStatusNotificationRequest . With status Downloaded	10. The CSMS responds with a FirmwareStatusNotificationResponse .
11. The Test System sends a FirmwareStatusNotificationRequest . With status SignatureVerified	12. The CSMS responds with a FirmwareStatusNotificationResponse .

Main (Test scenario)	
13. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installing</i>	14. The CSMS responds with a FirmwareStatusNotificationResponse .
15. The Test System sends a FirmwareStatusNotificationRequest . With status <i>InstallRebooting</i>	16. The CSMS responds with a FirmwareStatusNotificationResponse .
17. The Test System sends a BootNotificationRequest With reason <i>FirmwareUpdate</i>	18. The CSMS responds with a BootNotificationResponse
19. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest connectorStatus <i>Available</i> Message: NotifyEventRequest trigger <i>Delta</i> actualValue <i>"Available"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	20. The CSMS responds accordingly.
21. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installed</i>	22. The CSMS responds with a FirmwareStatusNotificationResponse .

Tool validations
* Step 18: Message BootNotificationResponse - status <i>Accepted</i>
Post scenario validations: N/a

TC_L_11_CSMS: Secure Firmware Update - Unable to cancel

Test case name	Secure Firmware Update - Unable to cancel
Test case Id	TC_L_11_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11,L01.FR.27</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to handle a Charging Station reporting the ongoing installation of a firmware cannot be canceled.
Prerequisite(s)	The CSMS is able to request a new firmware update, while there is already one ongoing on the Charging Station.

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): N/a

Main (Test scenario)	
Charging Station	CSMS
2. The Test System responds with a UpdateFirmwareResponse With status Accepted	1. The CSMS sends a UpdateFirmwareRequest
3. The Test System sends a FirmwareStatusNotificationRequest . With status Downloading	4. The CSMS responds with a FirmwareStatusNotificationResponse .
6. The Test System responds with a UpdateFirmwareResponse With status Rejected	5. The CSMS sends a UpdateFirmwareRequest
7. The Test System sends a FirmwareStatusNotificationRequest . With status Downloaded	8. The CSMS responds with a FirmwareStatusNotificationResponse .
9. The Test System sends a FirmwareStatusNotificationRequest . With status SignatureVerified	10. The CSMS responds with a FirmwareStatusNotificationResponse .
11. The Test System sends a FirmwareStatusNotificationRequest . With status Installing	12. The CSMS responds with a FirmwareStatusNotificationResponse .

Main (Test scenario)	
13. The Test System sends a FirmwareStatusNotificationRequest . With status <i>InstallRebooting</i>	14. The CSMS responds with a FirmwareStatusNotificationResponse .
15. The Test System sends a BootNotificationRequest With reason <i>FirmwareUpdate</i>	16. The CSMS responds with a BootNotificationResponse
17. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest connectorStatus <i>Available</i> Message: NotifyEventRequest trigger <i>Delta</i> actualValue <i>"Available"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	18. The CSMS responds accordingly.
19. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installed</i>	20. The CSMS responds with a FirmwareStatusNotificationResponse .

Tool validations
* Step 16: Message BootNotificationResponse - status <i>Accepted</i>
Post scenario validations: N/a

TC_L_13_CSMS: Secure Firmware Update - Unable to download/install firmware with ongoing transaction - AllowNewSessionsPendingFirmwareUpdate is false

Test case name	Secure Firmware Update - Unable to download/install firmware with ongoing transaction - AllowNewSessionsPendingFirmwareUpdate is false
Test case Id	TC_L_13_CSMS
Use case Id(s)	<u>L01</u>
Requirement(s)	<u>L01.FR.01,L01.FR.11</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to securely download and install a new firmware by sending an UpdateFirmwareRequest with a signingCertificate.
Purpose	To verify if the CSMS is able to handle a Charging Station setting connectors to Unavailable while preparing a firmware update when there is a transaction ongoing.
Prerequisite(s)	The CSMS is able to request a new firmware update when there is a transaction ongoing on the Charging Station.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a UpdateFirmwareResponse With status Accepted	1. The CSMS sends a UpdateFirmwareRequest
3. The Test System sends a FirmwareStatusNotificationRequest . With status DownloadScheduled	4. The CSMS responds with a FirmwareStatusNotificationResponse .
5. The Test System notifies the CSMS about the state change of all connectors that don't have a running transaction. Message: StatusNotificationRequest connectorStatus <i>Unavailable</i> Message: NotifyEventRequest trigger <i>Delta</i> actualValue <i>"Unavailable"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	6. The CSMS responds accordingly.
7. Execute Reusable State <i>StopAuthorized</i> <u>Note(s)</u> <i>Wait <configured transaction duration> before executing this step</i>	

Main (Test scenario)	
8. Execute Reusable State <i>EVConnectedPostSession</i>	
9. Execute Reusable State <i>EVDisconnected</i>	
10. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Downloading</i> <u>Note(s)</u> : - This step will be executed after the given <i>retrieveDateTime</i> from step 1 has been reached.	11. The CSMS responds with a FirmwareStatusNotificationResponse .
12. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Downloaded</i>	13. The CSMS responds with a FirmwareStatusNotificationResponse .
14. The Test System sends a FirmwareStatusNotificationRequest . With status <i>SignatureVerified</i>	15. The CSMS responds with a FirmwareStatusNotificationResponse .
16. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installing</i>	17. The CSMS responds with a FirmwareStatusNotificationResponse .
18. The Test System sends a FirmwareStatusNotificationRequest . With status <i>InstallRebooting</i>	19. The CSMS responds with a FirmwareStatusNotificationResponse .
20. The Test System sends a BootNotificationRequest With reason <i>FirmwareUpdate</i>	21. The CSMS responds with a BootNotificationResponse
22. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest connectorStatus <i>Available</i> Message: NotifyEventRequest trigger <i>Delta</i> actualValue <i>"Available"</i> component.name <i>"Connector"</i> variable.name <i>"AvailabilityState"</i>	23. The CSMS responds accordingly.
24. The Test System sends a FirmwareStatusNotificationRequest . With status <i>Installed</i>	25. The CSMS responds with a FirmwareStatusNotificationResponse .

Tool validations
* Step 1: Message UpdateFirmwareRequest - firmware.signingCertificate <i><configured signingCertificate></i> * Step 19: Message BootNotificationResponse - status <i>Accepted</i>

Tool validations**Post scenario validations:**

N/a

TC_L_17_CSMS: Publish Firmware - Published

Test case name	Publish Firmware - Published
Test case Id	TC_L_17_CSMS
Use case Id(s)	<u>L03</u>
Requirement(s)	N/a
System under test	CSMS
Description	The Local Controller downloads and publishes a firmware update at the specified URL. This allows the CSMS to send UpdateFirmwareRequests with the URI pointing to the Local Controller, to any Charging Station connected to the Local Controller. This allows the site to save bandwidth and data on the WAN interface.
Purpose	To verify if the CSMS is able to publish a firmware on the local controller as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a <code>PublishFirmwareRequest</code>
2. The Test System responds with a <code>PublishFirmwareResponse</code> with status <i>Accepted</i>	
3. The Test System sends a <code>PublishFirmwareStatusNotificationRequest</code> with status <i>Downloading</i>	4. The CSMS responds with a <code>PublishFirmwareStatusNotificationResponse</code>
5. The Test System sends a <code>PublishFirmwareStatusNotificationRequest</code> with status <i>Downloaded</i>	6. The CSMS responds with a <code>PublishFirmwareStatusNotificationResponse</code>
7. The Test System sends a <code>PublishFirmwareStatusNotificationRequest</code> with status <i>ChecksumVerified</i>	8. The CSMS responds with a <code>PublishFirmwareStatusNotificationResponse</code>
9. The Test System sends a <code>PublishFirmwareStatusNotificationRequest</code> with status <i>Published</i> AND location <i><Configured firmware_location></i>	10. The CSMS responds with a <code>PublishFirmwareStatusNotificationResponse</code>

Tool validations

* Step 1:

Message **PublishFirmwareRequest**

- **location** <Configured firmware_location>

Post scenario validations:

- N/a

TC_L_24_CSMS: Publish Firmware - Download failed

Test case name	Publish Firmware - Download failed
Test case Id	TC_L_24_CSMS
Use case Id(s)	<u>L03</u>
Requirement(s)	N/a
System under test	CSMS
Description	The Local Controller downloads and publishes a firmware update at the specified URL. This allows the CSMS to send UpdateFirmwareRequests with the URI pointing to the Local Controller, to any Charging Station connected to the Local Controller. This allows the site to save bandwidth and data on the WAN interface.
Purpose	To verify if the CSMS is able to publish a firmware on the local controller as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a PublishFirmwareResponse with status Accepted	1. The CSMS sends a PublishFirmwareRequest
3. The Test System sends a PublishFirmwareStatusNotificationRequest with status Downloading	4. The CSMS responds with a PublishFirmwareStatusNotificationResponse
5. The Test System sends a PublishFirmwareStatusNotificationRequest with status DownloadFailed	6. The CSMS responds with a PublishFirmwareStatusNotificationResponse

Tool validations

* Step 1:

Message **PublishFirmwareRequest**- **location** <Configured firmware_location>**Post scenario validations:**

- N/a

TC_L_19_CSMS: Publish Firmware - Invalid Checksum

Test case name	Publish Firmware - Invalid Checksum
Test case Id	TC_L_19_CSMS
Use case Id(s)	<u>L03</u>
Requirement(s)	N/a
System under test	CSMS
Description	The Local Controller downloads and publishes a firmware update at the specified URL. This allows the CSMS to send UpdateFirmwareRequests with the URI pointing to the Local Controller, to any Charging Station connected to the Local Controller. This allows the site to save bandwidth and data on the WAN interface.
Purpose	To verify if the CSMS is able to publish a firmware on the local controller as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a PublishFirmwareResponse with status Accepted	1. The CSMS sends a PublishFirmwareRequest
3. The Test System sends a PublishFirmwareStatusNotificationRequest with status Downloading	4. The CSMS responds with a PublishFirmwareStatusNotificationResponse
5. The Test System sends a PublishFirmwareStatusNotificationRequest with status Downloaded	6. The CSMS responds with a PublishFirmwareStatusNotificationResponse
7. The Test System sends a PublishFirmwareStatusNotificationRequest with status InvalidChecksum	8. The CSMS responds with a PublishFirmwareStatusNotificationResponse

Tool validations

* Step 1:

Message **PublishFirmwareRequest**- **location** <Configured firmware_location>

Tool validations**Post scenario validations:**

- N/a

TC_L_20_CSMS: Publish Firmware - PublishFailed

Test case name	Publish Firmware - PublishFailed
Test case Id	TC_L_20_CSMS
Use case Id(s)	<u>L03</u>
Requirement(s)	N/a
System under test	CSMS
Description	The Local Controller downloads and publishes a firmware update at the specified URL. This allows the CSMS to send UpdateFirmwareRequests with the URI pointing to the Local Controller, to any Charging Station connected to the Local Controller. This allows the site to save bandwidth and data on the WAN interface.
Purpose	To verify if the CSMS is able to publish a firmware on the local controller as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a PublishFirmwareResponse with status Accepted	1. The CSMS sends a PublishFirmwareRequest
3. The Test System sends a PublishFirmwareStatusNotificationRequest with status Downloading	4. The CSMS responds with a PublishFirmwareStatusNotificationResponse
5. The Test System sends a PublishFirmwareStatusNotificationRequest with status Downloaded	6. The CSMS responds with a PublishFirmwareStatusNotificationResponse
7. The Test System sends a PublishFirmwareStatusNotificationRequest with status ChecksumVerified	8. The CSMS responds with a PublishFirmwareStatusNotificationResponse
9. The Test System sends a PublishFirmwareStatusNotificationRequest with status PublishFailed	10. The CSMS responds with a PublishFirmwareStatusNotificationResponse

Tool validations

* Step 1:

Message **PublishFirmwareRequest**

- **location** <Configured firmware_location>

Post scenario validations:

- N/a

TC_L_21_CSMS: Unpublish Firmware - Unpublished

Test case name	Unpublish Firmware - Unpublished
Test case Id	TC_L_21_CSMS
Use case Id(s)	<u>L04</u>
Requirement(s)	N/a
System under test	CSMS
Description	Stop serving a firmware update to connected Charging Stations.
Purpose	To verify if the CSMS is able to unpublish a firmware on the local controller as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a UnpublishFirmwareResponse with status Unpublished	1. The CSMS sends a UnpublishFirmwareRequest

Tool validations

-N/a

Post scenario validations:

- N/a

TC_L_22_CSMS: Unpublish Firmware - NoFirmware

Test case name	Unpublish Firmware - NoFirmware
Test case Id	TC_L_22_CSMS
Use case Id(s)	<u>L04</u>
Requirement(s)	N/a
System under test	CSMS
Description	Stop serving a firmware update to connected Charging Stations.
Purpose	To verify if the CSMS is able to unpublish a firmware on the local controller as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a UnpublishFirmwareResponse with status NoFirmware	1. The CSMS sends a UnpublishFirmwareRequest

Tool validations

-N/a

Post scenario validations:

- N/a

TC_L_23_CSMS: Unpublish Firmware - Download Ongoing

Test case name	Unpublish Firmware - Download Ongoing
Test case Id	TC_L_23_CSMS
Use case Id(s)	<u>L04</u>
Requirement(s)	N/a
System under test	CSMS
Description	Stop serving a firmware update to connected Charging Stations.
Purpose	To verify if the CSMS is able to unpublish a firmware on the local controller as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a UnpublishFirmwareResponse with status <i>DownloadOngoing</i>	1. The CSMS sends a UnpublishFirmwareRequest

Tool validations

-N/a

Post scenario validations:

- N/a

14. M Certificate Management

TC_M_01_CSMS: Install CA certificate - CSMSRootCertificate

Test case name	Install CA certificate - CSMSRootCertificate
Test case Id	TC_M_01_CSMS
Use case Id(s)	<u>M05</u>
Requirement(s)	<u>M05.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to install new Root CA certificates using the InstallCertificateRequest message.
Purpose	To verify if the CSMS is able to request a Charging Station to install a new CSMSRootCertificate.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *CertificateInstalled* for certificateType *CSMSRootCertificate*

Tool validations

N.a

Post scenario validations:

N/a

TC_M_02_CSMS: Install CA certificate - ManufacturerRootCertificate

Test case name	Install CA certificate - ManufacturerRootCertificate
Test case Id	TC_M_02_CSMS
Use case Id(s)	<u>M05</u>
Requirement(s)	<u>M05.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to install new Root CA certificates using the InstallCertificateRequest message.
Purpose	To verify if the CSMS is able to request a Charging Station to install a new ManufacturerRootCertificate.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *CertificateInstalled* for certificateType *ManufacturerRootCertificate*

Tool validations

N/a

Post scenario validations:

N/a

TC_M_03_CSMS: Install CA certificate - V2GRootCertificate

Test case name	Install CA certificate - V2GRootCertificate
Test case Id	TC_M_03_CSMS
Use case Id(s)	<u>M05</u>
Requirement(s)	<u>M05.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to install new Root CA certificates using the InstallCertificateRequest message.
Purpose	To verify if the CSMS is able to request a Charging Station to install a new V2GRootCertificate.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *CertificateInstalled* for certificateType *V2GRootCertificate*

Tool validations

N/a

Post scenario validations:

N/a

TC_M_04_CSMS: Install CA certificate - MORootCertificate

Test case name	Install CA certificate - MORootCertificate
Test case Id	TC_M_04_CSMS
Use case Id(s)	<u>M05</u>
Requirement(s)	<u>M05.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to install new Root CA certificates using the InstallCertificateRequest message.
Purpose	To verify if the CSMS is able to request a Charging Station to install a new MORootCertificate.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

1. Execute **Reusable State** *CertificateInstalled* for certificateType *MORootCertificate*

Tool validations

N/a

Post scenario validations:

N/a

TC_M_05_CSMS: Install CA certificate - Failed

Test case name	Install CA certificate - Failed
Test case Id	TC_M_05_CSMS
Use case Id(s)	<u>M05</u>
Requirement(s)	<u>M05.FR.01,M05.FR.03</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to install new Root CA certificates using the InstallCertificateRequest message.
Purpose	To verify if the CSMS is able to handle a Charging Station reporting it failed to install the requested certificate.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Trigger the CSMS to send an InstallCertificateRequest with certificateType CSMSRootCertificate.

2. The Test System responds with a **InstallCertificateResponse**
With **status** is *Failed*

1. The CSMS sends a **InstallCertificateRequest**

Tool validations

* Step 1:

Message: **InstallCertificateRequest**
certificateType must be *CSMSRootCertificate*
certificate contains <A certificate>

Post scenario validations:

N/a

TC_M_12_CSMS: Retrieve certificates from Charging Station - CSMSRootCertificate

Test case name	Retrieve certificates from Charging Station - CSMSRootCertificate
Test case Id	TC_M_12_CSMS
Use case Id(s)	<u>M03</u>
Requirement(s)	<u>M03.FR.01</u>
System under test	CSMS
Description	The CSMS is able to retrieve the certificates installed at the Charging Station using the <code>GetInstalledCertificateIdsRequest</code> message. It supports all available hash algorithms, including SHA256, SHA384, and SHA512.
Purpose	To verify if the CSMS is able to retrieve the <code>hashData</code> from all <code>CSMSRootCertificates</code> stored at the Charging Station, using all available hash algorithms, including SHA256, SHA384, and SHA512.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**

1. Execute **Reusable State** `GetInstalledCertificates` for certificateType `CSMSRootCertificate`. The Test System responds with data hashed with SHA256.
2. Execute **Reusable State** `GetInstalledCertificates` for certificateType `CSMSRootCertificate`. The Test System responds with data hashed with SHA384.
3. Execute **Reusable State** `GetInstalledCertificates` for certificateType `CSMSRootCertificate`. The Test System responds with data hashed with SHA512.

Tool validations

N/a

Post scenario validations:

N/a

TC_M_13_CSMS: Retrieve certificates from Charging Station - ManufacturerRootCertificate

Test case name	Retrieve certificates from Charging Station - ManufacturerRootCertificate
Test case Id	TC_M_13_CSMS
Use case Id(s)	<u>M03</u>
Requirement(s)	<u>M03.FR.01</u>
System under test	CSMS
Description	The CSMS is able to retrieve the certificates installed at the Charging Station using the GetInstalledCertificateIdsRequest message.
Purpose	To verify if the CSMS is able to retrieve the hashData from all ManufacturerRootCertificate stored at the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

1. Execute **Reusable State** *GetInstalledCertificates* for certificateType *ManufacturerRootCertificate*

Tool validations

N/a

Post scenario validations:

N/a

TC_M_14_CSMS: Retrieve certificates from Charging Station - V2GRootCertificate

Test case name	Retrieve certificates from Charging Station - V2GRootCertificate
Test case Id	TC_M_14_CSMS
Use case Id(s)	<u>M03</u>
Requirement(s)	<u>M03.FR.01</u>
System under test	CSMS
Description	The CSMS is able to retrieve the certificates installed at the Charging Station using the GetInstalledCertificateIdsRequest message.
Purpose	To verify if the CSMS is able to retrieve the hashData from all V2GRootCertificate stored at the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

1. Execute **Reusable State** *GetInstalledCertificates* for certificateType *V2GRootCertificate*

Tool validations

N/a

Post scenario validations:

N/a

TC_M_15_CSMS: Retrieve certificates from Charging Station - V2GCertificateChain

Test case name	Retrieve certificates from Charging Station - V2GCertificateChain
Test case Id	TC_M_15_CSMS
Use case Id(s)	<u>M03</u>
Requirement(s)	<u>M03.FR.01,M03.FR.05</u>
System under test	CSMS
Description	The CSMS is able to retrieve the certificates installed at the Charging Station using the GetInstalledCertificateIdsRequest message.
Purpose	To verify if the CSMS is able to retrieve the hashData from all certificates that are part of a V2GCertificateChain stored at the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *GetInstalledCertificates* for certificateType *V2GCertificateChain*

Tool validations

N/a

Post scenario validations:

N/a

TC_M_16_CSMS: Retrieve certificates from Charging Station - MORootCertificate

Test case name	Retrieve certificates from Charging Station - MORootCertificate
Test case Id	TC_M_16_CSMS
Use case Id(s)	<u>M03</u>
Requirement(s)	<u>M03.FR.01</u>
System under test	CSMS
Description	The CSMS is able to retrieve the certificates installed at the Charging Station using the GetInstalledCertificateIdsRequest message.
Purpose	To verify if the CSMS is able to retrieve the hashData from all MORootCertificate stored at the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

1. Execute **Reusable State** *GetInstalledCertificates* for certificateType *MORootCertificate*

Tool validations

N/a

Post scenario validations:

N/a

TC_M_17_CSMS: Retrieve certificates from Charging Station - CSMSRootCertificate & ManufacturerRootCertificate

Test case name	Retrieve certificates from Charging Station - CSMSRootCertificate & ManufacturerRootCertificate
Test case Id	TC_M_17_CSMS
Use case Id(s)	<u>M03</u>
Requirement(s)	<u>M03.FR.01</u>
System under test	CSMS
Description	The CSMS is able to retrieve the certificates installed at the Charging Station using the GetInstalledCertificateIdsRequest message.
Purpose	To verify if the CSMS is able to retrieve the hashData from all CSMSRootCertificates and ManufacturerRootCertificate stored at the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

1. Execute **Reusable State** *GetInstalledCertificates* for certificateType *CSMSRootCertificate* AND *ManufacturerRootCertificate*

Tool validations

N/a

Post scenario validations:

N/a

TC_M_18_CSMS: Retrieve certificates from Charging Station - All certificateTypes

Test case name	Retrieve certificates from Charging Station - All certificateTypes
Test case Id	TC_M_18_CSMS
Use case Id(s)	<u>M03</u>
Requirement(s)	<u>M03.FR.01</u>
System under test	CSMS
Description	The CSMS is able to retrieve the certificates installed at the Charging Station using the <code>GetInstalledCertificateIdsRequest</code> message.
Purpose	To verify if the CSMS is able to retrieve the hashData from all Root CA and V2GCertificateChain certificates stored at the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to send a `GetInstalledCertificateIdsRequest` without `certificateType`.

2. The Test System responds with a **`GetInstalledCertificateIdsResponse`**
 With **status** is *Accepted*
certificateHashDataChain contains *<The hashData of all certificates stored at the Test System truststore>*

1. The CSMS sends a **`GetInstalledCertificateIdsRequest`**

Tool validations

* Step 1:

Message: **`GetInstalledCertificateIdsRequest`**

- **certificateType** is omitted

Post scenario validations:

N/a

TC_M_19_CSMS: Retrieve certificates from Charging Station - No matching certificate found

Test case name	Retrieve certificates from Charging Station - No matching certificate found
Test case Id	TC_M_19_CSMS
Use case Id(s)	<u>M03</u>
Requirement(s)	<u>M03.FR.01,M03.FR.02</u>
System under test	CSMS
Description	The CSMS is able to retrieve the certificates installed at the Charging Station using the <code>GetInstalledCertificateIdsRequest</code> message.
Purpose	To verify if the CSMS is able to handle a response from the Charging Station indicating it was not able to find a certificate for the requested criteria.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Trigger the CSMS to send a `GetInstalledCertificateIdsRequest` with `certificateType` `ManufacturerRootCertificate`.

2. The Test System responds with a `GetInstalledCertificateIdsResponse`
With `status` is `NotFound`
`certificateHashDataChain` is omitted.

1. The CSMS sends a `GetInstalledCertificateIdsRequest`

Tool validations

* Step 1:

Message: `GetInstalledCertificateIdsRequest`
- `certificateType` is `ManufacturerRootCertificate`

Post scenario validations:

N/a

TC_M_20_CSMS: Delete a certificate from a Charging Station - Success

Test case name	Delete a certificate from a Charging Station - Success
Test case Id	TC_M_20_CSMS
Use case Id(s)	M04
Requirement(s)	M04.FR.01,M04.FR.07
System under test	CSMS
Description	The CSMS is able to request the Charging Station to delete an installed certificate using the DeleteCertificateRequest message, using all available hash algorithms, including SHA256, SHA384, and SHA512.
Purpose	To verify if CSMS is able to request a Charging Station to delete an installed certificate, using all available hash algorithms, including SHA256, SHA384, and SHA512.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. *CertificateInstalled* with certificateType *CSMSRootCertificate*.

Manual Action: Request the CSMS to send a *DeleteCertificateRequest*.

3. The Test System responds with a **GetInstalledCertificateIdsResponse**
With **status** is *Accepted*
certificateHashDataChain contains an entry with following values:
certificateHashDataChain[0].certificateType is *CSMSRootCertificate*
certificateHashDataChain[0].certificateHashData.hashAlgorithm is *SHA256*

2. The CSMS sends a **GetInstalledCertificateIdsRequest**

5. The Test System responds with a **DeleteCertificateResponse**
With **status** is *Accepted*

4. The CSMS sends a **DeleteCertificateRequest**

Note(s):

- Steps 1 - 5 will be repeated for each hash algorithm (*SHA256, SHA384, SHA512*).

Tool validations

* Step 2:

Message: **GetInstalledCertificateIdsRequest**

- **certificateType** contains *CSMSRootCertificate* OR is *omitted*.

* Step 4:

Message: **DeleteCertificateRequest**

- **certificateHashData** is *<Returned certificateHashData at Step 3>*.

Post scenario validations:

N/a

TC_M_21_CSMS: Delete a certificate from a Charging Station - Failed

Test case name	Delete a certificate from a Charging Station - Failed
Test case Id	TC_M_21_CSMS
Use case Id(s)	<u>M04</u>
Requirement(s)	<u>M04.FR.01,M04.FR.07</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to delete an installed certificate using the DeleteCertificateRequest message.
Purpose	To verify if CSMS is able to handle a Charging Station that fails to delete an installed certificate.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

CertificateInstalled with certificateType CSMSRootCertificate.

Main (Test scenario)

Charging Station	CSMS
<u>Manual Action:</u> Request the CSMS to send a DeleteCertificateRequest.	
2. The Test System responds with a GetInstalledCertificateIdsResponse With status is <i>Accepted</i> certificateHashDataChain contains an entry with following values: certificateHashDataChain[0].certificateType is <i>CSMSRootCertificate</i> certificateHashDataChain[0].certificateHashData.hashAlgorithm is <i>SHA256</i>	1. The CSMS sends a GetInstalledCertificateIdsRequest
4. The Test System responds with a DeleteCertificateResponse With status is <i>Failed</i>	3. The CSMS sends a DeleteCertificateRequest

Tool validations

* Step 1:

Message: **GetInstalledCertificateIdsRequest**

- **certificateType** contains *CSMSRootCertificate* OR is *omitted*.

* Step 3:

Message: **DeleteCertificateRequest**

- **certificateHashData** contains *<Returned certificateHashData at Step 2>*.

Post scenario validations:

N/a

TC_M_24_CSMS: Get Charging Station Certificate status - Success

Test case name	Get Charging Station Certificate status - Success
Test case Id	TC_M_24_CSMS
Use case Id(s)	<u>M06</u>
Requirement(s)	<u>M06.FR.01,M06.FR.02,M06.FR.03,M06.FR.08,M06.FR.09</u>
System under test	CSMS
Description	The Charging Station is able to request the CSMS to get the status of a (V2G) Charging Station certificate.
Purpose	To verify if the CSMS is able to provide the status of a requested (V2G) Charging Station certificate.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

1. The Test System sends one or more subsequent **GetCertificateStatusRequests**
 With **ocspRequestData** contains <hashes from configured (V2G) certificate chain SubCA's>

CSMS

2. The CSMS responds with a **GetCertificateStatusResponse**

Tool validations

Step 2:

Message: **GetCertificateStatusResponse**
status *Accepted*

ocspResult <OCSPResponse class as defined in IETF RFC 6960. DER encoded (as defined in IETF RFC 6960), and then base64 encoded.>

Post scenario validations:

N/a

TC_M_26_CSMS: Certificate Installation EV - Success

Test case name	Certificate Installation EV - Success
Test case Id	TC_M_26_CSMS
Use case Id(s)	<u>M01</u>
Requirement(s)	<u>M01.FR.01</u>
System under test	CSMS
Description	The EV initiates installing a new certificate. The Charging Station forwards the request for a new certificate to the CSMS.
Purpose	To verify if the CSMS is able to return the Raw CertificateInstallationRes response for the EV to the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a Get15118EVCertificateRequest With action Install	2. The CSMS responds with a Get15118EVCertificateResponse

Tool validations

* Step 2:

Message: **Get15118EVCertificateResponse**- **status** *Accepted*- **exiResponse** *<Raw CertificateInstallationRes response for the EV, Base64 encoded.>***Post scenario validations:**

N/a

TC_M_28_CSMS: Certificate Update EV - Success

Test case name	Certificate Update EV - Success
Test case Id	TC_M_28_CSMS
Use case Id(s)	<u>M02</u>
Requirement(s)	<u>M02.FR.01</u>
System under test	CSMS
Description	The EV initiates updating the existing certificate. The Charging Station forwards the update request to the CSMS.
Purpose	To verify if the CSMS is able to return the Raw CertificateInstallationRes response for the EV to the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a Get15118EVCertificateRequest With action Update	2. The CSMS responds with a Get15118EVCertificateResponse

Tool validations

* Step 2:

Message: **Get15118EVCertificateResponse**- **status** *Accepted*- **exiResponse** *<Raw CertificateInstallationRes response for the EV, Base64 encoded.>***Post scenario validations:**

N/a

TC_M_100_CSMS: Certificate Installation EV - ISO 15118-20 - Success

Test case name	Certificate Installation EV - ISO 15118-20 - Success
Test case Id	TC_M_100_CSMS
Use case Id(s)	<u>M01</u>
Requirement(s)	<u>M01.FR.02</u> , <u>M01.FR.05</u>
System under test	CSMS
Description	The EV initiates installing a new certificate. The Charging Station forwards the request for a new certificate to the CSMS.
Purpose	To verify if the CSMS is able to return the Raw CertificateInstallationRes response for the EV to the Charging Station.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Note(s): 3 ContractCertificateChains will be communicated to the CSMS.

1. The Test System sends a **Get15118EVCertificateRequest**
With **action Install**
maximumContractCertificateChains 10
prioritizedEMAIDs <Configured list EMA IDs>

2. The CSMS responds with a **Get15118EVCertificateResponse**

3. The Test System sends a **Get15118EVCertificateRequest**
With **action Install**
maximumContractCertificateChains 10
prioritizedEMAIDs <Configured list EMA IDs>

4. The CSMS responds with a **Get15118EVCertificateResponse**

5. The Test System sends a **Get15118EVCertificateRequest**
With **action Install**
maximumContractCertificateChains 10
prioritizedEMAIDs <Configured list EMA IDs>

6. The CSMS responds with a **Get15118EVCertificateResponse**

Tool validations

* Step 2:

Message: **Get15118EVCertificateResponse**

- **status** must be *Accepted*
- **exiResponse** <Raw CertificateInstallationRes response for the EV, Base64 encoded.>
- **remainingContracts** must be 2

* Step 4:

Message: **Get15118EVCertificateResponse**

- **status** must be *Accepted*
- **exiResponse** <Raw CertificateInstallationRes response for the EV, Base64 encoded.>
- **remainingContracts** must be 1

* Step 6:

Message: **Get15118EVCertificateResponse**

- **status** must be *Accepted*
- **exiResponse** <Raw CertificateInstallationRes response for the EV, Base64 encoded.>
- **remainingContracts** must be 0

Post scenario validations:

N/a

TC_M_101_CSMS: Install CA certificate - OEMRootCertificate

Test case name	Install CA certificate - OEMRootCertificate
Test case Id	TC_M_101_CSMS
Use case Id(s)	<u>M05</u>
Requirement(s)	<u>M05.FR.01</u>
System under test	CSMS
Description	The CSMS is able to request the Charging Station to install new Root CA certificates using the InstallCertificateRequest message.
Purpose	To verify if the CSMS is able to request a Charging Station to install a new OEMRootCertificate.
Prerequisite(s)	- The CSMS supports ISO15118-20

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

1. Execute **Reusable State** *CertificateInstalled* for certificateType *OEMRootCertificate*

Tool validations

N/a

Post scenario validations:

N/a

15. N Diagnostics

TC_N_01_CSMS: Get Monitoring Report - with monitoringCriteria

Test case name	Get Monitoring Report - with monitoringCriteria
Test case Id	TC_N_01_CSMS
Use case Id(s)	<u>N02</u>
Requirement(s)	<u>N02.FR.05</u> , <u>N02.FR.10</u>
System under test	CSMS
Description	CSMS requests a report of monitors that match the component criteria.
Purpose	To test that CSMS supports requesting a monitoring report for the component criteria and that it handles an empty result set.
Prerequisite(s)	CS has implemented device model monitoring and MonitoringCtrlr.Enabled = true.

Before (Preparations)

Configuration State:

N/a

Memory State:

CSMS requests ClearVariableMonitoring ItemsPerMessage from CS.

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manually instruct CSMS to get a report of monitors for:

- all *DeltaMonitoring*

2. Test System responds with:
GetMonitoringReportResponse with:
Status *EmptyResultSet*

1. CSMS sends **GetMonitoringReportRequest**

Manually instruct CSMS to get a report of monitors for:

- all *ThresholdMonitoring*

4. Test System responds with:
GetMonitoringReportResponse with:
Status *Accepted*

3. CSMS sends **GetMonitoringReportRequest**

5. Test System responds with:
NotifyMonitoringReportRequest

6. CSMS sends **NotifyMonitoringReportResponse**

Step 5 and 6 are repeated as often as needed to report all configuration variables.

Tool validations

* Step 1:

Message: **GetMonitoringReportRequest**

- **monitoringCriteria** = *DeltaMonitoring*

- **componentVariable** is omitted.

* Step 3:

Message: **GetMonitoringReportRequest**

- **monitoringCriteria** = *ThresholdMonitoring*

- **componentVariable** is omitted.

Post scenario validations:

Check that CSMS shows the *Threshold* monitors.

TC_N_02_CSMS: Get Monitoring Report - with component/variable

Test case name	Get Monitoring Report - with component/variable
Test case Id	TC_N_02_CSMS
Use case Id(s)	<u>N02</u>
Requirement(s)	<u>N02.FR.05</u> , <u>N02.FR.10</u>
System under test	CSMS
Description	CSMS requests a report of monitors that match the the given list of components and variables.
Purpose	To test that CSMS supports requesting a monitoring report for a given component and variable and that it handles an empty result set.
Prerequisite(s)	CS has implemented device model monitoring and MonitoringCtrlr.Enabled = true.

Before (Preparations)**Configuration State:**

N/a

Memory State:

CSMS requests ClearVariableMonitoring ItemsPerMessage from CS.

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

Manually instruct CSMS to get a report of monitors for:
- the variable Power of ChargingStation

2. Test System responds with:
GetMonitoringReportResponse with:
Status *EmptyResultSet*

1. CSMS sends **GetMonitoringReportRequest**

Manually instruct CSMS to get a report of monitors for:
- the variable AvailabilityState of EVSE #1.

4. Test System responds with:
GetMonitoringReportResponse with:
Status *Accepted*

3. CSMS sends **GetMonitoringReportRequest**

5. Test System responds with:
NotifyMonitoringReportRequest

6. CSMS sends **NotifyMonitoringReportResponse**

Step 5 and 6 are repeated as often as needed to report all configuration variables.

Tool validations
* Step 1: Message: GetMonitoringReportRequest - componentVariable[0].component.name = <i>"ChargingStation"</i> - componentVariable[0].component.instance is omitted. - componentVariable[0].variable.name = <i>"Power"</i> - componentVariable[0].variable.instance is omitted. - monitoringCriteria is omitted.
* Step 3: Message: GetMonitoringReportRequest - componentVariable[1].component.name = <i>"EVSE"</i> - componentVariable[1].component.instance is omitted. - componentVariable[1].component.evse.id = <i>1</i> - componentVariable[1].variable.name = <i>"AvailabilityState"</i> - componentVariable[1].variable.instance is omitted. - monitoringCriteria is omitted.
Post scenario validations: Check that CSMS shows the monitor for AvailabilityState for EVSE #1.

TC_N_03_CSMS: Get Monitoring Report - with component criteria and component/variable

Test case name	Get Monitoring Report - with component criteria and component/variable
Test case Id	TC_N_03_CSMS
Use case Id(s)	<u>N02</u>
Requirement(s)	<u>N02.FR.05</u> , <u>N02.FR.10</u>
System under test	CSMS
Description	CSMS requests a report of monitors that match both the component criteria and the given list of components and variables.
Purpose	To test that CSMS supports requesting a monitoring report for both the component criteria and a given component and variable and that it handles an empty result set.
Prerequisite(s)	CS has implemented device model monitoring and MonitoringCtrlr.Enabled = true.

Before (Preparations)

Configuration State:

N/a

Memory State:

CSMS requests ClearVariableMonitoring ItemsPerMessage from CS.

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manually instruct CSMS to get a report of monitors for:

- all *DeltaMonitoring*
- and the variable *AvailabilityState* for EVSE #1.

2. Test System responds with:
GetMonitoringReportResponse with:
Status *EmptyResultSet*

1. CSMS sends **GetMonitoringReportRequest***Manually instruct CSMS to get a report of monitors for:*

- all *ThresholdMonitoring*
- and the variable *Power* of ChargingStation.

4. Test System responds with:
GetMonitoringReportResponse with:
Status *Accepted*

3. CSMS sends **GetMonitoringReportRequest**

5. Test System responds with:
NotifyMonitoringReportRequest

6. CSMS sends **NotifyMonitoringReportResponse***Step 5 and 6 are repeated as often as needed to report all configuration variables.*

Tool validations
* Step 1: Message: GetMonitoringReportRequest - monitoringCriteria = <i>DeltaMonitoring</i> - componentVariable[0].component.name = "EVSE" - componentVariable[0].component.evse.id = <configured evseId> - componentVariable[0].variable.name = "AvailabilityState" * Step 3: Message: GetMonitoringReportRequest - monitoringCriteria = <i>ThresholdMonitoring</i> - componentVariable[0].component.name = "ChargingStation" - componentVariable[0].variable.name = "Power"
Post scenario validations: Check that CSMS shows the <i>Threshold</i> monitors for Power for ChargingStation.

TC_N_60_CSMS: Get Monitoring Report - with component criteria and list of components/variables

Test case name	Get Monitoring Report - with component criteria and list of components/variables
Test case Id	TC_N_60_CSMS
Use case Id(s)	N02
Requirement(s)	N02.FR.05, N02.FR.10
System under test	CSMS
Description	CSMS requests a report of monitors that match both the component criteria and the given list of components and variables.
Purpose	To test that CSMS supports requesting a monitoring report for both the component criteria and a given list of components and optionally with variables and that it handles an empty result set.
Prerequisite(s)	CS has implemented device model monitoring and MonitoringCtrlr.Enabled = true.

Before (Preparations)

Configuration State:

N/a

Memory State:

CSMS requests ClearVariableMonitoring ItemsPerMessage from CS.

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manually instruct CSMS to get a report of monitors for:

- all *ThresholdMonitoring*
- and the variable *Power* of both *ChargingStation* and *EVSE #1*.

2. Test System responds with:
GetMonitoringReportResponse with:
Status *EmptyResultSet*

1. CSMS sends **GetMonitoringReportRequest**

Manually instruct CSMS to get a report of monitors for:

- all *DeltaMonitoring*
- and the variable *AvailabilityState* of both *ChargingStation* and *EVSE #1*.

4. Test System responds with:
GetMonitoringReportResponse with:
Status *Accepted*

3. CSMS sends **GetMonitoringReportRequest**

5. Test System responds with:
NotifyMonitoringReportRequest

6. CSMS sends **NotifyMonitoringReportResponse**

Step 5 and 6 are repeated as often as needed to report all configuration variables.

Tool validations

* Step 1:

Message: **GetMonitoringReportRequest**

- **monitoringCriteria** is *DeltaMonitoring*
- **componentVariable[0].component.name** = "*ChargingStation*"
- **componentVariable[0].variable.name** = "*AvailabilityState*"
- **componentVariable[1].component.name** = "*EVSE*"
- **componentVariable[1].component.evse.id** = *<configured evseId>*
- **componentVariable[1].variable.name** = "*AvailabilityState*"

* Step 3:

Message: **GetMonitoringReportRequest**

- **monitoringCriteria** = *ThresholdMonitoring*
- **componentVariable[0].component.name** = "*ChargingStation*"
- **componentVariable[0].variable.name** = "*AvailabilityState*"
- **componentVariable[1].component.name** = "*EVSE*"
- **componentVariable[1].component.evse.id** = *<configured evseId>*
- **componentVariable[1].variable.name** = "*AvailabilityState*"

Post scenario validations:

Check that CSMS shows the *Delta* monitors for *AvailabilityState* for both *ChargingStation* and *EVSE #1*.

TC_N_05_CSMS: Set Monitoring Base - success

Test case name	Set Monitoring Base - success
Test case Id	TC_N_05_CSMS
Use case Id(s)	<u>N03</u>
Requirement(s)	<u>N03.FR.03</u> , <u>N03.FR.04</u> , <u>N03.FR.05</u>
System under test	CSMS
Description	CSMS sends a SetMonitoringBaseRequest for <i>All</i> , <i>FactoryDefault</i> and <i>HardWiredOnly</i> .
Purpose	To test that CSMS supports all three monitoring base types.
Prerequisite(s)	CS has implemented device model monitoring and MonitoringCtrlr.Enabled = true.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: SetMonitoringBaseResponse	<i>Instruct CSMS to set a monitoring base of _All._</i> 1. CSMS sends SetMonitoringBaseRequest
4. Test System responds with: SetMonitoringBaseResponse	<i>Instruct CSMS to set a monitoring base of _FactoryDefault._</i> 3. Test System sends SetMonitoringBaseRequest
6. The Test System responds with: SetMonitoringBaseResponse	<i>Instruct CSMS to set a monitoring base of _HardWiredOnly._</i> 5. Test System sends SetMonitoringBaseRequest

Tool validations

* Step 1

Message: **SetMonitoringBaseRequest**- **monitoringBase** = *All*

* Step 3

Message: **SetMonitoringBaseRequest**- **monitoringBase** = *FactoryDefault*

* Step 6

Message: **SetMonitoringBaseRequest**- **monitoringBase** = *HardWiredOnly*

Tool validations**Post scenario validations:**

N/A

TC_N_08_CSMS: Set Variable Monitoring - One SetMonitoringData element

Test case name	Set Variable Monitoring - One SetMonitoringData element
Test case Id	TC_N_08_CSMS
Use case Id(s)	<u>N04</u>
Requirement(s)	<u>N04.FR.01</u> , <u>N04.FR.02</u> , <u>N04.FR.17</u>
System under test	CSMS
Description	CSMS sends a request to activate monitoring on one variable.
Purpose	To test that CSMS supports setting monitoring on one variable.
Prerequisite(s)	CS has implemented device model monitoring and MonitoringCtrlr.Enabled = true.

Before (Preparations)

Configuration State:

This test case activates monitoring on the following variable:

- Component "EVSE", evse <Configured evseId>, variable "AvailabilityState", monitor type *Delta*

It assumes, that no monitor is active on this variable prior to the test.

Note 1: this is a required variable for which a monitor can be expected to exist or it can be configured.

Note 2: Any other component/variable combination that supports monitoring could also be used for this test case.

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: Message: SetVariableMonitoringResponse with setMonitoringResult[0].status = Accepted	1. Request CSMS to install monitors on: - EVSE #<Configured evseId>, AvailabilityState, <i>Delta</i>, severity 8

Tool validations

* Step 1:

1. CSMS sends **SetVariableMonitoringRequest** with:

- **setMonitoringData[0].value** = 1, ← recommended value for *Delta* monitor
- **setMonitoringData[0].type** = *Delta*,
- **setMonitoringData[0].severity** = 8,
- **setMonitoringData[0].component.name** = "EVSE"
- **setMonitoringData[0].component.evse.id** = <Configured evseId>
- **setMonitoringData[0].variable.name** = "AvailabilityState"

Post scenario validations:

N/A

TC_N_09_CSMS: Set Variable Monitoring - Multiple elements on different component and variable

Test case name	Set Variable Monitoring - Multiple elements on different component and variable
Test case Id	TC_N_09_CSMS
Use case Id(s)	<u>N04</u>
Requirement(s)	<u>N04.FR.01</u> , <u>N04.FR.02</u> , <u>N04.FR.17</u>
System under test	CSMS
Description	CSMS sends a request to activate monitors on different variables.
Purpose	To test that CSMS supports setting of multiple monitors on different variables.
Prerequisite(s)	CS has implemented device model monitoring and MonitoringCtrlr.Enabled = true.

Before (Preparations)

Configuration State:

This test case activates monitors on the following variables:

- Component "EVSE", evse <Configured evseId>, variable "AvailabilityState", monitor type *Delta*
- Component "ChargingStation", variable "AvailabilityState", monitor type *Delta*

It assumes, that no monitor is active on these variables prior to the test.

Note 1: these are required variables for which a monitor can be expected to exist or it can be configured.

Note 2: Any other component/variable combination that supports monitoring could also be used for this test case.

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: Message: SetVariableMonitoringResponse with setMonitoringResult[0].status = Accepted	1. Request CSMS to install monitors on: - EVSE #<Configured evseId>, AvailabilityState, <i>Delta</i>, severity 8 - ChargingStation, AvailabilityState, <i>Delta</i>, severity 8

Tool validations

* Step 1:

1. CSMS sends **SetVariableMonitoringRequest** with:

- **setMonitoringData[0].value** = 1, ← recommended value for *Delta* monitor
- **setMonitoringData[0].type** = *Delta*,
- **setMonitoringData[0].severity** = 8,
- **setMonitoringData[0].component.name** = "EVSE"
- **setMonitoringData[0].component.evse.id** = <Configured evseId>
- **setMonitoringData[0].variable.name** = "AvailabilityState"

- **setMonitoringDate[1].value** = 1,
- **setMonitoringDate[1].type** = *Delta*,
- **setMonitoringDate[1].severity** = 8,
- **setMonitoringDate[1].component.name** = "ChargingStation"
- **setMonitoringDate[1].variable.name** = "AvailabilityState"

Post scenario validations:

N/A

TC_N_16_CSMS: Set Monitoring Level - Success

Test case name	Set Monitoring Level - Success
Test case Id	TC_N_16_CSMS
Use case Id(s)	<u>N05</u>
Requirement(s)	<u>N05.FR.01</u>
System under test	CSMS
Description	CSMS sets a monitoring level.
Purpose	To test that CSMS supports setting of a monitoring level.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

2. Test System responds with:
SetMonitoringLevelResponse with
Status is *Accepted*

1. Instruct CSMS to set a monitoring level with **severity** = *_4*

Tool validations

* Step 1:

Message: **SetMonitoringLevelRequest** with:
severity = *4*

Post scenario validations:

N/A

TC_N_17_CSMS: Set Monitoring Level - Out of range

Test case name	Set Monitoring Level - Out of range
Test case Id	TC_N_17_CSMS
Use case Id(s)	<u>N05</u>
Requirement(s)	<u>N05.FR.02</u>
System under test	CSMS
Description	CSMS sets a monitoring level.
Purpose	To test that CSMS supports the rejection of setting of a monitoring level.
Prerequisite(s)	The Test System will always reject the message, but normally this would only occur if the set severity level is out of range.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with: SetMonitoringLevelResponse with Status is <i>Rejected</i>	1. Instruct CSMS to set a monitoring level with severity = 4

Tool validations

* Step 1:

Message: **SetMonitoringLevelRequest** with:
severity = 4

Post scenario validations:

N/A

TC_N_18_CSMS: Clear Monitoring - Too many elements

Test case name	Clear Monitoring - Too many elements
Test case Id	TC_N_18_CSMS
Use case Id(s)	<u>N06</u>
Requirement(s)	<u>N06.FR.04</u>
System under test	CSMS
Description	CSMS is requested to clear more monitors than allowed in one request.
Purpose	To test that CSMS does not exceed the <code>ItemsPerMessageClearVariableMonitoring</code> amount of monitors in one request.
Prerequisite(s)	CS has implemented device model monitoring and <code>MonitoringCtrlr.Enabled = true</code> .

Before (Preparations)

Configuration State:

This test requests the value of `ItemsPerMessageClearVariableMonitoring` and then instructs the CSMS to clear (at least) one more monitor than allowed by this value. This value is 'read-only', so it cannot be manipulated in the test. As a consequence, if the Charging Station supports more monitor ids in the list, than can be set by the CSMS, then this cannot be tested.

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

2. The Test System responds with:
GetVariablesResponse

1. Instruct CSMS to send *GetVariablesRequest* with:
Component.name *MonitoringCtrlr*
Variable.name *ItemsPerMessage*
Variable.instance *ClearVariableMonitoring*.

4. The Test System responds with:
ClearVariableMonitoringResponse

3. Instruct CSMS to clear more monitors than allowed in *ItemsPerMessage*.
ClearVariableMonitoringRequest with a list of **ids**
Note: these monitor ids do not have to exist.

Tool validations

* Step 1:

Message: Two or more **ClearVariableMonitoringRequest**, so that the maximum number of `ItemsPerMessageClearVariableMonitoring` **ids** is never exceeded.

Test System will reply with a **ClearVariableMonitoringResponse** for each **ClearVariableMonitoringRequest**, but the content of the responses is irrelevant for the test.

Post scenario validations:

N/A

TC_N_21_CSMS: Alert Event - HardWiredMonitor

Test case name	Alert Event - HardWiredMonitor
Test case Id	TC_N_21_CSMS
Use case Id(s)	<u>N07</u>
Requirement(s)	<u>N07.FR.03</u>
System under test	CSMS
Description	Charging Station sends an NotifyEventRequest for a HardWiredMonitor.
Purpose	To test that the CSMS is able to handle a HardWiredMonitor.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**

1. Test System sends **NotifyEventRequest** message with **eventNotificationType = HardWiredMonitor**

2. CSMS returns **NotifyEventResponse** message.

Tool validations

* Step 2:

Message: **NotifyEventResponse** with empty body.

Post scenario validations:

N/A

TC_N_24_CSMS: Set Variable Monitoring - Periodic event

Test case name	Set Variable Monitoring - Periodic event
Test case Id	TC_N_24_CSMS
Use case Id(s)	<u>N08</u>
Requirement(s)	<u>N08.FR.02</u>
System under test	CSMS
Description	Charging Station sends a periodic NotifyEventRequest.
Purpose	To test that CSMS returns a NotifyEventResponse. <i>Note: this is identical to TC_N_21_CSMS, only with a periodic event.</i>
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

*Tester makes Test System send a NotifyEventRequest message.***1.** Test System sends **NotifyEventRequest** message.**2.** CSMS returns **NotifyEventResponse** message.Note(s):*- Step 1 and 2 will be repeated n times***Tool validations**

* Step 2:

Message: **NotifyEventResponse** with empty body.**Post scenario validations:**

N/A

TC_N_25_CSMS: Retrieve Log Information - Diagnostics Log - Success

Test case name	Retrieve Log Information - Diagnostics Log - Success
Test case Id	TC_N_25_CSMS
Use case Id(s)	<u>N01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers the functionality of getting log information from a Charging Station. The CSMS can request a Charging Station to upload a file with log information to a given location (URL). The format of this log file is not prescribed. The Charging Station successfully uploads a log file and gives information about the status of the upload by sending status notifications to the CSMS.
Purpose	To verify if the CSMS is able to request a charging station to successfully upload a log as described at the OCPP specification.
Prerequisite(s)	Charging Station has log information available.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetLogResponse with status <i>Accepted</i>	1. The CSMS sends a GetLogRequest
3. The Test System sends a LogStatusNotificationRequest with - status <i>Uploading</i> - requestId <i>Same Id as the GetLogRequest</i>	4. The CSMS responds with a LogStatusNotificationResponse .
5. The Test System sends a LogStatusNotificationRequest with - status <i>Uploaded</i> - requestId <i>Same Id as the GetLogRequest</i>	6. The CSMS responds with a LogStatusNotificationResponse .

Tool validations

* Step 1:

Message **GetLogRequest**- **logType** *DiagnosticsLog***Post scenario validations:**

- N/a

TC_N_27_CSMS: Get Customer Information - Accepted + data

Test case name	Get Customer Information - Accepted + data
Test case Id	TC_N_27_CSMS
Use case Id(s)	<u>N09</u>
Requirement(s)	<u>N09.FR.01</u> , <u>N09.FR.04</u>
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to retrieve IdToken customer information, for example to be compliant with local privacy laws. The Charging Station notifies the CSMS by sending one or more reports.
Purpose	To verify if the CSMS sends the request correctly and responds on the notifies as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a CustomerInformationResponse with status <i>Accepted</i>	1. The CSMS sends a CustomerInformationRequest
3. The Test System sends a NotifyCustomerInformationRequest	4. The CSMS responds with a NotifyCustomerInformationResponse .

Tool validations

* Step 1:

Message **CustomerInformationRequest**- **report** *true*- **idToken.idToken** <Configured valid_idtoken_idtoken>- **idToken.type** <Configured valid_idtoken_type>**Post scenario validations:**

- N/a

TC_N_28_CSMS: Get Customer Information - Accepted + no data

Test case name	Get Customer Information - Accepted + no data
Test case Id	TC_N_28_CSMS
Use case Id(s)	<u>N09</u>
Requirement(s)	<u>N09.FR.01</u> , <u>N09.FR.04</u>
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to retrieve IdToken customer information, for example to be compliant with local privacy laws. The Charging Station notifies the CSMS by sending one or more reports.
Purpose	To verify if the CSMS sends the request correctly and responds on the notifies as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a CustomerInformationResponse with status <i>Accepted</i>	1. The CSMS sends a CustomerInformationRequest
3. The Test System sends a NotifyCustomerInformationRequest	4. The CSMS responds with a NotifyCustomerInformationResponse .

Tool validations

* Step 1:

Message **CustomerInformationRequest**- **report** *true*- **idToken.idToken** <Configured valid_idtoken_idtoken>- **idToken.type** <Configured valid_idtoken_type>**Post scenario validations:**

- N/a

TC_N_29_CSMS: Get Customer Information - Not Accepted

Test case name	Get Customer Information - Not Accepted
Test case Id	TC_N_29_CSMS
Use case Id(s)	<u>N09</u>
Requirement(s)	<u>N09.FR.01</u> , <u>N09.FR.04</u>
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to retrieve IdToken customer information, but the Charging Station rejects the request.
Purpose	To verify if the CSMS sends the request correctly as described at the OCPP specification, and can handle the Charging Station rejecting the request.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a CustomerInformationResponse with status <i>Rejected</i>	1. The CSMS sends a CustomerInformationRequest

Tool validations

* Step 1:

Message **CustomerInformationRequest**- **report** *true*- **idToken.idToken** <Configured valid_idtoken_idtoken>- **idToken.type** <Configured valid_idtoken_type>

Post scenario validations:

- N/a

TC_N_30_CSMS: Clear Customer Information - Clear and report + data

Test case name	Clear Customer Information - Clear and report + data
Test case Id	TC_N_30_CSMS
Use case Id(s)	<u>N10</u>
Requirement(s)	<u>N10.FR.08</u>
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to clear (and retrieve) IdToken customer information, for example to be compliant with local privacy laws. The Charging Station notifies the CSMS by sending one or more reports.
Purpose	To verify if the CSMS sends the request correctly and responds on the notifies as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a CustomerInformationResponse with status <i>Accepted</i>	1. The CSMS sends a CustomerInformationRequest
3. The Test System sends a NotifyCustomerInformationRequest	4. The CSMS responds with a NotifyCustomerInformationResponse .

Tool validations*** Step 1:**Message **CustomerInformationRequest**- **report** *true*- **clear** *true*- **idToken.idToken** <Configured valid_idtoken_idtoken>- **idToken.type** <Configured valid_idtoken_type>**Post scenario validations:**

- N/a

TC_N_31_CSMS: Clear Customer Information - Clear and report + no data

Test case name	Clear Customer Information - Clear and report + no data
Test case Id	TC_N_31_CSMS
Use case Id(s)	<u>N10</u>
Requirement(s)	<u>N10.FR.08</u>
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to clear (and retrieve) IdToken customer information, for example to be compliant with local privacy laws. The Charging Station notifies the CSMS by sending one or more reports.
Purpose	To verify if the CSMS sends the request correctly and responds on the notifies as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a CustomerInformationResponse with status <i>Accepted</i>	1. The CSMS sends a CustomerInformationRequest
3. The Test System sends a NotifyCustomerInformationRequest	4. The CSMS responds with a NotifyCustomerInformationResponse .

Tool validations

* Step 1:

Message **CustomerInformationRequest**- **report** *true*- **clear** *true*- **idToken.idToken** <Configured valid_idtoken_idtoken>- **idToken.type** <Configured valid_idtoken_type>**Post scenario validations:**

- N/a

TC_N_32_CSMS: Clear Customer Information - Clear and no report

Test case name	Clear Customer Information - Clear and no report
Test case Id	TC_N_32_CSMS
Use case Id(s)	<u>N10</u>
Requirement(s)	<u>N10.FR.08</u>
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to clear IdToken customer information, for example to be compliant with local privacy laws.
Purpose	To verify if the CSMS sends the request correctly.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a CustomerInformationResponse with status <i>Accepted</i>	1. The CSMS sends a CustomerInformationRequest
3. The Test System sends a NotifyCustomerInformationRequest	4. The CSMS responds with a NotifyCustomerInformationResponse

Tool validations

* Step 1:

Message **CustomerInformationRequest**- **report** *false*- **clear** *true*- **idToken.idToken** <Configured valid_idtoken_idtoken>- **idToken.type** <Configured valid_idtoken_type>**Post scenario validations:**

- N/a

TC_N_62_CSMS: Clear Customer Information - Clear and report - customerIdentifier

Test case name	Clear Customer Information - Clear and report - customerIdentifier
Test case Id	TC_N_62_CSMS
Use case Id(s)	<u>N10</u>
Requirement(s)	<u>N10.FR.08</u>
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to clear (and retrieve) raw customer information, for example to be compliant with local privacy laws. The Charging Station notifies the CSMS by sending one or more reports.
Purpose	To verify if the CSMS sends the request correctly and responds to the notifications as described in the OCPP specification.
Prerequisite(s)	The CSMS supports retrieving / deleting CustomerInformation - CustomerIdentifier

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State: N/a

Main (Test scenario)	
Charging Station	CSMS
2. The Test System responds with a CustomerInformationResponse with status <i>Accepted</i>	1. The CSMS sends a CustomerInformationRequest
3. The Test System sends a NotifyCustomerInformationRequest	4. The CSMS responds with a NotifyCustomerInformationResponse

Tool validations
* Step 1: Message CustomerInformationRequest - report <i>true</i> - clear <i>true</i> - customerIdentifier <i>"OpenChargeAlliance"</i> - idToken is omitted - customerCertificate is omitted
Post scenario validations: - N/a

TC_N_63_CSMS: Clear Customer Information - Clear and report - customerCertificate

Test case name	Clear Customer Information - Clear and report - customerCertificate
Test case Id	TC_N_63_CSMS
Use case Id(s)	<u>N10</u>
Requirement(s)	<u>N10.FR.08</u>
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to clear (and retrieve) a customer certificate, for example to be compliant with local privacy laws. The Charging Station notifies the CSMS by sending one or more reports.
Purpose	To verify if the CSMS sends the request correctly and responds to the notifications as described in the OCPP specification.
Prerequisite(s)	The CSMS supports retrieving / deleting CustomerInformation - CustomerCertificate

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a CustomerInformationResponse with status <i>Accepted</i>	1. The CSMS sends a CustomerInformationRequest with specific hash data <i><customer certificate hash data></i> .
3. The Test System sends a NotifyCustomerInformationRequest	4. The CSMS responds with a NotifyCustomerInformationResponse

Tool validations

* Step 1:

Message **CustomerInformationRequest**- **report** *true*- **clear** *true*- **customerCertificate** contains *<customer certificate hash data>*

Post scenario validations:

- N/a

TC_N_34_CSMS: Retrieve Log Information - Rejected

Test case name	Retrieve Log Information - Rejected
Test case Id	TC_N_34_CSMS
Use case Id(s)	<u>N01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers the functionality of getting log information from a Charging Station. The CSMS can request a Charging Station to upload a file with log information to a given location (URL). The format of this log file is not prescribed. The Charging Station successfully uploads a log file and gives information about the status of the upload by sending status notifications to the CSMS.
Purpose	To verify if the CSMS is able to request a charging station to successfully upload a log as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetLogResponse with status <i>Rejected</i>	1. The CSMS sends a GetLogRequest

Tool validations

N/a

Post scenario validations:

- N/a

TC_N_35_CSMS: Retrieve Log Information - Security Log - Success

Test case name	Retrieve Log Information - Security Log - Success
Test case Id	TC_N_35_CSMS
Use case Id(s)	<u>N01</u>
Requirement(s)	
System under test	CSMS
Description	This test case covers the functionality of getting log information from a Charging Station. The CSMS can request a Charging Station to upload a file with log information to a given location (URL). The format of this log file is not prescribed. The Charging Station successfully uploads a log file and gives information about the status of the upload by sending status notifications to the CSMS.
Purpose	To verify if the CSMS is able to request a charging station to successfully upload a log as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

Charging Station has log information available.

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetLogResponse with status <i>Accepted</i>	1. The CSMS sends a GetLogRequest
3. The Test System sends a LogStatusNotificationRequest with - status <i>Uploading</i> - requestId <i>Same Id as the GetLogRequest</i>	4. The CSMS responds with a LogStatusNotificationResponse .
5. The Test System sends a LogStatusNotificationRequest with - status <i>Uploaded</i> - requestId <i>Same Id as the GetLogRequest</i>	6. The Test System responds with a LogStatusNotificationResponse .

Tool validations

* Step 1:

Message **GetLogRequest**- **logType** *SecurityLog***Post scenario validations:**

- N/a

TC_N_36_CSMS: Retrieve Log Information - Second Request

Test case name	Retrieve Log Information - Second Request
Test case Id	TC_N_36_CSMS
Use case Id(s)	<u>N01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case covers the functionality of getting log information from a Charging Station. The CSMS can request a Charging Station to upload a file with log information to a given location (URL). The format of this log file is not prescribed. The Charging Station successfully uploads a log file and gives information about the status of the upload by sending status notifications to the CSMS.
Purpose	To verify if the CSMS is able to request a second request while the charging station is uploading a log as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

Charging Station has log information available.

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
	1. The CSMS sends a GetLogRequest
2. The Test System responds with a GetLogResponse with status Accepted	
3. The Test System sends a LogStatusNotificationRequest with - status Uploading - requestId Same Id as the GetLogRequest from Step 1	4. The CSMS responds with a LogStatusNotificationResponse .
6. The Test System responds with a GetLogResponse with status AcceptedCanceled	5. The CSMS sends a GetLogRequest
7. The Test System sends a LogStatusNotificationRequest with - status AcceptedCanceled - requestId Same Id as the GetLogRequest from Step 1	8. The CSMS responds with a LogStatusNotificationResponse .
9. The Test System sends a LogStatusNotificationRequest with - status Uploading - requestId Same Id as the GetLogRequest from Step 5	10. The CSMS responds with a LogStatusNotificationResponse .

Main (Test scenario)	
11. The Test System sends a LogStatusNotificationRequest with - status <i>Uploaded</i> - requestId <i>Same Id as the GetLogRequest from Step 5</i>	12. The CSMS responds with a LogStatusNotificationResponse .
Tool validations	
N/a	
Post scenario validations:	
- N/a	

TC_N_44_CSMS: Clear Monitoring - Rejected

Test case name	Clear Monitoring - Rejected
Test case Id	TC_N_44_CSMS
Use case Id(s)	<u>N06</u>
Requirement(s)	N/a
System under test	CSMS
Description	A monitoring setting can be cleared (removed) by sending a <code>ClearVariableMonitoringRequest</code> with the id of the monitoring setting.
Purpose	To verify if the CSMS is able to correctly read the respond from a charging station on a request to clear a monitor that cannot be cleared as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

2. The Test System responds with a `ClearVariableMonitoringResponse` with `clearMonitoringResult[0].status` *Rejected*

CSMS

1. The CSMS sends a `ClearVariableMonitoringRequest`

Tool validations

N/a

Post scenario validations:

- N/a

TC_N_46_CSMS: Clear Customer Information - Update Local Authorization List

Test case name	Clear Customer Information - Update Local Authorization List
Test case Id	TC_N_46_CSMS
Use case Id(s)	N10
Requirement(s)	N10.FR.02, N10.FR.08, D01.FR.01, D01.FR.06, D01.FR.18,
System under test	CSMS
Description	The CSMS sends a message to the Charging Station to clear (and retrieve) raw customer information, for example to be compliant with local privacy laws. The Charging Station notifies the CSMS by sending one or more reports.
Purpose	To verify if the CSMS updates the local authorization list when customer information, which was present in the local authorization list, has been removed as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

A local authorization list with <Configured valid_idtoken_idtoken> is configured.

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual action: Trigger CSMS to CustomerInformationRequest to both report and clear token <Configured valid_idtoken_idtoken> and <Configured valid_idtoken_type>

2. The Test System responds with a **CustomerInformationResponse** with **status Accepted**

1. The CSMS sends a **CustomerInformationRequest**

3. The Test System sends a **NotifyCustomerInformationRequest**

4. The CSMS responds with a **NotifyCustomerInformationResponse** .

Manual action: If not triggered automatically, trigger CSMS to send SendLocalListRequest with **version** = <configured local list version> + 1 and **updateType** = Differential and **localAuthorizationList** = [{idToken = {<Configured valid_idtoken>, <Configured valid_idtoken_type>}}]

6 The Test System responds with a **SendLocalListResponse** with **status Accepted**

5. The CSMS sends a **SendLocalListRequest**

Tool validations
* Step 1: Message CustomerInformationRequest - report <i>true</i> AND - clear <i>true</i> AND - idToken.idToken <i><Configured valid_idtoken_idtoken></i> - idToken.type <i><Configured valid_idtoken_type></i> * Step 5: Message SendLocalListRequest - updateType <i>Differential</i> - versionNumber <i><configured local list version> + 1</i> - localAuthorizationListp[0].idToken contains <i><configured_valid_idtoken_idtoken></i> and <i><configured valid_idtoken_type></i> - localAuthorizationList[0].idTokenInfo <i><omitted></i>
Post scenario validations: - All messages have been received

TC_N_47_CSMS: Get Monitoring report - Report all

Test case name	Get Monitoring report - Report all
Test case Id	TC_N_47_CSMS
Use case Id(s)	N02
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS requests the Charging Station to send a report about configured monitoring settings per component and variable. Optionally, this list can be filtered on monitoringCriteria and componentVariables.
Purpose	To verify if the CSMS is able to send a get monitor request omitting the monitoringCriteria and componentVariable as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

2. The Test System responds with a **GetMonitoringReportResponse**

3. The Test System sends a **NotifyMonitoringReportRequest**

CSMS

1. The CSMS sends a **GetMonitoringReportRequest**

4. The CSMS responds with a **NotifyMonitoringReportResponse** .

Note(s):

- If **tb** is True at Step 3 then step 3 and 4 will be repeated

Tool validations

* Step 1:

Message **GetMonitoringReportRequest**

- **monitoringCriteria** omitted AND

- **componentVariable** omitted.

Post scenario validations:

- N/a

TC_N_48_CSMS: Alert Event - Variable monitoring on write only

Test case name	Alert Event - Variable monitoring on write only
Test case Id	TC_N_48_CSMS
Use case Id(s)	<u>N07</u>
Requirement(s)	N/a
System under test	CSMS
Description	NotifyEventRequest reports every Component/Variable for which a VariableMonitoring setting was triggered. Only the VariableMonitoring settings that are responsible for triggering an event are included.
Purpose	To verify if the CSMS is able to read a request from a trigger from a variablemonitor which is write only as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

1. The Test System sends a **NotifyEventRequest** with **eventData.actualValue** empty

CSMS

2. The CSMS responds with a **NotifyEventResponse** .

Tool validations

N/a

Post scenario validations:

- N/a

TC_N_49_CSMS: Alert Event - LowerThreshold/UpperThreshold cleared after reboot

Test case name	Alert Event - LowerThreshold/UpperThreshold cleared after reboot
Test case Id	TC_N_49_CSMS
Use case Id(s)	<u>N07</u>
Requirement(s)	N/a
System under test	CSMS
Description	NotifyEventRequest reports every Component/Variable for which a VariableMonitoring setting was triggered. Only the VariableMonitoring settings that are responsible for triggering an event are included.
Purpose	To verify if the CSMS is able to read a request when a trigger is cleared after a reboot as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

1. The Test System sends a **NotifyEventRequest** with **eventData.cleared true**

CSMS

2. The CSMS responds with a **NotifyEventResponse** .

Tool validations

N/a

Post scenario validations:

- N/a

TC_N_50_CSMS: Alert Event - Periodic Triggered

Test case name	Alert Event - Periodic Triggered
Test case Id	TC_N_50_CSMS
Use case Id(s)	<u>N07</u>
Requirement(s)	N/a
System under test	CSMS
Description	NotifyEventRequest reports every Component/Variable for which a VariableMonitoring setting was triggered. Only the VariableMonitoring settings that are responsible for triggering an event are included.
Purpose	To verify if the CSMS is able to read a request when a trigger reason is periodic after a reboot as described at the OCPP specification.
Prerequisite(s)	n/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

1. The Test System sends a **NotifyEventRequest** with **eventData.trigger** *Periodic*

CSMS

2. The CSMS responds with a **NotifyEventResponse** .

Tool validations

N/a

Post scenario validations:

- N/a

TC_N_100_CSMS: Retrieve Log Information - DataCollectorLog - Success

Test case name	Retrieve Log Information - DataCollectorLog - Success
Test case Id	TC_N_100_CSMS
Use case Id(s)	<u>N01</u>
Requirement(s)	...
System under test	Charging Station
Description	This test case covers the functionality of getting log information from a Charging Station. The CSMS can request a Charging Station to upload a file with log information to a given location (URL). The format of this log file is not prescribed. The Charging Station successfully uploads a log file and gives information about the status of the upload by sending status notifications to the CSMS.
Purpose	To verify if the CSMS is able to successfully retrieve a log of logType <i>DataCollectorLog</i> as described at the OCPP specification.
Prerequisite(s)	- CSMS supports DataCollectorLog. - A diagnostics logging server has been setup supporting one of the file transfer protocols supported by the Charging Station (This is configured at the configuration variable FileTransferProtocols). - Charging station supports DataCollector (Component DataCollector available).

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
<u>Manual Action:</u> Request the CSMS to retrieve the DataCollectorLog	
2. The Test System responds with a GetLogResponse	1. The CSMS sends a GetLogRequest
3. The Test System sends a LogStatusNotificationRequest with status <i>Uploading</i> requestId <GetLogRequest.requestId of step 1>	4. The CSMS responds with a LogStatusNotificationResponse .
5. The Test System sends a LogStatusNotificationRequest with status <i>Uploaded</i> requestId <GetLogRequest.requestId of step 1>	6. The CSMS responds with a LogStatusNotificationResponse .

Tool validations

* Step 1:

Message **GetLogRequest**- **logType** *DataCollectorLog*

Tool validations**Post scenario validations:**

N/a

TC_N_102_CSMS: Retrieve Log Information - Authentication - HTTP

Test case name	Retrieve Log Information - Authentication - HTTP
Test case Id	TC_N_102_CSMS
Use case Id(s)	<u>N01</u>
Requirement(s)	<u>N01.FR.26</u> , <u>N01.FR.27</u> , <u>N01.FR.28</u>
System under test	CSMS
Description	This test case covers the functionality of getting log information from a Charging Station. The CSMS can request a Charging Station to upload a file with log information to a given location (URL). The format of this log file is not prescribed. The Charging Station successfully uploads a log file and gives information about the status of the upload by sending status notifications to the CSMS.
Purpose	To verify if the CSMS is able to successfully request and process a log of logType <i>DiagnosticsLog</i> as described at the OCPP specification.
Prerequisite(s)	- CSMS supports HTTP file transfer protocol. - A diagnostics logging server has been set up for HTTP and accepts specific userinfo credentials .

Before (Preparations)**Configuration State:**

CSMS configured to prevent unauthorised uploading to HTTP log location.

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
<u>Manual action:</u> Trigger CSMS to GetLogRequest with logType <i>DiagnosticsLog</i> and remoteLocation with userinfo for HTTP Basic Authentication	
2. Test System responds with a GetLogResponse with status Accepted	1. CSMS sends a GetLogRequest
3. The Test System uploads log file to <GetLogRequest.log.remoteLocation of step 1 without userinfo> using HTTP POST	4. The CSMS responds using HTTP
5. The Test System uploads file to <GetLogRequest.log.remoteLocation of step 1> using HTTP POST	6. The CSMS responds using HTTP
7. Test System sends a LogStatusNotificationRequest with status Uploading requestId <GetLogRequest.requestId of step 1>	8. CSMS responds with a LogStatusNotificationResponse
9. Test System sends a LogStatusNotificationRequest with status Uploaded requestId <GetLogRequest.requestId of step 1>	10. CSMS responds with a LogStatusNotificationResponse

Tool validations

* Step 1:

Message **GetLogRequest**

- **logType** must be *DiagnosticsLog*
- **log.remoteLocation** must
 - contain a valid URL
 - specify scheme *HTTP*
 - contain userinfo for HTTP Basic Authentication
 - not contain a URL fragment component
 - not contain a URL query components that ends in a slash character ("/")

* Step 4:

- HTTP status code must be *401 Unauthorized*

* Step 6:

- HTTP status code must be in *2xx*-range

Post scenario validations:

N/a

TC_N_103_CSMS: Retrieve Log Information - Authentication - HTTPS

Test case name	Retrieve Log Information - Authentication - HTTPS
Test case Id	TC_N_102_CSMS
Use case Id(s)	<u>N01</u>
Requirement(s)	<u>N01.FR.26</u> , <u>N01.FR.27</u> , <u>N01.FR.28</u>
System under test	CSMS
Description	This test case covers the functionality of getting log information from a Charging Station. The CSMS can request a Charging Station to upload a file with log information to a given location (URL). The format of this log file is not prescribed. The Charging Station successfully uploads a log file and gives information about the status of the upload by sending status notifications to the CSMS.
Purpose	To verify if the CSMS is able to successfully request and process a log of logType <i>DiagnosticsLog</i> as described at the OCPP specification.
Prerequisite(s)	- CSMS supports HTTPS file transfer protocol. - A diagnostics logging server has been set up for HTTPS and accepts specific userinfo credentials .

Before (Preparations)**Configuration State:**

CSMS configured to prevent unauthorised uploading to HTTPS log location.

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
<u>Manual action:</u> Trigger CSMS to GetLogRequest with logType <i>DiagnosticsLog</i> and remoteLocation with userinfo for HTTP Basic Authentication	
2. Test System responds with a GetLogResponse with status Accepted	1. CSMS sends a GetLogRequest
3. The Test System uploads log file to <GetLogRequest.log.remoteLocation of step 1 without userinfo> using HTTP POST	4. The CSMS responds using HTTP
5. The Test System uploads file to <GetLogRequest.log.remoteLocation of step 1> using HTTP POST	6. The CSMS responds using HTTP
7. Test System sends a LogStatusNotificationRequest with status Uploading requestId <GetLogRequest.requestId of step 1>	8. CSMS responds with a LogStatusNotificationResponse
9. Test System sends a LogStatusNotificationRequest with status Uploaded requestId <GetLogRequest.requestId of step 1>	10. CSMS responds with a LogStatusNotificationResponse

Tool validations

* Step 1:

Message **GetLogRequest**

- **logType** must be *DiagnosticsLog*
- **log.remoteLocation** must
 - contain a valid URL
 - specify scheme *HTTPS*
 - contain userinfo for HTTP Basic Authentication
 - not contain a URL fragment component
 - not contain a URL query components that ends in a slash character ("/")

* Step 4:

- HTTP status code must be *401 Unauthorized*

* Step 6:

- HTTP status code must be in 2xx-range

Post scenario validations:

N/a

TC_N_104_CSMS: Get Monitoring report - TargetDeltaMonitoring

Test case name	Get Monitoring report - TargetDeltaMonitoring
Test case Id	TC_N_104_CSMS
Use case Id(s)	<u>N02</u>
Requirement(s)	...
System under test	CSMS
Description	CSMS requests a report of monitors that match the component criteria.
Purpose	To test that CSMS supports requesting a monitoring report for the component criteria and that it handles an empty result set.
Prerequisite(s)	CS has implemented device model monitoring and MonitoringCtrlr.Enabled = true.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual action: Trigger CSMS to get a report of monitors for DeltaMonitoring

2. Test System responds with a **GetMonitoringReportResponse** with **Status Accepted**

1. CSMS sends **GetMonitoringReportRequest**

Main (Test scenario)	
3. Test System responds with a NotifyMonitoringReportRequest with requestId <GetMonitoringReportRequest.requestId from step 1> tb false seqNo 0 monitor[0].component.name EVSE monitor[0].component.evse.id 1 monitor[0].variable.name Power monitor[0].variableMonitoring[0].type TargetDeltaRelative monitor[0].variableMonitoring[0].transaction = false monitor[0].variableMonitoring[0].value = 5 monitor[0].variableMonitoring[0].eventNotificationType = <not omitted> monitor[0].variableMonitoring[1].type TargetDelta monitor[0].variableMonitoring[1].transaction = false monitor[0].variableMonitoring[1].value = 5 monitor[0].variableMonitoring[1].eventNotificationType = <not omitted>	4. CSMS sends NotifyMonitoringReportResponse
Tool validations	
* Step 1: Message: GetMonitoringReportRequest - monitoringCriteria = DeltaMonitoring	
Post scenario validations: N/a	

TC_N_105_CSMS: Set Variable Monitoring - Set Frequent Periodic Variable Monitoring - Periodic

Test case name	Set Variable Monitoring - Set Frequent Periodic Variable Monitoring - Periodic
Test case Id	TC_N_105_CSMS
Use case Id(s)	<u>N11</u> , <u>N14</u> , <u>N15</u>
Requirement(s)	<u>N11.FR.06</u> , <u>N11.FR.08</u> , <u>N14.FR.01</u> , <u>N15.FR.02</u>
System under test	CSMS
Description	To give the CSMS the ability to request efficient frequent periodic monitoring of variables.
Purpose	To verify if the CSMS can accept and receive periodic event streams.
Prerequisite(s)	- The CSMS supports periodic event streams.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request CSMS to install a monitor on:**component.name** EVSE**EVSE.id** <Configured evseId>**variable.name** Power**type** Periodic**severity** 8**value** 5**periodicEventStream.interval** 5**periodicEventStream.values** 10

2. Test System responds with:

Message: **SetVariableMonitoringResponse** with**setMonitoringResult[0].status** = *Accepted*3. Test System sends a **OpenPeriodicEventStreamRequest** with
constantStreamData.id 2**constantStreamData.variableMonitoringId** <monitor id from
step 1>**constantStreamData.params.interval** 5**constantStreamData.params.values** 101. CSMS sends a **SetVariableMonitoringRequest**4. CSMS responds with a **OpenPeriodicEventStreamResponse**Note: The Test System waits 5 seconds.

Main (Test scenario)	
5. Test System sends a NotifyPeriodicEventStream using RFC framework SEND	
<u>Manual Action:</u> Request the CSMS to reconfigure the periodic event stream to an interval of 10s and values 20	
7. Test System responds with a AdjustPeriodicEventStreamResponse with status Accepted	6. CSMS sends a AdjustPeriodicEventStreamRequest
<u>Note:</u> The Test System waits 10 seconds.	
8. Test System sends a NotifyPeriodicEventStream using RFC framework SEND	
9. Test System sends a ClosePeriodicEventStreamRequest with constantStreamData.id 2	10. CSMS responds with a ClosePeriodicEventStreamResponse

Tool validations
* Step 1: Message SetVariableMonitoringRequest with: - setMonitoringData[0].type = <i>Periodic</i>, - setMonitoringData[0].severity = 8, - setMonitoringData[0].component.name = "EVSE" - setMonitoringData[0].component.evse.id = <Configured evseId> - setMonitoringData[0].variable.name = <i>Power</i> - setMonitoringData[0].value = 5 - setMonitoringData[0].periodicEventStream.interval 5 - setMonitoringData[0].periodicEventStream.values 10 * Step 4: Message OpenPeriodicEventStreamResponse - status must be <i>Accepted</i> * Step 6: Message AdjustPeriodicEventStreamRequest id must be 2 params.interval must be 10 params.values must be 20 Post scenario validations: - CSMS does not respond with any message when receiving message NotifyPeriodicEventStream of step 5.

TC_N_107_CSMS: Set Variable Monitoring - Get Periodic Event Streams - Goodflow

Test case name	Set Variable Monitoring - Get Periodic Event Streams - Goodflow
Test case Id	TC_N_107_CSMS
Use case Id(s)	<u>N12</u>
Requirement(s)	<u>N12.FR.01</u>
System under test	CSMS
Description	To get a list of existing event streams
Purpose	To verify if the CSMS can request retrieving a list of periodic event streams.
Prerequisite(s)	- The CSMS supports periodic event streams.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to retrieve a list of existing event streams from the Charging Station

2. Test System responds with a **GetPeriodicEventStreamResponse** with
constantStreamData[0].id 2
constantStreamData[0].variableMonitoringId 3
constantStreamData[0].params.interval 10
constantStreamData[0].params.values 30
constantStreamData[1].id 3
constantStreamData[1].variableMonitoringId 4
constantStreamData[1].params.interval 10
constantStreamData[1].params.values 30

1. CSMS sends a **GetPeriodicEventStreamRequest**

Tool validations

N/a

Post scenario validations:

N/a

16. O Display Message

TC_O_01_CSMS: Set Display Message - Success

Test case name	Set Display Message - Success
Test case Id	TC_O_01_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	<u>O01.FR.04</u>
System under test	CSMS
Description	This test case describes how the CSMS can be requested to send an SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send the request according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a SetDisplayMessageRequest.

2. The Test System responds with a **SetDisplayMessageResponse** with:
status *Accepted*

1. The CSMS sends a **SetDisplayMessageRequest** with:
state *<Configured display message state>*

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**

- **message.id** *<Generated Id>*

- **message.priority** *<Configured Priority>*

- **message.message.format** *<Configured Format>*

- **message.state** *<Configured State>*

Post scenario validations:

- N/a

DRAFT

TC_O_02_CSMS: Get all Display Messages - Success

Test case name	Get all Display Messages - Success
Test case Id	TC_O_02_CSMS
Use case Id(s)	<u>O03</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can request all the installed DisplayMessages configured via OCPP in a Charging Station. The Charging Station can remove messages when they are out-dated, or transactions have ended. It can be very useful for a CSO to be able to view to current list of messages, so the CSO knows which messages are (still) configured.
Purpose	To verify if the CSMS is able to send the request to get the DisplayMessages according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

A display message is configured.

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetDisplayMessagesResponse with status <i>Accepted</i>	1. The CSMS sends a GetDisplayMessagesRequest
3. The Test System sends a NotifyDisplayMessagesRequest	4. The CSMS responds with a NotifyDisplayMessagesResponse

Tool validations

* Step 1:

Message **GetDisplayMessagesRequest**- **requestId** <Generated Id>- **id** <Omitted>- **priority** <Omitted>- **state** <Omitted>**Post scenario validations:**

- N/a

TC_O_03_CSMS: Get all Display Messages - No DisplayMessages configured

Test case name	Get all Display Messages - No DisplayMessages configured
Test case Id	TC_O_03_CSMS
Use case Id(s)	<u>O03</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can request all the installed DisplayMessages configured via OCPP in a Charging Station. The Charging Station can remove messages when they are out-dated, or transactions have ended. It can be very useful for a CSO to be able to view to current list of messages, so the CSO knows which messages are (still) configured.
Purpose	To verify if the CSMS can request to get all display messages according to the DisplayMessage mechanism as described in the OCPP specification when no messages are configured.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetDisplayMessagesResponse with status <i>Unknown</i>	1. The CSMS sends a GetDisplayMessagesRequest

Tool validations*** Step 1:**Message **GetDisplayMessagesRequest**- **requestId** <Generated request id>**Post scenario validations:**

- N/a

TC_O_04_CSMS: Clear Display Message - Success

Test case name	Clear Display Message - Success
Test case Id	TC_O_04_CSMS
Use case Id(s)	<u>O05</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can remove a specific message, configured via OCPP in a Charging Station.
Purpose	To verify if the CSMS is able to request the Charging Station to clear a message according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

A display message is configured.

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Note: As a help method, a *GetDisplayMessagesRequest* is requested first for CSMS's that implemented their *ClearDisplayMessage* as a combined feature.

2. The Test System responds with a **ClearDisplayMessageResponse** with **status Accepted**

1. The CSMS sends a **ClearDisplayMessageRequest**

Tool validations

* Step 1:

Message **ClearDisplayMessageRequest**- **id** <Generated Id from set display message>**Post scenario validations:**

- N/a

TC_O_05_CSMS: Clear Display Message - Unknown Key

Test case name	Clear Display Message - Unknown Key
Test case Id	TC_O_05_CSMS
Use case Id(s)	<u>O05</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can remove a specific message, configured via OCPP in a Charging Station.
Purpose	To verify if the CSMS is able to request the Charging Station to clear a message according to the mechanism as described in the OCPP specification.
Prerequisite(s)	If the CSMS supports sending a ClearDisplayMessageRequest with an unknown id.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a ClearDisplayMessageResponse with status <i>Unknown</i>	1. The CSMS sends a ClearDisplayMessageRequest

Tool validations

N/a

Post scenario validations:

- N/a

TC_O_06_CSMS: Set Display Message - Specific transaction - Success

Test case name	Set Display Message - Specific transaction - Success
Test case Id	TC_O_06_CSMS
Use case Id(s)	O02
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can set a message to be displayed on a Charging Station for a specific transaction. Depending on the given parameters the message shall be displayed a certain way on the Charging Station.
Purpose	To verify if the CSMS is able to send a display message correctly according the mechanism as described in the OCPP specification for a specific transaction.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a display message for a specific transaction.

2. The Test System responds with a **SetDisplayMessageResponse** with **status Accepted**

1. The CSMS sends a **SetDisplayMessageRequest**

3. Execute **Reusable State EVDisconnected**

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.transactionId** Same ID as previously returned by the Charging Station AND- **message.priority** <Configured Priority>

Post scenario validations:

- N/a

TC_O_07_CSMS: Get a Specific Display Message - Id

Test case name	Get a Specific Display Message - Id
Test case Id	TC_O_07_CSMS
Use case Id(s)	<u>004</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can request specific installed DisplayMessages configured via OCPP in a Charging Station. The Charging Station can remove messages when they are out-dated, or transactions have ended. It can be very useful for a CSO to be able to view to current list of messages, so the CSO knows which messages are (still) configured.
Purpose	To verify if the CSMS is able to request a specific id message from the charging station according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

A display message is configured.

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetDisplayMessagesResponse with <i>status Accepted</i>	1. The CSMS sends a GetDisplayMessagesRequest
3. The Test System sends a NotifyDisplayMessagesRequest	4. The CSMS responds with a NotifyDisplayMessagesResponse .

Tool validations

* Step 1:

Message **GetDisplayMessagesRequest**- **id** <Configured_Id>- **priority** <Omitted>- **state** <Omitted>- **requestId** <Generated Id>**Post scenario validations:**

- N/a

TC_O_08_CSMS: Get a Specific Display Message - Priority

Test case name	Get a Specific Display Message - Priority
Test case Id	TC_O_08_CSMS
Use case Id(s)	<u>004</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can request specific installed DisplayMessages configured via OCPP in a Charging Station. The Charging Station can remove messages when they are out-dated, or transactions have ended. It can be very useful for a CSO to be able to view to current list of messages, so the CSO knows which messages are (still) configured.
Purpose	To verify if the CSMS is able to request specific priority messages from the charging station according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

A message with <Configured_Priority> is configured

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetDisplayMessagesResponse with status <i>Accepted</i>	1. The CSMS sends a GetDisplayMessagesRequest
3. The Test System sends a NotifyDisplayMessagesRequest	4. The CSMS responds with a NotifyDisplayMessagesResponse

Tool validations

* Step 1:

Message **GetDisplayMessagesRequest**- **priority** <Configured_Priority>- **id** <Omitted>- **state** <Omitted>- **requestId** <Generated Id>**Post scenario validations:**

- N/a

TC_O_09_CSMS: Get a Specific Display Message - State

Test case name	Get a Specific Display Message - State
Test case Id	TC_O_09_CSMS
Use case Id(s)	<u>004</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can request specific installed DisplayMessages configured via OCPP in a Charging Station. The Charging Station can remove messages when they are out-dated, or transactions have ended. It can be very useful for a CSO to be able to view to current list of messages, so the CSO knows which messages are (still) configured.
Purpose	To verify if the CSMS is able to request specific state messages from the charging station according to the mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

A message with <Configured_State> is configured

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetDisplayMessagesResponse with status Accepted	1. The CSMS sends a GetDisplayMessagesRequest
3. The Test System sends a NotifyDisplayMessagesRequest	4. The CSMS responds with a NotifyDisplayMessagesResponse .

Tool validations

* Step 1:

Message **GetDisplayMessagesRequest**- **state** <Configured_State>- **priority** <Omitted>- **id** <Omitted>- **requestId** <Generated Id>**Post scenario validations:**

- N/a

TC_O_10_CSMS: Set Display Message - Specific transaction - UnknownTransaction

Test case name	Set Display Message - Specific transaction - UnknownTransaction
Test case Id	TC_O_10_CSMS
Use case Id(s)	O02
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSMS can attempt to set a DisplayMessage for a transactionId that the CS does not know. The CS will respond with a SetDisplayMessageResponse status of UnknownTransaction.
Purpose	To verify if the CSMS is able to send a display message correctly according the mechanism as described in the OCPP specification for a specific transaction.
Prerequisite(s)	If the CSMS supports sending a SetDisplayMessageRequest with a transactionId for a transaction that does not exist.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a display message for a specific transaction.

2. The Test System responds with a **SetDisplayMessageResponse** with status *UnknownTransaction*

1. The CSMS sends a **SetDisplayMessageRequest**

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.transactionId** not omit AND- **message.priority** <Configured Priority>

Post scenario validations:

- N/a

TC_O_11_CSMS: Get a Specific Display Message - Unknown parameters

Test case name	Get a Specific Display Message - Unknown parameters
Test case Id	TC_O_11_CSMS
Use case Id(s)	<u>004</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can request specific installed DisplayMessages configured via OCPP in a Charging Station. The Charging Station can remove messages when they are out-dated, or transactions have ended. It can be very useful for a CSO to be able to view to current list of messages, so the CSO knows which messages are (still) configured.
Purpose	To verify if the CSMS is able to request a specific id message from the charging station according to the mechanism as described in the OCPP specification.
Prerequisite(s)	If the CSMS is able to send a GetDisplayMessage with an unknown id.

Before (Preparations)**Configuration State:**

N/a

Memory State:

A display message is configured.

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a GetDisplayMessagesResponse with status <i>Unknown</i>	1. The CSMS sends a GetDisplayMessagesRequest

Tool validations

* Step 1:

Message **GetDisplayMessagesRequest**- **id** <A different generated Id>- **requestId** <Generated Id>**Post scenario validations:**

- N/a

TC_O_12_CSMS: Set Display Message - Replace DisplayMessage

Test case name	Set Display Message - Replace DisplayMessage
Test case Id	TC_O_12_CSMS
Use case Id(s)	O06
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how a CSO can replace a DisplayMessage that is previously configured in a Charging Station. Replace the message content, but also all the given parameters with the new one.
Purpose	To verify if the CSMS is able to request to replace a display message according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

A display message is configured.

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action:

- Request the CSMS to sent a display message with the same id as already configured one

2. The Test System responds with a **SetDisplayMessageResponse** with **status Accepted**

1. The CSMS sends a **SetDisplayMessageRequest** with:
message.id <Configured_Id>
message.priority <Configured Priority>

Tool validations

* Step 2:

Message **SetDisplayMessageRequest**- **message.id** <Configured_Id>- **message.priority** <Configured Priority>

Post scenario validations:

- N/a

TC_O_13_CSMS: Set Display Message - Display message at StartTime

Test case name	Set Display Message - Display message at StartTime
Test case Id	TC_O_13_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	<u>O01.FR.05</u>
System under test	CSMS
Description	This test case describes how the CSMS can be requested to send a SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send the request with a startTime according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to send a SetDisplayMessageRequest with a startTime.

2. The Test System responds with a **SetDisplayMessageResponse** with status *Accepted*

1. The CSMS sends a **SetDisplayMessageRequest**

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.id** <Generated Id>- **message.startDateTime** <Configured startDateTime>**Post scenario validations:**

- N/a

TC_O_14_CSMS: Set Display Message - Remove message after EndTime

Test case name	Set Display Message - Remove message after EndTime
Test case Id	TC_O_14_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	<u>O01.FR.05</u>
System under test	CSMS
Description	This test case describes how the CSMS can be requested to send a SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send the request with a endTime according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to send a SetDisplayMessageRequest with a endTime.

2. The Test System responds with a **SetDisplayMessageResponse** with status *Accepted*

1. The CSMS sends a **SetDisplayMessageRequest**

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.id** <Generated Id>- **message.endTime** <Configured endTime>**Post scenario validations:**

- N/a

TC_O_17_CSMS: Set Display Message - NotSupportedPriority

Test case name	Set Display Message - NotSupportedPriority
Test case Id	TC_O_17_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to sent an SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send a display message with a specific priority, on which the Charging station responds not supported, according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetDisplayMessageResponse with status NotSupportedPriority	1. The CSMS sends a SetDisplayMessageRequest

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.id** <Generated Id>- **message.priority** <Configured priority>**Post scenario validations:**

- N/a

TC_O_18_CSMS: Set Display Message - NotSupportedState

Test case name	Set Display Message - NotSupportedState
Test case Id	TC_O_18_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to sent an SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send a display message with a specific state, on which the Charging station responds not supported, according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetDisplayMessageResponse with status NotSupportedState	1. The CSMS sends a SetDisplayMessageRequest

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.id** <Generated Id>- **message.state** <Configured state>**Post scenario validations:**

- N/a

TC_O_19_CSMS: Set Display Message - NotSupportedMessageFormat

Test case name	Set Display Message - NotSupportedMessageFormat
Test case Id	TC_O_19_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to sent an SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send a display message with a specific MessageFormat, on which the Charging station responds not supported, according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetDisplayMessageResponse with status NotSupportedMessageFormat	1. The CSMS sends a SetDisplayMessageRequest

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.id** <Generated Id>**Post scenario validations:**

- N/a

TC_O_25_CSMS: Set Display Message - Send Specific state

Test case name	Set Display Message - Send Specific state
Test case Id	TC_O_25_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to sent an SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send a display messages with a "Charging" state according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetDisplayMessageResponse with status <i>Accepted</i>	1. The CSMS sends a SetDisplayMessageRequest

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.id** <Configured_Id>- **message.state** <Configured State>**Post scenario validations:**

- N/a

TC_O_26_CSMS: Set Display Message - Rejected

Test case name	Set Display Message - Rejected
Test case Id	TC_O_26_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to send a SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send the request according to the DisplayMessage mechanism as described in the OCPP specification which gets rejected.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to send a SetDisplayMessageRequest with a Normal Cycle priority.

2. The Test System responds with a **SetDisplayMessageResponse** with *status Rejected*

1. The CSMS sends a **SetDisplayMessageRequest**

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**- **message.id** <Generated Id>- **message.priority** <Configured Priority>**Post scenario validations:**

- N/a

TC_O_27_CSMS: Set Display Message - Specific transaction - Display message at StartTime

Test case name	Set Display Message - Specific transaction - Display message at StartTime
Test case Id	TC_O_27_CSMS
Use case Id(s)	O02
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to send a SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send the request with a startTime for a specific transaction according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a1

Reusable State:

State is *EnergyTransferStarted*

Main (Test scenario)

Charging Station	CSMS
2. The Test System responds with a SetDisplayMessageResponse with status <i>Accepted</i>	1. The CSMS sends a SetDisplayMessageRequest

Tool validations

* Step 1:

Message SetDisplayMessageRequest

- message.state is <omitted>
- message.startDateTime is <Configured startDateTime>
- message.endDateTime is <omitted>
- message.transactionId is <Generated transactionId from Before>

Post scenario validations:

- N/a

TC_O_28_CSMS: Set Display Message - Specific transaction - Remove message after EndTime

Test case name	Set Display Message - Specific transaction - Remove message after EndTime
Test case Id	TC_O_28_CSMS
Use case Id(s)	<u>O02</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to send a SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send the request with an endTime for a specific transaction according to the DisplayMessage mechanism as described in the OCPP specification.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:State is *EnergyTransferStarted***Main (Test scenario)**

Charging Station	CSMS
2. The Test System responds with a SetDisplayMessageResponse with status <i>Accepted</i>	1. The CSMS sends a SetDisplayMessageRequest

Tool validations*** Step 1:**Message **SetDisplayMessageRequest**

- **message.state** is *<omitted>*
- **message.startDateTime** is *<omitted>*
- **message.endDateTime** is *<Configured endDateTime>*
- **message.transactionId** is *<Generated transactionId from Before>*

Post scenario validations:

- N/a

TC_O_100_CSMS: Set Display Message - unsupported language

Test case name	Set Display Message - unsupported language
Test case Id	TC_O_100_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to send a SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to handle a rejection request for DisplayMessage mechanism as result of an unsupported language status.
Prerequisite(s)	CSMS support multiple languages.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to send a SetDisplayMessageRequest with a certain language. Test System will respond that the language is not supported.

2. The Test System responds with a **SetDisplayMessageResponse** with status *LanguageNotSupported*

1. The CSMS sends a **SetDisplayMessageRequest**

Manual Action: Verify that CSMS UI reports that message was not accepted due to unknown language

Tool validations

* Step 1:

Message **SetDisplayMessageRequest**

- **message.id** <Generated Id>

- **message.priority** <Configured Priority>

- **message.message.language** <Not omitted>

Post scenario validations:

- N/a

TC_O_101_CSMS: Set Display Message - Language preference of the EV Driver

Test case name	Set DisplayMessage - Language preference of the EV Driver
Test case Id	TC_O_101_CSMS
Use case Id(s)	<u>O01</u>
Requirement(s)	N/a
System under test	CSMS
Description	This test case describes how the CSMS can be requested to send a SetDisplayMessageRequest to the charging station. Depending on the given parameters the message shall be displayed a certain way and at a certain moment on the Charging Station. These messages are displayed additionally on a Charging Station and are not part of the firmware.
Purpose	To verify if the CSMS is able to send a DisplayMessage with the default and a different language.
Prerequisite(s)	CSMS support multiple languages.

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State:

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a SetDisplayMessageRequest with an additional language (other than the default language), that is supported on the charging station.

2. The Test System responds with a SetDisplayMessageResponse with status *Accepted*

1. The CSMS sends a SetDisplayMessageRequest

Tool validations

* Step 1:

Message SetDisplayMessageRequest

- message.id <Generated Id>

- message.priority <Configured Priority>

- message.message.language <Not omitted>

- message.messageExtra[0].language <Different from message.language>

Post scenario validations:

- N/a

17. P Data Transfer

TC_P_02_CSMS: Data Transfer to the CSMS - Rejected / Unknown VendorId / Unknown MessageId

Test case name	Data Transfer to the CSMS - Rejected / Unknown VendorId / Unknown MessageId
Test case Id	TC_P_02_CSMS
Use case Id(s)	<u>P02</u>
Requirement(s)	<u>P02.FR.06</u> , <u>P02.FR.07</u>
System under test	CSMS
Description	The DataTransfer message to send information for functions that are not supported by OCPP.
Purpose	To verify whether the CSMS is able to handle receiving a DataTransferRequest, even if it does not support any vendor-specific implementations.
Prerequisite(s)	N/a

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a DataTransferRequest with vendorId <Configured vendorId> messageId <Configured messageId>	2. The CSMS responds with a DataTransferResponse

Tool validations

* Step 2:

Message: **DataTransferResponse**

- **status** must be *UnknownVendorId* OR *UnknownMessageId* OR *Rejected* (Rejected will also be allowed, because there are implementers that like to just reject the message when the Charging Station does not support any vendor-specific features).

Post scenario validations:

N/a

TC_P_03_CSMS: CustomData - Receive custom data

Test case name	CustomData - Receive custom data
Test case Id	TC_P_03_CSMS
Use case Id(s)	N/a
Requirement(s)	N/a
System under test	CSMS
Description	Checks if the CSMS is able to receive custom data.
Purpose	To verify whether the CSMS is able to handle receiving custom data.
Prerequisite(s)	N/a

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station	CSMS
1. The Test System sends a StatusNotificationRequest with customData <customData>	2. The CSMS responds with a StatusNotificationResponse
3. The Test System sends a TransactionEventRequest with customData customData transactionInfo.customData <customData>	4. The CSMS responds with a TransactionEventResponse

Tool validations

N/a

Post scenario validations:

N/a

18. Q Bidirectional Power Transfer

TC_Q_103_CSMS: V2X Authorisation - ISO15118-20 - Charging needs rejected

Test case name	V2X Authorisation - ISO15118-20 - Charging needs rejected
Test case Id	TC_Q_103_CSMS
Use case Id(s)	Q01
Requirement(s)	Q01.FR.04
System under test	CSMS
Description	Authorization of an EV by the CSMS to start a V2X power transfer loop.
Purpose	To check if the CSMS rejects a <code>NotifyEVChargingNeedsRequest</code> when the EV requests a <code>EnergyTransferModes</code> that is not allowed.
Prerequisite(s)	CSMS supports ISO15118-20

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is ISO15118-20 transaction just started exposing:

- `<allowedEnergyTransferModes>`
- `<transactionId>`

Main (Test scenario)

Charging Station

CSMS

Agree on energy transfer mode

1. Test System sends a `NotifyEVChargingNeedsRequest` with `evseId` `<Configured evseId>`
`chargingNeeds.requestedEnergyTransfer` one different than mentioned in `<allowedEnergyTransferModes>` or if omitted `AC_BPT`
`chargingNeeds.availableEnergyTransfer` [`AC_single_phase`, `AC_two_phase`, `AC_three_phase`, `AC_BPT`, `AC_BPT_DER`, `AC_DER`]
`chargingNeeds.controlMode` `ScheduledControl`
`chargingNeeds.v2xChargingParameters.maxChargePower` `1234W`
`chargingNeeds.v2xChargingParameters.maxDischargePower` `0W`

2. CSMS SHALL respond with a `NotifyEVChargingNeedsResponse`

Tool validations

* Step 2:

Message: **NotifyEVChargingNeedsResponse**

- **status** must be *Rejected*

Post scenario validations:

N/a

TC_Q_107_CSMS: V2X Authorisation - ISO15118-20 - Charging only (V2X control) before starting V2X

Test case name	V2X Authorisation - ISO15118-20 - Charging only (V2X control) before starting V2X
Test case Id	TC_Q_107_CSMS
Use case Id(s)	Q02
Requirement(s)	Q02.FR.01, Q02.FR.02, Q02.FR.05
System under test	CSMS
Description	To start a transaction with operationMode ChargingOnly, such that it can become a bidirectional session at a later time.
Purpose	To check if the CSMS supports starting in a non-bidirectional way.
Prerequisite(s)	CSMS supports ISO15118-20

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is ISO15118-20 transaction just started exposing:

- <allowedEnergyTransferModes>

- <transactionId>

Main (Test scenario)

Charging Station

CSMS

Agree on energy transfer mode

1. The Test System sends a **NotifyEVChargingNeedsRequest** with
evseId <Configured evseId>
chargingNeeds.requestedEnergyTransfer one of non-bidirectional mode in <allowedEnergyTransferModes> received in AuthorizeResponse or if omitted *AC_three_phase*
chargingNeeds.availableEnergyTransfer [AC_single_phase, AC_two_phase, AC_three_phase, AC_BPT, AC_BPT_DER, AC_DER]
chargingNeeds.controlMode ScheduledControl
chargingNeeds.v2xChargingParameters.maxChargePower 1234W
chargingNeeds.v2xChargingParameters.maxDischargePower 1234W

2. The CSMS responds with a
NotifyEVChargingNeedsResponse

Setting charging profile

4. The Test System responds with a
SetChargingProfileResponse with
status Accepted

3. The CSMS sends a **SetChargingProfileRequest**

Main (Test scenario)	
<u>Manual Action:</u> Trigger CSMS to allow V2X after initial start	
6. The Test System sends a NotifyAllowedEnergyTransferResponse with status Accepted	5. The CSMS sends a NotifyAllowedEnergyTransferRequest
7. The Test System sends a NotifyEVChargingNeedsRequest with evseId <Configured evseId> chargingNeeds.requestedEnergyTransfer one of bidirectional mode in <allowedEnergyTransferModes> received in NotifyAllowedEnergyTransferRequest chargingNeeds.availableEnergyTransfer [AC_single_phase, AC_two_phase, AC_three_phase, AC_BPT, AC_BPT_DER, AC_DER] chargingNeeds.controlMode ScheduledControl chargingNeeds.v2xChargingParameters.maxChargePower 1234W chargingNeeds.v2xChargingParameters.maxDischargePower 1234W	8. The CSMS responds with a NotifyEVChargingNeedsResponse
Setting charging profile	
10. The Test System responds with a SetChargingProfileResponse with status Accepted	9. The CSMS sends a SetChargingProfileRequest
11. The Test System sends a TransactionEventRequest with eventType Updated triggerReason OperationModeChanged transactionInfo.transactionId <transactionId> transactionInfo.operationMode <Operation mode from step 9>	12. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 2:

Message: **NotifyEVChargingNeedsResponse**

- **status** must be *Accepted* or *Processing*

* Step 3:

Message: **SetChargingProfileRequest**

- **evseId** must be *<Configured evseId>*

- **chargingProfile.transactionId** must be *<transactionId>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].operationMode** must be *ChargingOnly* | *<omitted>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].Limit** must NOT be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].dischargeLimit** must be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].dischargeLimit_L2** must be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].dischargeLimit_L3** must be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpoint** must be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpoint_L2** must be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpoint_L3** must be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setPointReactive** must be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setPointReactive_L2** must be *<omitted>* | *<empty>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setPointReactive_L3** must be *<omitted>* | *<empty>*

* Step 5:

Message: **NotifyAllowedEnergyTransferRequest**

- **transactionId** must be *<transactionId>*

- **allowedEnergyTransfer** must contain one of *[DC_BPT, AC_BPT, AC_BPT_DER]*

* Step 7:

Message: **NotifyEVChargingNeedsResponse**

- **status** must be *Accepted* or *Processing*

* Step 9:

Message: **SetChargingProfileRequest**

- **evseId** must be *<Configured evseId>*

- **chargingProfile.transactionId** must be *<transactionId>*

- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].operationMode** must NOT be *ChargingOnly* | *<empty>*

Post scenario validations:

N/a

TC_Q_108_CSMS: V2X Authorisation - ISO15118-20 - Central V2X control with charging schedule

Test case name	V2X Authorisation - ISO15118-20 - Central V2X control with charging schedule
Test case Id	TC_Q_108_CSMS
Use case Id(s)	Q03
Requirement(s)	Q03.FR.01, Q03.FR.02
System under test	CSMS
Description	To allow the CSMS to control the charge and discharge behaviour of an EV with power profiles.
Purpose	To verify the CSMS is able to set a charging profile with a setpoint in the ISO15118-20 transaction.
Prerequisite(s)	CSMS supports ISO15118-20

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is ISO15118-20 transaction started in V2X AC_BPT energy transfer mode exposing:

- <allowedEnergyTransferModes>

- <transactionId>

Main (Test scenario)

Charging Station	CSMS
2. Test System responds with a SetChargingProfileResponse with status <i>Accepted</i>	1. CSMS sends a SetChargingProfileRequest

Tool validations

* Step 1:

Message: **SetChargingProfileRequest**

- **evseId** must be <Configured evseId>
- **chargingProfile.transactionId** must be <transactionId>
- **chargingProfile.chargingProfilePurpose** must be *TxProfile*
- **chargingProfile.chargingSchedule.chargingRateUnit** must be *W*
- **chargingProfile.chargingSchedule.chargingSchedulePeriod[0].startPeriod** must be *0*
- **chargingProfile.chargingSchedule.chargingSchedulePeriod[0].operationMode** must be *CentralSetpoint*
- **chargingProfile.chargingSchedule.chargingSchedulePeriod[0].setpoint** must be *10.000*
- **chargingProfile.chargingSchedule.chargingSchedulePeriod[0].numberPhases** may be present
- **chargingProfile.chargingSchedule.chargingSchedulePeriod[0].phaseToUse** may be present
- all other fields in **chargingProfile.chargingSchedule.chargingSchedulePeriod[0]** must be absent

Post scenario validations:

N/a

TC_Q_109_CSMS: Central V2X control with dynamic CSMS setpoint - push

Test case name	Central V2X control with dynamic CSMS setpoint - push
Test case Id	TC_Q_109_CSMS
Use case Id(s)	<u>Q04</u> , <u>K28</u>
Requirement(s)	<u>K28.FR.01</u> , <u>K28.FR.02</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to specify the power or current drawn by EV.
Purpose	To verify if the CSMS is able to support DynamicControl charging profiles and push changes to the CS.
Prerequisite(s)	- CSMS supports Bidirectional Power Transfer

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a TxDefaultProfile dynamic charging profile to the CS which will be periodically updated every <Configured dynamic profile update interval> seconds by CSMS with limit 6 * <limit multiplier>

Note: Check [csKSmartChargingChargingProfileLimits] for <limit multiplier>

2. The Test System responds with a SetChargingProfileResponse with status Accepted

1. The CSMS sends a SetChargingProfileRequest

4. The Test System responds with a UpdateDynamicScheduleResponse with status Accepted

3. The CSMS sends a UpdateDynamicScheduleRequest

6. The Test System responds with a UpdateDynamicScheduleResponse with status Accepted

5. The CSMS sends a UpdateDynamicScheduleRequest

Manual Action: Trigger the CSMS to clear the chargingProfile from step 1.

Main (Test scenario)	
8. The Test System responds with a ClearChargingProfileResponse with status <i>Accepted</i>	7. The CSMS sends a ClearChargingProfileRequest

Tool validations
* Step 1: Message: SetChargingProfileRequest - chargingProfile.chargingProfileKind must be <i>Dynamic</i> - chargingProfile. must be <i>TxDefaultProfile</i> - chargingProfile.chargingSchedule[0].chargingSchedulePeriods must be size <i>1</i> - chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].startPeriod must be <i>0</i> - chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].limit must be <i>6 * <limit multiplier></i> Note: Check [<i>csKSmartChargingChargingProfileLimits</i>] for <i><limit multiplier></i> * Step 3: Message: UpdateDynamicScheduleRequest - chargingProfileId must be <i><SetChargingProfileRequest.chargingProfile.id from step 1></i> - scheduleUpdate.limit must be <i><not omitted></i> * Step 5: Message: UpdateDynamicScheduleRequest - chargingProfileId must be <i><SetChargingProfileRequest.chargingProfile.id from step 1></i> - scheduleUpdate.limit must be <i><not omitted></i> * Step 7: Message: ClearChargingProfileRequest - chargingProfileId <i><chargingProfileId from step 1 chargingProfile></i> - chargingProfileCriteria <i>omit</i>
Post scenario validations:
N/a

TC_Q_110_CSMS: Central V2X control with dynamic CSMS setpoint - pull

Test case name	Central V2X control with dynamic CSMS setpoint - pull
Test case Id	TC_Q_110_CSMS
Use case Id(s)	<u>Q04</u> , <u>K28</u>
Requirement(s)	<u>K28.FR.01</u> , <u>K28.FR.02</u> , <u>K28.FR.07</u> , <u>K28.FR.12</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to support DynamicControl charging profiles and a CS is able to pull from CSMS.
Prerequisite(s)	- CSMS supports Bidirectional Power Transfer

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): N/a

Main (Test scenario)	
Charging Station	CSMS
<u>Manual Action</u> : Request the CSMS to send a TxDefaultProfile dynamic charging profile to the CS with - dynUpdateInterval 15	
2. The Test System responds with a SetChargingProfileResponse with status Accepted	1. The CSMS sends a SetChargingProfileRequest
3. The Test System sends a PullDynamicScheduleUpdateRequest with chargingProfileId <unknown chargingProfileId>	4. The CSMS responds with a PullDynamicScheduleUpdateResponse
5. The Test System sends a PullDynamicScheduleUpdateRequest with chargingProfileId <SetChargingProfileRequest.chargingProfile.id from step 1>	6. The CSMS responds with a PullDynamicScheduleUpdateResponse

Tool validations

* Step 1:

Message: **SetChargingProfileRequest**

- **chargingProfile.chargingProfileKind** must be *Dynamic*
- **chargingProfile.dynUpdateInterval** must be *15*
- **chargingProfile.dynUpdateTime** must be *<not omitted>*
- **chargingProfile.chargingSchedule** must be size = *1*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriods** must be size = *1*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].startPeriod** must be *0*

* Step 4:

Message: **PullDynamicScheduleUpdateResponse**

- **status** must be *Rejected*

* Step 6:

Message: **PullDynamicScheduleUpdateResponse**

- **status** must be *Accepted*
- **scheduleUpdate** must be *<not omitted>*
- **scheduleUpdate** must at least have a single field set

Post scenario validations:

N/a

TC_Q_111_CSMS: External V2X control - with a charging profile from CSMS - setpoint

Test case name	External V2X control - with a charging profile from CSMS - setpoint
Test case Id	TC_Q_111_CSMS
Use case Id(s)	Q05
Requirement(s)	...
System under test	CSMS
Description	CSMS explicitly allows an External System to control the charge and discharge behaviour of an EV for a certain period of time.
Purpose	To verify if the CSMS is able to configure the CS to use an External V2X setpoint with relative or dynamic charging profile.
Prerequisite(s)	- CSMS supports Bidirectional Power Transfer

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual action: trigger CSMS to set a TxDefaultProfile, Relative Charging Profile with chargingSchedulePeriod with operationMode ExternalSetpoint

2. The Test System responds with a **SetChargingProfileResponse** with **status Accepted**

1. The CSMS sends a **SetChargingProfileRequest**

Manual action: trigger CSMS to set a TxDefaultProfile, Dynamic Charging Profile with chargingSchedulePeriod with operationMode ExternalSetpoint

4. The Test System responds with a **SetChargingProfileResponse** with **status Accepted**

3. The CSMS sends a **SetChargingProfileRequest**

Tool validations

* Step 1: Message: **SetChargingProfileRequest**

- **evseId** must be *<Configured evseId>*
- **chargingProfile.chargingProfilePurpose** must be *TxDefaultProfile*
- **chargingProfile.chargingProfileKind** *Relative*
- **chargingProfile.dynUpdateTime** must be *<omitted>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].operationMode** must be *ExternalSetpoint*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpoint** must be *<not omitted>*

* Step 3: Message: **SetChargingProfileRequest**

- **evseId** must be *<Configured evseId>*
- **chargingProfile.chargingProfilePurpose** must be *TxDefaultProfile*
- **chargingProfile.chargingProfileKind** *Dynamic*
- **chargingProfile.dynUpdateTime** must be *<Datetime of moment of step 3>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].operationMode** must be *ExternalSetpoint*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpoint** must be *<not omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod** size must be *1*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].startPeriod** must be *0*

Post scenario validations:

N/a

TC_Q_112_CSMS: External V2X control - with a charging profile from CSMS - limit

Test case name	External V2X control - with a charging profile from CSMS - limit
Test case Id	TC_Q_112_CSMS
Use case Id(s)	Q05
Requirement(s)	...
System under test	CSMS
Description	CSMS explicitly allows an External System to control the charge and discharge behaviour of an EV for a certain period of time.
Purpose	To verify if the CSMS is able to configure the CS to use an External V2X limit.
Prerequisite(s)	- CSMS supports Bidirectional Power Transfer

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual action: trigger CSMS to set a TxDefaultProfile, Relative Charging Profile with chargingSchedulePeriod with operationMode ExternalLimit

2. The Test System responds with a **SetChargingProfileResponse** with **status Accepted**

1. The CSMS sends a **SetChargingProfileRequest**

Manual action: trigger CSMS to set a TxDefaultProfile, Dynamic Charging Profile with chargingSchedulePeriod with operationMode ExternalLimit

4. The Test System responds with a **SetChargingProfileResponse** with **status Accepted**

3. The CSMS sends a **SetChargingProfileRequest**

Tool validations

* Step 1: Message: **SetChargingProfileRequest**

- **evseId** must be *<Configured evseId>*
- **chargingProfile.chargingProfilePurpose** must be *TxDefaultProfile*
- **chargingProfile.chargingProfileKind** *Relative*
- **chargingProfile.dynUpdateTime** must be *<omitted>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].operationMode** must be *ExternalLimits*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].limit** must be *<not omitted>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpoint** must be *<omitted>*

* Step 3: Message: **SetChargingProfileRequest**

- **evseId** must be *<Configured evseId>*
- **chargingProfile[0].chargingProfilePurpose** must be *TxDefaultProfile*
- **chargingProfile.chargingProfileKind** *Dynamic*
- **chargingProfile.dynUpdateTime** must be *<Datetime of moment of step 3>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].operationMode** must be *ExternalLimits*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].limit** must be *<not omitted>* _
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpoint** must be *<omitted>* _
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod** size must be *1*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].startPeriod** must be *0*

Post scenario validations:

N/a

TC_Q_117_CSMS: Frequency Support - Central V2X control - push

Test case name	Frequency Support - Central V2X control - push
Test case Id	TC_Q_117_CSMS
Use case Id(s)	<u>Q07</u> , <u>Q04</u> , <u>K28</u>
Requirement(s)	<u>K28.FR.01</u> , <u>K28.FR.02</u>
System under test	CSMS
Description	To enable the CSMS to influence the charging power or current drawn from a specific EVSE or the entire Charging Station over a period of time. The CSMS sends a SetChargingProfileRequest to the Charging Station to influence the power or current drawn by EVs. The CSMS calculates a ChargingSchedule to stay within certain limits, which MAY be imposed by any external system.
Purpose	To verify if the CSMS is able to support DynamicControl charging profiles and push changes to the CS.
Prerequisite(s)	- CSMS supports Bidirectional Power Transfer

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a TxDefaultProfile dynamic charging profile to the CS and update the schedule every 15 seconds with

- operationMode CentralFrequency

- setpoint -6 * <limit multiplier>

Note: Check [csKSmartChargingChargingProfileLimits] for <limit multiplier>

2. The Test System responds with a **SetChargingProfileResponse** with status *Accepted*

1. The CSMS sends a **SetChargingProfileRequest**

4. The Test System responds with a **UpdateDynamicScheduleResponse** with status *Accepted*

3. The CSMS sends a **UpdateDynamicScheduleRequest**

26. 6. 9. 오후 6:02	OCA OCPP 2.1: Part 6 - Test case document Charging Station Management System for OCTT
Tool validations	
* Step 1: Message: SetChargingProfileRequest - chargingProfile.chargingProfileKind must be <i>Dynamic</i> - chargingProfile. must be <i>TxDefaultProfile</i> - chargingProfile.chargingSchedule[0].chargingSchedulePeriods must be size <i>1</i> - chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].startPeriod must be <i>0</i> - chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].setpoint must be <i>-6 * <limit multiplier></i> Note: Check [csKSmartChargingChargingProfileLimits] for <limit multiplier> * Step 3: Message: UpdateDynamicScheduleRequest - chargingProfileId must be <i><SetChargingProfileRequest.chargingProfile.id from step 1></i> - scheduleUpdate.limit must be <i><not omitted></i>	
Post scenario validations: N/a	

TC_Q_120_CSMS: Frequency Support - Local V2X control - AFRR support

Test case name	Frequency Support - Local V2X control - AFRR support
Test case Id	TC_Q_120_CSMS
Use case Id(s)	<u>Q08</u>
Requirement(s)	<u>Q08.FR.01</u> , <u>Q08.FR.02</u> , <u>Q08.FR.05</u> , <u>Q08.FR.12</u>
System under test	CSMS
Description	To allow an EV to be used for aFRR frequency control, depending on a aFRR signal request that CSMS receives from an external actor.
Purpose	To verify if the CSMS is able to produce a charging profile for aFRR frequency support.
Prerequisite(s)	- CSMS supports Bidirectional Power Transfer - CSMS has Frequency support

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual action: trigger CSMS to set a TxDefaultProfile Charging Profile with chargingSchedulePeriod with operationMode LocalFrequency and v2xBaseline and v2xFreqWattCurve and v2xSignalWattCurve.

2. The Test System responds with a **SetChargingProfileResponse** with **status Accepted**

1. The CSMS the sends a **SetChargingProfileRequest**

Manual action: trigger CSMS to send out a AFRRSignalRequest with a given value for signal

4. The Test System responds with a **AFRRSignalResponse** with **status Accepted**

3. The CSMS sends a **AFRRSignalRequest**

Tool validations

* Step 1:

Message: **SetChargingProfileRequest**

- **chargingProfile.chargingProfilePurpose** must be *TxDefaultProfile*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].operationMode** must be *LocalFrequency*
- **chargingProfile.chargingSchedule[0].chargingRateUnit** must be *W*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].limit** must be *<omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].limit_L2** must be *<omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].limit_L3** must be *<omitted>*

- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].dischargeLimit** must be *<omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].dischargeLimit_L2** must be *<omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].dischargeLimit_L3** must be *<omitted>*

- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].setpoint** must be *<omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].setpoint_L2** must be *<omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].setpoint_L3** must be *<omitted>*

- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].setpointReactive** must be *<omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].setpointReactive_L2** must be *<omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].setpointReactive_L3** must be *<omitted>*

- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].v2xBaseline** must be *<not omitted>*

- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].v2xFreqWattCurve** size must be *>= 2*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].v2xFreqWattCurve[*].frequency** must be *<not omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].v2xFreqWattCurve[*].power** must be *<not omitted>*

- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].v2xSignalWattCurve** size must be *>= 2*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].v2xSignalWattCurve[*].signal** must be *<not omitted>*
- **chargingProfile.chargingSchedule[0].chargingSchedulePeriod[0].v2xSignalWattCurve[*].power** must be *<not omitted>*

* Step 3:

Message: **AFRRSignalRequest**

- **signal** must *<be the value that was provided in manual action before step 3.>*

Post scenario validations:

N/a

TC_Q_121_CSMS: Frequency Support - Local V2X control

Test case name	Frequency Support - Local V2X control
Test case Id	TC_Q_121_CSMS
Use case Id(s)	Q08
Requirement(s)	Q08.FR.01, Q08.FR.02, Q08.FR.05
System under test	CSMS
Description	To allow an EV to be used for frequency control, depending on local frequency readings.
Purpose	To check if the CSMS supports Local V2X control for frequency support.
Prerequisite(s)	- CSMS supports Bidirectional Power Transfer - CSMS has Frequency support

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

State is ISO15118-20 transaction just started exposing:

- <allowedEnergyTransferModes>

- <transactionId>

Main (Test scenario)

Charging Station

CSMS

Agree on energy transfer mode

1. The Test System sends a **NotifyEVChargingNeedsRequest** with
evseId = <Configured evseId>
chargingNeeds.requestedEnergyTransfer *AC_BPT*
chargingNeeds.availableEnergyTransfer [*AC_single_phase, AC_two_phase, AC_three_phase, AC_BPT, AC_BPT_DER, AC_DER*]
chargingNeeds.controlMode *DynamicControl*
chargingNeeds.v2xChargingParameters.maxChargePower *1234W*
chargingNeeds.v2xChargingParameters.maxDischargePower *4321W*

2. The CSMS responds with a
NotifyEVChargingNeedsResponse

4. And Test System responds with a
SetChargingProfileResponse with
status *Accepted*

3. The CSMS sends a **SetChargingProfileRequest**

Main (Test scenario)

Manual Action: Request the CSMS to send a LocalFrequency charging profile to the CS

- **evseId** <Configured evseId>
- **chargingProfile.transactionId** <transactionId>
- **chargingProfile.chargingProfilePurpose** TxProfile
- **chargingProfile.chargingSchedule.chargingRateUnit** W
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.operationMode** must be LocalFrequency
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[0].frequency** 49.5
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[0].power** -1000
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[1].frequency** 50.0
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[1].power** 0
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[2].frequency** 50.5
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[2].power** 1000
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xBaseline** 7000

5. The CSMS sends a **SetChargingProfileRequest**

6. And Test System responds with a **SetChargingProfileResponse** with **status Accepted**

Tool validations

* Step 2:

Message: **NotifyEVChargingNeedsResponse**

- **status** must be *Accepted* or *Processing*

* Step 5:

Message: **SetChargingProfileRequest**

- **evseId** must be <Configured evseId>
- **chargingProfile.transactionId** must be <transactionId>
- **chargingProfile.chargingProfilePurpose** must be TxProfile
- **chargingProfile.chargingSchedule.chargingRateUnit** must be W
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.operationMode** must be LocalFrequency
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.limit** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.limit_L2** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.limit_L3** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.dischargeLimit** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.dischargeLimit_L2** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.dischargeLimit_L3** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.setpoint** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.setpoint_L2** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.setpoint_L3** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.setpointReactive** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.setpointReactive_L2** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.setpointReactive_L3** must be <omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[0].frequency** must be <not omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[0].power** must be <not omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[1].frequency** must be <not omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[1].power** must be <not omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[2].frequency** must be <not omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xFreqWattCurve[2].power** must be <not omitted>
- **chargingProfile.chargingSchedule.chargingSchedulePeriod.v2xBaseline** must be <not omitted>

Post scenario validations:

N/a

TC_Q_124_CSMS: Local V2X control for load balancing - good flow

Test case name	Local V2X control for load balancing - good flow
Test case Id	TC_Q_124_CSMS
Use case Id(s)	Q09
Requirement(s)	...
System under test	CSMS
Description	To allow the EV to be utilized for locally controlled load balancing.
Purpose	To verify if the CSMS is able to configure a CS for locally controlled load balancing
Prerequisite(s)	- CSMS supports Bidirectional Power Transfer

Before (Preparations)

Configuration State:

- CS reports **V2XChargingCtrlr.SupportedOperationModes** that contains *LocalLoadBalancing*

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual action: trigger CSMS to set local load-balancing thresholds:

- *UpperThreshold* = 1000, *LowerThreshold* = -1000

- *UpperOffset* = 200, *LowerOffset* = 0

Main (Test scenario)	
2. The Test System responds with a SetVariablesResponse with setVariableResult[0].component.name <i>V2XChargingCtrlr</i> setVariableResult[0].component.evse.id <i>1</i> setVariableResult[0].variable.name <i>V2XLocalLoadBalancing</i> setVariableResult[0].variable.instance <i>UpperThreshold</i> setVariableResult[0].attributeStatus <i>Accepted</i> setVariableResult[1].component.name <i>V2XChargingCtrlr</i> setVariableResult[1].component.evse.id <i>1</i> setVariableResult[1].variable.name <i>V2XLocalLoadBalancing</i> setVariableResult[1].variable.instance <i>LowerThreshold</i> setVariableResult[1].attributeStatus <i>Accepted</i> setVariableResult[2].component.name <i>V2XChargingCtrlr</i> setVariableResult[2].component.evse.id <i>1</i> setVariableResult[2].variable.name <i>V2XLocalLoadBalancing</i> setVariableResult[2].variable.instance <i>UpperOffset</i> setVariableResult[2].attributeStatus <i>Accepted</i> setVariableResult[3].component.name <i>V2XChargingCtrlr</i> setVariableResult[3].component.evse.id <i>1</i> setVariableResult[3].variable.name <i>V2XLocalLoadBalancing</i> setVariableResult[3].variable.instance <i>LowerOffset</i> setVariableResult[3].attributeStatus <i>Accepted</i>	1. The CSMS sends a SetVariablesRequest <u>Note:</u> CSMS may send these variables in one or multiple requests
CSMS sets a Charging Profile with LocalLoadBalancing	
<u>Manual action:</u> trigger CSMS to set a TxDefaultProfile Charging Profile with chargingSchedulePeriod with operationMode LocalLoadBalancing	
4. The Test System responds with a SetChargingProfileResponse with status <i>Accepted</i>	3. The CSMS the sends a SetChargingProfileRequest

Tool validations

* Step 1:

Message **SetVariablesRequest** with

```
setVariableData[0].component.name V2XChargingCtrlr
setVariableData[0].component.evse.id 1
setVariableData[0].variable.name V2XLocalLoadBalancing
setVariableData[0].variable.instance UpperThreshold
setVariableData[0].attributeValue 2000
setVariableData[1].component.name V2XChargingCtrlr
setVariableData[1].component.evse.id 1
setVariableData[1].variable.name V2XLocalLoadBalancing
setVariableData[1].variable.instance LowerThreshold
setVariableData[1].attributeValue -1000
setVariableData[2].component.name V2XChargingCtrlr
setVariableData[2].component.evse.id 1
setVariableData[2].variable.name V2XLocalLoadBalancing
setVariableData[2].variable.instance UpperOffset
setVariableData[2].attributeValue 200
setVariableData[3].component.name V2XChargingCtrlr
setVariableData[3].component.evse.id 1
setVariableData[3].variable.name V2XLocalLoadBalancing
setVariableData[3].variable.instance LowerOffset
setVariableData[3].attributeValue 0
```

Note: Alternatively these may be sent as individual requests

* Step 3:

Message: **SetChargingProfileRequest**

- **evseId** must be *<Configured evseId>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].operationMode** must be *LocalLoadBalancing*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].limit(L2/L3)** must be *<omitted>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].dischargingLimit(L2/L3)** must be *<omitted>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpoint(L2/L3)** must be *<omitted>*
- **chargingProfile.chargingSchedule[*].chargingSchedulePeriod[*].setpointReactive(L2/L3)** must be *<omitted>*

Post scenario validations:

N/a

TC_Q_125_CSMS: Idle operationMode - Idle, minimizing energy consumption - Idle with EvseSleep

Test case name	Idle operationMode - Idle, minimizing energy consumption - Idle with EvseSleep
Test case Id	TC_Q_150_CS
Use case Id(s)	Q10
Requirement(s)	Q10.FR.01
System under test	CSMS
Description	To request the EV to not perform any charging or discharging. Preconditioning of the vehicle is allowed.
Purpose	To verify that the CSMS is able to send a Charging Profile which Minimizes the energy consumption.
Prerequisite(s)	- CSMS supports ISO15118-20 - CS supports <i>V2XChargingCtrlr.V2XSupportedOperationModes</i> containing <i>Idle</i> - CS supports <i>SmartChargingCtrlr.SupportsFeature[EvseSleep]</i> = true

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

1. Execute **Reusable State** *ISO15118-20* transaction started in *V2X AC_BPT* energy transfer mode*Without EVSE sleeping*Manual action: trigger CSMS to set a TxProfile Charging Profile with *chargingSchedulePeriod* with *operationMode Idle* and *evseSleep false*3. The Test System responds with a **SetChargingProfileResponse** with **status** must be *Accepted*2. The CSMS the sends a **SetChargingProfileRequest**

4. The Test System sends a **TransactionEventRequest** with
eventType *Updated*
triggerReason *OperationModeChanged*
transactionInfo.transactionId *<transactionId>*
transactionInfo.operationMode *Idle*
evseSleep *false*

5. The CSMS responds with a **TransactionEventResponse***With EVSE sleeping*Manual action: trigger CSMS to set a TxProfile Charging Profile with *chargingSchedulePeriod* with *operationMode Idle* and *evseSleep true*

Main (Test scenario)	
7. The Test System responds with a SetChargingProfileResponse	6. The CSMS the sends a SetChargingProfileRequest
8. The Test System sends a TransactionEventRequest with eventType <i>Updated</i> triggerReason <i>MeterValuePeriodic</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.operationMode <i>Idle</i> evseSleep <i>true</i>	9. The CSMS responds with a TransactionEventResponse

Tool validations
* Step 2: Message: SetChargingProfileRequest - chargingProfile.chargingProfilePurpose must be <i>TxProfile</i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.operationMode must be <i>Idle</i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.evseSleep must be <i>false</i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.limit must be <i><omitted></i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.dischargingLimit must be <i><omitted></i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.setpoint must be <i><omitted></i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.setpointReactive must be <i><omitted></i> * Step 6: Message: SetChargingProfileRequest - chargingProfile.chargingProfilePurpose must be <i>TxProfile</i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.operationMode must be <i>Idle</i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.evseSleep must be <i>true</i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.limit must be <i><omitted></i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.dischargingLimit must be <i><omitted></i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.setpoint must be <i><omitted></i> - chargingProfile.chargingSchedule.chargingSchedulePeriod.setpointReactive must be <i><omitted></i> Post scenario validations: N/a

19. R DER Control

TC_R_107_CSMS: Configure DER control settings at CS

Test case name	Configure DER control settings at CS
Test case Id	TC_R_107_CSMS
Use case Id(s)	<u>R04</u>
Requirement(s)	<u>R04.FR.12</u> , <u>R04.FR.14</u>
System under test	CSMS
Description	Setting some DER controls and reboot the charging station. After reboot, retrieving the configured DER controls.
Purpose	To check if the CSMS is able to set DER Controls.
Prerequisite(s)	CSMS supports DER Control The Test System must report support for the following: component.name <i>ACDERCtrlr</i> variable.name <i>ModesSupported</i> variableAttribute.value [<i>FreqWatt, FreqDroop, LimitMaxDischarge, Gradients, EnterService</i>]

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

*Test ClearDERControl*Manual action: trigger CSMS to send a *ClearDERControlRequest* with *isDefault true*

2. The Test System responds with a **ClearDERControlResponse** with
status *Accepted*

1. The CSMS sends a **ClearDERControlRequest**

Manual action: trigger CSMS to send a *ClearDERControlRequest* with *isDefault false*

4. The Test System responds with a **ClearDERControlResponse** with
status *Accepted*

3. The CSMS sends a **ClearDERControlRequest**

*Test SetDERControl*Manual action: trigger CSMS to send a *SetDERControlRequest* with *isDefault true*, *controlType FreqDroop* with *priority 6*

Main (Test scenario)	
6. The Test System responds with a SetDERControlResponse with status Accepted	5. The CSMS sends a SetDERControlRequest
<u>Manual action:</u> trigger CSMS to send a SetDERControlRequest with isDefault false controlType FreqWatt with a startTime, duration of 900 and priority 4	
8. The Test System responds with a SetDERControlResponse with status Accepted	7. The CSMS sends a SetDERControlRequest
<u>Manual action:</u> trigger CSMS to send a SetDERControlRequest for controlType EnterService with priority = 1	
10. The Test System responds with a SetDERControlResponse with status Accepted	9. The CSMS sends a SetDERControlRequest
<i>Test GetDERControl</i>	
<u>Manual action:</u> trigger CSMS to send a GetDERControlRequest with isDefault true	
12. The Test System responds with a GetDERControlResponse with status Accepted	11. The CSMS sends a GetDERControlRequest
13. The Test System sends a ReportDERControlRequest with - enterService : - enterService.id = "enterservice_1" - enterService.*enterService.highVoltage = 250 - enterService.*enterService.lowVoltage = 210 - enterService.*enterService.highFreq = 50.5 - enterService.*enterService.lowFreq 49.5 - enterService.*enterService.priority = 1 - freqDroop : - freqDroop.id = "freqdroop_1" - freqDroop.isDefault = true - freqDroop.isSuperseded = false - freqDroop.freqDroop.priority must be 6 - freqDroop.freqDroop.overFreq = 50.5 - freqDroop.freqDroop.underFreq = 49.5 - freqDroop.freqDroop.overDroop = 0.05 - freqDroop.freqDroop.underDroop = 0.05 - freqDroop.freqDroop.responseTime = 10	14. The CSMS sends a ReportDERControlResponse
<u>Manual action:</u> trigger CSMS to send a GetDERControlRequest with isDefault false	
16. The Test System responds with a GetDERControlResponse with status Accepted	15. The CSMS sends a GetDERControlRequest

Main (Test scenario)	
17. The Test System sends a ReportDERControlRequest with - curve : - curve.id = "freqwatt_1" - curve.curveType = "FreqWatt" - curve.isDefault = false - curve.isSuperseded = false - curve.curve.priority = 4 - curve.curve.yUnit = "PctMaxW" - curve.curve.curvedata[0].x = 49 - curve.curve.curvedata[0].y = 75 - curve.curve.curvedata[1].x = 49.5 - curve.curve.curvedata[1].y = 90 - curve.curve.curvedata[2].x = 50.5 - curve.curve.curvedata[2].y = 100 - curve.curve.curvedata[3].x = 51 - curve.curve.curvedata[3].y = 100 - curve.curve.startTime = <current time> - curve.curve.duration = 900	18. The CSMS sends a ReportDERControlResponse
<u>Manual action</u>: trigger CSMS to send a <i>GetDERControlRequest</i> with controlType VoltVar	
20. The Test System responds with a GetDERControlResponse with status <i>NotFound</i>	19. The CSMS sends a GetDERControlRequest
<u>Manual action</u>: trigger CSMS to send a <i>GetDERControlRequest</i> with controlId from step 7	
22. The Test System responds with a GetDERControlResponse with status <i>Accepted</i>	21. The CSMS sends a GetDERControlRequest
23. The Test System sends a ReportDERControlRequest with - curve : - curve.id = "freqwatt_1" - curve.curveType = "FreqWatt" - curve.isDefault = false - curve.isSuperseded = false - curve.curve.priority = 4 - curve.curve.yUnit = "PctMaxW" - curve.curve.curvedata[0].x = 49 - curve.curve.curvedata[0].y = 75 - curve.curve.curvedata[1].x = 49.5 - curve.curve.curvedata[1].y = 90 - curve.curve.curvedata[2].x = 50.5 - curve.curve.curvedata[2].y = 100 - curve.curve.curvedata[3].x = 51 - curve.curve.curvedata[3].y = 100 - curve.curve.startTime = <current time> - curve.curve.duration = 900	24. The CSMS sends a ReportDERControlResponse

Tool validations

* Step 1:

Message: **ClearDERControlRequest**

- **isDefault** *true*

* Step 3:

Message: **ClearDERControlRequest**

- **isDefault** *false*

* Step 5:

Message: **SetDERControlRequest**

- **isDefault** must be *true*

- **controlType** must be *FreqDroop*

- **freqDroop.priority** must be *6*

- **freqDroop.startTime** must be *<omitted>*

- **freqDroop.duration** must be *<omitted>*

- *Other optional fields must be omitted*

* Step 7:

Message: **SetDERControlRequest**

- **isDefault** must be *false*

- **controlType** must be *FreqWatt*

- **curve.priority** must be *4*

- **curve.startTime** must be *<not omitted>*

- **curve.duration** must be *900*

- **curve.curveData[*].x** must be *<not omitted>*

- **curve.curveData[*].y** must be *<not omitted>* - *Other optional fields must be omitted*

* Step 9:

Message: **SetDERControlRequest**

- **isDefault** must be *false*

- **controlType** must be *EnterService*

- **enterService** must be *<not omitted>*

- **enterService.priority** must be *1* - *Other optional fields must be omitted*

* Step 11:

Message: **GetDERControlRequest**

- **isDefault** must be *true*

* Step 15:

Message: **GetDERControlRequest**

- **isDefault** must be *false*

* Step 19:

Message: **GetDERControlRequest**

- **controlType** must be *VoltVar*

* Step 21:

Message: **GetDERControlRequest**

- **controlId** must be *<controlId from step 7>*

Post scenario validations:

N/a

TC_R_108_CSMS: Charging station reporting a DER event

Test case name	Charging station reporting a DER event
Test case Id	TC_R_108_CSMS
Use case Id(s)	<u>R05</u>
Requirement(s)	
System under test	CSMS
Description	Configure a DER Control which will always trigger and halves the charging rate of the charging profile when a transaction starts, then add another DER Control to test overriding the previous one. After tha clearing all DER controls to test it reports the alert should be cleared.
Purpose	To check if the CSMS supports receiving DER events.
Prerequisite(s)	CSMS supports DER Control

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Report DER alarm

1. The Test System sends a **NotifyDERAlarmRequest** with
controlType *LVMustTrip*
gridEventFault *UnderVoltage*
alarmEnded *false*
timestamp *<Current DateTime>*

2. The CSMS responds with a **NotifyDERAlarmResponse**

Note: The Test System waits 5 seconds

Clear DER alarm

3. The Test System sends a **NotifyDERAlarmRequest** with
controlType *LVMustTrip*
gridEventFault *UnderVoltage*
alarmEnded *true*
timestamp *<Current DateTime>*

4. The CSMS responds with a **NotifyDERAlarmResponse**

Tool validations

N/a

Tool validations**Post scenario validations:**

N/a

20. S Battery Swapping

TC_S_102_CSMS: Battery Swap - Remote Start - not enough batteries

Test case name	Battery Swap - Remote Start - not enough batteries
Test case Id	TC_S_102_CSMS
Use case Id(s)	<u>S01</u>
Requirement(s)	
System under test	CSMS
Description	CSMS is able to support battery swapping.
Purpose	To verify whether the CSMS is able successful support battery swapping with not enough batteries.
Prerequisite(s)	N/a

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): N/a

Main (Test scenario)	
Charging Station	CSMS
<u>Manual Action:</u> Let CSMS trigger a request for swapping batteries	
2. The Test System responds with a RequestBatterySwapResponse with status <i>Rejected</i> statusInfo.reasonCode <i>NoBatteryAvailable</i>	1. The CSMS sends a RequestBatterySwapRequest

Tool validations
* Step 1: Message: RequestBatterySwapRequest - requestId must be <not omitted> - idToken.idToken must be <Configured valid_idtoken_idtoken>
Post scenario validations: N/a

TC_S_103_CSMS: Battery Swap - Remote Start - enough batteries available

Test case name	Battery Swap - Remote Start - enough batteries available
Test case Id	TC_S_103_CSMS
Use case Id(s)	<u>S02</u> , <u>S03</u> , <u>S04</u>
Requirement(s)	
System under test	CSMS
Description	CSMS is able to support battery swapping.
Purpose	To verify whether the CSMS is able successful support battery swapping.
Prerequisite(s)	Charging Station supports battery swapping. There are enough batteries available for swapping. BatterySwapCtrlr.Idtoken is <not set>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Let CSMS trigger a request for swapping batteries

2. The Test System responds with a **RequestBatterySwapResponse** with status *Accepted*

1. The CSMS sends a **RequestBatterySwapRequest**

Note: Set of two (virtual) batteries are inserted at this point in scenario.

3. The Test System notifies the CSMS about the status change of the slot.
connectorStatus *Occupied*

4. The CSMS responds accordingly.

5. The Test System sends a **TransactionEventRequest** with
eventType *Started*
triggerReason *CablePluggedIn*
idToken.idToken ""
idToken.type *NoAuthorization*
evse <i>
evse.connectorId 1
transactionInfo.transactionId <transactionId>
transactionInfo.chargingState *EVConnected*

6. The CSMS responds with a **TransactionEventResponse**

Note: Steps 3 through 6 are repeated for each battery inserted

Main (Test scenario)	
7. The Test System sends a BatterySwapRequest with eventType <i>BatteryIn</i> requestId <i><requestId></i> idToken.idToken <i><Configured valid_idtoken_idtoken></i> batteryData[0].evseId <i>1</i> batteryData[0].serialNumber <i>1234</i> batteryData[0].soC <i>23</i> batteryData[0].soH <i>85</i> batteryData[1].evseId <i>2</i> batteryData[1].serialNumber <i>5678</i> batteryData[1].soC <i>45</i> batteryData[1].soH <i>87</i>	8. The CSMS responds with a BatterySwapResponse
<u>Note:</u> Set of two batteries are extracted at this point in scenario.	
9. The Test System notifies the CSMS about the status change of the slot. connectorStatus <i>Available</i>	10. The CSMS responds accordingly.
11. The Test System sends a TransactionEventRequest with eventType <i>Ended</i> triggerReason <i>EVCommunicationLost</i> idToken.idToken <i>""</i> idToken.type <i>NoAuthorization</i> transactionInfo.transactionId <i><transactionId></i> transactionInfo.chargingState <i>Idle</i> transactionInfo.stoppedReason <i>EVDisconnected</i>	12. The CSMS responds with a TransactionEventResponse
<u>Note:</u> Steps 9 through 12 are repeated for each battery taken out	
13. The Test System sends a BatterySwapRequest with eventType <i>BatteryOut</i> requestId <i><requestId></i> idToken.idToken <i><Configured valid_idtoken_idtoken></i> batteryData[0].evseId <i>3</i> batteryData[0].serialNumber <i>4321</i> batteryData[0].soC <i>80</i> batteryData[0].soH <i>95</i> batteryData[1].evseId <i>4</i> batteryData[1].serialNumber <i>8765</i> batteryData[1].soC <i>85</i> batteryData[1].soH <i>78</i>	14. The CSMS responds with a BatterySwapResponse

Tool validations

* Step 1:

Message: **RequestBatterySwapRequest**

- **requestId** must be *<not omitted>*

- **idToken.idToken** must be *<Configured valid_idtoken_idtoken>*

* Step 6:

Message: **TransactionEventResponse**

- **idTokenInfo.status** must be *Accepted*

* Step 12:

Message: **TransactionEventResponse**

- **idTokenInfo.status** must be *Accepted*

Post scenario validations:

N/a

21. Reusable states

Testcases can refer to a reusable state at the before or main stage. The steps described at the reusable state will be executed and then it will return to the testcase that called the reusable state.

Booted

State	Booted
System under test	CSMS
Description	This state will simulate that the Charging Station is completely power cycled. The Test System end in a state where it is "booted" back up and is in idle mode.

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): N/a

Main (Test scenario)	
Charging Station	CSMS
1. The Test System sends a BootNotificationRequest with reason <i>PowerUp</i> chargingStation.model <Configured model> chargingStation.vendorName <Configured vendorName>	2. The CSMS responds with a BootNotificationResponse
3. The Test System notifies the CSMS about the current state of all connectors. Message: StatusNotificationRequest with connectorStatus <i>Available</i> Message: NotifyEventRequest with trigger <i>Delta</i> actualValue "Available" component.name "Connector" variable.name "AvailabilityState"	4. The CSMS responds accordingly.

Tool validations
* Step 2: Message: BootNotificationResponse - status <i>Accepted</i>
Post scenario validations: State is <i>Booted</i>

Reserved

State	Reserved
System under test	CSMS
Description	This state will simulate a reservation for a specified evse.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Trigger the CSMS to send a *ReserveNowRequest* for specific EVSE.

2. The Test System responds with a **ReserveNowResponse**
With status *Accepted*

1. The CSMS sends a **ReserveNowRequest**

3. The Test System notifies the CSMS about the current state of the connector(s) of the Specified EVSE

4. The CSMS responds accordingly.

Message: **StatusNotificationRequest**
with **connectorStatus** *Reserved*
Message: **NotifyEventRequest**
with **trigger** *Delta*
actualValue *"Reserved"*
component.name *"Connector"*
variable.name *"AvailabilityState"*

Tool validations

* Step 1:

Message: **ReserveNowRequest**- **evseId** must be *<Specified evseId>*- **connectorType** must be omitted- **idToken.idToken** *<Configured valid_idtoken_idtoken>*- **idToken.type** *<Configured valid_idtoken_type>***Post scenario validations:****State** is *Reserved*

Unavailable

State	Unavailable
System under test	CSMS
Description	This state will simulate that Charging Station / EVSEs / connectors are set to AvailabilityState Unavailable.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Request the CSMS to change the availability of the specified components to Inoperative.

2. The Test System responds with a **ChangeAvailabilityResponse** with **status** *Accepted*

1. The CSMS sends a **ChangeAvailabilityRequest**

3. The Test System notifies the CSMS about the current state of all connectors belonging to the specified EVSE (and optionally also from the EVSE itself).

Message: **StatusNotificationRequest**

- **connectorStatus** *Unavailable*

Message: **NotifyEventRequest**

- **trigger** *Delta*

- **actualValue** *"Unavailable"*

- **component.name** *"ChargingStation" / EVSE / Connector*

- **variable.name** *"AvailabilityState"*

4. The CSMS responds accordingly.

Tool validations

* Step 1:

Message **ChangeAvailabilityRequest**

- **operationalStatus** *Inoperative*

- **evse** *<Specified evseId>*

- **connectorId** *omitted*

Post scenario validations:

State is *Unavailable*

EVConnectedPreSession

State	EVConnectedPreSession
System under test	CSMS
Description	This state will simulate that the EV and EVSE of the simulated Charging Station are connected.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS**

1. The Test System notifies the CSMS about the status change of the connector

Message: **StatusNotificationRequest**

- **connectorStatus** is *Occupied*

Message: **NotifyEventRequest**

- **trigger** is *Delta*

- **actualValue** is *Occupied*

- **component.name** is *Connector*

- **variable.name** is *AvailabilityState*

2. The CSMS responds accordingly.

3. The Test System sends a **TransactionEventRequest**

With **triggerReason** is *CablePluggedIn*

transactionInfo.chargingState is *EVConnected*

evse.id <Configured evseId>

evse.connectorId <Configured connectorId>

If **State** is *Authorized* then

eventType is *Updated*

else

eventType is *Started*

4. The CSMS responds with a **TransactionEventResponse**

Tool validations

N/a

Post scenario validations:

State is *EVConnectedPreSession*

Authorized

State	Authorized
System under test	CSMS
Description	This state will simulate that the EV Driver is locally authorizing to start a transaction on the simulated Charging Station.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station****CSMS****1.** The Test System sends an **AuthorizeRequest**With **idToken.idToken** <Configured valid_idtoken_idtoken>**idToken.type** <Configured valid_idtoken_type>**2.** The CSMS responds with an **AuthorizeResponse****3.** The Test System sends a **TransactionEventRequest**With **triggerReason** is *Authorized***idToken.idToken** <Configured valid_idtoken_idtoken>**idToken.type** <Configured valid_idtoken_type>If **State** is *EVConnectedPreSession* then**eventType** is *Updated*

else

eventType is *Started***4.** The CSMS responds with a **TransactionEventResponse****Tool validations**

* Step 2:

Message: **AuthorizeResponse**- **idTokenInfo.status** must be *Accepted*

* Step 4:

Message: **TransactionEventResponse**- **idTokenInfo.status** must be *Accepted***Post scenario validations:****State** is *Authorized*

EnergyTransferStarted

State	EnergyTransferStarted
System under test	CSMS
Description	This state will simulate that there is transferring energy between the EV and EVSE of the simulated Charging Station.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):If **State** is NOT *Authorized* then execute **Reusable State** *Authorized*If **EVConnected** is *true*, then proceed to part 2

Else proceed to part 1.

Main (Part 1) (Test scenario)

Charging Station	CSMS
1. The Test System notifies the CSMS about the status change of the connector. Message: StatusNotificationRequest - connectorStatus is <i>Occupied</i> Message: NotifyEventRequest - trigger is <i>Delta</i> - actualValue is <i>Occupied</i> - component.name is <i>Connector</i> - variable.name is <i>AvailabilityState</i>	2. The CSMS responds accordingly.
3. The Test System sends a TransactionEventRequest With triggerReason is <i>CablePluggedIn</i> transactionInfo.chargingState is <i>EVConnected</i> evse.id < <i>Configured evseId</i> > evse.connectorId < <i>Configured connectorId</i> > eventType is <i>Updated</i>	4. The CSMS responds with a TransactionEventResponse

Tool validations

N/a

Main (Part 2) (Test scenario)

Charging Station	CSMS
5. The Test System sends a TransactionEventRequest With triggerReason is <i>ChargingStateChanged</i> transactionInfo.chargingState is <i>Charging</i> eventType is <i>Updated</i>	6. The CSMS responds with a TransactionEventResponse

Tool validations

N/a

Post scenario validations:**State** is *EnergyTransferStarted***EVConnected** is *true*

EnergyTransferSuspended

State	EnergyTransferSuspended
System under test	CSMS
Description	This state will simulate that the Charging Station is in a state where the energy transfer is suspended by the EV.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):If **State** is NOT *EnergyTransferStarted* then execute **Reusable State** *EnergyTransferStarted***Main (Test scenario)****Charging Station**

CSMS

Notes(s): The tool will wait for <Configured Transaction Duration> seconds**1.** The Test System sends a **TransactionEventRequest**With **triggerReason** is *ChargingStateChanged***transactionInfo.chargingState** is *SuspendedEV***2.** The CSMS responds with a **TransactionEventResponse****Tool validations**

N/a

Post scenario validations:**State** is *EnergyTransferSuspended*

StopAuthorized

State	StopAuthorized
System under test	CSMS
Description	This state will simulate that the Charging Station is in a state where the charging session is authorized to stop.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):If **State** is NOT *EnergyTransferStarted* then execute **Reusable State** *EnergyTransferStarted***Main (Test scenario)****Charging Station**

CSMS

Notes(s): The tool will wait for <Configured Transaction Duration> seconds**1.** The Test System sends a **TransactionEventRequest**With **triggerReason** is *StopAuthorized***eventType** is *Updated***2.** The CSMS responds with a **TransactionEventResponse****Tool validations**

* Step 2:

Message: **TransactionEventResponse**- **idTokenInfo.status** must be *Accepted***Post scenario validations:****State** is *StopAuthorized*

EVConnectedPostSession

State	EVConnectedPostSession
System under test	CSMS
Description	This state will simulate that the Charging Station is in a state where the energy transfer has been stopped and the transaction is NOT authorized to resume energy transfer without re-authorization.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):If **State** is NOT *StopAuthorized* then execute **Reusable State** *StopAuthorized***Main (Test scenario)****Charging Station**

CSMS

1. The Test System sends a **TransactionEventRequest**

With **triggerReason** is *ChargingStateChanged*
transactionInfo.chargingState is *EVConnected*
eventType is *Updated*

2. The CSMS responds with a **TransactionEventResponse****Tool validations**

N/a

Post scenario validations:**State** is *EVConnectedPostSession*

EVDisconnected

State	EVDisconnected
System under test	CSMS
Description	This state will simulate that the EV and EVSE of the simulated Charging Station are disconnected, after the charging session is authorized to stop.

Before (Preparations)
Configuration State: N/a
Memory State: N/a
Reusable State(s): If State is NOT EVConnectedPostSession then execute Reusable State EVConnectedPostSession

Main (Test scenario)	
Charging Station	CSMS
1. The Test System notifies the CSMS about the status change of the connector. Message: StatusNotificationRequest - connectorStatus is Available Message: NotifyEventRequest - trigger is Delta - actualValue is Available - component.name is Connector - variable.name is AvailabilityState	2. The CSMS responds accordingly.
3. The Test System sends a TransactionEventRequest With triggerReason is EVCommunicationLost transactionInfo.chargingState is Idle transactionInfo.stoppedReason is EVDisconnected eventType is Ended	4. The CSMS responds with a TransactionEventResponse

Tool validations
N/a
Post scenario validations: State is EVDisconnected

GetInstalledCertificates

State	GetInstalledCertificates
System under test	CSMS
Description	The hashData from installed certificates of the specified type will be retrieved from the Charging Station

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Trigger the CSMS to send a *GetInstalledCertificateIdsRequest* with *certificateType_ <Specified certificateType>*

2. The Test System responds with a **GetInstalledCertificateIdsResponse**
 With **status** is *Accepted*
certificateHashDataChain contains an entry with following values:
certificateHashDataChain[0].certificateType is *<Specified certificateType>*
certificateHashDataChain[0].certificateHashData contains *<HashData from the configured certificate of the specified certificateType>*

1. The CSMS sends a **GetInstalledCertificateIdsRequest**

Tool validations

* Step 1:

Message: **GetInstalledCertificateIdsRequest**

- **certificateType** must be *<Specified certificateType>*

Post scenario validations:

Certificate of the specified *certificateType* is retrieved from the Charging Station.

CertificateInstalled

State	CertificateInstalled
System under test	CSMS
Description	A pre configured certificate of the specified certificateType will be installed.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)**Charging Station**

CSMS

Manual Action: Trigger the CSMS to send an InstallCertificateRequest with certificateType <Specified certificateType>

2. The Test System responds with a **InstallCertificateResponse**
With **status** is *Accepted*

1. The CSMS sends a **InstallCertificateRequest**

Tool validations

* Step 1:

Message: **InstallCertificateRequest**

- **certificateType** must be <Specified certificateType>

- **certificate** must be <The configured certificate of the specified certificateType.>

Post scenario validations:

Certificate of the specified certificateType is stored at the Charging Station.

ISO15118SmartCharging

State	ISO15118SmartCharging
System under test	CSMS
Description	A version for the ISO15118 can be specified as argument to this reusable state. This version is exposed as variable <iso15118Version> and can either be iso15118-2 or iso15118-20. If not specified iso15118-2 will be assumed.

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Before (Preparations)	
Reusable State(s): N/a	
Main (Test scenario)	
Charging Station	CSMS
1. The Test System sends a NotifyEVChargingNeedsRequest with evseId <Configured evseId> maxScheduleTuples 2 if <iso15118Version> is iso15118-2: chargingNeeds.requestedEnergyTransfer AC_three_phase chargingNeeds.acChargingParameters.energyAmount 50000 chargingNeeds.acChargingParameters.evMinCurrent 0 chargingNeeds.acChargingParameters.evMaxCurrent 32 chargingNeeds.acChargingParameters.evMaxVoltage 230 end if if <iso15118Version> is iso15118-20: chargingNeeds.requestedEnergyTransfer AC_single_phase chargingNeeds.availableEnergyTransfer [AC_single_phase, AC_three_phase, AC_BPT] chargingNeeds.controlMode ScheduledControl chargingNeeds.v2xChargingParameters.maxChargePower 10000 chargingNeeds.v2xChargingParameters.maxDischargePower 10000 end if	2. The CSMS responds with a NotifyEVChargingNeedsResponse .
4. The Test System responds with a SetChargingProfileResponse with: status Accepted	3. The CSMS sends a SetChargingProfileRequest <u>Note(s):</u> - If NotifyEVChargingNeedsResponseStatus was Processing, the Test System will wait 60 seconds for the request
5. The Test System sends a NotifyEVChargingScheduleRequest with evseId <Configured evseId> chargingSchedule <ChargingSchedule provided at step 3>	6. The CSMS responds with a NotifyEVChargingScheduleResponse .
7. The Test System sends a TransactionEventRequest with triggerReason <ChargingStateChanged> transactionInfo.chargingState <Charging>	8. The CSMS responds with a TransactionEventResponse .

Tool validations

* Step 2:

Message: **NotifyEVChargingNeedsResponse**

- **Status** must be *Accepted* or *Processing*

* Step 3:

Message: **SetChargingProfileRequest**

- **chargingProfilePurpose** must be *<TxProfile>*

- **transactionId** must be *<Provided transactionId from before>*

* Step 4:

Message: **NotifyEVChargingScheduleResponse**

- **status** must be *<Accepted>*

N/a

RenewChargingStationCertificate

State	RenewChargingStationCertificate
System under test	CSMS
Description	The ChargingStationCertificate is renewed using <u>A02/A03</u>

Before (Preparations)

Configuration State:

N/a

Memory State:

N/a

Reusable State(s):

N/a

Main (Test scenario)

Charging Station

CSMS

Manual Action: Request the CSMS to send a Trigger Message Request with **requestedMessage** SignChargingStationCertificate

2. The Test System sends a **TriggerMessageResponse** with **status** *Accepted*

1. The CSMS sends a **TriggerMessageRequest** With **requestedMessage** SignChargingStationCertificate

3 The Test System sends a **SignCertificateRequest**

4. The CSMS responds with a **SignCertificateResponse** with **status** *Accepted*

6. The Test System sends a **CertificateSignedResponse** with **status** *Accepted*

5. The CSMS sends a **CertificateSignedRequest** With **certificateChain** <Certificate generated from the received CSR from step 3 and signed by the configured CSMS Root certificate>
certificateType ChargingStationCertificate

Tool validations

* Step 1:

Message: **TriggerMessageRequest**

- **requestedMessage** must be SignChargingStationCertificate

* Step 4:

Message: **SignCertificateResponse**

- **status** must be *Accepted*

* Step 5:

Message: **CertificateSignedRequest**

- **certificateChain** <Certificate generated from the received CSR from step 3 and signed by the configured CSMS Root certificate>

- **certificateType** must be ChargingStationCertificate

Post scenario validations:

N/a

ISO15118-20 transaction started in V2X AC_BPT energy transfer mode

State	ISO15118-20 transaction started in V2X AC_BPT energy transfer mode
System under test	CSMS
Description	Starts ISO15118-20 transaction and upgrades to V2X AC_BPT energy transfer mode
Exposes	<allowedEnergyTransferModes> <transactionId>

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):**State** is ISO15118-20 transaction just started exposing:

- <allowedEnergyTransferModes>

- <transactionId>

Main (Test scenario)**Charging Station**

CSMS

*Agree on energy transfer mode***1. The Test System sends a NotifyEVChargingNeedsRequest**
with:**evseId** = <Configured evseId>**chargingNeeds.requestedEnergyTransfer** = AC_BPT**chargingNeeds.availableEnergyTransfer** = [AC_single_phase,
AC_two_phase, AC_three_phase, AC_BPT, AC_BPT_DER, AC_DER]**chargingNeeds.controlMode** = ScheduledControl**chargingNeeds.v2xChargingParameters.maxChargePower** =
1234W**chargingNeeds.v2xChargingParameters.maxDischargePower** =
0W**2. The CSMS responds with a**
NotifyEVChargingNeedsResponse**Tool validations**

* Step 2:

Message: **NotifyEVChargingNeedsResponse**- **status** must be *Accepted* or *Processing***Post scenario validations:**

N/a

ISO15118-20 transaction just started

State	ISO15118-20 transaction just started
System under test	CSMS
Description	Starts ISO15118-20 transaction
Exposes	<allowedEnergyTransferModes> <transactionId>
Prerequisite(s)	Test System reported support for V2XChargingCtrlr.V2XEnabled for configured EVSEs with value <i>true</i>

Before (Preparations)**Configuration State:**

N/a

Memory State:

N/a

Reusable State(s):State is *EVConnectedPreSession***Main (Test scenario)**

Charging Station	CSMS
1. The Test System sends an AuthorizeRequest with idToken.type <i>eMAID</i> idToken.type <Configured ISO15118 idToken> iso15118CertificateHashData <hashData's from MO certificate chain>	2. The CSMS responds with an AuthorizeResponse - allowedEnergyTransfer [<allowedEnergyTransferModes>]
3. The Test System sends a TransactionEventRequest with eventType <i>Started</i> transactionInfo.transactionId <transactionId> idToken.additionalInfo.additionalIdToken <Configured EVCCID> idToken.additionalInfo.additionalInfo.type <i>EVCCID</i>	4. The CSMS responds with a TransactionEventResponse

Tool validations

* Step 2:

Message: **AuthorizeResponse**- **idTokenInfo.status** must be *Accepted*- **certificateStatus** must be *Accepted*

* Step 4:

Message: **TransactionEventResponse**- **idTokenInfo.status** must be *Accepted***Post scenario validations:**

N/a

Last updated 2026-05-22 05:26:20 UTC

DRAFT